

TUGAS AKHIR - EF234801

**ANALISIS EFEKTIVITAS TEKNIK RESAMPLING DALAM
PENANGANAN IMBALANCED DATA UNTUK
MENINGKATKAN SENSITIVITAS MODEL ENSEMBLE
LEARNING PADA DATASET PENIPUAN DALAM
TRANSAKSI KARTU KREDIT**

Davin Fisabilillah Reynard Putra

NRP 5025221137

Dosen Pembimbing

Prof. Dr. Eng. Chastine Fatichah, S.Kom., M.Kom.

NIP 197512202001122002

Dosen Ko-Pembimbing

Ilham Gurat Adillion, S.Kom., M.Eng.

NIP 1995202411009

Program Studi S-1 Teknik Informatika

Departemen Teknik Informatika

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026

Halaman ini sengaja dikosongkan.



TUGAS AKHIR - EF234801

**ANALISIS EFEKTIVITAS TEKNIK RESAMPLING DALAM
PENANGANAN IMBALANCED DATA UNTUK
MENINGKATKAN SENSITIVITAS MODEL ENSEMBLE
LEARNING PADA DATASET PENIPUAN DALAM
TRANSAKSI KARTU KREDIT**

Davin Fisabilillah Reynard Putra

NRP 5025221137

Dosen Pembimbing

Prof. Dr. Eng. Chastine Fatichah, S.Kom., M.Kom.

NIP 197512202001122002

Dosen Ko-Pembimbing

Ilham Gurat Adillion, S.Kom., M.Eng.

NIP 1995202411009

Program Studi S-1 Teknik Informatika

Departemen Teknik Informatika

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026

Halaman ini sengaja dikosongkan.



FINAL PROJECT - EF234801

ANALYSIS OF THE EFFECTIVENESS OF RESAMPLING TECHNIQUES IN HANDLING IMBALANCED DATA TO INCREASE THE SENSITIVITY OF ENSEMBLE LEARNING MODELS ON FRAUD IN CREDIT CARD TRANSACTIONS DATASET

Davin Fisabilillah Reynard Putra

NRP 5025221137

Advisor

Prof. Dr. Eng. Chastine Fatichah, S.Kom., M.Kom.

NIP 197512202001122002

Co-Advisor

Ilham Gurat Adillion, S.Kom., M.Eng.

NIP 1995202411009

Study Program Bachelor of Informatics

Department of Informatics

Faculty of Intelligent Electrical and Informatics Technology

Institut Teknologi Sepuluh Nopember

Surabaya

2026

Halaman ini sengaja dikosongkan

LEMBAR PENGESAHAN

ANALISIS EFEKTIVITAS TEKNIK RESAMPLING DALAM PENANGANAN IMBALANCED DATA UNTUK MENINGKATKAN SENSITIVITAS MODEL ENSEMBLE LEARNING PADA DATASET PENIPUAN DALAM TRANSAKSI KARTU KREDIT

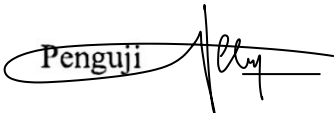
TUGAS AKHIR

Diajukan untuk memenuhi salah satu syarat
memperoleh gelar Sarjana Komputer pada
Program Studi S-1 Teknik Informatika
Departemen Teknik Informatika
Fakultas Teknologi Elektro dan Informatika Cerdas
Institut Teknologi Sepuluh Nopember

Oleh : Davin Fisabilillah Reynard Putra

NRP. 5025221137

Disetujui oleh Tim Penguji Tugas Akhir :

- | | |
|---|---|
| 1. Prof. Dr. Eng. Chastine Fatichah, S.Kom., M.Kom. | Pembimbing |
| 2. Ilham Gurat Adillion, S.Kom., M.Eng. | Ko-pembimbing |
| 3. Aldinata Rizky Revanda, S.Kom., M.Kom. | Penguji  |
| 4. Imam Mustafa Kamal, S.ST., Ph.D. | Penguji  |

SURABAYA

Juli, 2026

Halaman ini sengaja dikosongkan.

APPROVAL SHEET

ANALYSIS OF THE EFFECTIVENESS OF RESAMPLING TECHNIQUES IN HANDLING IMBALANCED DATA TO INCREASE THE SENSITIVITY OF ENSEMBLE LEARNING MODELS ON FRAUD IN CREDIT CARD TRANSACTIONS DATASET

FINAL PROJECT

Submitted to fulfill one of the requirements
for obtaining a Bachelor of Computer Science degree at
Undergraduate Study Program of Informatics
Department of Informatics
Faculty of Intelligent Electrical and Informatics Technology
Institut Teknologi Sepuluh Nopember

By: Davin Fisabilillah Reynard Putra

NRP. 5025221137

Approved by Final Project Examiner Team:

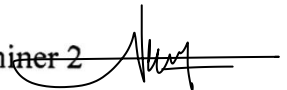
1. Prof. Dr. Eng. Chastine Fatichah, S.Kom., M.Kom. Advisor

2. Ilham Gurat Adillion, S.Kom., M.Eng. Co-advisor

3. Aldinata Rizky Revanda, S.Kom., M.Kom.

Examiner 

4. Imam Mustafa Kamal, S.ST., Ph.D.

Examiner 2 

SURABAYA

July, 2026

Halaman ini sengaja dikosongkan.

PERNYATAAN ORISINALITAS

Yang bertanda tangan di bawah ini:

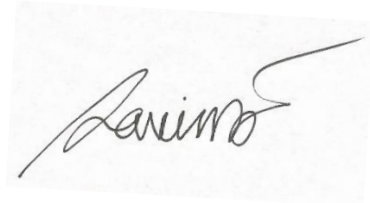
Nama mahasiswa / NRP : Davin Fisabilillah Reynard Putra/5025221137
Program studi : Teknik Informatika
Dosen Pembimbing / NIP : Prof. Dr. Eng. Chastine Fatichah, S.Kom., M.Kom./
197512202001122002
Ilham Gurat Adillion, S.Kom., M.Eng./ 1995202411009

dengan ini menyatakan bahwa Tugas Akhir dengan judul “ANALISIS EFEKTIVITAS TEKNIK RESAMPLING DALAM PENANGANAN IMBALANCED DATA UNTUK MENINGKATKAN SENSITIVITAS MODEL ENSEMBLE LEARNING PADA DATASET PENIPUAN DALAM TRANSAKSI KARTU KREDIT” adalah hasil karya sendiri, bersifat orisinal, dan ditulis dengan mengikuti kaidah penulisan ilmiah.

Bilamana di kemudian hari ditemukan ketidaksesuaian dengan pernyataan ini, maka saya bersedia menerima sanksi sesuai dengan ketentuan yang berlaku di Institut Teknologi Sepuluh Nopember.

Mengetahui
Dosen Pembimbing

Surabaya, 9 Juli 2026
Mahasiswa



Prof. Dr. Eng. Chastine Fatichah, S.Kom.,
M.Kom
NIP. 197512202001122002

Davin Fisabilillah Reynard Putra
NRP. 5025221137

Dosen Ko-pembimbing

Ilham Gurat Adillion, S.Kom., M.Eng.
NIP. 1995202411009

Halaman ini sengaja dikosongkan.

STATEMENT OF ORIGINALITY

The undersigned:

Student Name / Student ID : Davin Fisabilillah Reynard Putra/5025221137
Study Program : Bachelor of Informatics
Advisor / Employee ID : Prof. Dr. Eng. Chastine Fatichah, S.Kom., M.Kom./
197512202001122002
Co-advisor / Employee ID : Ilham Gurat Adillion, S.Kom., M.Eng./ 1995202411009

hereby declares that the Final Project entitled “ANALYSIS OF THE EFFECTIVENESS OF RESAMPLING TECHNIQUES IN HANDLING IMBALANCED DATA TO INCREASE THE SENSITIVITY OF ENSEMBLE LEARNING MODELS ON FRAUD IN CREDIT CARD TRANSACTIONS DATASET” is my own work, is original, and was written in accordance with the rules of scientific writing.

If any discrepancies with this statement are found in the future, I am willing to accept sanctions in accordance with the provisions of Institut Teknologi Sepuluh Nopember.

Acknowledged
Advisor

Surabaya, 9 July 2026
Student



Prof. Dr. Eng. Chastine Fatichah, S.Kom.,
M.Kom
NIP. 197512202001122002

Davin Fisabilillah Reynard Putra
NRP. 5025221137

Co-advisor

Ilham Gurat Adillion, S.Kom., M.Eng.
NIP. 1995202411009

Halaman ini sengaja dikosongkan.

PERNYATAAN KODE ETIK
PENGUNAAN AI GENERATIF
Code of Conduct Statement: Generative AI or AI-Assisted Usage

Saya yang bertanda tangan di bawah ini:
I, the undersigned:

Nama Mahasiswa / NRP : Davin Fisabilillah Reynard Putra/5025221137
Full Name / Student ID

Program Studi : S-1 Teknik Informatika
Study Program

Judul Tugas Akhir : Analisis Efektivitas Teknik Resampling dalam
Final Project Title Penanganan Imbalanced Data untuk Meningkatkan Sensitivitas Model Ensemble Learning pada Dataset Penipuan dalam Transaksi Kartu Kredit

dengan ini menyatakan bahwa pada Tugas Akhir dengan judul di atas tersebut:
Hereby declare that in the Final Project with the above title:

No.	Pernyataan <i>Statement</i>	(✓)
1	Saya hanya menggunakan AI generatif sebagai alat bantu untuk memperbaiki tata bahasa. AI generatif tidak digunakan untuk membuat isi Tugas Akhir. <i>I only used generative AI as a tool to improve the readability or language of the text in my Final Project. It was not used to generate a complete text of my work.</i>	✓
2	Saya telah memeriksa dan/atau memperbaiki seluruh bagian dari Tugas Akhir saya yang dibantu oleh AI generatif agar sesuai dengan baku mutu penulisan karya ilmiah. <i>I have reviewed and refined all aspects of my work that generative AI assists with, ensuring it adheres to the standards of academic writing.</i>	✓
3	Saya tidak menggunakan AI generatif untuk pembuatan data primer, grafik dan/atau tabel pada Tugas Akhir saya. <i>I did not use generative AI to generate primary data, figures, and/or tables in my work.</i>	✓
4	Saya telah memberikan atribusi/pengakuan terhadap alat AI yang digunakan, secara rinci pada suatu bagian pada lampiran. <i>I have acknowledged the use of generative AI in any part of the work in the specific appendix page.</i>	✓
5	Saya memastikan tidak ada plagiarisme, termasuk hal yang berasal dari penggunaan AI generatif. <i>I have ensured that there is no plagiarism issue in the work, including any parts generated by AI.</i>	✓

Surabaya, 9 Juli 2026
Mahasiswa



Davin Fisabilillah Reynard
Putra
NRP. 5025221137

Halaman ini sengaja dikosongkan.

ABSTRAK

ANALISIS EFEKTIVITAS TEKNIK RESAMPLING DALAM PENANGANAN IMBALANCED DATA UNTUK MENINGKATKAN SENSITIVITAS MODEL ENSEMBLE LEARNING PADA DATASET PENIPUAN DALAM TRANSAKSI KARTU KREDIT

Nama Mahasiswa / NRP : Davin Fisabilillah Reynard Putra / 5025221137
Departemen : Teknik Informatika FTEIC - ITS
Dosen Pembimbing : Prof. Dr. Eng. Chastine Fatichah, S.Kom., M.Kom.
Dosen Ko-pembimbing : Ilham Gurat Adillion, S.Kom., M.Eng.

Abstrak

Deteksi *fraud* pada transaksi kartu kredit merupakan permasalahan penting dalam *machine learning* karena karakteristik datanya yang sangat tidak seimbang (*imbalanced data*), sehingga model tidak dapat mempelajari dengan baik pola transaksi *fraud* dikarenakan volume datanya yang sangat sedikit. Penelitian ini bertujuan menganalisis efektivitas berbagai teknik *resampling* dalam menangani *imbalanced data* untuk meningkatkan sensitivitas model *ensemble learning* pada kasus deteksi *fraud* transaksi kartu kredit. Penelitian menggunakan dataset *Credit Card Fraud Detection* yang terdiri dari 284.807 transaksi dengan proporsi *fraud* sebesar 0,172%. Tahapan penelitian meliputi *data preprocessing*, penerapan delapan teknik *resampling* yaitu *Random Oversampling*, *Random Undersampling*, SMOTE, ADASYN, *Tomek Links*, *Edited Nearest Neighbor* (ENN), SMOTE-ENN, dan SMOTE-Tomek, serta pelatihan model *Random Forest*, *Gradient Boosting*, *XGBoost*, *Soft Voting*, dan *Stacking*. Evaluasi dilakukan menggunakan metrik *recall*, *precision*, *pr-auc*, dan *F1-Score* dengan *recall* sebagai metrik utama. Selain itu, dilakukan *hyperparameter tuning* menggunakan *RandomizedSearchCV* pada lima kombinasi terbaik berdasarkan *F1-Score*. Hasil penelitian menunjukkan bahwa efektivitas teknik *resampling* berbeda pada setiap model. Teknik ENN menghasilkan rata-rata *F1-Score* tertinggi dengan nilai 0,8314. Untuk tujuan *fraud detection* yang memprioritaskan *recall* atau performa model dalam mendeteksi transaksi *fraud*, kombinasi *XGBoost* dan ENN menjadi yang paling efektif dengan *recall* sebesar 0,8367, *precision* senilai 0,9111, *pr-auc* sebesar 0,8517, dan *F1-Score* dengan nilai 0,8723. Penelitian ini menyimpulkan bahwa teknik pembersihan data seperti ENN dan *Tomek Links* lebih efektif dibandingkan teknik yang menyeimbangkan distribusi kelas yang agresif dalam meningkatkan kinerja sistem *deteksi fraud*.

Kata kunci: Fraud Detection, Imbalanced Data, Resampling, Ensemble Learning, XGBoost, Edited Nearest Neighbor, Recall.

Halaman ini sengaja dikosongkan.

ABSTRACT

ANALYSIS OF THE EFFECTIVENESS OF RESAMPLING TECHNIQUES IN HANDLING IMBALANCED DATA TO INCREASE THE SENSITIVITY OF ENSEMBLE LEARNING MODELS ON FRAUD IN CREDIT CARD TRANSACTIONS DATASET

Full Name / Student ID : Davin Fisabilillah Reynard Putra / 5025221137
Department : Informatics ELECTICS - ITS
Advisor : Prof. Dr. Eng. Chastine Fatichah, S.Kom., M.Kom.
Co-advisor : Ilham Gurat Adillion, S.Kom., M.Eng.

Abstract

Credit card fraud detection is a major challenge in machine learning due to the highly imbalanced nature of the transaction data, where fraudulent transactions represent only a small portion of all observations. The imbalance of the data volume between classes causes the models to not properly learn the patterns and characteristics of fraudulent transactions due to the limited amount of data. This study aims to analyze the effectiveness of various resampling techniques in handling imbalanced data to improve the sensitivity of ensemble learning models for credit card fraud detection. The research used the Credit Card Fraud Detection dataset consisting of 284,807 transactions, with fraudulent transactions accounting for only 0.172% of the data. The methodology included data preprocessing, the application of eight resampling techniques, namely Random Oversampling, Random Undersampling, SMOTE, ADASYN, Tomek Links, Edited Nearest Neighbor (ENN), SMOTE-ENN, and SMOTE-Tomek, followed by the training of Random Forest, Gradient Boosting, XGBoost, and Stacking models. Model performance was evaluated using Recall, Precision, pr-auc, and F1-Score, with Recall serving as the primary metric. Hyperparameter tuning using RandomizedSearchCV was also conducted on the top five model-resampling combinations. The results show that the effectiveness of resampling techniques varies across models. ENN achieved the highest average F1-Score with the score of 0.8314. From a fraud detection perspective where recall is prioritized because it measures the model's performance based on the ability to detect fraudulent transactions, the XGBoost and ENN combination proved to be the most effective, achieving a Recall of 0.8367, Precision of 0.9111, PR-AUC of 0.8517, and F1-Score of 0.8723. The study concludes that data cleaning techniques such as ENN and Tomek Links are more effective than aggressive class-balancing approaches in improving fraud detection performance.

Keywords: *Fraud detection, Imbalanced data, Resampling, Ensemble learning, XGBoost, Edited Nearest Neighbor, Recall.*

Halaman ini sengaja dikosongkan.

KATA PENGANTAR

Puji syukur penulis panjatkan kehadirat Allah Subhanahu wa ta'ala atas segala rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan Tugas Akhir ini dengan judul "Analisis Efektivitas Teknik Resampling dalam Penanganan Imbalanced Data untuk Meningkatkan Sensitivitas Model Ensemble Learning pada Dataset Penipuan dalam Transaksi Kartu Kredit" sebagai salah satu syarat untuk memperoleh gelar Sarjana Komputer pada Departemen Teknik Informatika di Institut Teknologi Sepuluh Nopember.

Penulisan Tugas Akhir ini tidak akan terlaksana dengan baik tanpa bimbingan, dukungan, dan motivasi dari berbagai pihak. Oleh karena itu, penulis ingin menyampaikan ucapan terima kasih yang sebesar-besarnya kepada:

1. Ibu, Bapak, dan seluruh keluarga tercinta yang senantiasa memberikan doa, kasih sayang, dukungan moral maupun materiil, serta kepercayaan yang tidak pernah putus kepada penulis. Terima kasih atas segala pengorbanan, kesabaran, dan motivasi yang diberikan sehingga penulis dapat menyelesaikan pendidikan dan Tugas Akhir ini dengan baik.
2. Ibu Prof. Dr. Eng. Chastine Fatichah, S.Kom., M.Kom. dan Bapak Ilham Gurat Adillion, S.Kom., M.Eng. selaku Dosen Pembimbing I dan II yang telah meluangkan waktunya untuk memberikan bimbingan, saran, serta kesabaran dalam mengarahkan penulis selama proses penyusunan Tugas Akhir ini.
3. Bapak Aldinata Rizky Revanda, S.Kom., M.Kom. dan Bapak Imam Mustafa Kamal, S.ST., Ph.D. selaku Dosen Penguji yang telah memberikan masukan, kritik, dan saran konstruktif demi perbaikan Tugas Akhir ini.
4. Seluruh Dosen, Tenaga Kependidikan, dan Staf Departemen Teknik Informatika yang telah memberikan bekal ilmu dan bantuan selama masa perkuliahan.
5. Teman-teman Page yang telah menjadi bagian penting dalam perjalanan hidup penulis sejak masa sekolah menengah pertama hingga saat ini. Terima kasih atas persahabatan, dukungan, serta berbagai kenangan yang telah memberikan warna dan semangat dalam setiap tahap kehidupan penulis.
6. Teman-teman Nonton yang senantiasa menemani dan memberikan dukungan selama perjalanan perkuliahan. Kebersamaan, canda tawa, dan berbagai pengalaman yang telah dilalui bersama menjadi salah satu sumber motivasi bagi penulis dalam menyelesaikan masa studi ini.
7. Real Madrid, Scuderia Ferrari Formula 1, Los Angeles Lakers, serta Marc Márquez, yang secara tidak langsung memberikan inspirasi kepada penulis. Mereka juga menjadi sumber hiburan dan antusiasme yang selalu menemani penulis selama proses perkuliahan ini.

Penulis menyadari bahwa Tugas Akhir ini masih jauh dari sempurna. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun untuk perbaikan di masa mendatang.

Penulis berharap Tugas Akhir ini dapat bermanfaat sebagai referensi bagi penelitian selanjutnya dan perkembangan ilmu pengetahuan dan teknologi khususnya di bidang industri keuangan.

DAFTAR ISI

LEMBAR PENGESAHAN	i
APPROVAL SHEET.....	iii
PERNYATAAN ORISINALITAS	v
STATEMENT OF ORIGINALITY	vii
PERNYATAAN KODE ETIK PENGGUNAAN AI GENERATIF	ix
ABSTRAK	xi
ABSTRACT	xiii
KATA PENGANTAR.....	xv
DAFTAR ISI	xvii
DAFTAR GAMBAR.....	xxi
DAFTAR TABEL	xxiii
DAFTAR KODE SEMU	xxv
BAB 1 PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Batasan Masalah	3
1.4 Tujuan	3
1.5 Manfaat	4
1.5.1 Manfaat Teoritis	4
1.5.2 Manfaat Praktis.....	4
1.5.3 Manfaat Sosial	4
BAB 2 TINJAUAN PUSTAKA.....	5
2.1 Hasil Penelitian Terdahulu.....	5
2.2 Hasil Penelitian State of The Art	9
2.3 Dasar Teori.....	13
2.3.1 Penipuan dalam Transaksi Kartu Kredit.....	13
2.3.2 Data Preprocessing	14
2.3.3 Single Classifier.....	17
2.3.4 Ensemble Learning.....	19
2.3.5 Teknik Resampling.....	25
2.3.6 Hyperparameter Tuning.....	33

2.3.7	Cross Validation	35
2.3.8	Metrik Evaluasi	36
BAB 3	METODOLOGI	41
3.1	Metode yang digunakan	41
3.1.1	Dataset Credit Card Fraud Detection	41
3.1.2	Load Data	42
3.1.3	Exploratory Data Analysis (EDA).....	43
3.1.4	Data Preprocessing	47
3.1.5	Data Resampling	49
3.1.6	Model Training.....	53
3.1.7	Model Testing.....	55
3.1.8	Result Evaluation.....	56
3.1.9	Hyperparameter Tuning.....	57
3.1.10	Analisis and Evaluasi	63
3.1.11	Skenario Pengujian	64
3.2	Bahan dan peralatan yang digunakan.....	65
3.3	Lingkungan Pengembangan Penelitian	66
BAB 4	HASIL DAN PEMBAHASAN	67
4.1	Hasil penelitian	67
4.1.1	Hasil Pengujian Model Baseline (Tanpa Penerapan Teknik Resampling).....	67
4.1.2	Hasil Penerapan Teknik Resampling terhadap Distribusi Data.....	68
4.1.3	Hasil Pengujian Model dengan Data Resampling	70
4.1.4	Pemeringkatan Kombinasi Model dan Teknik Resampling Berdasarkan Metrik Evaluasi	76
4.1.5	Hasil Pengujian Model dengan Hyperparameter Tuning	80
4.2	Pembahasan.....	86
4.2.1	Analisis Pengaruh Teknik Resampling terhadap Distribusi Data	86
4.2.2	Pengaruh Teknik Resampling terhadap Kinerja Model Ensemble Learning ...	92
4.2.3	Analisis Performa Model Ensemble Learning.....	97
4.2.4	Pengaruh Hyperparameter Tuning terhadap Kinerja Model Ensemble Learning	101
4.2.5	Analisis Kombinasi Model dan Teknik Resampling Terbaik	104
BAB 5	KESIMPULAN DAN SARAN	111
5.1	Kesimpulan	111
5.2	Saran	112

DAFTAR PUSTAKA.....	113
LAMPIRAN	119
BIODATA PENULIS.....	153

Halaman ini sengaja dikosongkan

DAFTAR GAMBAR

Gambar 2.1 Konsep principal component analysis	16
Gambar 2.2 Hyperplane svm.....	18
Gambar 2.3 Konsep random forest.....	21
Gambar 2.4 Konsep Gradient boosting	23
Gambar 2.5 Konsep hard dan soft voting.....	24
Gambar 2.6 Konsep stacking.....	25
Gambar 2.7 Konsep random oversampling	26
Gambar 2.8 Konsep SMOTE.....	27
Gambar 2.9 Contoh penerapan ADASYN	28
Gambar 2.10 Konsep Random Undersampling.....	29
Gambar 2.11 Contoh Tomek Links	30
Gambar 2.12 Contoh ENN	30
Gambar 2.13 Contoh SMOTE-ENN	32
Gambar 2.14 Contoh SMOTE-Tomek Links	33
Gambar 2.15 Perbandingan GridSearch dengan RandomSearch	35
Gambar 2.16 Visualisasi CrossValidation.....	36
Gambar 2.17 Confusion Matrix.....	37
Gambar 3.1 Diagram Alir Penelitian.....	41
Gambar 3.2 Distribusi kelas target	42
Gambar 3.3 Contoh entri data	42
Gambar 3.4 Statistik deskriptif dataset.....	43
Gambar 3.5 Countplot fitur target	44
Gambar 3.6 Countplot fitur amount	44
Gambar 3.7 Countplot fitur time	45
Gambar 3.8 Histogram fitur time	45
Gambar 3.9 Histogram fitur amount.....	46
Gambar 3.10 Correlation heatmap matrix	47
Gambar 3.11 Alur tahapan preprocessing	47
Gambar 4.1 Perbandingan Rasio Fraud.....	87
Gambar 4.2 Perbandingan Distribusi Data.....	87
Gambar 4.3 Perbandingan persebaran Data setelah SMOTE.....	88
Gambar 4.4 Perbandingan persebaran Data setelah ADASYN.....	88
Gambar 4.5 Perbandingan persebaran Data setelah Random Oversampling	89
Gambar 4.6 Persebaran Data Sebelum Sampling.....	89
Gambar 4.7 Persebaran Data setelah Random Undersampling.....	90
Gambar 4.8 Persebaran Data setelah Tomek Links.....	90
Gambar 4.9 Persebaran Data setelah ENN	91
Gambar 4.10 Persebaran Data setelah SMOTE-ENN	91
Gambar 4.11 Persebaran Data setelah SMOTE-Tomek.....	92
Gambar 4.12 Heatmap berdasarkan F1-Score	93
Gambar 4.13 Scatter Plot Trade-off Precision dan Recall.....	94
Gambar 4.14 Heatmap Perubahan Recall terhadap Baseline	96
Gambar 4.15 Rata-rata Metrik Berdasarkan Teknik Resampling	96

Gambar 4.16 Rata-rata perubahan Metrik terhadap Baseline.....	96
Gambar 4.17 Rata-rata Nilai Metrik Berdasarkan Model	97
Gambar 4.18 Stability Plot Model Berdasarkan F1-Score	100
Gambar 4.19 Stability Plot Model Berdasarkan Recall.....	100
Gambar 4.20 Selisih Nilai Sebelum dan Sesudah Tuning.....	101
Gambar 4.21 Perbandingan Sebelum dan Sesudah Tuning.....	102
Gambar 4.22 Kombinasi dengan F1-Score Terbaik	105
Gambar 4.23 Confusion Matrix Kombinasi Terbaik.....	107
Gambar 4.24 Grafik PR-AUC Kombinasi Terbaik	109

DAFTAR TABEL

Tabel 2.1 Parameter RandomSearchCV	34
Tabel 3.1 Parameter SMOTE	50
Tabel 3.2 Parameter ADASYN	50
Tabel 3.3 Parameter Random Oversampling	50
Tabel 3.4 Parameter Random Undersampling	51
Tabel 3.5 Parameter Tomek Links	51
Tabel 3.6 Paramater ENN	51
Tabel 3.7 Parameter SMOTE-ENN	52
Tabel 3.8 Parameter SMOTE-Tomek Links	53
Tabel 3.9 Parameter RandomsearchCV	57
Tabel 3.10 Ruang pencarian parameter Random Forest	58
Tabel 3.11 Ruang pencarian parameter XGBoost	59
Tabel 3.12 Ruang pencarian parameter GradientBoosting	60
Tabel 3.13 Ruang pencarian parameter Soft Voting	61
Tabel 3.14 Ruang pencarian parameter Stacking	62
Tabel 3.15 Spesifikasi perangkat lunak	65
Tabel 3.16 Spesifikasi perangkat keras	66
Tabel 3.17 Pustaka Pengembangan Penelitian	66
Tabel 4.1 Hasil pengujian baseline	67
Tabel 4.2 Distribusi data sebelum dan sesudah resampling	69
Tabel 4.3 Hasil pengujian seluruh kombinasi	70
Tabel 4.4 Hasil pengujian per Teknik Resampling	73
Tabel 4.5 Hasil pengujian per Model	74
Tabel 4.6 Pemeringkatan berdasarkan F1-Score	76
Tabel 4.7 Pemeringkatan berdasarkan Recall	77
Tabel 4.8 Pemeringkatan berdasarkan Precision	79
Tabel 4.9 Hasil Skenario Hyperparamter Tuning	80
Tabel 4.10 Parameter terbaik Baseline-Random Forest	82
Tabel 4.11 Parameter Terbaik Tomek Links-Random Forest	83
Tabel 4.12 Parameter Terbaik Random Oversampling-Random Forest	83
Tabel 4.13 Parameter Terbaik ENN-XGBoost	84
Tabel 4.14 Parameter Terbaik Base Learner ENN-Stacking Random Forest	85
Tabel 4.15 Base Learner ENN-Stacking XGBoost	85
Tabel 4.16 Base Learner ENN-Stacking Gradient Boosting	85

Halaman ini sengaja dikosongkan

DAFTAR KODE SEMU

Kode Semu 3.1 Cek Kualitas Data	48
Kode Semu 3.2 Pemisahan fitur prediktor dan target.....	48
Kode Semu 3.3 Train-test split.....	49
Kode Semu 3.4 Feature scaling	49
Kode Semu 3.5 Resampling data.....	53
Kode Semu 3.6 Training Model	55
Kode Semu 3.7 Model testing	56
Kode Semu 3.8 Hyperparameter tuning	63

Halaman ini sengaja dikosongkan

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Seiring dengan meningkatnya adopsi teknologi digital dan *e-commerce*, penggunaan kartu kredit untuk transaksi online terus berkembang pesat. Namun, kemudahan ini juga membuka peluang bagi para penipu untuk melakukan aktivitas penipuan yang merugikan institusi keuangan dan pemilik kartu. Data dari *British Ministry of Finance Annual Fraud Reporting* menunjukkan bahwa pada tahun 2022, total 1,2 miliar *pounds* diperoleh secara ilegal melalui operasi kriminal, dengan kerugian mencapai 2.300 *pounds* setiap menitnya (Ahmed et al., 2025). *Remote purchase fraud*, di mana penipu menggunakan informasi kartu yang dicuri untuk bertransaksi secara online atau melalui telepon/pos, menjadi penyebab umum kerugian finansial yang sangat besar dengan total kerugian sebesar 395.7 juta *pounds* (Khalid et al., 2024).

Meningkatnya kasus penipuan dalam transaksi kartu kredit telah mendorong beberapa penelitian untuk menerapkan metode *machine learning* tradisional dan algoritma berbasis AI yang lebih canggih dalam deteksi fraud. Namun, salah satu hambatan signifikan dalam menyelesaikan masalah klasifikasi transaksi penipuan dan transaksi sah adalah distribusi data yang sangat tidak seimbang (*imbalanced*). Sebagai contoh, dalam dataset yang akan digunakan di penelitian ini, distribusi kelas transaksi *fraud* sangat kecil dibandingkan dengan total transaksinya dengan hanya 0,172% dari 284.807 transaksi adalah transaksi *fraud*, yang artinya hanya ada 492 data (Prasad et al., 2023). Ketidakseimbangan data kelas ini menciptakan tantangan serius karena algoritma pembelajaran akan memiliki kesulitan dalam melatih model pada kelas minoritas. Model pembelajaran akan cenderung memprioritaskan (*bias*) kelas mayoritas (*non-fraud*), yang dapat mengakibatkan *overfitting* dan kinerja evaluasi yang buruk (Lokanan & Sharma, 2022).

Kesalahan dalam klasifikasi terus menjadi tantangan signifikan dalam mengidentifikasi *fraud* dalam transaksi kartu kredit (Barongo & Mbelwa, 2024). Penting untuk meminimalkan transaksi *fraud* yang tidak terdeteksi atau false *negatives* guna menghindari ketidaknyamanan bagi pelanggan dan mempertahankan kepercayaan dalam sistem institusi penyedia layanan. Optimalisasi *trade-off* antara prediksi positif yang salah dan negatif yang salah (kasus *fraud* yang tidak terdeteksi) merupakan isu yang sangat sensitif yang memerlukan penyesuaian presisi dari algoritma deteksi (Ahmed et al., 2025). Oleh karena itu penggunaan sensitivitas (*recall*) menjadi metrik penting karena tingginya biaya ketika ada kasus *fraud* yang terlewatkan.

Untuk mengatasi tantangan penanganan imbalanced data, berbagai teknik *resampling* telah diuji dalam berbagai penelitian. Teknik *oversampling* seperti *Synthetic Minority Oversampling Technique (SMOTE)* telah terbukti efektif dalam mengatasi *imbalanced data* dengan menghasilkan sampel sintetis untuk kelas minoritas berdasarkan interpolasi (Odeyale et al., 2024). Sebaliknya, teknik *undersampling* mengurangi sampel kelas mayoritas untuk menciptakan keseimbangan dataset. Penelitian oleh Ahmed et al. (2025) menunjukkan bahwa teknik hybrid yang menggabungkan *SMOTE* dengan *Edited Nearest Neighbor (ENN)* dapat meningkatkan keseimbangan dataset dan meminimalkan kemungkinan *overfitting* terhadap instansi yang berisi noise atau menyesatkan. *SMOTE-ENN* secara khusus telah menunjukkan efektivitas yang lebih besar dalam menangani data imbalanced dibandingkan dengan teknik *oversampling* dan *undersampling* tradisional, karena *ENN* juga menghilangkan sampel yang

overlap dan berisi *noise* setelah *SMOTE* menghasilkan sampel sintetis (Bonde & Bichanga, 2025).

Selain penanganan *imbalanced data*, *ensemble learning* menjadi salah satu pendekatan yang sangat efektif untuk *fraud detection*. Dalam penelitian ini, *Ensemble learning models* akan digunakan untuk mengetahui performanya ketika diimplementasikan teknik resampling untuk penanganan ketidakseimbangan datanya. Algoritma yang akan dibandingkan adalah *Random Forest*, *XGBoost*, *Gradient Boosting*, *Stacking*, dan *Soft Voting Classifiers* yang menggabungkan kekuatan dari beberapa *learner* untuk meningkatkan kemampuan prediktif keseluruhan model (Chhabra et al., 2024). Penelitian oleh Bagga et al. (2020) menunjukkan bahwa pendekatan *ensemble* menghasilkan kinerja yang sangat baik untuk deteksi *fraud* pada dataset yang sangat *imbalanced* khususnya dalam performa untuk mendeteksi kelas positif (*true positive rate*) atau *recall*. Penelitian oleh Azim Mim et al. (2024) menguji pendekatan *soft voting classifier ensemble learning* untuk mendeteksi *fraud* kartu kredit pada data *imbalanced*, membandingkan berbagai teknik *resampling* termasuk *hybrid sampling*, *undersampling*, dan *oversampling*. Penelitian tersebut menemukan bahwa metode *soft voting* memberikan kinerja yang lebih baik dibandingkan *individual classifiers*. Penelitian oleh Qaddoura & Biltawi (2022) menunjukkan bahwa teknik *oversampling* dapat meningkatkan kinerja model, meskipun hasilnya masih tergantung pada algoritma *machine learning* yang digunakan.

Meskipun sudah ada penelitian mengenai topik integrasi *resampling techniques* dengan *ensemble learning*, masih terdapat *gap* dalam literatur mengenai analisis efektivitas komparatif berbagai teknik *resampling* khususnya *hybrid resampling* seperti *SMOTE-ENN*, *SMOTE-Tomek Links*, serta teknik *undersampling* dan *oversampling* untuk meningkatkan sensitivitas (*recall*) model *ensemble learning* pada dataset *fraud* kartu kredit. Mayoritas studi terfokus pada akurasi atau kombinasi metrik tanpa memberikan perhatian khusus terhadap *recall* sebagai prioritas utama dalam mendeteksi penipuan, sedangkan data *fraud* yang tidak terdeteksi (*false negatives*) memiliki konsekuensi finansial yang jauh lebih serius dibandingkan *false positives* (Chung & Lee, 2023)

Penelitian ini dirancang untuk mengisi *gap* tersebut dengan melakukan analisis komprehensif terhadap efektivitas berbagai teknik *resampling* baik *hybrid* maupun *non-hybrid* dalam meningkatkan sensitivitas model *ensemble learning* pada dataset *fraud* kartu kredit yang sangat *imbalanced*. Dengan fokus pada sensitivitas sebagai metrik utama evaluasi, penelitian ini diharapkan dapat memberikan rekomendasi praktis mengenai kombinasi teknik *resampling* dan *ensemble architecture* yang paling efektif untuk aplikasi *real-world fraud detection systems* dalam institusi keuangan.

1.2 Rumusan Masalah

Permasalahan utama dalam penelitian ini adalah mengetahui pengaruh teknik resampling dalam menangani ketidakseimbangan kelas pada dataset penipuan transaksi kartu kredit agar model *ensemble learning* mampu mendeteksi kelas transaksi penipuan (kelas minoritas) dengan sensitivitas yang lebih tinggi tanpa menurunkan kualitas prediksi secara keseluruhan. Berdasarkan latar belakang tersebut, rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana perbandingan kinerja model *Ensemble Learning* pada kondisi *baseline* (tanpa *resampling*) dibandingkan dengan setelah penerapan teknik *resampling* pada

dataset penipuan dalam transaksi kartu kredit yang sangat *imbalanced*, khususnya ditinjau dari aspek sensitivitas (*recall*) untuk deteksi kelas *fraud*?

2. Seberapa efektif masing-masing teknik *resampling* (*hybrid sampling*, *undersampling*, dan *oversampling*) dalam mengatasi *imbalanced data*?
3. Kombinasi teknik *resampling* dan model *Ensemble Learning* manakah yang menghasilkan performa paling optimal dalam mendeteksi *fraud*?
4. Bagaimana pengaruh penerapan *hyperparameter tuning* terhadap performa model *Ensemble Learning* dalam mendeteksi transaksi *fraud* jika dibandingkan dengan model yang menggunakan parameter *default*?

1.3 Batasan Masalah

Penelitian ini memiliki beberapa batasan yang perlu diperhatikan untuk memastikan fokus dan keterbatasan dalam proses dan hasil penelitian. Batasan-batasan ini ditetapkan untuk mengarahkan penelitian agar tetap berada dalam lingkup yang dapat dikelola dan menghasilkan hasil yang relevan. Adapun batasan masalah dalam penelitian ini adalah sebagai berikut:

1. Data yang digunakan adalah dataset dari kaggle dengan judul *Credit Card Fraud Detection* yang dikumpulkan oleh *Machine Learning Group* - Université Libre de Bruxelles.
2. Penelitian berfokus pada kasus klasifikasi biner, yaitu *fraud* (1) dan *non-fraud* (0), tanpa membahas multi-class fraud atau jenis fraud berdasarkan kategori tertentu.
3. Teknik *hybrid sampling* yang akan digunakan adalah SMOTE-ENN dan SMOTE-Tomek Links.
4. Teknik *oversampling* yang akan digunakan adalah SMOTE (*Synthetic Minority Over-sampling Technique*), ADASYN (*Adaptive Synthetic Sampling*), dan Random Oversampling.
5. Teknik *undersampling* yang akan digunakan adalah Random Undersampling, Tomek Links, dan Edited Nearest Neighbor (ENN).
6. Algoritma *Machine Learning* yang diujikan dibatasi pada *Random Forest*, *Gradient Boosting*, *XGBoost*, *Soft Voting Classifier* (Kombinasi *Logistic Regression*, *Decision Tree*, *Support Vector Machine (SVM)*), dan *Stacking* (Stacked Generalization) dengan *base learner* *Random Forest*, *XGBoost*, dan *Gradient Boosting* serta *meta learner* *Logistic Regression*.
7. Hyperparameter tuning dilakukan menggunakan metode *RandomizedSearchCV* pada lima kombinasi model dan teknik *resampling* yang memperoleh nilai F1-Score tertinggi pada tahap evaluasi awal dengan parameter *default*.
8. Metrik yang digunakan untuk mengevaluasi performa model adalah *recall*, *precision*, *F1-score*, *accuracy*, *pr-auc*, serta *confusion matrix*.
9. Penelitian tidak melakukan rekayasa fitur berbasis domain karena fitur pada dataset sudah didominasi oleh hasil PCA dan keterbatasan informasi asli transaksi.

1.4 Tujuan

Penelitian ini bertujuan untuk menganalisis efektivitas teknik *resampling* dalam menangani *imbalanced data* untuk meningkatkan sensitivitas model *ensemble learning* pada pendeteksian penipuan transaksi kartu kredit. Adapun tujuan spesifik dari penelitian ini adalah sebagai berikut:

1. Menganalisis perbandingan kinerja model *Ensemble Learning* kondisi *baseline* dengan kondisi setelah penerapan teknik *resampling* untuk membuktikan efek peningkatan nilai sensitivitas (*recall*) pada deteksi transaksi *fraud*.
2. Mengevaluasi efektivitas dari berbagai teknik *resampling* yang meliputi *hybrid sampling*, *undersampling*, dan *oversampling* dalam menangani ketidakseimbangan data yang ekstrem
3. Mengidentifikasi kombinasi teknik *resampling* dan model *Ensemble Learning* yang menghasilkan performa pendeteksian transaksi penipuan paling optimal.
4. Menganalisis pengaruh penerapan hyperparameter tuning terhadap performa model *Ensemble Learning* melalui perbandingan hasil model sebelum dan sesudah proses tuning

1.5 Manfaat

Penelitian Tugas Akhir ini diharapkan dapat memberikan berbagai manfaat, baik secara teoritis maupun praktis, dalam analisis efektivitas teknik *resampling* dalam menangani *imbalanced data* untuk meningkatkan sensitivitas model *ensemble learning* pada pendeteksian penipuan transaksi kartu kredit. Adapun manfaat penelitian ini adalah sebagai berikut.

1.5.1 Manfaat Teoritis

1. Memberikan kontribusi literasi ilmiah terkait penanganan *imbalanced data* melalui teknik *resampling* serta pengaruhnya terhadap peningkatan sensitivitas pada kasus klasifikasi *fraud* transaksi kartu kredit.
2. Menjadi referensi akademik mengenai evaluasi komparatif kinerja beberapa model *ensemble learning* pada dataset *fraud* yang sangat tidak seimbang, terutama dari sudut pandang metrik yang berorientasi pada kelas minoritas.
3. Mengidentifikasi faktor-faktor yang mempengaruhi kinerja model *ensemble learning*.

1.5.2 Manfaat Praktis

1. Menjadi acuan bagi proses *modeling pipeline* yang dapat diadaptasi untuk kasus serupa, sehingga mempercepat eksperimen dan pengembangan model deteksi *fraud*.
2. Memberikan rekomendasi teknis bagi pengembang sistem deteksi *fraud* terkait kombinasi teknik *resampling* dan model *ensemble learning* yang lebih efektif dalam meningkatkan kemampuan menangkap kasus *fraud*.
3. Membantu meningkatkan kualitas pengambilan keputusan dalam proses investigasi transaksi dengan menyediakan model yang lebih sensitif terhadap indikasi *fraud*, tanpa hanya bergantung pada akurasi yang berpotensi menyesatkan pada dataset sangat *imbalanced*.

1.5.3 Manfaat Sosial

1. Berkontribusi dalam upaya pengurangan kerugian finansial yang dialami nasabah maupun institusi keuangan akibat transaksi *fraud*, melalui peningkatan deteksi dini pada kelas *fraud*.
2. Meningkatkan rasa aman dan kepercayaan masyarakat terhadap ekosistem transaksi digital karena adanya pendekatan ilmiah untuk memperkuat sistem deteksi penipuan.
3. Mendukung terciptanya layanan keuangan digital yang lebih andal dan inklusif, karena risiko *fraud* yang lebih terkendali dapat memperkuat keberlanjutan penggunaan transaksi non-tunai di masyarakat.

BAB 2 TINJAUAN PUSTAKA

2.1 Hasil Penelitian Terdahulu

Subbab ini menyajikan ringkasan hasil penelitian terdahulu yang relevan dengan topik yang diangkat pada penelitian ini yaitu deteksi *fraud* transaksi kartu kredit, serta penggunaan teknik *resampling* untuk menangani *imbalanced data* dan penerapan model *ensemble learning*, serta *gap* yang menjadi dasar kebaruan penelitian ini.

Judul Paper	Dataset	Metode	Hasil Penelitian	Analisis Gap terhadap Penelitian ini
<i>A credit card fraud detection approach based on ensemble machine learning classifier with hybrid data sampling</i> (Ahmed et al., 2025)	Dataset Credit Card Fraud Detection ULB Machine Learning Group	Hybrid Sampling: SMOTE (Oversampling) + ENN (Edited Nearest Neighbors) untuk membersihkan data dari noise. Classifier: Ensemble Model (AdaBoost, Random Forest, KNN) dengan Soft Voting Classifier.	Model yang Diusulkan (Ensemble + SMOTE + ENN): • <i>Accuracy</i> : 0,99997 (Tertinggi dibanding model lain) - <i>Precision</i> : 0,999 - <i>Recall</i> : 1 (Mampu mengeliminasi <i>false negatives</i> secara total) - <i>F1-Score</i> : 0,999	Studi tersebut menekankan satu skema <i>hybrid sampling</i> (SMOTE-ENN) dalam rancangan yang diusulkan. Penelitian ini memperluas analisis dengan membandingkan beberapa teknik <i>resampling</i> (<i>oversampling</i> , <i>undersampling</i> , dan <i>hybrid sampling</i>) dan beberapa tipe <i>ensemble learning</i> (<i>bagging</i> , <i>boosting</i> , <i>voting</i> , dan <i>stacking</i>) dengan fokus utama peningkatan sensitivitas (<i>recall</i>) kelas <i>fraud</i> .
<i>A soft voting ensemble learning approach for credit card fraud detection</i> (Azim Mim et al., 2024)	Dataset Credit Card Fraud Detection ULB Machine Learning Group	- Penanganan Imbalance Data: Membandingkan <i>Under-sampling</i> (Random Under-sampling), <i>Over-sampling</i> (SMOTE, ADASYN), dan <i>Hybrid Sampling</i> (SMOTE-Tomek, SMOTE-ENN). - Classifier: Kombinasi <i>Soft Voting</i> dari XGBoost, MLP, dan KNN	Model Terbaik (XGBoost + MLP + KNN pada data Under-sampled): - <i>Recall</i> : 0,9694 - <i>False Negative Rate</i> (FNR): 0,0306 Kombinasi VC2 (RF + XGBoost + KNN pada data asli): - Presisi (<i>Precision</i>): 0,9870 - <i>F1-Score</i> : 0,8764 - AUROC: 0,9936	Penelitian tersebut berpusat pada pendekatan <i>soft voting</i> sebagai metode <i>ensemble</i> utama. Penelitian ini menambahkan ruang lingkup perbandingan ke beberapa tipe <i>ensemble</i> lain (Random Forest, XGBoost, Gradient Boosting, dan <i>stacking</i>) sekaligus menganalisis kontribusi setiap teknik <i>resampling</i> terhadap peningkatan <i>recall</i> kelas <i>fraud</i> .

Judul Paper	Dataset	Metode	Hasil Penelitian	Analisis Gap terhadap Penelitian ini
<i>A voting ensemble machine learning based credit card fraud detection using highly imbalance data</i> (Chhabra et al., 2024)	Dataset Credit Card Fraud Detection ULB Machine Learning Group	- Penanganan Imbalance Data: Random Over-sampling (ROS) dan Random Under-sampling (RUS). - Classifier Base: Random Forest (RF), Logistic Regression (LR), dan K-Nearest Neighbor (KNN). - Metrik Kombinasi: Evaluasi bobot prediktif $\langle w_1, w_2, w_3 \rangle$ menggunakan skema <i>Hard</i> dan <i>Soft Voting</i>.	Hasil Terbaik (Kombinasi ROS + Bobot $\langle 2, 1, 1 \rangle$ pada RF, LR, KNN): - Akurasi Pelatihan (<i>Training Accuracy</i>): 100% - Akurasi Pengujian (<i>Testing Accuracy</i>): 99,99% - Menghasilkan <i>Zero False Predictions</i> (nol prediksi salah) pada klasifikasi sampel fraud baik di pengujian <i>Hard</i> maupun <i>Soft Voting</i>	Studi tersebut menekankan efektivitas <i>voting ensemble</i> pada kondisi <i>imbalanced</i> . Penelitian ini memperluas fokus dengan membandingkan <i>voting</i> terhadap <i>ensemble</i> lain (<i>bagging</i> , <i>boosting</i> , dan <i>stacking</i>) dan menguji lebih banyak variasi <i>resampling</i> (termasuk <i>hybrid</i> seperti SMOTE-ENN dan SMOTE-Tomek Links) untuk mengoptimalkan sensitivitas.
<i>A Support Vector Machine Credit Card Fraud Detection Model based on High Imbalance Dataset</i> (Odeyale et al., 2024)	Dataset Credit Card Fraud Detection ULB Machine Learning Group	- Penanganan Imbalance Data: SMOTE (Synthetic Minority Oversampling Technique). - Seleksi Fitur: ANOVA F-Test (mengurangi fitur dari 29 menjadi 20 kolom). - Classifier: Support Vector Machine (SVM) Linear Kernel.	Model yang Diusulkan (SVM + SMOTE): - Akurasi (<i>Accuracy</i>): 93,9% (Akurasi data tidak seimbang 99,8% turun menjadi 93,9% namun performa deteksi riil jauh lebih konsisten) - Presisi (<i>Precision</i>): 97,3% - Recall: 90,3% - F1-Score: 93,5%	Penelitian tersebut berfokus pada satu model utama (SVM) dan satu pendekatan <i>resampling</i> dominan (SMOTE). Penelitian ini berbeda karena mengevaluasi beberapa model <i>ensemble learning</i> dan membandingkan teknik <i>resampling</i> yang lebih beragam (<i>oversampling</i> , <i>undersampling</i> , dan <i>hybrid sampling</i>) untuk peningkatan sensitivitas (<i>recall</i>) pada kelas <i>fraud</i> .

Judul Paper	Dataset	Metode	Hasil Penelitian	Analisis Gap terhadap Penelitian ini
<i>Analysis and visualization of fraud detection patterns through data mining and classification using MLP and hybrid deep learning model</i> (T. A. Khan et al., 2025)	Data Sintetis Simulasi Sparkov Data Generation	- Penanganan Imbalance Data: SMOTE. - Model 1: Multilayer Perceptron (MLP). - Model 2: Hybrid 1D Convolutional Neural Network (1D-CNN) + Long Short-Term Memory (LSTM).	Model MLP: Akurasi: 99,64%, Presisi: 99,47%, Recall: 99,63%, F1-Score: 99,64%. Model Hybrid 1D-CNN-LSTM: Akurasi: 99,76%, Presisi: 98,94%, Recall: 99,57%, F1-Score: 99,46%.	Studi tersebut berorientasi pada pemodelan deep learning dan analisis pola <i>fraud</i> , dengan <i>resampling</i> sebagai langkah pendukung. Penelitian ini memusatkan kontribusi pada evaluasi efektivitas teknik <i>resampling</i> untuk meningkatkan sensitivitas model-model <i>ensemble learning</i> (bukan deep learning) pada dataset <i>fraud</i> kartu kredit yang sangat <i>imbalanced</i> .
<i>Hybrid feature selection framework for enhanced credit card fraud detection using machine learning models.</i> (Siam et al., 2025)	Lima dataset fraud dengan karakteristik berbeda	Hybrid Feature Selection Framework: - Langkah 1: Eliminasi redundansi dengan <i>Pearson Correlation</i> (Threshold ± 0.95). - Langkah 2: Seleksi relevansi fitur gabungan melalui <i>Information Gain</i> (IG) dan <i>Random Forest Importance</i> (RFI) via operasi <i>Union</i> ($F = F2 \cup F3$). - Klasifikasi: RF, Extra Trees (ET), XGBoost (XGBC), AdaBoost, dan CatBoost.	Model Terbaik pada Dataset Utama (ECC-2013): ADASYN + Extra Trees (ET): - Akurasi: 99,97% - F1-Score: 89,95% - AUC: 98,72% - MCC: 89,99% • Ensemble Vote (pada German Credit Data): - Akurasi: 84,00%, F1-Score: 89,04%, AUC: 78.30%	Studi tersebut berfokus dalam strategi peningkatan performa pada level fitur (<i>feature-level</i>) melalui seleksi fitur. Penelitian ini menekankan strategi pada level data (<i>data-level</i>) melalui <i>resampling</i> (<i>oversampling</i> , <i>undersampling</i> , dan <i>hybrid sampling</i>) untuk memperbaiki sensitivitas (<i>recall</i>) pada kelas <i>fraud</i> , sehingga menawarkan sudut pandang solusi yang berbeda.

Berdasarkan Tabel Penelitian Terdahulu, terlihat bahwa fokus utama riset global dalam deteksi *credit card fraud* bertumpu pada penyelesaian masalah ketidakseimbangan kelas (*class imbalance*) yang sangat ekstrem melalui pendekatan level data (*data-level approach*) dan level model. Penelitian oleh Ahmed et al. (2025) membuktikan bahwa penggunaan teknik *hybrid sampling* berupa kombinasi SMOTE dan *Edited Nearest Neighbor* (ENN) mampu menghasilkan batas keputusan (*decision boundary*) yang bersih dari *noise*, sehingga ketika dipadukan dengan *Soft Voting Classifier* berbasis AdaBoost, RF, dan KNN dapat meningkatkan nilai akurasi hingga 0,99997 serta mengeliminasi kesalahan *false negative* secara total (*recall* = 1). Pentingnya metode *voting* heterogen juga diperkuat oleh Penelitian Azim Mim et al. (2024) yang mengombinasikan XGBoost, MLP, dan KNN pada data *under-sampled* untuk meminimalkan *False Negative Rate* (FNR) menjadi 0,0306. Di sisi lain, Penelitian Chhabra et al. (2024) menawarkan perspektif optimasi bobot prediktif $\langle w_1, w_2, w_3 \rangle$ pada skema *weighted voting ensemble* homogen, di mana integrasi *Random Oversampling* (ROS) dengan bobot dominan pada algoritma Random Forest berhasil menekan angka kesalahan klasifikasi hingga mencapai *zero false predictions* pada data pengujian.

Selain algoritma berbasis pohon keputusan (*tree-based*), penggunaan model linear seperti *Support Vector Machine* (SVM) yang dikombinasikan dengan teknik seleksi fitur juga dieksplorasi oleh Penelitian Odeyale et al. (2024). Melalui uji ANOVA F-Test untuk mereduksi fitur dari 29 menjadi 20 kolom dan penyeimbangan data menggunakan SMOTE, model SVM mampu menghasilkan nilai presisi sebesar 97,3% dan *recall* 90,3%. Terakhir, Penelitian oleh T. A. Khan et al. (2025) mencoba keluar dari batasan *machine learning* konvensional dengan mengusulkan arsitektur *deep learning* melalui arsitektur MLP dan *hybrid* 1D-CNN-LSTM yang dikombinasikan dengan SMOTE, di mana model *hybrid* tersebut terbukti sangat tangguh dalam menangkap karakteristik temporal data transaksi dan mencapai akurasi pengujian tertinggi sebesar 99,76%. Komprehensivitas eksperimen ini diperluas oleh Siam et al. (2025) yang merancang kerangka seleksi fitur *hybrid* berbasis korelasi Pearson, *Information Gain* (IG), dan *Random Forest Importance* (RFI) pada 5 dataset berbeda, yang menyimpulkan bahwa kombinasi ADASYN dan *Extra Trees* (ET) memberikan performa paling adaptif pada data hasil transformasi PCA dengan nilai MCC mencapai 89,99%.

Penelitian Tugas Akhir ini menggunakan dataset *Credit Card Fraud Detection* dari ULB *ML Group* yang serupa dengan dataset utama yang digunakan oleh penelitian oleh Ahmed et al. (2025), Azim Mim et al. (2024), Odeyale et al. (2024), Chhabra et al. (2024), dan Siam et al. (2025). Jika dibandingkan dengan pengujian baseline (tanpa *resampling*) dengan model Random Forest pada penelitian ini sudah memiliki kinerja dasar yang cukup tinggi dengan *recall* 0,8163 dan *F1-score* 0,8743. Namun, melalui serangkaian eksperimen penerapan teknik *resampling*, ditemukan bahwa integrasi XGBoost dan teknik *undersampling* ENN tanpa penggunaan *oversampling* agresif justru menjadi konfigurasi paling optimal dalam deteksi transaksi *fraud*. Kombinasi XGBoost-ENN ini berhasil mencapai nilai *recall* sebesar 0,8367, *precision* sebesar 0,9111, dan *F1-score* sebesar 0,8723 dengan parameter default.

Hasil ini memberikan performa yang jauh lebih realistis dan seimbang dibandingkan dengan hasil penelitian Ahmed et al. (2025) yang mencatatkan *recall* sempurna senilai 1 namun rentan terhadap *overfitting* akibat sintesis kelas minoritas yang terlalu padat. Penggunaan ENN secara mandiri sebagai alat pembersih data terbukti jauh lebih efektif dalam mempertahankan nilai *precision* dibandingkan metode *Random Undersampling* (RUS) yang digunakan pada model *Soft Voting* oleh Azim Mim et al. (2024) atau Chhabra et al. (2024). Meskipun RUS pada

penelitian ini mencatatkan nilai *recall* murni tertinggi (0,8980), ia mengorbankan nilai presisi secara masif sehingga nilai *F1-score* rata-ratanya anjlok ke angka 0,1825 akibat banyaknya *false positive*. Pada skenario selanjutnya ketika dioptimasi menggunakan *hyperparameter tuning* menggunakan RandomizedSearchCV, kombinasi Random Oversampling (ROS) dan Random Forest serta XGBoost-ENN pada penelitian ini mampu mendongkrak *recall* hingga mencapai batas optimal 0,8469, mengungguli sensitivitas dasar dari mayoritas model tunggal maupun ensemble parameter default yang diajukan oleh penelitian-penelitian sebelumnya.

Kontribusi utama dari penelitian ini terletak pada perluasan ruang lingkup (*scope*) analisis melalui evaluasi komparatif yang komprehensif, terstruktur, dan berskala besar pada level data (*data-level approach*) dan level pemodelan (*ensemble architecture*). Penelitian ini memberikan pembaruan ilmiah dengan menghadirkan representasi metodologis yang utuh dari setiap kategori *ensemble learning*, yang meliputi rumpun *bagging* (Random Forest), *boosting* (Gradient Boosting dan XGBoost), *voting* (Soft Voting), serta *stacking* (Stacked Generalization). Keragaman arsitektur ini diintegrasikan secara sistematis dengan representasi dari seluruh kategori penanganan data tidak seimbang (*class imbalance*), yang mencakup teknik *oversampling* (Random Oversampling, SMOTE, ADASYN), *undersampling* (Random Undersampling, Tomek Links, ENN), hingga pendekatan *hybrid sampling* (SMOTE-Tomek dan SMOTE-ENN). Kehadiran multiteknik ini secara masif melampaui keterbatasan riset-riset terdahulu yang umumnya hanya berfokus pada pengujian satu atau dua metode secara parsial.

Selain itu, kontribusi krusial dari penelitian ini adalah penekanan orientasi pemodelan yang secara ketat berfokus pada aspek sensitivitas (*recall*). Dalam ekosistem industri penyedia layanan keuangan, kegagalan dalam mengidentifikasi transaksi penipuan (*false negative*) memiliki konsekuensi kerugian finansial dan penurunan reputasi yang jauh lebih fatal dibandingkan dengan kesalahan deteksi palsu (*false positive*). Melalui fokus pada orientasi sensitivitas ini, penelitian berhasil membuktikan bahwa optimalisasi sistem deteksi *fraud* tidak selalu bergantung pada penyeimbangan kuantitas distribusi kelas secara agresif, melainkan pada kualitas kebersihan data (*data cleaning*) di sekitar batas keputusan (*decision boundary*). Temuan mengenai efektivitas kombinasi XGBoost dengan *Edited Nearest Neighbor* (ENN) serta implementasi *hyperparameter tuning* berbasis *scoring recall* dalam penelitian ini menyajikan sebuah rekomendasi *pipeline* yang aplikatif, adaptif, dan kokoh untuk diimplementasikan pada sistem deteksi penipuan kartu kredit di dunia nyata.

2.2 Hasil Penelitian State of The Art

Subbab ini menyajikan penjelasan mendalam terhadap literatur mutakhir atau *state-of-the-art* (SOTA) dalam domain deteksi penipuan transaksi kartu kredit. Seiring dengan eskalasi kecanggihan cara kerja kejahatan finansial, arah riset global pada tahun-tahun terakhir telah bergeser secara signifikan dari algoritma *machine learning* konvensional tunggal menuju pemanfaatan arsitektur kecerdasan buatan yang jauh lebih kompleks. Semua penelitian SOTA ini mengevaluasi efektivitas metode mereka menggunakan jenis *dataset* publik orisinal yang sama dengan yang diuji dalam penelitian Tugas Akhir ini, yaitu *European Credit Card Dataset* yang dirilis oleh *Machine Learning Group - Université Libre de Bruxelles* (ULB) dengan total populasi 284.807 transaksi dan tingkat ketimpangan kelas minoritas sebesar 0,172%. Melalui kesamaan basis data uji ini, perbandingan performa secara langsung dapat diformulasikan guna mengidentifikasi batas-batas keunggulan, trade-off metrik evaluasi, serta celah riset (*research*

gap) yang melandasi urgensi dan kontribusi kebaruan dari penelitian yang diajukan dalam penelitian ini.

Judul Paper	Dataset	Metode	Hasil Penelitian	Analisis Gap terhadap Penelitian ini
<i>A behavior-aware intelligent decision framework for real-time credit card fraud detection using deep learning and adaptive user profiling.</i> (Onwudebelu et al., 2026)	Dataset Credit Card Fraud Detection ULB Machine Learning Group	Integrasi arsitektur 7-layer <i>Multilayer Perceptron</i> (MLP). - <i>Sliding-window-based personalization mechanism</i> untuk profiling perilaku adaptif pengguna. - SMOTE untuk penyeimbangan data. - SHAP untuk atribusi fitur (<i>explainability</i>).	- <i>Precision</i> : 91% - <i>Recall</i> : 77% - <i>F1-score</i> : 84% - AUC-ROC: 94% - Pengurangan <i>false-positive rate</i> sebesar 35,7% - Rata-rata latensi: 67,2 ms/transaksi.	Penelitian Onwudebelu et al. berfokus pada arsitektur <i>deep learning</i> tunggal yang membutuhkan komputasi tambahan untuk melacak profil perilaku personal per individu guna menekan <i>false positive</i> . Sementara penelitian ini mengeksplorasi manipulasi level data melalui <i>multi-techniques resampling</i> (8 variasi teknik) langsung pada fungsi keputusan model-model <i>ensemble learning</i> konvensional untuk memaksimalkan <i>recall</i> secara efisien.
<i>A hybrid deep learning framework using synthetic oversampling, autoencoder, convolutional neural networks, and an attention mechanism for credit card fraud detection.</i> (Zavvar et al., 2026)	Dataset Credit Card Fraud Detection ULB Machine Learning Group	- Kerangka kerja <i>hybrid</i> SMOTE-AE-CNN-Att. - <i>Autoencoder</i> (AE) untuk <i>denoising</i> data hulu. - 1D-CNN untuk ekstraksi fitur spasial sekuensial. - <i>Attention mechanism</i> untuk pembobotan ulang fitur (<i>reweighting</i>). - Klasifikasi akhir menggunakan model ML tradisional	Model terbaik (Gaussian Naïve Bayes): - <i>Accuracy</i> : 99,94% - <i>Precision</i> : 90,07% - <i>Recall</i> : 91,13% - <i>F1-score</i> : 90,60%.	Penelitian Zavvar et al. membangun ekstraktor fitur <i>deep learning hybrid</i> yang berlapis-lapis dan sangat kompleks sebelum masuk ke model pengklasifikasi statis. Sebaliknya, penelitian ini mengkaji efektivitas relatif teknik <i>resampling</i> (<i>hybrid</i> , <i>under-</i> , dan <i>over-sampling</i>) langsung pada arsitektur <i>ensemble learning</i> serta dikombinasikan dengan <i>hyperparameter tuning</i> berbasis optimasi <i>recall</i> .

Judul Paper	Dataset	Metode	Hasil Penelitian	Analisis Gap terhadap Penelitian ini
<i>Leveraging GraphSAGE and large language models with cross-attention for transaction fraud detection: Evidence from credit card and PaySim datasets</i> (Mohawesh et al., 2026)	Dataset Credit Card Fraud Detection ULB Machine Learning Group dan PaySim mobile money simulator.	- Pendekatan <i>multimodal hybrid stream</i>. - GraphSAGE Encoder untuk mengekstrak informasi struktural jaringan transaksi. - DistilBERT Encoder untuk mengekstrak semantik metadata kategori. - <i>Multi-head cross-attention fusion mechanism</i> sebagai penyatu kedua representasi GraphSAGE dan DistilBERT. - Kelas diseimbangkan dengan SMOTE dan diklasifikasikan dengan MLP.	Hasil Klasifikasi terbaik pada Data ULB: - <i>Accuracy</i>: 89,1% - <i>Precision</i>: 84,2% - <i>Recall</i>: 85,9% - <i>F1-score</i>: 85,0% - AUC: 85,5%.	Riset Mohawesh et al. bergantung pada data topologi jaringan graf dan pemodelan bahasa alami (LLM) yang membutuhkan beban komputasi tinggi (akselerasi GPU). Riset saya beroperasi murni pada level data struktural tabular orisinal dengan membuktikan bahwa penyeimbangan dan pembersihan kualitas data lewat ENN mampu mengoptimalkan sensitivitas model <i>ensemble</i> modern tanpa kompleksitas konversi graf atau <i>embedding</i> bahasa.
<i>TS-GNN: Modeling temporal-semantic discrepancy for graph-based credit card fraud detection</i> (J. Zhang et al., 2026)	Dataset Credit Card Fraud Detection ULB Machine Learning Group, IEEE-CIS, YelpChi, dan DGraph.	- TS-GNN (Temporal-Semantic Graph Neural Network). - <i>Temporal-Semantic Quantifier</i> (TSDQ) untuk mengukur konsistensi perilaku lintas	Hasil Klasifikasi pada Data ULB: - AUC: 95,99% - <i>F1-score</i>: 91,10% - <i>Recall</i>: 88,78%.	TS-GNN dirancang khusus untuk memitigasi manipulasi hubungan antar-entitas (<i>relational camouflage</i>) pada struktur graf non-Euclidean. Sementara penelitian ini berfokus pada manipulasi level data langsung pada ruang matriks data tabular orisinal menggunakan

Judul Paper	Dataset	Metode	Hasil Penelitian	Analisis Gap terhadap Penelitian ini
		modal via fungsi basis frekuensi. - <i>Discrepancy-Aware Attention</i> (DAT) untuk menyaring pesan dari tetangga yang terindikasi melakukan kamufase. - <i>Anomaly-Retaining Gated Unit</i> (ARGU) untuk mencegah dilusi fitur anomali di lapisan graf yang dalam. - Penanganan imbalance data via <i>downsampling</i> (rasio 1:3).		kombinasi teknik <i>resampling</i> (<i>hybrid</i>, <i>under</i>-, dan <i>over-sampling</i>) guna meningkatkan performa sensitivitas arsitektur <i>ensemble learning</i> homogen dan heterogen.

Perbandingan hasil kuantitatif antara penelitian *state-of-the-art* (SOTA) di tabel dengan penelitian Tugas Akhir ini menunjukkan adanya variasi performa yang dipengaruhi secara fundamental oleh pemodelan dan kompleksitas arsitektur yang digunakan. Jika dibandingkan dengan penelitian tabular berbasis *deep learning* yang diajukan oleh Onwudebelu et al. (2026), kombinasi parameter default *XGBoost* dan ENN pada penelitian ini menghasilkan *recall* sebesar 0,8367 dan *F1-score* sebesar 0,8723, yang kemudian meningkat secara optimal setelah *hyperparameter tuning* berbasis *scoring recall* hingga mencapai *recall* sebesar 0,8469. Performa ini mengungguli model *personalized Multilayer Perceptron* (MLP) milik Onwudebelu et al. yang mencatatkan *recall* sebesar 77% dan *F1-score* 84%. Secara teoretis, keunggulan ini terjadi karena algoritma berbasis pohon keputusan (*tree-based ensemble*) seperti *XGBoost* dan *Random Forest* lebih adaptif dalam membangun garis pemisah keputusan (*decision boundary*) yang non-linear pada data tabular terstruktur dibandingkan dengan arsitektur jaringan saraf tiruan (*neural network*) tunggal yang rentan mengalami saturasi gradien atau terjebak dalam minimum lokal ketika dihadapkan pada ketidakseimbangan kelas yang ekstrem.

Namun, jika dihadapkan pada model SOTA berbasis representasi hulu yang masif seperti kerangka kerja SMOTE-AE-CNN-Att dari penelitian oleh Zavvar et al. (2026), hasil penelitian ini secara hasil berada di bawah model penelitian tersebut yang mencapai *recall* 91,13% dan *F1-score* 90,60%. Berdasarkan landasan teori *representation learning*, performa superior

penelitian Zavvar et al. didorong oleh pemanfaatan *Autoencoder* sebagai filter *denoising* yang dikombinasikan dengan arsitektur 1D-CNN untuk mengekstrak fitur laten spasial berdimensi tinggi, serta diperkuat mekanisme atensi (*attention*) sebelum masuk ke proses klasifikasi. Proses reduksi *noise* dan ekstraksi fitur laten berlapis tersebut secara signifikan mempermudah algoritma pengklasifikasi dalam mengenali pola anomali dibandingkan dengan pemodelan yang dilakukan langsung pada matriks fitur orisinal tabular. Sifat *trade-off* yang tidak dapat dihindari dari kompleksitas arsitektur penelitian Zavvar et al. adalah tingginya beban komputasi dan latensi operasional, sedangkan kerangka kerja *ensemble* pada penelitian ini menawarkan solusi yang jauh lebih ringan (*lightweight deployment*) dengan stabilitas performa yang tetap kompetitif.

Di sisi lain, perbandingan dengan kelompok pemodelan berbasis graf, yaitu penelitian Mohawesh et al. (2026) dengan *F1-score* 85,0% dan *recall* 85,9% serta TS-GNN milik Zhang et al. (2026) dengan *F1-score* 91,10% dan *recall* 88,78%, memperlihatkan batasan teoretis dari pemodelan berbasis data tabular biasa. Kedua penelitian SOTA tersebut mentransformasikan data transaksi kartu kredit menjadi struktur graf *non-Euclidean*. Berdasarkan teori *Graph Neural Networks* (GNN), representasi graf memungkinkan lapisan *message passing* seperti GraphSAGE untuk secara dinamis mengagregasikan informasi struktural multistap (*multi-hop relational dependencies*) dan mendeteksi pola kamufase hubungan ataupun lingkaran penipuan (*fraud rings*) antar-akun yang berkolusi. Informasi topologi jaringan dan relasi antar-simpul ini secara fundamental hilang pada pemodelan berbasis data tabular konvensional yang mengasumsikan sampel data bersifat independen dan terdistribusi secara identik (*independent and identically distributed / i.i.d.*). Akibatnya, model graf mampu menangkap sinyal penipuan kolektif secara lebih sensitif yang berujung pada perolehan *recall* yang lebih tinggi. Walaupun begitu, hasil penelitian Tugas Akhir ini membuktikan bahwa melalui penerapan teknik *data cleaning* murni seperti *Edited Nearest Neighbor* (ENN) yang dipadukan secara tepat dengan algoritma *tree boosting* (XGBoost), model tabular konvensional tetap mampu memberikan akurasi tinggi (99,96%) serta *F1-score* (0,8723) dan *precision* (0,9111) yang sangat kuat tanpa memerlukan beban komputasi besar untuk mengonstruksi graf maupun melakukan *embedding* bahasa alami.

2.3 Dasar Teori

Subbab ini menguraikan tinjauan pustaka serta konsep-konsep teoretis yang menjadi landasan utama dalam penyusunan penelitian. Pembahasan diawali dengan penjelasan mengenai *Credit Card Fraud* dan tantangan utamanya, yaitu ketidakseimbangan data (*class imbalance*). Selanjutnya, dijelaskan mengenai teknik pra-pemrosesan data termasuk transformasi PCA dan strategi penanganan data melalui berbagai teknik *Resampling* baik *Undersampling*, *Oversampling*, maupun *Hybrid*. Landasan teori juga mencakup penjelasan mendalam mengenai algoritma *Ensemble Learning* yang digunakan, meliputi model tipe *Bagging*, *Boosting*, *Voting*, dan *Stacking*, serta diakhiri dengan penjelasan mengenai metrik evaluasi kinerja model yang relevan untuk kasus klasifikasi pada data tidak seimbang.

2.3.1 Penipuan dalam Transaksi Kartu Kredit

Transaksi kartu kredit merupakan bentuk pembayaran nontunai oleh nasabah yang diberikan kartu yang diterbitkan oleh sebuah institusi keuangan, kemudian menggunakan instrumen kartu tersebut untuk melakukan pembayaran kepada pihak penjual atau menarik uang tunai, dan prosesnya melibatkan validasi agar transaksi dapat disetujui oleh penerbit kartu

(Odeyale et al., 2024). Dalam praktiknya, transaksi kartu dapat terjadi pada skenario *card-present* ketika kartu fisik ada saat transaksi maupun *card-not-present/CNP* ketika penjual tidak melihat kartu fisik, misalnya pada transaksi online atau via telepon (Bodker et al., 2022). Pada transaksi CNP, karena kartu dan pemegang kartu tidak hadir secara fisik untuk verifikasi langsung, risiko penyalahgunaan cenderung lebih tinggi dan memerlukan kontrol keamanan tambahan dari penjual maupun institusi keuangan (Bodker et al., 2022).

Penipuan (*fraud*) dalam transaksi kartu kredit pada dasarnya mengacu pada penggunaan kartu atau detail pembayaran milik pihak lain secara tidak sah untuk melakukan transaksi tanpa pengetahuan maupun persetujuan pemiliknya (Ahmed et al., 2025). Penipuan kartu kredit dapat terjadi melalui berbagai cara, seperti pencurian kartu secara fisik, pencurian data/detil kartu seperti nomor kartu dan informasi verifikasi, hingga penyalahgunaan detail kartu pada transaksi online yang dilakukan tanpa izin pemegang kartu (Btoush et al., 2023). Khusus pada CNP fraud, pelaku biasanya terlebih dahulu memperoleh detail pembayaran secara ilegal contohnya melalui kebocoran data, *phishing*, atau *skimming*, lalu memanfaatkannya untuk transaksi CNP dan mengupayakan keuntungan sekaligus menghindari deteksi (Bodker et al., 2022). Meningkatnya penggunaan pembayaran online juga berdampak pada meningkatnya peluang terjadinya *cyber fraud* pada kartu kredit sehingga pencegahan dan deteksi *fraud* menjadi isu penting bagi industri perbankan dan sistem pembayaran digital (Btoush et al., 2023).

Karakteristik data fraud pada transaksi kartu kredit umumnya memiliki distribusi kelas yang sangat timpang (*highly imbalanced*), yaitu jumlah transaksi fraud jauh lebih sedikit dibanding transaksi normal sehingga model cenderung bias ke kelas mayoritas apabila tidak ditangani dengan tepat (Paldino et al., 2022). Kemudahan dalam menggunakan kartu kredit untuk transaksi meningkatkan tingkat kesulitan dalam mendeteksi perilaku penipuan. (Shah & Passi, 2021). Selain itu, data fraud bersifat dinamis karena pola perilaku nasabah dan strategi fraudster dapat berubah dari waktu ke waktu (*concept drift*), sehingga distribusi data yang dipelajari model dapat berbeda dengan kondisi data terbaru (Paldino et al., 2022). Data fraud juga dapat memiliki *class overlap* dimana tidak selalu ada batas yang jelas antara fraud dan transaksi normal dan sering menghadapi keterlambatan ketersediaan label karena pelaporan fraud dapat terjadi beberapa hari setelah transaksi (Paldino et al., 2022).

2.3.2 Data Preprocessing

Data preprocessing adalah tahap penyiapan data sebelum pemodelan yang mencakup serangkaian prosedur untuk meningkatkan kualitas data dan kesesuaian format dan bentuk data agar kinerja model *machine learning* lebih baik (Maharana et al., 2022). Pada umumnya, preprocessing mencakup proses seperti pembersihan data, transformasi data, reduksi dimensi, dan *data scaling* agar data lebih informatif serta stabil untuk dilatih oleh model (Maharana et al., 2022). Dalam penelitian ini, *preprocessing* difokuskan pada penyeragaman skala fitur menggunakan *feature scaling* (StandardScaler) agar proses pembelajaran dan evaluasi model berjalan lebih efektif. Selain itu, teknik *Principal Component Analysis* (PCA) digunakan pada fitur dataset oleh pihak pengumpul data untuk menjaga privasi pihak-pihak yang melakukan transaksi. Kedua teknik tersebut akan dijelaskan lebih lanjut.

1. Principal Component Analysis (PCA)

Principal Component Analysis (PCA) merupakan metode reduksi dimensi yang mentransformasikan fitur asli menjadi sekumpulan fitur baru (*principal components*) yang saling ortogonal, dengan komponen pertama menangkap variasi terbesar pada data (Dorabiala

et al., 2024). Prinsip utama PCA adalah memilih arah proyeksi yang memaksimalkan variasi data. Secara umum principal component dengan *eigen value* terbesar merepresentasikan arah dengan variasi terbesar (Dorabiala et al., 2024). Dengan demikian, PCA dapat digunakan untuk merangkum informasi penting dari banyak fitur ke lebih sedikit komponen, sehingga kompleksitas data berkurang dan proses pelatihan model menjadi lebih efisien (Dorabiala et al., 2024). Secara ringkas, PCA dapat dipahami melalui tahapan matematis berbasis kovarians dan eigen decomposition seperti yang ditunjukkan pada **Persamaan 2.1** sampai dengan **2.4** berikut.

A. Pusatkan data (mengurangi rata-rata setiap fitur).

$$X_c = X - \mu \quad (2.1)$$

Di mana:

- X : matriks data asli berukuran $n \times p$ (n = jumlah sampel, p = jumlah fitur).
- μ : vektor rata-rata setiap fitur (berukuran $1 \times p$), dihitung dari seluruh sampel.
- X_c : matriks data yang sudah *mean-centered* (setiap fitur dikurangi rata-ratanya), berukuran $n \times p$.

B. Hitung matriks kovarians

$$C = \frac{1}{n-1} X_c^T X_c \quad (2.2)$$

Di mana:

- C : matriks kovarians berukuran $p \times p$.
- n : jumlah sampel/observasi.
- X_c^T : transpose dari X_c (berukuran $p \times n$).

C. Eigen decomposition

$$C v_i = \lambda_i v_i \quad (2.3)$$

Di mana:

- v_i : eigenvector ke- i dari matriks kovarians (arah principal component), berukuran $p \times 1$.
- λ_i : eigenvalue ke- i , menyatakan besar variasi yang dijelaskan oleh eigenvector v_i .
- i : indeks komponen utama

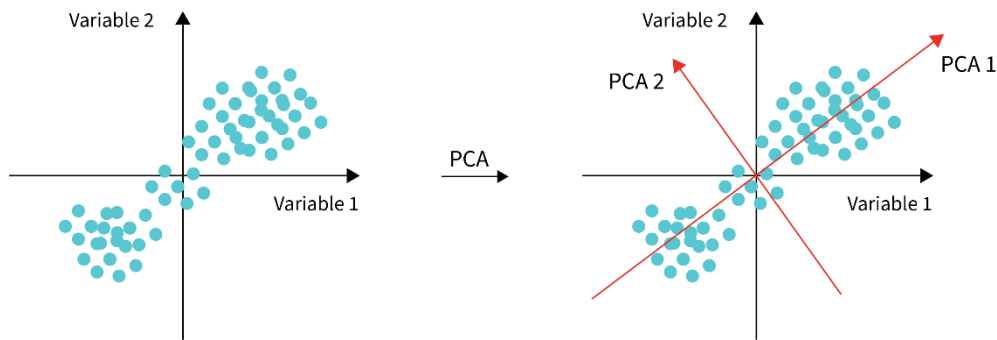
D. Proyeksi ke k komponen utama

$$Z = X_c W_k; W_k = [v_1, \dots, v_k] \quad (2.4)$$

Di mana:

- k : jumlah komponen utama yang dipilih (dengan $k \leq p$).
- W_k : matriks *loading* berukuran $p \times k$ yang berisi k eigenvector terpilih (dari eigenvalue terbesar).
- Z : matriks data hasil transformasi PCA berukuran $n \times k$.

Formulasi tersebut selaras dengan gagasan bahwa eigenvector dengan eigenvalue terbesar menghasilkan proyeksi dengan variasi maksimum (Dorabiala et al., 2024). **Gambar 2.1** menggambarkan konsep PCA sebagai transformasi sumbu koordinat dari fitur asli menjadi sumbu baru yang disebut *principal components*. PCA membentuk sumbu baru PCA 1 dan PCA 2 yang merupakan kombinasi linear dari variabel asli, di mana PCA 1 diarahkan mengikuti sebaran data terbesar dan PCA 2 tegak lurus terhadap PCA 1 untuk menangkap variasi berikutnya.



Gambar 2.1 Konsep principal component analysis

Sumber: (S. S. Khan et al., 2024)

2. Feature Scaling

Feature scaling adalah proses mengubah skala fitur numerik agar memiliki skala yang sebanding, sehingga fitur tertentu tidak mendominasi pembelajaran model hanya karena memiliki rentang nilai yang lebih besar (Maharana et al., 2022). Salah satu teknik yang umum dipakai dan akan digunakan dalam penelitian ini adalah standardisasi dengan target *zero mean* dan *unit variance*, yang pada implementasi praktisnya sering dilakukan melalui *StandardScaler* (Chauhan & Lee, 2022). *StandardScaler* mentransformasikan nilai fitur menggunakan rata-rata (μ) dan simpangan baku (σ) sehingga setiap fitur menjadi lebih terpusat dan memiliki skala yang seragam (Bonamutial & Prasetyo, 2023).

Rumus standardisasi adalah seperti yang ditunjukkan di **Persamaan 2.5**

$$z = \frac{x - \mu}{\sigma} \quad (2.5)$$

Di mana:

- x : nilai asli suatu fitur sebelum standardisasi.
- μ : rata-rata (mean) dari fitur tersebut pada data.
- σ : simpangan baku (standard deviation) dari fitur tersebut pada data.
- z : nilai fitur setelah standardisasi.

2.3.3 Single Classifier

Single classifier adalah model klasifikasi *supervised* yang membangun satu fungsi keputusan $f(x; \theta)$ untuk memetakan fitur input x ke label kelas, dengan θ sebagai parameter model yang dipelajari dari data *training*. *Training single classifier* umumnya dipandang sebagai proses optimisasi untuk meminimalkan fungsi kerugian (*loss*) rata-rata pada data *train* (*empirical risk minimization*), sehingga model memperoleh parameter yang menghasilkan kesalahan klasifikasi serendah mungkin pada sampel pelatihan dan diharapkan dapat melakukan generalisasi pada data baru. Pada data yang distribusinya tidak seimbang, teori evaluasi *single classifier* menekankan bahwa ukuran kinerja tidak cukup hanya akurasi, karena bias ke kelas mayoritas dapat menghasilkan akurasi tinggi namun gagal mengenali kelas minoritas. (Arshad et al., 2025)

1. Logistic Regression

Logistic Regression adalah model diskriminatif untuk klasifikasi biner yang memodelkan probabilitas kelas tertentu melalui fungsi sigmoid dengan memetakan kombinasi linear fitur ke rentang 0–1, sehingga keputusan klasifikasi dibangun dari *decision boundary* linear pada ruang fitur (Liang et al., 2022). Logistic Regression dapat dipahami melalui konsep *log-odds* yang linear terhadap fitur, sehingga perubahan pada suatu fitur memiliki interpretasi langsung terhadap perubahan peluang kelas (Liang et al., 2022). Dalam praktik *trainingnya*, parameter Logistic Regression diperoleh dengan optimisasi misalnya melalui pendekatan maksimum varian turunannya dan sering dipadukan dengan regularisasi untuk mengendalikan kompleksitas dan meningkatkan generalisasi (Quan, 2022). Logistic Regression dipilih karena mampu menghasilkan probabilitas klasifikasi yang terkalibrasi dengan baik sehingga sesuai digunakan pada mekanisme Soft Voting dan Stacking. Selain itu, model ini mewakili pendekatan berbasis probabilitas yang melengkapi karakteristik base learner lainnya.

Rumus dasar Logistic Regression ditunjukkan di **Persamaan 2.6**

$$p(y = 1 | x) = \sigma(z) = \frac{1}{1 + e^{-z}}, z = w^T x + b \quad (2.6)$$

Di mana:

- x : vektor fitur suatu sampel.

- w : vektor bobot/koeffisien yang dipelajari model.
- b : bias/intersep.
- z : skor linear sebelum dipetakan oleh sigmoid.
- $p(y = 1 | x)$: probabilitas sampel termasuk kelas positif.

2. Support Vector Machine

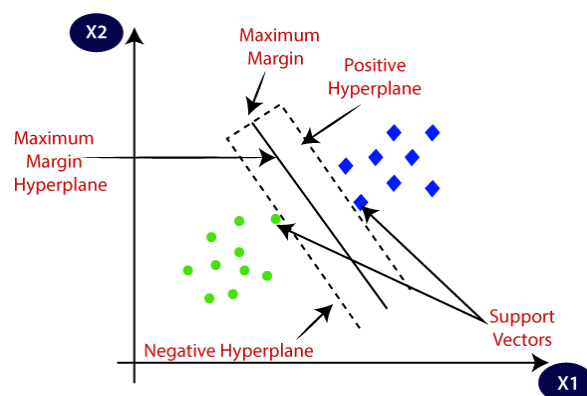
Support Vector Machine (SVM) adalah *classifier* yang membangun *hyperplane* pemisah dengan prinsip *maximum margin*, yaitu mencari batas keputusan yang memaksimalkan margin antar kelas agar meningkatkan generalisasi (Cervantes et al., 2020). Dalam pengembangan teori SVM modern, variasi seperti *certainty-aware* SVM menunjukkan bahwa SVM dapat diperluas dengan mekanisme tambahan untuk mengelola ketidakpastian prediksi dan isu *fairness*, namun tetap berakar pada konsep margin dan pemisahan kelas (Chen et al., 2025). SVM juga sering diformulasikan melalui *hinge loss* pada skenario *soft-margin* sebagai cara memberi penalti ketika sampel melanggar margin, sehingga masalah klasifikasi dapat diselesaikan sebagai optimisasi konveks (Tufail et al., 2025). Support Vector Machine dipilih karena mampu membentuk *hyperplane* optimal yang memaksimalkan pemisahan antar kelas pada ruang fitur. Model ini mewakili pendekatan berbasis jarak dan pemisahan kelas sehingga dapat melengkapi karakteristik prediksi dari base learner lainnya.

Rumus hinge loss ditunjukkan di **Persamaan 2.7**

$$L(y) = \max(0, 1 - t \cdot y) \quad (2.7)$$

Di mana:

- t : label sebenarnya ($\{-1, +1\}$ pada formulasi SVM).
- y : skor keputusan (output fungsi keputusan sebelum threshold).
- $L(y)$: penalti kesalahan; bernilai 0 jika sampel berada cukup jauh di sisi yang benar (margin terpenuhi).



Gambar 2.2 Hyperplane svm

Sumber: Alqahtani et al., 2023

3. Decision Tree

Decision Tree adalah model non-parametrik yang membentuk aturan keputusan dengan membagi data secara rekursif ke dalam subset yang semakin homogen terhadap kelas target, sehingga menghasilkan struktur pohon dari *root*, *node*, *leaf* (Mienye & Jere, 2024). Secara teori, proses pembentukan pohon umumnya bersifat *greedy (top-down)*, yakni memilih split terbaik di setiap node berdasarkan ukuran ketidakmurnian (*impurity*) seperti Gini atau entropy, lalu mengulangi proses hingga kondisi berhenti terpenuhi (Mienye & Jere, 2024). Decision Tree juga memiliki aspek teori model *complexity control* melalui *pruning*/regularisasi struktur misalnya membatasi kedalaman pohon karena pohon yang terlalu dalam cenderung *overfitting* terhadap data *training* (Souza et al., 2025). Decision Tree dipilih karena mampu membentuk aturan keputusan (*decision rules*) yang mudah diinterpretasikan serta menangkap hubungan nonlinier antar fitur. Model ini mewakili pendekatan berbasis aturan sehingga memberikan perspektif klasifikasi yang berbeda dari model penyusun voting lainnya.

Rumus Entropy ditunjukkan di **Persamaan 2.8**

$$H(S) = - \sum_{c=1}^c p_c \log(p_c) \quad (2.8)$$

Keterangan variabel:

- S : himpunan sampel pada suatu node.
- C : jumlah kelas (pada fraud: 2 kelas).
- p_c : proporsi sampel pada node yang termasuk kelas c .

Rumus Gini impurity ditunjukkan di **Persamaan 2.9**

$$G(S) = 1 - \sum_{c=1}^c p_c^2 \quad (2.9)$$

Keterangan variabel:

- $G(S)$: tingkat ketidakmurnian kelas pada node S (lebih kecil berarti lebih murni).
- p_c : proporsi kelas c pada node S .

2.3.4 Ensemble Learning

Ensemble learning adalah pendekatan yang membangun beberapa model (*base learner*) lalu menggabungkan prediksinya untuk menghasilkan prediksi akhir. Secara teori, ensemble memanfaatkan keragaman antar model dalam perbedaan data *train*, subset fitur, atau konfigurasi sehingga kesalahan prediksi dari satu model dapat dibenarkan oleh model lain saat

diagregasi. Metode ensemble learning dikelompokkan ke keluarga utama seperti bagging, boosting, voting, dan stacking dengan mekanisme pembentukan keragaman serta aturan penggabungan yang berbeda (Mienye & Sun, 2022). Pada permasalahan klasifikasi dengan class imbalance, kombinasi metode ensemble dengan teknik penyeimbangan data (*data augmentation/resampling*) sering dilaporkan mampu meningkatkan kinerja, khususnya pada pengenalan kelas minoritas (A. A. Khan et al., 2024). Pada penelitian ini, terdapat beberapa model yang diuji dari berbagai tipe *ensemble learning* yang akan dijelaskan cara kerjanya.

Rumus agregasi *ensemble* ditunjukkan di **Persamaan 2.10**

$$\hat{y} = g(\hat{y}_1, \hat{y}_2, \dots, \hat{y}_M) \quad (2.10)$$

Di mana:

- $\hat{y}$: prediksi akhir dari ensemble.
- $\hat{y}_m$: prediksi dari base learner ke- m .
- M : jumlah base learner di dalam ensemble.
- $g(\cdot)$: fungsi agregasi/penggabungan.

1. Bagging (Boosting Agregating)

Bagging (*bootstrap aggregating*) adalah metode ensemble yang melatih beberapa model pada data hasil *bootstrap sampling* yaitu sampling dengan pengembalian untuk menurunkan varians dan meningkatkan stabilitas prediksi. Setelah semua model dilatih, hasil prediksi digabungkan melalui aturan agregasi seperti voting untuk klasifikasi atau rata-rata untuk regresi. Secara konsep, bagging menekankan paralelisme karena setiap *base learner* dapat dilatih secara independen pada *bootstrap sample* yang berbeda (Mienye & Sun, 2022)

Rumus *majority voting* untuk klasifikasi ditunjukkan di **Persamaan 2.11**

$$\hat{y} = \text{mode}\{\hat{y}_1, \hat{y}_2, \dots, \hat{y}_M\} \quad (2.11)$$

Di mana:

- $\hat{y}$: label kelas hasil prediksi akhir bagging.
- $\hat{y}_m$: label kelas dari base learner ke- m .
- M : jumlah base learner yang dibangun.
- $\text{mode}\{\cdot\}$: fungsi yang memilih label yang paling sering muncul (mayoritas).

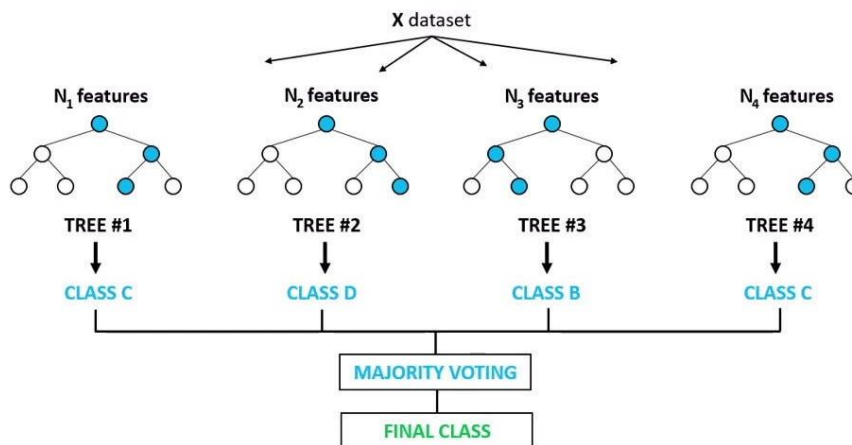
Rumus Rata-rata untuk regresi ditunjukkan di **Persamaan 2.12**

$$\hat{y} = \frac{1}{M} \sum_{m=1}^M \hat{y}_m \quad (2.12)$$

Di mana:

- $\hat{y}$: nilai prediksi akhir regresi.
- $\hat{y}_m$: nilai prediksi dari base learner ke- m .
- M : jumlah base learner.

Random Forest merupakan algoritma yang digunakan dalam penelitian ini merepresentasikan kategori *Bagging* dalam *ensemble learning*. Random Forest adalah *ensemble* berbasis *decision tree* yang secara umum dapat dipahami sebagai bagging pohon keputusan yang ditambah randomisasi fitur pada saat pemilihan split untuk meningkatkan keragaman antar pohon. Randomisasi fitur ini membantu mengurangi korelasi antar pohon dan menjadi salah satu alasan mengapa Random Forest cenderung lebih stabil dibanding *single decision tree*. Pada penerapan di data dunia nyata, Random Forest sering digunakan karena mampu menangani hubungan non-linear dan interaksi fitur melalui agregasi banyak pohon (Sun et al., 2024). *Random Forest* dipilih sebagai representasi kategori *bagging* pada penelitian ini karena memiliki kemampuan yang baik dalam menangani data berdimensi tinggi, tahan terhadap noise, serta relatif tidak mudah mengalami overfitting melalui mekanisme bagging. Selain itu, Random Forest banyak digunakan sebagai model pembanding (baseline ensemble) pada penelitian klasifikasi data tidak seimbang sehingga sesuai untuk mengevaluasi efektivitas berbagai teknik resampling.



Gambar 2.3 Konsep random forest

Sumber: (Feudjio et al., 2021)

2. Boosting

Boosting adalah metode *ensemble* yang membangun model secara berurutan atau sekuensial, di mana model berikutnya diarahkan untuk memperbaiki kesalahan model

sebelumnya (Mienye & Sun, 2022). Berbeda dari bagging yang independen, boosting bersifat serial sehingga konstruksi model saat ini dipengaruhi oleh hasil model sebelumnya (Xu et al., 2023). Dalam kerangka teori model aditif, boosting membentuk prediktor akhir sebagai penjumlahan kontribusi banyak learner lemah (*weak learners*) (Mienye & Sun, 2022).

Rumus model aditif untuk boosting ditunjukkan di **Persamaan 2.13**

$$F_T(x) = \sum_{t=1}^T f_t(x) \quad (2.13)$$

Di mana:

- $F_T(x)$: model akhir setelah T iterasi boosting.
- T : jumlah iterasi / jumlah weak learner yang ditambahkan.
- $f_t(x)$: weak learner pada iterasi ke- t .
- x : vektor fitur input.

a. Gradient Boosting

Gradient Boosting adalah boosting berbasis pohon yang membangun model secara serial, di mana setiap pohon baru ditambahkan untuk memperbaiki kesalahan prediksi model sebelumnya. Gradient Boosting umumnya dibangun dengan mekanisme optimisasi berbasis gradien dari fungsi *loss*, sehingga penambahan *learner* baru diarahkan untuk menurunkan *loss* secara bertahap. Penggunaan *learning rate* (η) memperkenalkan mekanisme *shrinkage* yang memperlambat penambahan kontribusi *learner* baru dan sering dipakai untuk mengontrol kompleksitas serta mengurangi *overfitting*. Dalam kasus deteksi fraud, penelitian oleh Xu et al (2023) menekankan bahwa boosting decision trees banyak digunakan karena memberikan performa dan stabilitas yang kuat untuk klasifikasi dikotomis (Xu et al., 2023). Gradient Boosting dipilih karena mampu membangun model secara bertahap dengan memperbaiki kesalahan model sebelumnya pada setiap iterasi. Kehadirannya dalam penelitian ini bertujuan untuk membandingkan performa boosting konvensional dengan XGBoost yang merupakan pengembangan lebih lanjut dari metode boosting.

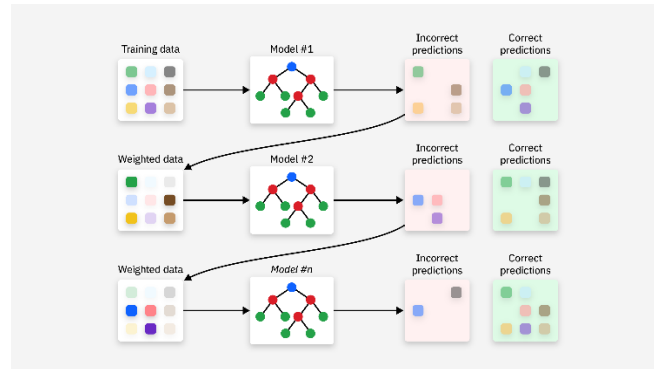
Rumus pembaruan model dengan *Learning Rate* ditunjukkan di **Persamaan 2.14**

$$F_t(x) = F_{t-1}(x) + \eta f_t(x) \quad (2.14)$$

Di mana:

- $F_t(x)$: model pada iterasi ke- t .
- $F_{t-1}(x)$: model pada iterasi ke- $t - 1$.
- η : learning rate (*shrinkage*) yang mengatur kontribusi learner baru.

- $f_t(x)$: learner yang ditambahkan pada iterasi ke- t
- x : vektor fitur input.



Gambar 2.4 Konsep Gradient boosting

Sumber: Murel, 2026, <https://www.ibm.com/think/topics/ensemble-learning>

b. eXtreme Gradient Boosting (XGBoost)

XGBoost adalah implementasi *extreme gradient boosting* yang termasuk keluarga *gradient boosting* berbasis pohon, dan dipahami sebagai model aditif yang menambahkan pohon secara bertahap untuk meminimalkan fungsi objektif. Karakteristik penting XGBoost adalah adanya regularisasi pada objektif untuk mengendalikan kompleksitas model misalnya kompleksitas pohon, sehingga generalisasi dapat meningkat. Secara konseptual, XGBoost tetap mengikuti prinsip boosting sekuensial, tetapi dirancang agar pelatihan lebih efisien dan stabil melalui formulasi objektif dan strategi optimisasi yang matang. Dalam praktik aplikasi data finansial, XGBoost sering digunakan karena memberikan keseimbangan antara performa prediksi dan kemampuan interpretasi relatif melalui struktur pohon (Liu et al., 2022). XGBoost dipilih karena merupakan algoritma *boosting* yang mampu menghasilkan performa tinggi melalui mekanisme optimasi gradien, regularisasi, dan pengendalian kompleksitas model. Kemampuan tersebut menjadikan XGBoost banyak digunakan pada berbagai penelitian klasifikasi, termasuk deteksi fraud, sehingga relevan untuk mengevaluasi pengaruh teknik resampling pada dataset yang sangat tidak seimbang.

Rumus objektif regularisasi ditunjukkan di **Persamaan 2.15**

$$\text{Obj} = \sum_{i=1}^n L(y_i, \hat{y}_i) + \sum_{t=1}^T \Omega(f_t) \quad (2.15)$$

Di mana:

- Obj : fungsi objektif yang diminimalkan.
- n : jumlah sampel.
- $L(y_i, \hat{y}_i)$: loss untuk sampel ke- i .

- y_i : label sebenarnya sampel ke- i .
- $\hat{y}_i$: prediksi model untuk sampel ke- i .
- $\Omega(f_t)$: penalti regularisasi untuk learner/pohon ke- t .
- T : jumlah pohon/iterasi boosting.

3. Soft Voting

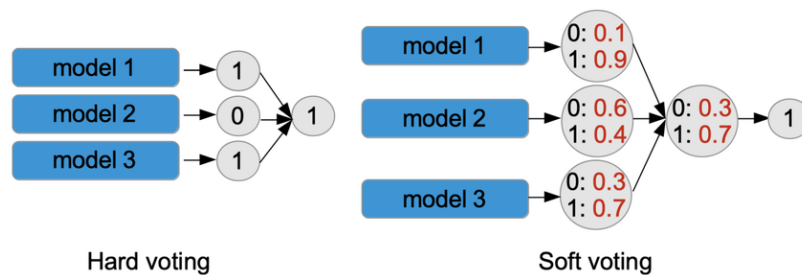
Voting *ensemble* menggabungkan prediksi dari beberapa *classifier*. Pada *soft voting classifier*, agregasi dilakukan pada level probabilitas sehingga keputusan akhir didasarkan pada probabilitas gabungan. Berbeda dari hard voting yang menggabungkan label diskrit, *soft voting* mengasumsikan bahwa probabilitas antar model cukup sebanding untuk diakumulasikan (Mienye & Sun, 2022). Pada penelitian ini digunakan Soft Voting dibandingkan Hard Voting karena Soft Voting mempertimbangkan probabilitas prediksi dari setiap *base classifier* sebelum menentukan kelas akhir, sedangkan Hard Voting hanya menggunakan hasil voting berdasarkan label prediksi. Pada kasus deteksi fraud dengan distribusi kelas yang sangat tidak seimbang, informasi probabilitas memberikan representasi tingkat keyakinan model yang lebih baik sehingga keputusan akhir menjadi lebih akurat dan tidak hanya bergantung pada jumlah suara mayoritas.

Rumus Soft Voting (rata-rata probabilitas) ditunjukkan di **Persamaan 2.16**

$$p(y = 1 | x) = \frac{1}{M} \sum_{m=1}^M p_m(y = 1 | x) \quad (2.16)$$

Di mana:

- $p(y = 1 | x)$: probabilitas akhir kelas positif setelah agregasi soft voting.
- $p_m(y = 1 | x)$: probabilitas kelas positif yang dihasilkan base learner ke- m .
- M : jumlah base learner.
- x : vektor fitur input.



Gambar 2.5 Konsep hard dan soft voting

Sumber: (Rizka & Sari, 2025)

4. Stacking

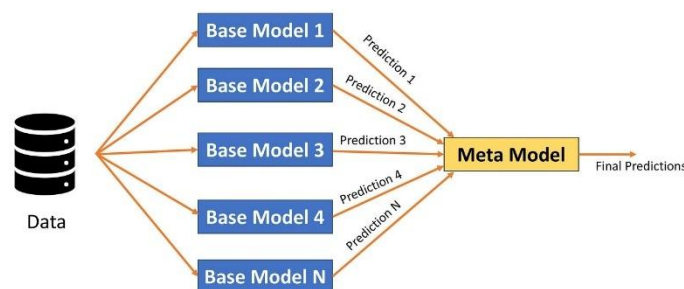
Stacking adalah *ensemble* dua tingkat yang menggunakan beberapa *base learner* pada level pertama dan *meta-learner* pada level kedua untuk menggabungkan keluaran *base learner* secara terpelajar (Mienye & Sun, 2022). Pendekatan ini melatih *meta-learner* menggunakan prediksi *base learner* sebagai fitur (*meta-features*), sehingga *meta-learner* belajar pola kesalahan dan keunggulan relatif antar *base learner* (Zian et al., 2021). Stacking dipilih karena mampu mengombinasikan keunggulan beberapa algoritma *ensemble* melalui proses pembelajaran bertingkat menggunakan *meta learner*. Dengan memanfaatkan karakteristik yang berbeda dari setiap *base learner*, Stacking diharapkan menghasilkan performa yang lebih baik dibandingkan penggunaan satu model *ensemble* secara individual dalam mendeteksi transaksi fraud.

Rumus *Stacking Meta-learner* ditunjukkan di **Persamaan 2.17**

$$\hat{y} = h(z), z = [\hat{y}_1, \hat{y}_2, \dots, \hat{y}_M] \quad (2.17)$$

Di mana:

- $\hat{y}$: prediksi akhir dari stacking.
- $h(\cdot)$: meta-learner (model level 2).
- z : vektor *meta-features* yang dibentuk dari keluaran base learner.
- $\hat{y}_m$: prediksi (atau probabilitas) base learner ke- m .
- M : jumlah base learner.



Gambar 2.6 Konsep stacking

Sumber: Salunke, 2024, <https://medium.com/data-and-beyond/stacking-ensemble-technique-in-ml-6d0a98a12607>

2.3.5 Teknik Resampling

Resampling adalah teknik *data-level* untuk menangani ketidakseimbangan kelas dengan memodifikasi komposisi data *train* melalui *oversampling* (menambah kelas minoritas), *undersampling* (mengurangi kelas mayoritas), atau hybrid sampling (kombinasi *oversampling* dan *undersampling*). *Oversampling* dapat dilakukan dengan menduplikasi sampel minoritas atau dengan membuat sampel sintetis, sedangkan *undersampling* mengurangi jumlah sampel mayoritas agar proporsinya lebih seimbang. Berbagai metode hybrid seperti SMOTE–Tomek

Links dan SMOTE-ENN menggabungkan pembuatan sampel sintetis dengan pembersihan noise/overlap untuk memperbaiki kualitas batas pemisah antar kelas. (Jiang et al., 2025)

1. Oversampling

a. Random Oversampling

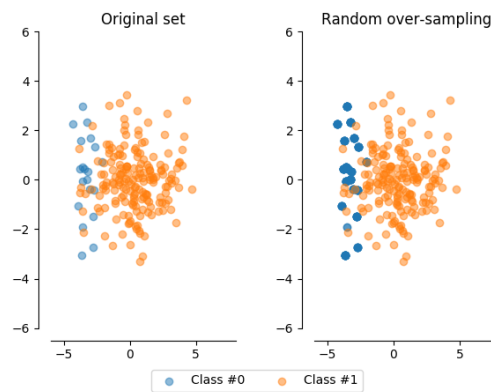
Random oversampling adalah salah satu jenis teknik sampling oversampling yang menyeimbangkan data dengan menduplikasi sampel kelas minoritas secara acak sampai mencapai proporsi kelas yang diinginkan. Secara teori, kelemahan ROS adalah risiko overfitting karena adanya duplikasi sampel minoritas yang sama sehingga model dapat menghafal pola tertentu. (Jiang et al., 2025)

Rumus jumlah sampel minoritas setelah sampling ditunjukkan di **Persamaan 2.18**

$$N'_{\min} = N_{\min} + G \quad (2.18)$$

Di mana:

- $N'_{\min}$: jumlah sampel minoritas setelah oversampling.
- $N_{\min}$: jumlah sampel minoritas sebelum oversampling.
- G : jumlah sampel minoritas yang ditambahkan melalui duplikasi acak.



Gambar 2.7 Konsep random oversampling

Sumber: Lemaitre, 2016, https://glemaitre.github.io/imbalanced-learn/auto_examples/over-sampling/plot_random_over_sampling.html

b. Synthetic Minority Oversampling Technique (SMOTE)

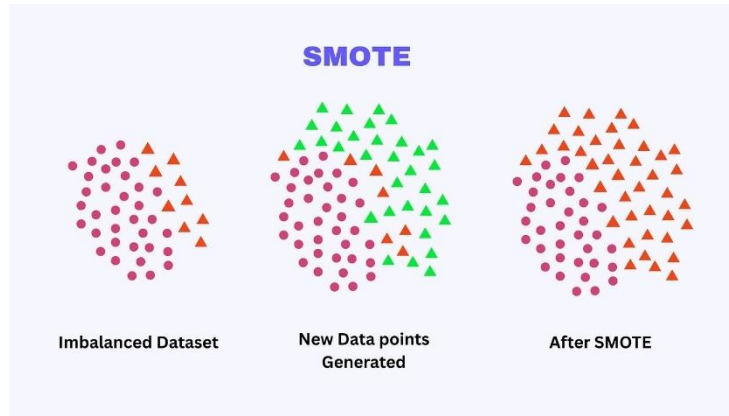
SMOTE adalah *oversampling* sintetis berbasis *k-nearest neighbor* yang membentuk sampel minoritas baru pada ruang fitur tanpa hanya menduplikasi data. SMOTE menghasilkan data sintetis pada ruang fitur menggunakan tetangga terdekat dan bilangan acak, sehingga mengurangi overfitting yang sering muncul pada oversampling dengan duplikasi. (Swana et al., 2022)

Rumus interpolasi sintetis SMOTE ditunjukkan di **Persamaan 2.19**

$$S_{\text{syn}} = r (S_{kNN} - S_f) + S_f \quad (2.19)$$

Di mana:

- S_{syn} : sampel sintetis yang dihasilkan.
- S_f : sampel fitur minoritas yang dipilih sebagai titik acuan.
- S_{kNN} : tetangga terdekat (kNN) dari S_f .
- r : bilangan acak 0–1.



Gambar 2.8 Konsep SMOTE

Sumber: Subramaniam, 2025, <https://towardsai.com/p/artificial-intelligence/handling-imbalanced-datasets-in-machine-learning-smote-oversampling-undersampling-explained>

c. Adaptive Synthetic (ADASYN)

ADASYN (Adaptive Synthetic Minority Over-Sampling Technique) adalah metode oversampling sintetis yang menghasilkan sampel minoritas baru secara adaptif berdasarkan tingkat kesulitan lokal pada ruang fitur. Ide adaptif pada ADASYN menekankan bahwa titik minoritas yang berada di wilayah dengan dominasi tetangga mayoritas (*borderline/overlap*) cenderung membutuhkan lebih banyak sampel sintetis dibanding titik minoritas yang berada di wilayah yang relatif aman. (Ileberi & Sun, 2024)

Rumus tingkat kesulitan lokal ditunjukkan di **Persamaan 2.20**

$$r_i = \frac{\Delta_i}{K} \quad (2.20)$$

Di mana:

- r_i : tingkat kesulitan lokal sampel minoritas ke- i .
- Δ_i : jumlah tetangga dari K tetangga terdekat yang berasal dari kelas mayoritas (atau bukan minoritas).
- K : jumlah tetangga terdekat yang digunakan.

Rumus normalisasi tingkat kesulitan ditunjukkan di **Persamaan 2.21**

$$\hat{r}_i = \frac{r_i}{\sum_{j=1}^{m_s} r_j} \quad (2.21)$$

Di mana:

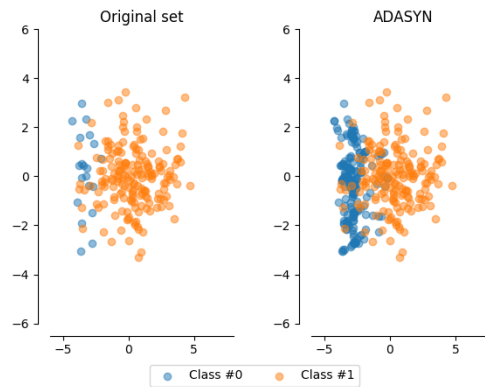
- $\hat{r}_i$: tingkat kesulitan ter-normalisasi untuk sampel ke- i .
- m_s : jumlah sampel minoritas.
- r_j : tingkat kesulitan lokal sampel minoritas ke- j .

Rumus jumlah sampel sintetis per titik ditunjukkan di **Persamaan 2.22**

$$g_i = \hat{r}_i \cdot G \quad (2.22)$$

Di mana:

- g_i : jumlah sampel sintetis yang dibuat dari sampel minoritas ke- i .
- $\hat{r}_i$: tingkat kesulitan ter-normalisasi untuk sampel minoritas ke- i .
- G : total jumlah sampel sintetis yang akan dihasilkan.



Gambar 2.9 Contoh penerapan ADASYN

Sumber: Danquah, 2020

2. Undersampling

a. Random Undersampling

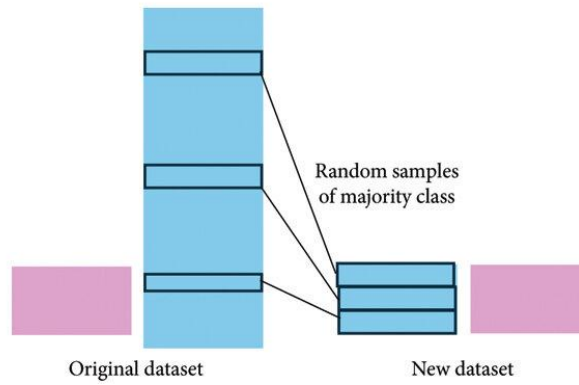
Random undersampling (RUS) adalah teknik *undersampling* yang menghapus sebagian sampel dari kelas mayoritas secara acak untuk menurunkan dominasi kelas mayoritas. Kelemahan teoretis *Random undersampling* adalah risiko hilangnya informasi penting karena penghapusan dilakukan tanpa mempertimbangkan struktur lokal/cluster atau posisi borderline. (Jiang et al., 2025)

Rumus ukuran kelas mayoritas setelah *sampling* ditunjukkan di **Persamaan 2.23**

$$N'_{\text{maj}} = \lfloor \rho \cdot N_{\text{min}} \rfloor \quad (2.23)$$

Di mana:

- N'_{maj} : jumlah sampel mayoritas setelah undersampling.
- N_{min} : jumlah sampel kelas minoritas (sebelum resampling).
- ρ : rasio target mayoritas terhadap minoritas (misalnya $\rho = 1$ untuk 1:1).
- $\lfloor \cdot \rfloor$: fungsi pembulatan ke bawah agar hasil berupa bilangan bulat.



Gambar 2.10 Konsep Random Undersampling

Sumber: Alsorory & Alshraideh, 2024

b. Tomek Links

Tomek Links adalah teknik *undersampling*/pembersihan data yang mengidentifikasi pasangan sampel beda kelas yang paling dekat, yang sering diinterpretasikan sebagai indikasi *overlap* atau *borderline* antarkelas. Dalam praktik yang umum, sampel mayoritas yang termasuk dalam pasangan Tomek link dihapus agar overlap berkurang dan batas kelas lebih jelas. (Jiang et al., 2025)

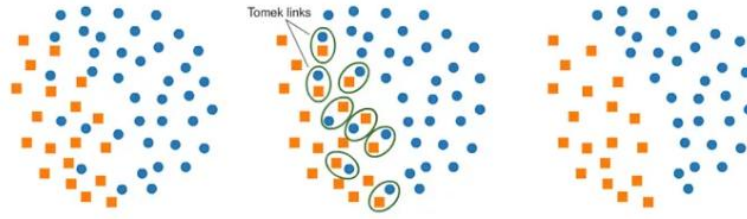
Rumus Tomek Links untuk menemukan kondisi pasangan terdekat beda kelas ditunjukkan di **Persamaan 2.24**

$$\neg \exists xz: d(xa, xz) < d(xa, xb) \text{ atau } d(xb, xz) < d(xa, xb) \quad (2.24)$$

Di mana:

- x_a, x_b : dua sampel beda kelas yang diuji sebagai kandidat Tomek link.
- x_z : sampel lain pembanding untuk memastikan tidak ada yang lebih dekat daripada pasangan Tomek link.

- $d(\cdot, \cdot)$: metrik jarak pada ruang fitur.



Gambar 2.11 Contoh Tomek Links

Sumber: (H. Liu et al., 2024)

c. Edited Nearest Neighbor (ENN)

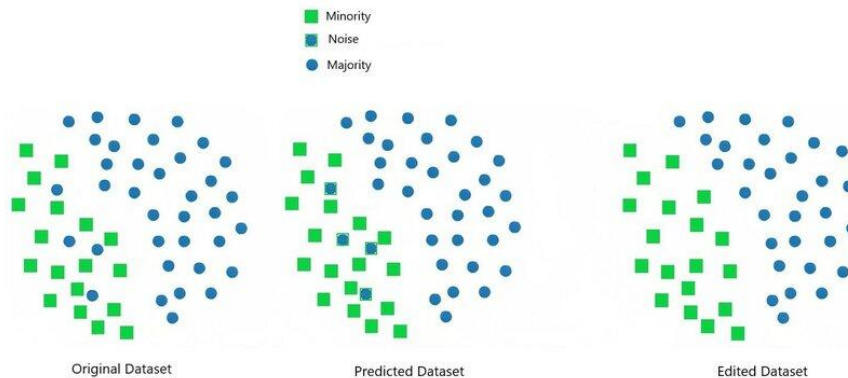
ENN adalah metode *editing* yang menghapus sampel yang labelnya tidak konsisten dengan tetangga terdekatnya, sehingga bertujuan mengurangi noise dan memperjelas batas keputusan. Secara teori, ENN menggunakan prinsip voting mayoritas pada k tetangga terdekat untuk menentukan apakah suatu sampel sesuai dengan lingkungan lokalnya. (Jiang et al., 2025)

Aturan ENN ditunjukkan di **Persamaan 2.25**

$$\text{Hapus } x_i \text{ jika } y_i \neq \text{mode}\{y_{i1}, y_{i2}, \dots, y_{ik}\} \quad (2.25)$$

Di mana:

- x_i : sampel yang diperiksa.
- y_i : label kelas sampel x_i .
- $y_{i1}, \dots, y_{ik}$: label milik k tetangga terdekat sampel x_i .
- $\text{mode}\{\cdot\}$: label mayoritas dari tetangga.
- k : jumlah tetangga terdekat yang dipakai ENN.



Gambar 2.12 Contoh ENN

Sumber: (Kumari et al., n.d.)

3. Hybrid Sampling

a. SMOTE-ENN

SMOTE-ENN adalah teknik hybrid yang menggabungkan SMOTE yang cara kerjanya membuat sampel minoritas sintetis dan ENN yang menghapus sampel yang dianggap noise/inkonsisten berdasarkan tetangga terdekat dalam satu pipeline resampling. Secara teori, tahap SMOTE meningkatkan kepadatan lokal kelas minoritas, sedangkan tahap ENN melakukan *editing* untuk mengurangi *noise* dan *overlap* di sekitar batas kelas dengan menghapus sampel yang berkonflik dengan tetangga terdekatnya. (Jiang et al., 2025)

Rumus interpolasi sintetis SMOTE ditunjukkan di **Persamaan 2.26**

$$S_{\text{syn}} = r (S_{kNN} - S_f) + S_f \quad (2.26)$$

Di mana:

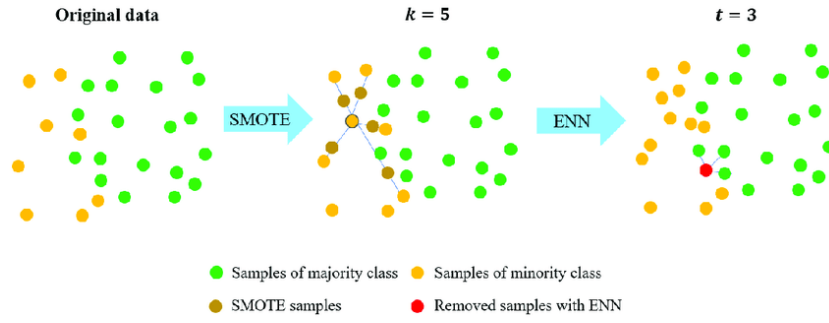
- S_{syn} : sampel sintetis yang dihasilkan.
- S_f : sampel fitur (titik minoritas) yang menjadi acuan pembentukan sampel sintetis.
- S_{kNN} : sampel tetangga terdekat (k-nearest neighbor) dari S_f pada kelas minoritas.
- r : bilangan acak pada rentang 0 sampai 1.

Aturan ENN yang merupakan penghapusan berbasis mayoritas tetangga ditunjukkan di **Persamaan 2.27**

$$\text{Hapus } x_i \text{ jika } y_i \neq \text{mode}\{y_{i1}, y_{i2}, \dots, y_{ik}\} \quad (2.27)$$

Di mana:

- x_i : sampel yang dievaluasi oleh ENN.
- y_i : label kelas dari x_i .
- $y_{i1}, \dots, y_{ik}$: label dari k tetangga terdekat sampel x_i .
- $\text{mode}\{\cdot\}$: label yang paling sering muncul pada himpunan tetangga (voting mayoritas).
- k : jumlah tetangga terdekat yang digunakan pada ENN.



Gambar 2.13 Contoh SMOTE-ENN

Sumber: (Lin & Zeng, 2021)

b. SMOTE–Tomek Links

SMOTE–Tomek Links adalah teknik hybrid yang menggabungkan *oversampling* sintetis dengan SMOTE dan *undersampling*/pembersihan *overlap* dengan Tomek Links untuk mengurangi tumpang tindih kelas setelah penambahan sampel minoritas. Secara konsep, Tomek Links mengidentifikasi pasangan sampel beda kelas yang saling menjadi tetangga terdekat dan sering dipakai untuk menghapus sampel mayoritas yang berada di area borderline sehingga overlap berkurang. (Jiang et al., 2025)

Rumus interpolasi sintetis SMOTE ditunjukkan di **Persamaan 2.28**

$$S_{\text{syn}} = r (S_{kNN} - S_f) + S_f \quad (2.28)$$

Di mana:

- S_{syn} : sampel sintetis yang dihasilkan.
- S_f : sampel fitur (titik minoritas) yang menjadi acuan pembentukan sampel sintetis. (
- S_{kNN} : sampel tetangga terdekat dari S_f (umumnya dari kelas minoritas).
- r : bilangan acak 0–1 yang mengatur posisi interpolasi.

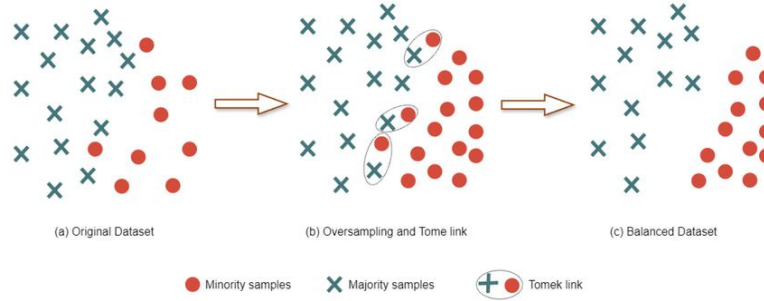
Jika dua sampel beda kelas x_a dan x_b membentuk Tomek link, maka tidak ada sampel lain x_z yang lebih dekat ke salah satu dari keduanya dibanding jarak antar pasangan tersebut (Swana et al., 2022). Rumus Tomek Links untuk menemukan kondisi pasangan terdekat beda kelas ditunjukkan di **Persamaan 2.29**

$$\neg \exists xz: d(xa, xz) < d(xa, xb) \text{ atau } d(xb, xz) < d(xa, xb) \quad (2.29)$$

Di mana:

- x_a : sampel dari satu kelas (mayoritas).
- x_b : sampel dari kelas lain (minoritas).

- x_z : sampel lain dalam dataset yang digunakan untuk mengecek apakah pasangan (x_a, x_b) benar-benar pasangan terdekat lintas kelas.
- $d(\cdot, \cdot)$: fungsi jarak pada ruang fitur.



Gambar 2.14 Contoh SMOTE-Tomek Links

Sumber: Y. Zhang et al., 2024

2.3.6 Hyperparameter Tuning

Hyperparameter tuning merupakan proses optimasi parameter eksternal pada algoritma *machine learning* yang bertujuan memperoleh konfigurasi model dengan performa terbaik berdasarkan metrik evaluasi tertentu (Franceschi et al., 2025). Berbeda dengan parameter model yang dipelajari secara otomatis selama proses pelatihan, hyperparameter ditentukan sebelum proses training dimulai dan berperan dalam mengatur perilaku serta kompleksitas model (Franceschi et al., 2025).

Pada algoritma *machine learning*, hyperparameter dapat berupa jumlah pohon keputusan ($n_estimators$), kedalaman maksimum pohon (max_depth), laju pembelajaran ($learning_rate$), jumlah minimum sampel untuk melakukan pemisahan node ($min_samples_split$), serta berbagai parameter lain yang memengaruhi proses pembelajaran model (Kette, 2025). Pemilihan nilai hyperparameter yang kurang tepat dapat menyebabkan model mengalami *underfitting* maupun *overfitting* sehingga kemampuan generalisasi model terhadap data baru menjadi kurang optimal (Arnold et al., 2024).

Hyperparameter tuning dilakukan untuk mencari kombinasi *hyperparameter* yang menghasilkan performa terbaik terhadap suatu fungsi evaluasi tertentu (Franceschi et al., 2025). Pada penelitian ini, fungsi evaluasi yang digunakan adalah metrik *recall* karena penelitian berfokus pada kemampuan model dalam mendeteksi transaksi fraud yang termasuk kelas minoritas. Secara matematis, proses hyperparameter tuning dapat dipandang sebagai permasalahan optimasi yang bertujuan mencari kombinasi hyperparameter terbaik dari ruang pencarian yang tersedia (Franceschi et al., 2025).

$$\lambda^* = \arg \max_{\lambda \in \Lambda} f(\lambda) \quad (2.30)$$

Di mana:

- λ^* : kombinasi hyperparameter terbaik.

- Λ : ruang pencarian hyperparameter (*hyperparameter search space*).
- $f(\lambda)$: fungsi evaluasi model berdasarkan kombinasi hyperparameter yang digunakan.

Berdasarkan **Persamaan 2.30**, proses optimasi dilakukan dengan mengevaluasi berbagai kombinasi hyperparameter hingga diperoleh konfigurasi yang menghasilkan nilai evaluasi terbaik sesuai tujuan penelitian.

1. *RandomizedSearchCV*

RandomizedSearchCV merupakan metode *hyperparameter tuning* yang melakukan pencarian kombinasi parameter secara acak dari ruang pencarian yang telah ditentukan sebelumnya (Rizky et al., 2024). Metode ini merupakan implementasi dari *algoritma Random Search* yang tersedia pada pustaka *Scikit-Learn* dan banyak digunakan karena mampu menghasilkan performa yang kompetitif dengan biaya komputasi yang lebih rendah dibandingkan *Grid Search* (Subaşi, 2024).

Berbeda dengan *Grid Search* yang mengevaluasi seluruh kemungkinan kombinasi *hyperparameter*, *Random Search* hanya memilih sejumlah kombinasi parameter secara acak berdasarkan jumlah iterasi yang ditentukan (n_iter) (Subaşi, 2024). Pendekatan ini memungkinkan eksplorasi ruang parameter yang lebih luas tanpa harus melakukan evaluasi terhadap seluruh kombinasi yang tersedia.

Secara umum, proses pemilihan kombinasi parameter pada *Random Search* dapat dinyatakan seperti di **Persamaan 2.31**

$$\lambda_i \sim P(\Lambda) \quad (2.31)$$

Di mana:

- λ_i : kombinasi *hyperparameter* ke- i .
- $P(\Lambda)$: distribusi probabilitas ruang pencarian *hyperparameter*.

Pada setiap iterasi, *RandomizedSearchCV* akan memilih satu kombinasi *hyperparameter* secara acak, melatih model menggunakan kombinasi tersebut, kemudian mengevaluasi performanya menggunakan metode *cross validation* (Rizky et al., 2024). Setelah seluruh iterasi selesai dilakukan, kombinasi parameter yang menghasilkan nilai evaluasi terbaik akan dipilih sebagai konfigurasi optimal.

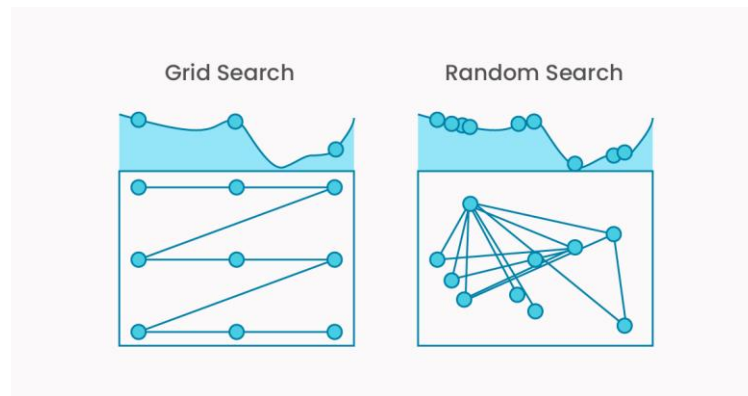
Beberapa parameter utama yang digunakan pada *RandomizedSearchCV* dapat dilihat pada **Tabel 2.1**

Tabel 2.1 Parameter RandomSearchCV

Parameter	Keterangan
estimator	Model yang akan dilakukan proses tuning
param_distributions	Ruang pencarian hyperparameter yang akan dieksplorasi.

n_iter	Jumlah kombinasi parameter yang diuji.
scoring	Metrik evaluasi yang digunakan untuk menentukan parameter terbaik.
cv	Jumlah fold pada proses cross validation.
random_state	Nilai seed untuk memastikan reproduksibilitas eksperimen.
n_jobs	Menggunakan seluruh core prosesor yang tersedia untuk mempercepat proses tuning.
verbose	Menampilkan informasi proses tuning selama eksekusi.

Pada penelitian ini, *RandomizedSearchCV* digunakan untuk mengoptimalkan lima kombinasi model dan teknik resampling yang memperoleh nilai *F1-score* tertinggi pada tahap evaluasi awal.



Gambar 2.15 Perbandingan GridSearch dengan RandomSearch

Sumber: Simple ML, 2025, <https://ai.plainenglish.io/boost-your-ml-models-with-gridsearchcv-randomizedsearchcv-0bea9d173361>

2.3.7 Cross Validation

Cross Validation merupakan teknik evaluasi model yang digunakan untuk mengukur kemampuan generalisasi model terhadap data yang belum pernah dilihat sebelumnya (Karl et al., 2022). Teknik ini dilakukan dengan membagi data pelatihan menjadi beberapa bagian (*fold*) kemudian melatih dan mengevaluasi model secara berulang menggunakan kombinasi data yang berbeda pada setiap iterasi (Ünal et al., 2024).

Salah satu metode yang paling umum digunakan adalah *K-Fold Cross Validation*. Pada metode ini, data dibagi menjadi *K* bagian dengan ukuran yang relatif sama. Pada setiap iterasi, satu fold digunakan sebagai data validasi sedangkan fold lainnya digunakan sebagai data pelatihan hingga seluruh fold pernah menjadi data validasi satu kali (Ünal et al., 2024).

Pada skenario pengujian performa model yang menggunakan *hyperparameter tuning*, digunakan *Stratified K-Fold Cross Validation*. Metode ini mempertahankan proporsi kelas pada setiap fold sehingga distribusi kelas fraud dan non-fraud tetap konsisten selama proses validasi (Ünal et al., 2024). Pendekatan ini sangat penting untuk menghindari kondisi di mana suatu fold memiliki jumlah sampel fraud yang terlalu sedikit sehingga hasil evaluasi menjadi tidak

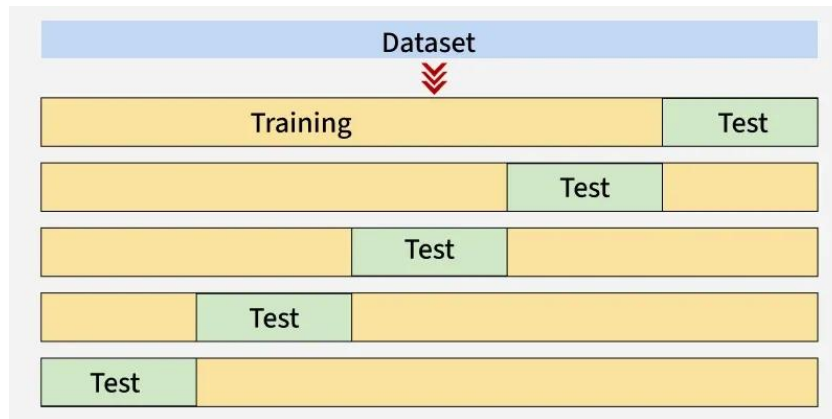
representatif. Nilai performa akhir dari proses cross validation diperoleh melalui rata-rata hasil evaluasi seluruh fold yang digunakan seperti yang ditunjukkan pada **Persamaan 2.32**.

$$CV_{score} = \frac{1}{K} \sum_{i=1}^K Score_i \quad (2.32)$$

Di mana:

- CV_{score} : nilai rata-rata performa *cross validation*.
- K : jumlah fold.
- $Score_i$: nilai evaluasi pada fold ke- i .

Pada penelitian ini digunakan *Stratified 3-Fold Cross Validation*, sehingga data pelatihan dibagi menjadi lima fold dengan proporsi kelas fraud dan non-fraud yang tetap terjaga. Nilai *recall* rata-rata dari lima fold tersebut digunakan sebagai dasar pemilihan kombinasi *hyperparameter* terbaik pada proses *RandomizedSearchCV*.



Gambar 2.16 Visualisasi CrossValidation

Sumber: (Abi Nader & Soufi, 2026)

2.3.8 Metrik Evaluasi

1. Confusion Matrix

Confusion matrix adalah representasi kinerja classifier biner melalui matriks $N \times N$ yang digunakan untuk mengevaluasi kinerja suatu model klasifikasi, di mana N merupakan jumlah kelas target. *Confusion matrix* sepenuhnya ditentukan oleh empat komponen yaitu *True Positive* (TP), *True Negative* (TN), *False Positive* (FP), dan *False Negative* (FN). *True Positive* merupakan jumlah sampel positif yang diprediksi positif, *True Negative* adalah sampel negatif yang diprediksi negatif, *False Positive* merupakan sampel negatif yang diprediksi positif, dan *False Negative* sebagai sampel positif yang diprediksi negatif. Matriks ini juga dapat dinyatakan dengan jumlah aktual positif (AP) dan aktual negatif (AN) serta estimasi positif (EP) dan estimasi negatif (EN), dengan hubungan $AP + AN = EP + EN = n$ (jumlah seluruh sampel). **Gambar 2.15** menunjukkan contoh bentuk *confusion matrix*. (Lavazza & Morasca, 2023)

		Real Label	
		Positive	Negative
Predicted Label	Positive	True Positive(TP)	False Positive(FP)
	Negative	False Negative(FN)	True Negative(TN)

Gambar 2.17 Confusion Matrix

Sumber: Nouri et al., 2025

2. Recall

Recall atau sering disebut juga *True Positive Rate* atau *sensitivity*/sensitivitas adalah metrik kelengkapan deteksi positif, yaitu dari semua positif aktual, berapa yang berhasil terdeteksi sebagai positif. Recall berfokus pada kemampuan model menangkap kasus positif dan sensitif terhadap banyaknya *False Negative*. Recall tinggi berarti model jarang melewatkan sampel positif yang terlihat melalui *False Negative* rendah relatif terhadap *True Positive*. (Lavazza & Morasca, 2023)

Pada deteksi *fraud*, *False Negative* berarti transaksi fraud yang lolos sebagai transaksi normal, yang secara operasional biasanya berdampak langsung pada kerugian finansial, risiko *chargeback*, dan potensi kerusakan reputasi. Karena itu, banyak sistem *fraud detection* menargetkan *recall* tinggi agar transaksi fraud tertangkap sebanyak mungkin, lalu menyeimbangkannya dengan *precision* agar jumlah alarm palsu (*False Positive*) tetap terkendali.

Rumus Recall dapat ditunjukkan di **Persamaan 2.33**

$$Recall = \frac{TP}{TP + FN} \quad (2.33)$$

Di mana:

- *Recall*: nilai recall/TPR.
- *TP*: true positive.
- *FN*: false negative.

3. Accuracy

Accuracy didefinisikan sebagai proporsi prediksi yang benar atau *True Positive* dan *True Negative* terhadap seluruh sampel. Akurasi mengukur seberapa sering model benar secara

keseluruhan tanpa membedakan jenis kesalahan (*False Positive* dan *False Negative*) Karena menggabungkan semua kondisi menjadi satu angka, akurasi dapat menutupi kelemahan model pada kelas tertentu ketika distribusi kelas tidak seimbang. (Lavazza & Morasca, 2023)

Rumus Accuracy dapat ditunjukkan di **Persamaan 2.34**

$$Accuracy = \frac{TP + TN}{n} \quad (2.34)$$

Di mana:

- *Accuracy*: nilai accuracy.
- *TP*: true positive.
- *TN*: true negative.
- *n*: jumlah seluruh sampel.

4. Precision

Precision atau sering disebut *Positive Predictive Value/PPV* adalah ukuran ketepatan prediksi positif, yaitu dari semua yang diprediksi positif, berapa yang benar-benar positif. *Precision* berfokus pada kualitas alarm positif dan sensitif terhadap banyaknya *False Positive*. *Precision* tinggi berarti model jarang memberi prediksi positif yang keliru yang dapat ditunjukkan melalui *False Positive* yang lebih rendah relatif terhadap *True Positive*. (Lavazza & Morasca, 2023)

Rumus Precision dapat ditunjukkan di **Persamaan 2.35**

$$Precision = \frac{TP}{TP + FP} \quad (2.35)$$

Di mana:

- *Precision*: nilai precision/PPV.
- *TP*: true positive.
- *FP*: false positive.

5. F1-Score

F1-score atau F-measure adalah rata-rata harmonik dari *precision* dan *recall* untuk merangkum keduanya dalam satu nilai. Konsep rata-rata harmonik membuat F1 memberikan penalti lebih besar ketika salah satu dari *precision* atau *recall* sangat rendah, sehingga mendorong keseimbangan di antara keduanya (Lavazza & Morasca, 2023). F1-score juga

merupakan metrik penting untuk membandingkan performa klasifikasi biner secara lebih adil pada kondisi tertentu. (Gyu Kim & Park, 2025)

Rumus F1-score dapat ditunjukkan di **Persamaan 2.36**

$$F1 = \frac{2TP}{2TP + FN + FP} \quad (2.36)$$

Di mana:

- $F1$: nilai F1-score.
- TP : true positive.
- FP : false positive.
- FN : false negative.

6. Precision-Recall Area Under Curve (PR-AUC)

Precision-Recall Area Under Curve (PR-AUC) atau *Area Under the Precision-Recall Curve* (AUPRC) merupakan metrik evaluasi yang mengukur kemampuan model klasifikasi biner dalam membedakan kelas positif melalui hubungan antara precision dan recall pada berbagai nilai *threshold*. Berbeda dengan metrik yang hanya mengevaluasi model pada satu *threshold* tertentu, PR-AUC memberikan gambaran menyeluruh mengenai performa model di seluruh kemungkinan *threshold* sehingga lebih representatif untuk mengevaluasi kualitas model klasifikasi (Mullick et al., 2020).

Kurva Precision-Recall dibentuk dengan memplot nilai *Recall* pada sumbu horizontal (x-axis) dan *Precision* pada sumbu vertikal (y-axis). Setiap titik pada kurva diperoleh dari perubahan nilai *threshold* probabilitas prediksi model. Semakin tinggi posisi kurva mendekati sudut kanan atas, semakin baik kemampuan model dalam mempertahankan nilai precision dan recall secara bersamaan (Richardson et al., 2024).

Dalam implementasi komputasi, nilai PR-AUC dihitung sebagai luas daerah di bawah kurva Precision-Recall umumnya dihitung menggunakan *trapezoidal rule*, yaitu penjumlahan luas trapesium antar pasangan titik pada kurva Precision-Recall, sehingga dapat dituliskan seperti di **Persamaan 2.37**.

$$PR-AUC \approx \sum_{i=1}^{n-1} (R_{i+1} - R_i) \times \frac{P_{i+1} + P_i}{2} \quad (2.37)$$

Di mana:

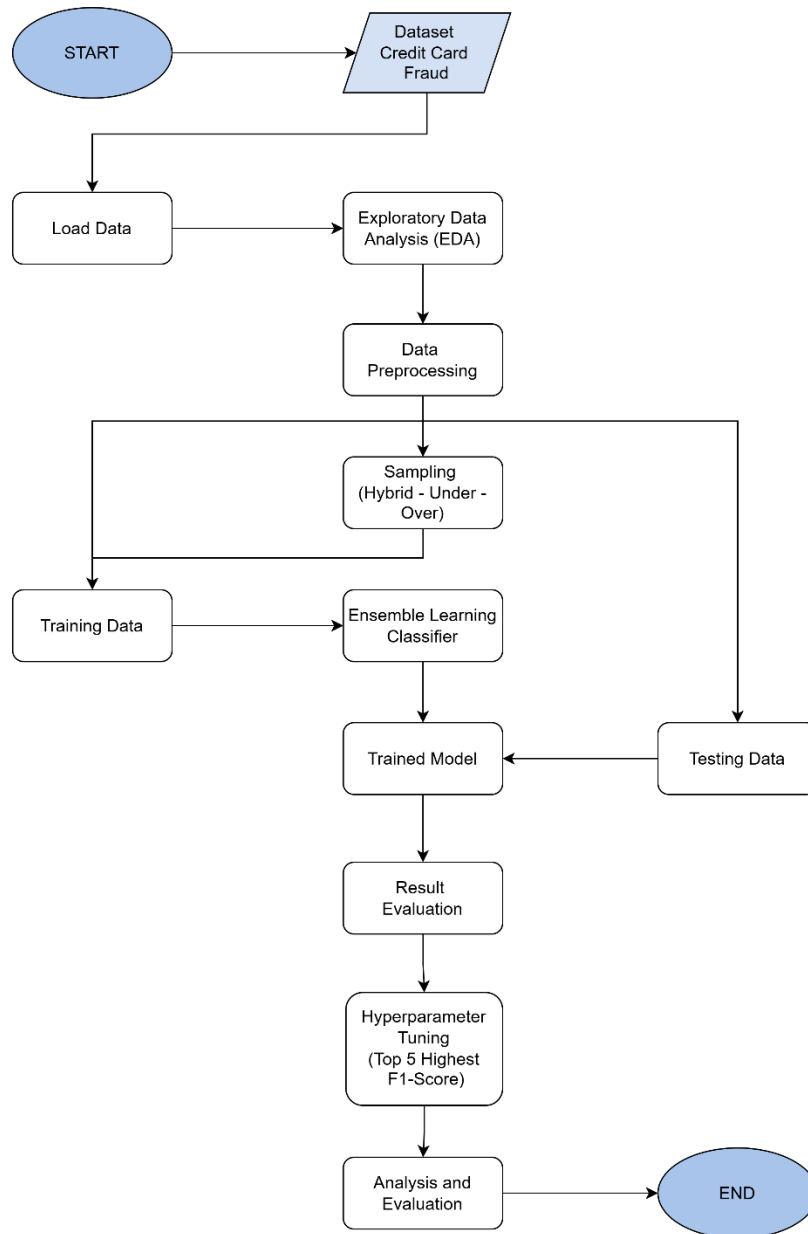
- P_i = nilai precision pada titik ke- i ,
- R_i = nilai recall pada titik ke- i .

Halaman ini sengaja dikosongkan

BAB 3 METODOLOGI

3.1 Metode yang digunakan

Subbab ini akan menjelaskan metode yang digunakan dalam penelitian ini yang bertujuan untuk menganalisis efektivitas teknik *resampling* dalam menangani *imbalanced* data di dataset penipuan dalam transaksi kartu kredit pada model *ensemble learning*. **Gambar 3.1** menunjukkan diagram alir tahapan proses pengembangan model dimana masing-masing tahapan akan dijelaskan di subbab berikut.



Gambar 3.1 Diagram Alir Penelitian

3.1.1 Dataset Credit Card Fraud Detection

Pada penelitian ini, dataset yang digunakan adalah *Credit Card Fraud Detection* yang tersedia di Kaggle. Dataset ini dikumpulkan oleh sebuah organisasi yang ada di Université Libre de Bruxelles (ULB) yang bernama Université Libre de Bruxelles (ULB) Machine Learning

Group. Pemilihan dataset ini untuk dijadikan bahan penelitian didasari oleh relevansi dan validitas dataset yang cocok dengan topik penelitian. Dataset ini sudah digunakan oleh jurnal-jurnal yang mengangkat topik penipuan dalam transaksi kartu kredit sehingga keakuratan dan reputasinya sudah teruji. Dataset berisi transaksi kartu kredit dunia nyata oleh pengguna kartu kredit di Eropa pada bulan September 2023 selama periode 2 hari.

Dataset memiliki total 284.807 data. Dari keseluruhan transaksi tersebut, hanya 492 transaksi teridentifikasi sebagai fraud sehingga proporsi kelas fraud hanya sekitar 0,172% dan menunjukkan kondisi data yang sangat imbalanced. **Gambar 3.2** menunjukkan jumlah dan proporsi kelas target berdasarkan hasil eksplorasi dataset.

```
Distribusi kelas Target (Persentase):
Class
0    0.998273
1    0.001727
Name: proportion, dtype: float64
Distribusi kelas Target:
Class
0    284315
1     492
Name: count, dtype: int64
```

Gambar 3.2 Distribusi kelas target

Dataset ini memiliki total 31 kolom, yang terdiri dari 28 fitur (V1–V28) yang merupakan hasil transformasi *Principal Component Analysis* (PCA) yang dilakukan untuk menjaga privasi pengguna kartu kredit yang ada di dataset, serta tiga kolom tambahan yaitu *Time*, *Amount*, dan *Class*. Fitur *Time* berisi jarak yang berlalu antara setiap transaksi dan transaksi pertama di dataset dalam satuan detik. Fitur *Amount* adalah nominal yang dikeluarkan dalam transaksi tersebut. Fitur *Class* adalah fitur target klasifikasi yang bernilai 1 jika merupakan penipuan dan 0 jika bukan merupakan penipuan. Seluruh kolom prediktor (V1–V28, *Time*, *Amount*) bersifat numerik kontinu dan memiliki tipe data *float*. Sementara itu, kolom *Class* direpresentasikan sebagai bilangan bulat (integer) karena hanya memuat kategori diskrit 0 atau 1.

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	V23	V24	V25	V26	V27	V28	Amount	Class
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	-0.189115	0.133558	-0.021053	149.62	0
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	0.125895	-0.008983	0.014724	2.69	0
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	-0.139097	-0.055353	-0.059752	378.66	0
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	-0.221929	0.062723	0.061458	123.50	0
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	0.502292	0.219422	0.215153	69.99	0

Gambar 3.3 Contoh entri data

3.1.2 Load Data

Tahap *load data* merupakan langkah awal dalam alur penelitian yang bertujuan untuk memasukkan dataset *Credit Card Fraud Detection* dari sumber eksternal yang berbentuk file *Comma Separated Value* (CSV) yang didapatkan dari Kaggle ke dalam lingkungan pemrograman sehingga dapat dilakukan pengujian dan diolah lebih lanjut. Pada tahap ini, file dataset dibaca menggunakan pustaka pemrosesan data dan disimpan dalam struktur dataframe

yang memuat seluruh 31 fitur, yaitu 28 fitur hasil transformasi PCA (V1–V28) beserta fitur Time, Amount, dan Class sebagai fitur target. Dataset yang telah dimuat kemudian dicek kembali ukurannya (jumlah baris dan kolom), tipe data masing-masing fitur, distribusi label Class (0 untuk non-fraud dan 1 untuk fraud), serta beberapa sampel baris teratas untuk memastikan proses pembacaan file berjalan dengan benar dan tidak terjadi kesalahan format sebelum dilanjutkan ke tahap *exploratory data analysis* dan *preprocessing* berikutnya.

3.1.3 Exploratory Data Analysis (EDA)

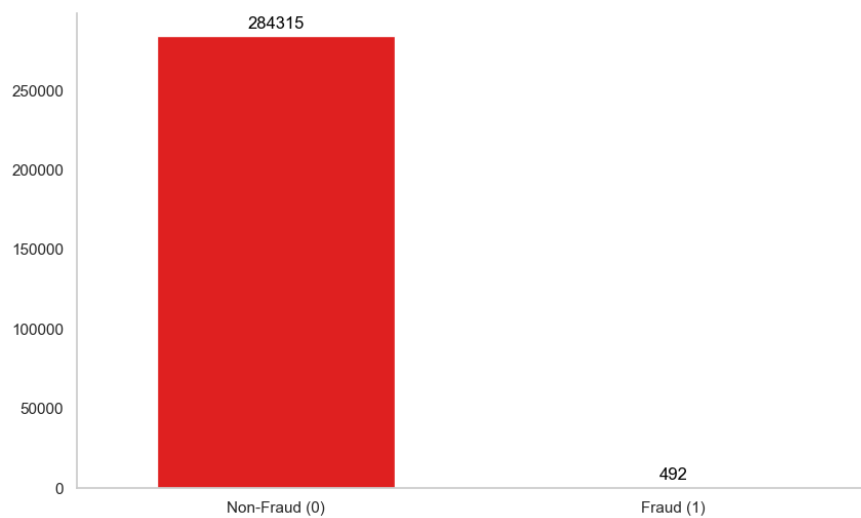
Tahap *Exploratory Data Analysis* (EDA) dilakukan untuk memahami karakteristik dan pola dasar pada dataset sebelum dilakukan pemodelan lebih lanjut. Pada tahap ini, analisis dimulai dari statistik deskriptif untuk setiap fitur numerik (rata-rata, standar deviasi, nilai minimum–maksimum, dan kuartil) yang ditunjukkan pada **Gambar 3.4**.

	count	mean	std	min	25%	50%	75%	max
Time	284807.0	9.481386e+04	47488.145955	0.000000	54201.500000	84692.000000	139320.500000	172792.000000
V1	284807.0	1.168375e-15	1.958696	-56.407510	-0.920373	0.018109	1.315642	2.454930
V2	284807.0	3.416908e-16	1.651309	-72.715728	-0.598550	0.065486	0.803724	22.057729
V3	284807.0	-1.379537e-15	1.516255	-48.325589	-0.890365	0.179846	1.027196	9.382558
V4	284807.0	2.074095e-15	1.415869	-5.683171	-0.848640	-0.019847	0.743341	16.875344
V5	284807.0	9.604066e-16	1.380247	-113.743307	-0.691597	-0.054336	0.611926	34.801666
V6	284807.0	1.487313e-15	1.332271	-26.160506	-0.768296	-0.274187	0.398565	73.301626
V7	284807.0	-5.556467e-16	1.237094	-43.557242	-0.554076	0.040103	0.570436	120.589494
V8	284807.0	1.213481e-16	1.194353	-73.216718	-0.208630	0.022358	0.327346	20.007208
V9	284807.0	-2.406331e-15	1.098632	-13.434066	-0.643098	-0.051429	0.597139	15.594995
V10	284807.0	2.239053e-15	1.088850	-24.588262	-0.535426	-0.092917	0.453923	23.745136
V11	284807.0	1.673327e-15	1.020713	-4.797473	-0.762494	-0.032757	0.739593	12.018913
V12	284807.0	-1.247012e-15	0.999201	-18.683715	-0.405571	0.140033	0.618238	7.848392
V13	284807.0	8.190001e-16	0.995274	-5.791881	-0.648539	-0.013568	0.662505	7.126883
V14	284807.0	1.207294e-15	0.958596	-19.214325	-0.425574	0.050601	0.493150	10.526766
V15	284807.0	4.887456e-15	0.915316	-4.498945	-0.582884	0.048072	0.648821	8.877742
V16	284807.0	1.437716e-15	0.876253	-14.129855	-0.468037	0.066413	0.523296	17.315112
V17	284807.0	-3.772171e-16	0.849337	-25.162799	-0.483748	-0.065676	0.399675	9.253526
V18	284807.0	9.564149e-16	0.838176	-9.498746	-0.498850	-0.003636	0.500807	5.041069
V19	284807.0	1.039917e-15	0.814041	-7.213527	-0.456299	0.003735	0.458949	5.591971
V20	284807.0	6.406204e-16	0.770925	-54.497720	-0.211721	-0.062481	0.133041	39.420904
V21	284807.0	1.654067e-16	0.734524	-34.830382	-0.228395	-0.029450	0.186377	27.202839
V22	284807.0	-3.568593e-16	0.725702	-10.933144	-0.542350	0.006782	0.528554	10.503090
V23	284807.0	2.578648e-16	0.624460	-44.807735	-0.161846	-0.011193	0.147642	22.528412
V24	284807.0	4.473266e-15	0.605647	-2.836627	-0.354586	0.040976	0.439527	4.584549
V25	284807.0	5.340915e-16	0.521278	-10.295397	-0.317145	0.016594	0.350716	7.519589
V26	284807.0	1.683437e-15	0.482227	-2.604551	-0.326984	-0.052139	0.240952	3.517346
V27	284807.0	-3.660091e-16	0.403632	-22.565679	-0.070840	0.001342	0.091045	31.612198
V28	284807.0	-1.227390e-16	0.330083	-15.430084	-0.052960	0.011244	0.078280	33.847808
Amount	284807.0	8.834962e+01	250.120109	0.000000	5.600000	22.000000	77.165000	25691.160000
Class	284807.0	1.727486e-03	0.041527	0.000000	0.000000	0.000000	0.000000	1.000000

Gambar 3.4 Statistik deskriptif dataset

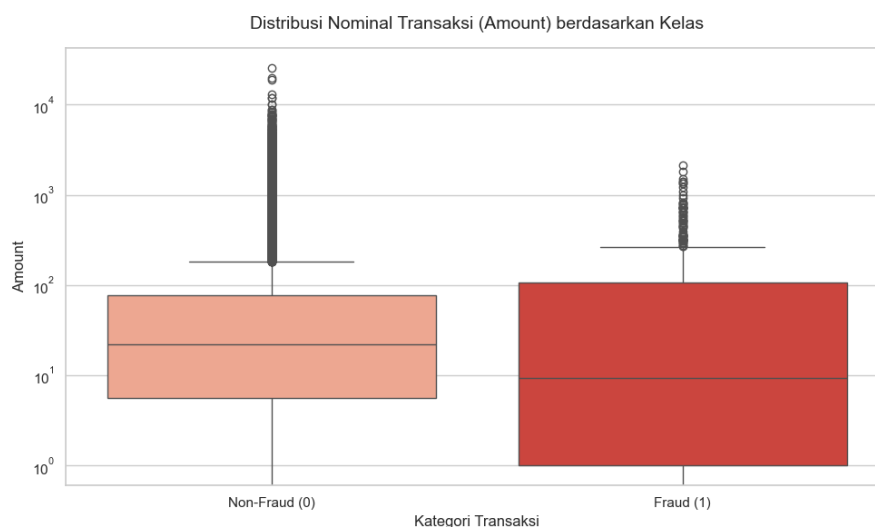
Gambar 3.5 menunjukkan distribusi fitur target yang divisualisasikan menggunakan diagram batang (*countplot*) untuk mengidentifikasi proporsi kelas fitur target. Visualisasi tersebut memperlihatkan adanya ketimpangan data yang ekstrem (*extreme class imbalance*), di mana kelas minoritas yang merepresentasikan kasus *fraud* hanya berjumlah 492 sampel, sedangkan kelas mayoritas yang merepresentasikan kasus *non-fraud* memiliki 284.315 data. Informasi dari grafik ini menjadi bukti yang kuat bahwa dataset memerlukan penanganan

khusus melalui teknik *resampling* guna mencegah bias pada model klasifikasi akibat dominasi kelas mayoritas.

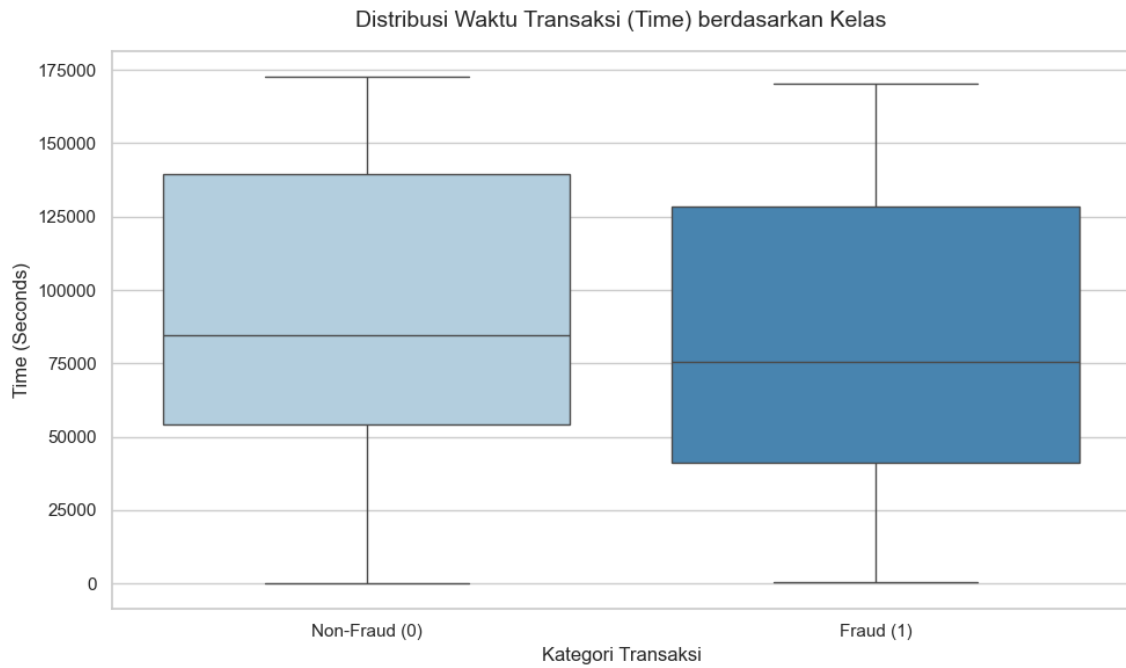


Gambar 3.5 Countplot fitur target

Fitur `Amount` dan `Time` merupakan fitur yang tidak di implementasikan PCA pada dataset ini. Fitur tersebut merepresentasikan jarak transaksi dengan transaksi pertama dalam satuan detik dan jumlah nominal yang dikeluarkan. Kedua fitur tersebut dapat dilakukan eksplorasi menggunakan grafik untuk memperlihatkan informasi-informasi yang dapat membantu proses pembangunan model. **Gambar 3.6** dan **3.7** menunjukkan distribusi data kedua fitur tersebut berdasarkan kelas targetnya. Grafik tersebut merupakan *boxplot* yang membantu memperlihatkan distribusi data, rata-rata, nilai tengah, kuartil, dan data *outlier*. Dari gambar tersebut, dapat dilihat bahwa tidak ada *outlier* untuk fitur `Time` tetapi untuk fitur `Amount` masih ada banyak *outlier* khususnya untuk data kelas *non-fraud* (0)

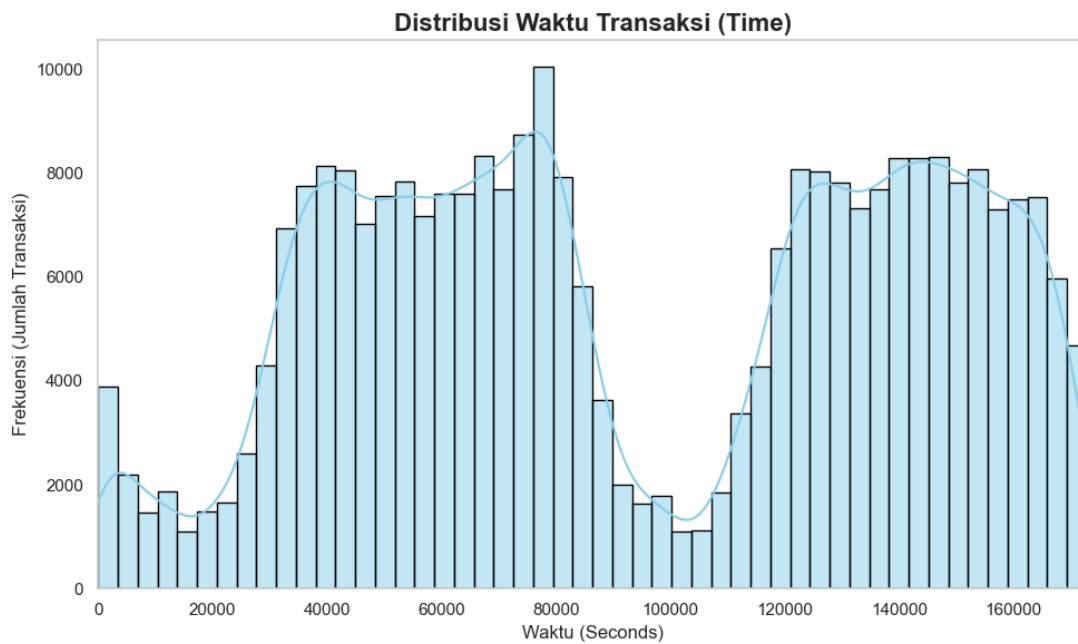


Gambar 3.6 Countplot fitur amount

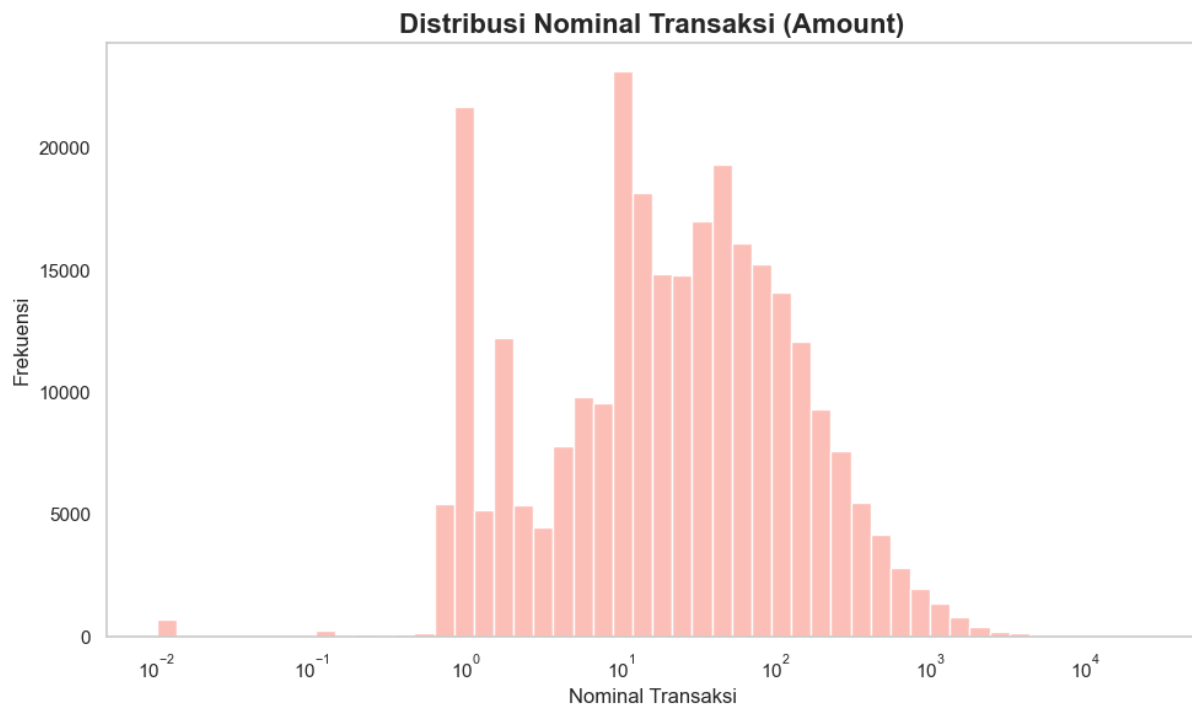


Gambar 3.7 Countplot fitur time

Tahapan eksplorasi selanjutnya dilakukan untuk mengetahui distribusi data dalam bentuk histogram. Grafik histogram membantu untuk memahami pola penyebaran data dalam bentuk batang. Data dikelompokkan ke dalam rentang (*bins*) dan menunjukkan seberapa sering data jatuh di setiap rentang, sehingga bentuk sebaran data terlihat jelas. **Gambar 3.8** dan **3.9** menunjukkan visualisasi data melalui histogram.



Gambar 3.8 Histogram fitur time

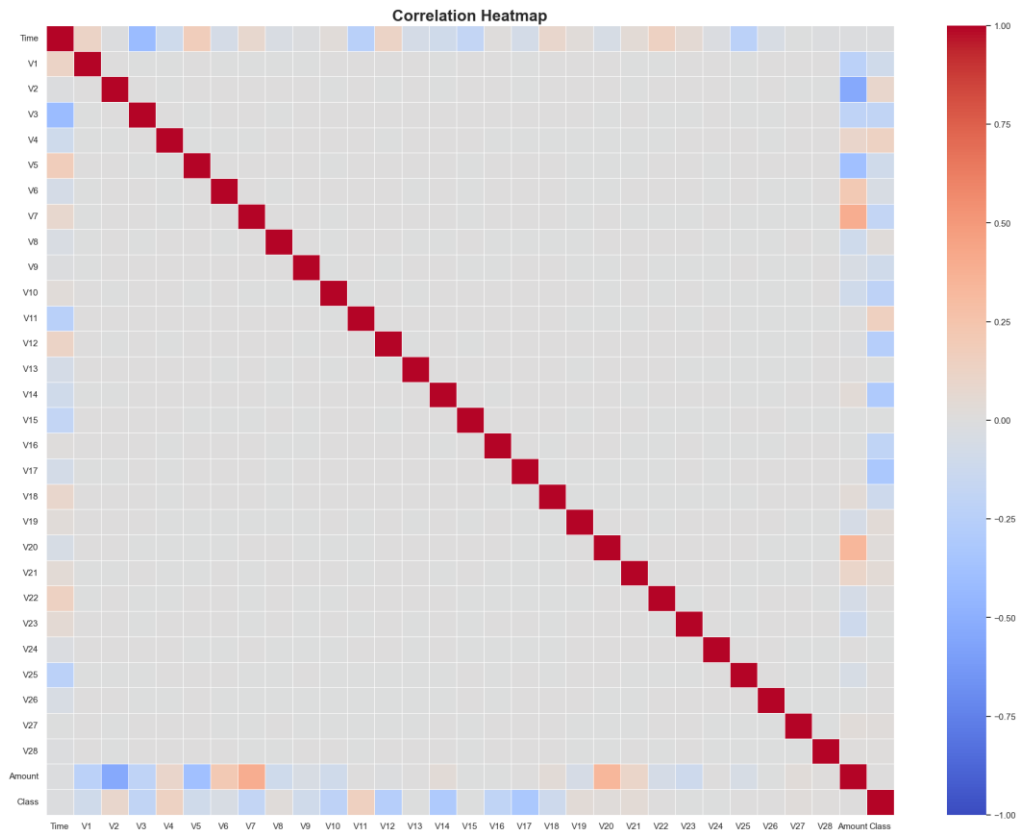


Gambar 3.9 Histogram fitur amount

Eksplorasi selanjutnya adalah mencari hubungan korelasi antar fitur, terutama untuk menemukan informasi mengenai fitur yang paling berkorelasi ke fitur target. Hal tersebut dapat membantu dalam menentukan fitur yang difokuskan dalam mempersiapkan pembuatan model. Alat yang digunakan untuk menemukan informasi tersebut pada dataset adalah *correlation heatmap matrix*. Visualisasi heatmap matriks korelasi menyajikan analisis hubungan linear antar variabel dalam dataset.

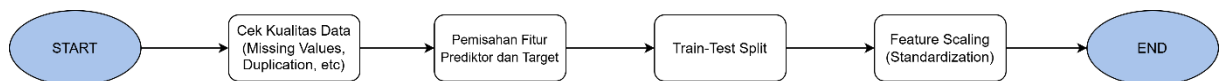
Pada Gambar 3.10, observasi pada area tengah matriks memperlihatkan warna abu-abu atau netral (nilai korelasi mendekati 0) pada interaksi antar fitur V1 hingga V28. Hal ini mengonfirmasi sifat data hasil transformasi *Principal Component Analysis* (PCA) yang bersifat ortogonal, di mana tidak terdapat multikolinearitas antar komponen utamanya. Fokus analisis pada baris dan kolom fitur target *Class* menunjukkan adanya variasi pengaruh fitur terhadap penentuan label *fraud*. Terlihat adanya korelasi negatif yang cukup signifikan (ditandai dengan warna biru) pada fitur-fitur seperti V10, V12, V14, dan V17, yang mengindikasikan bahwa semakin rendah nilai fitur tersebut, semakin tinggi kemungkinan transaksi tergolong *fraud*.

Sebaliknya, terdapat korelasi positif (berwarna kemerahan) pada fitur seperti V4 dan V11. Sementara itu, analisis terhadap variabel yang tidak ditransformasi PCA yaitu *Time* dan *Amount* menunjukkan karakteristik yang berbeda dibandingkan fitur hasil PCA. Pada visualisasi *heatmap*, interaksi antara kedua fitur ini dengan variabel target *Class* ditandai dengan warna yang cenderung netral, yang mengindikasikan korelasi linear yang sangat lemah atau mendekati nol. Hal ini menyiratkan bahwa secara statistik individual, besaran nominal transaksi maupun waktu terjadinya transaksi bukanlah penentu tunggal yang dominan dalam membedakan transaksi sah dan penipuan.



Gambar 3.10 Correlation heatmap matrix

3.1.4 Data Preprocessing



Gambar 3.11 Alur tahapan preprocessing

Tahap pra-pemrosesan yang divisualisasikan alurnya melalui **Gambar 3.11** bertujuan untuk menyiapkan data agar berada dalam kondisi yang optimal sebelum digunakan pada proses pemodelan. Tahap ini terdiri atas empat langkah utama, yaitu pemeriksaan kualitas data, pemisahan fitur prediktor dan target, pembagian data, serta penyesuaian skala fitur (*feature scaling*).

Langkah pertama adalah pemeriksaan kualitas data untuk memastikan integritas dataset sebelum digunakan dalam proses pelatihan model. Proses ini meliputi pengecekan terhadap nilai yang hilang (*missing values*) dan duplikasi data yang berpotensi menimbulkan bias pada hasil pelatihan. Meskipun dataset yang bersumber dari *Université Libre de Bruxelles* ini telah melalui tahap kurasi awal, verifikasi ulang tetap dilakukan untuk menjamin dataset bebas dari permasalahan teknis. Mengingat dataset terdiri dari data numerik hasil transformasi PCA, fokus utama pada tahap ini adalah memastikan konsistensi tipe data pada seluruh 31 kolom yang tersedia. Implementasi pemeriksaan kualitas data ditunjukkan pada **Kode Semu 3.1**.

Input: Dataset Credit Card Fraud Detection	
1.	BEGIN
2.	READ dataset dan simpan ke dataframe df
3.	COUNT missing values untuk setiap atribut pada df
4.	STORE informasi missing values
5.	COUNT duplicated records pada df
6.	STORE informasi duplicated records
7.	DISPLAY informasi kualitas data
8.	END
Output: Informasi missing values, Informasi data duplikat	

Kode Semu 3.1 Cek Kualitas Data

Setelah kualitas data dipastikan memadai, dilakukan pemisahan antara fitur prediktor dan fitur target. Pada penelitian ini, fitur prediktor terdiri atas seluruh atribut transaksi, yaitu V1 hingga V28, Time, dan Amount, sedangkan fitur Class digunakan sebagai target klasifikasi yang merepresentasikan status transaksi, yaitu fraud (1) atau non-fraud (0). Pemisahan ini dilakukan agar proses pembagian data dan pemodelan dapat dilakukan secara sistematis menggunakan variabel input dan output yang terdefinisi dengan jelas. Implementasi pemisahan fitur prediktor dan target ditunjukkan pada **Kode Semu 3.2**.

Input: Dataset Credit Card Fraud Detection	
1.	BEGIN
2.	SELECT seluruh kolom selain Class sebagai fitur
3.	STORE atribut terpilih ke variabel X
4.	SELECT kolom Class sebagai target
5.	STORE atribut terpilih ke variabel y
6.	VERIFY jumlah data pada X dan y
7.	END
Output: Variabel fitur (X), Variabel target (y)	

Kode Semu 3.2 Pemisahan fitur prediktor dan target

Langkah selanjutnya adalah pembagian data (*train-test split*). Dataset dibagi menjadi dua bagian terpisah, yaitu data latih (*training set*) dan data uji (*testing set*). Mengingat kondisi dataset yang sangat tidak seimbang (*imbalanced*) dengan persentase kelas positif hanya 0,172%, metode pembagian yang digunakan adalah *Stratified Train-Test Split*. Metode ini memastikan bahwa rasio antara kelas *fraud* dan *non-fraud* pada data *train* dan data *test* tetap terjaga proporsionalitasnya sesuai populasi asli. Proses ini dilakukan sebelum penerapan teknik *resampling*. Hal ini bertujuan untuk menjaga kemurnian data *test* agar tetap merepresentasikan kondisi dunia nyata yang *imbalanced*, serta mencegah terjadinya kebocoran data (*data leakage*) yang dapat menyebabkan evaluasi model menjadi bias dan terlalu optimis. Implementasi *train-test split* ditunjukkan pada **Kode Semu 3.3**.

Input: Variabel fitur (X), Variabel target (y)	
1.	BEGIN
2.	SET test_size = 0.2
3.	SET random_state = 42
4.	APPLY Stratified Train-Test Split
5.	USE X sebagai variabel fitur
6.	USE y sebagai variabel target
7.	MAINTAIN distribusi kelas menggunakan stratify = y
8.	GENERATE X_train dan X_test

9.	GENERATE y_train dan y_test
10.	COUNT distribusi kelas pada y_train
11.	COUNT distribusi kelas pada y_test
12.	Display distribusi kelas data train dan data test
13.	END
Output: X_train, X_test, y_train, y_test	
Kode Semu 3.3 Train-test split	

Tahap terakhir dalam pra-pemrosesan ini adalah penyesuaian skala fitur (*Feature Scaling*) menggunakan metode *StandardScaler*. Dataset ini memiliki karakteristik fitur yang dengan rentang nilai yang berbeda-beda. 28 fitur utama (V1–V28) telah ditransformasi menggunakan PCA sehingga memiliki distribusi nilai yang sudah terstandarisasi, kemudian terdapat dua fitur yang tidak di transformasi, yaitu *Time* dan *Amount*, yang nilainya tidak berskala. Fitur *Amount* menunjukkan nominal transaksi yang memiliki rentang nilai yang sangat bervariasi dan cenderung memiliki outlier ekstrem, sedangkan fitur *Time* merepresentasikan durasi dalam satuan detik. Ketimpangan skala antara fitur PCA dan fitur *Time* dan *Amount* ini dapat menurunkan performa algoritma optimasi pada model Machine Learning. Oleh karena itu, teknik standarisasi *StandardScaler* diterapkan pada data *training* untuk menjaga konsistensi skala pada seluruh fitur prediktor dan menghindari dominasi fitur tertentu selama proses pembelajaran model. Parameter hasil standarisasi yang diperoleh dari data latih kemudian diterapkan pada data uji menggunakan transformasi yang sama. Dengan demikian, seluruh fitur memiliki skala yang lebih seragam sehingga proses pembelajaran model dapat berlangsung secara lebih stabil dan efektif. Implementasi *feature scaling* ditunjukkan pada **Kode Semu 3.4**.

Input: X_train, X_test	
1.	BEGIN
2.	INITIALIZE StandardScaler
3.	FIT StandardScaler menggunakan X_train
4.	CALCULATE rata-rata setiap fitur pada X_train
5.	CALCULATE standar deviasi setiap fitur pada X_train
6.	TRANSFORM X_train menggunakan parameter hasil fitting
7.	STORE hasil transformasi ke X_train_scaled
8.	TRANSFORM X_test menggunakan parameter yang sama
9.	STORE hasil transformasi ke X_test_scaled
10.	END
Output: X_train_scaled, X_test_scaled	
Kode Semu 3.4 Feature scaling	

3.1.5 Data Resampling

Pada dataset yang digunakan dalam penelitian ini, jumlah transaksi fraud hanya sekitar 0,172% dari total transaksi sehingga model cenderung lebih banyak mempelajari karakteristik kelas mayoritas dibandingkan kelas minoritas. Kondisi tersebut dapat menyebabkan model menghasilkan nilai akurasi yang tinggi, namun memiliki kemampuan yang rendah dalam mendeteksi transaksi fraud. Untuk mengatasi bias klasifikasi yang disebabkan oleh ketidakseimbangan ekstrem pada dataset, penelitian ini menerapkan strategi penanganan pada level data (*data-level approach*) melalui teknik *resampling*. Resampling bertujuan memodifikasi distribusi data pelatihan sehingga model memperoleh representasi yang lebih baik terhadap karakteristik kelas minoritas tanpa mengubah distribusi data pengujian. Seluruh proses resampling diterapkan hanya pada data latih (*training set*) setelah proses *train-test split*

selesai dilakukan. Pendekatan ini bertujuan untuk mencegah terjadinya *data leakage* serta menjaga agar data uji tetap merepresentasikan kondisi nyata yang bersifat tidak seimbang.

Teknik resampling yang digunakan dikelompokkan menjadi tiga kategori, yaitu *oversampling*, *undersampling*, dan *hybrid sampling*. Pada kategori *oversampling*, penelitian ini menggunakan *Random Oversampling* (ROS), *Synthetic Minority Oversampling Technique* (SMOTE), dan *Adaptive Synthetic Sampling* (ADASYN). *Random Oversampling* bekerja dengan menambahkan sampel kelas minoritas melalui proses duplikasi data secara acak hingga distribusi kelas menjadi lebih seimbang. Berbeda dengan ROS, SMOTE menghasilkan sampel sintetis baru melalui proses interpolasi antara suatu sampel minoritas dengan sejumlah tetangga terdekatnya (*k-nearest neighbor*). Sementara itu, ADASYN mengembangkan konsep SMOTE dengan menghasilkan lebih banyak data sintetis pada area yang dianggap lebih sulit dipelajari oleh model sehingga proses pembelajaran menjadi lebih adaptif terhadap batas keputusan antar kelas. Parameter yang digunakan pada teknik-teknik *oversampling* ditunjukkan pada **Tabel 3.1-3.3**.

Tabel 3.1 Parameter SMOTE

Parameter	Nilai	Keterangan
sampling_strategy	0,3	Rasio perbandingan kelas mayoritas dan minoritas
k_neighbors	3	Jumlah tetangga terdekat pembentuk data sintetis
random_state	42	Seed yang menjamin reproduksibilitas eksperimen

Tabel 3.2 Parameter ADASYN

Parameter	Nilai	Keterangan
sampling_strategy	0,3	Rasio perbandingan kelas mayoritas dan minoritas
n_neighbors	3	Jumlah tetangga terdekat pembentuk data sintetis
random_state	42	Seed yang menjamin reproduksibilitas eksperimen

Tabel 3.3 Parameter Random Oversampling

Parameter	Nilai	Keterangan
sampling_strategy	0,3	Rasio perbandingan kelas mayoritas dan minoritas
random_state	42	Seed yang menjamin reproduksibilitas eksperimen

Pemilihan parameter pada teknik *oversampling* dilakukan dengan mempertimbangkan karakteristik dataset yang sangat tidak seimbang serta kebutuhan untuk menjaga representasi distribusi data asli. Nilai *sampling_strategy* sebesar 0,3 dipilih agar jumlah sampel kelas minoritas meningkat secara bertahap hingga mencapai 30% dari jumlah kelas mayoritas.

Pendekatan ini bertujuan mengurangi ketidakseimbangan kelas tanpa menghasilkan data sintetis secara berlebihan. Pada SMOTE dan ADASYN, nilai $k_neighbors/n_neighbors$ sebesar 3 digunakan untuk membentuk sampel sintetis berdasarkan tetangga terdekat yang memiliki karakteristik serupa. Jumlah tetangga yang relatif kecil dipilih agar data sintetis yang dihasilkan tetap merepresentasikan pola lokal kelas minoritas dan mengurangi kemungkinan terbentuknya sampel yang berada pada wilayah kelas mayoritas. Selain itu, *random_state* sebesar 42 digunakan untuk memastikan reproduksibilitas proses pembangkitan sampel sehingga hasil eksperimen dapat direplikasi secara konsisten.

Pada kategori *undersampling*, penelitian ini menggunakan *Random Undersampling* (RUS), *Tomek Links*, dan *Edited Nearest Neighbour* (ENN). *Random Undersampling* bekerja dengan mengurangi jumlah sampel kelas mayoritas secara acak hingga mencapai distribusi tertentu. *Tomek Links* digunakan untuk mengidentifikasi pasangan data dari kelas berbeda yang saling berdekatan dan menghapus sampel mayoritas yang berada pada area ambigu. Sementara itu, ENN melakukan pembersihan data dengan menghapus sampel yang memiliki mayoritas tetangga terdekat dari kelas yang berbeda sehingga dapat mengurangi *noise* dan memperjelas batas keputusan (*boundary lines*) klasifikasi. Parameter yang digunakan pada teknik-teknik *undersampling* ditunjukkan pada **Tabel 3.4-3.6**.

Tabel 3.4 Parameter Random Undersampling

Parameter	Nilai	Keterangan
sampling_strategy	0,3	Rasio perbandingan kelas mayoritas dan minoritas
random_state	42	Seed yang menjamin reproduksibilitas eksperimen

Tabel 3.5 Parameter Tomek Links

Parameter	Nilai	Keterangan
sampling_strategy	auto	Menghapus sampel ambigu pada batas kelas selain minoritas
n_jobs	-1	Menggunakan seluruh inti prosesor

Tabel 3.6 Paramater ENN

Parameter	Nilai	Keterangan
n_neighbors	3	Jumlah tetangga untuk proses editing
kind_sel	mode	Strategi kapan sebuah data dianggap sebagai "noise" dan harus dihapus menggunakan prinsip pemungutan suara terbanyak tetangganya.

Parameter	Nilai	Keterangan
n_jobs	-1	Menggunakan seluruh inti prosesor

Pada teknik *undersampling*, pemilihan parameter difokuskan pada upaya mengurangi dominasi kelas mayoritas tanpa menghilangkan informasi penting secara berlebihan. Untuk *Random Undersampling*, nilai *sampling_strategy* sebesar 0,3 digunakan agar jumlah data mayoritas dikurangi hingga menghasilkan rasio yang lebih seimbang terhadap kelas minoritas, namun tetap mempertahankan sebagian besar informasi yang terkandung dalam data mayoritas. Pada *Tomek Links*, parameter *sampling_strategy* = *auto* dipilih karena metode ini secara teoritis dirancang untuk menghapus pasangan sampel ambigu yang berada di area perbatasan antar kelas, sehingga dapat meningkatkan kualitas pemisahan kelas tanpa memerlukan penentuan rasio secara manual. Sementara itu, pada ENN digunakan *n_neighbors* sebesar 3 dan *kind_sel* = *mode*, sehingga suatu sampel akan dihapus apabila labelnya tidak sesuai dengan mayoritas label dari tetangga terdekatnya. Konfigurasi ini dipilih karena mampu mengurangi data yang berpotensi menjadi noise sekaligus mempertahankan struktur utama distribusi data. Penggunaan *n_jobs* = -1 bertujuan memanfaatkan seluruh inti prosesor yang tersedia untuk meningkatkan efisiensi komputasi.

Penelitian ini juga menerapkan pendekatan *hybrid sampling* yang menggabungkan teknik *oversampling* dan *undersampling*. Kombinasi yang digunakan adalah SMOTE-ENN dan SMOTE-Tomek. Pada kedua metode tersebut, proses *oversampling* menggunakan SMOTE dilakukan terlebih dahulu untuk menambah jumlah sampel kelas minoritas. Selanjutnya dilakukan proses pembersihan data menggunakan ENN atau Tomek Links untuk menghilangkan sampel yang berpotensi menjadi *noise* atau berada pada area tumpang tindih antar kelas. Parameter yang digunakan pada teknik-teknik *hybrid sampling* ditunjukkan pada **Tabel 3.7-3.8**.

Tabel 3.7 Parameter SMOTE-ENN

Parameter	Nilai	Keterangan
sampling_strategy	0,5	Rasio perbandingan kelas mayoritas dan minoritas
k_neighbors (SMOTE)	3	Jumlah tetangga terdekat pembentuk data sintetis
n_neighbors (ENN)	3	Jumlah tetangga untuk proses editing
kind_sel (ENN)	mode	Strategi kapan sebuah data dianggap sebagai "noise" dan harus dihapus menggunakan prinsip pemungutan suara terbanyak tetangganya.
random_state	42	Seed yang menjamin reproduksibilitas eksperimen

Tabel 3.8 Parameter SMOTE-Tomek Links

Parameter	Nilai	Keterangan
sampling_strategy	0,5	Rasio perbandingan kelas mayoritas dan minoritas
k_neighbors (SMOTE)	3	Jumlah tetangga terdekat pembentuk data sintetis
sampling_strategy (Tomek)	auto	Menghapus sampel ambigu pada batas kelas selain minoritas
random_state	42	Seed yang menjamin reproduksibilitas eksperimen

Pada teknik *hybrid sampling*, parameter dipilih untuk mengombinasikan keunggulan *oversampling* dan *undersampling* dalam satu proses. Nilai *sampling_strategy* sebesar 0,5 digunakan pada tahap SMOTE untuk meningkatkan jumlah sampel kelas minoritas secara terkontrol sehingga ketidakseimbangan kelas dapat dikurangi. Parameter *k_neighbors* sebesar 3 digunakan pada SMOTE untuk membentuk data sintetis berdasarkan pola lokal kelas minoritas yang paling relevan. Pada SMOTE-ENN, parameter *n_neighbors* sebesar 3 dan *kind_sel = mode* diterapkan pada tahap ENN untuk menghapus sampel yang dianggap tidak konsisten dengan lingkungan sekitarnya sehingga kualitas data hasil *oversampling* dapat ditingkatkan. Sementara itu, pada SMOTE-Tomek, parameter *sampling_strategy = auto* pada Tomek Links digunakan untuk menghilangkan pasangan sampel yang berada pada batas keputusan antar kelas dan berpotensi menimbulkan ambiguitas. Seluruh metode hybrid menggunakan *random_state* sebesar 42 guna menjamin konsistensi dan reproduksibilitas proses resampling pada setiap percobaan.

Alur implementasi semua teknik resampling melalui cara dan proses yang sama sehingga direpresentasikan pada satu kode semu. Implementasi resampling data ditunjukkan pada **Kode Semu 3.5**.

Input: X_train_scaled, y_train
1. BEGIN 2. INITIALIZE teknik resampling 3. SET parameter resampling 4. APPLY resampling pada X_train_scaled dan y_train 5. GENERATE data hasil resampling 6. STORE fitur hasil resampling ke X_train_(nama teknik) 7. STORE target hasil resampling ke y_train_(nama teknik) 8. COUNT distribusi kelas sebelum proses resampling 9. COUNT distribusi kelas setelah proses resampling 10. DISPLAY distribusi kelas sebelum dan setelah resampling 11. END
Output: X_train_(nama teknik), y_train_(nama teknik), Distribusi kelas sebelum resampling, Distribusi kelas setelah resampling

Kode Semu 3.5 Resampling data

3.1.6 Model Training

Tahap *model training* merupakan proses pembentukan model klasifikasi menggunakan data latih yang telah melalui tahapan pra-pemrosesan dan resampling. Pada tahap ini, algoritma

machine learning mempelajari pola dan karakteristik yang membedakan transaksi fraud dan non-fraud berdasarkan fitur-fitur yang tersedia pada dataset. Proses pelatihan dilakukan pada seluruh dataset hasil resampling serta dataset tanpa resampling sebagai model baseline. Penggunaan model baseline bertujuan sebagai acuan untuk mengevaluasi sejauh mana teknik resampling mampu meningkatkan kemampuan model dalam mendeteksi kelas minoritas. Penelitian ini berfokus pada penerapan metode *Ensemble Learning*. Pendekatan ini mengombinasikan beberapa model pembelajaran untuk menghasilkan performa yang lebih baik dibandingkan penggunaan model tunggal. Melalui penggabungan beberapa model, ensemble learning mampu meningkatkan kemampuan generalisasi, mengurangi variansi maupun bias model, serta menghasilkan prediksi yang lebih stabil pada data yang belum pernah dilihat sebelumnya.

Model pertama yang digunakan adalah *Random Forest*, yang mewakili pendekatan *bagging ensemble*. Random Forest membangun sejumlah pohon keputusan (*decision tree*) secara paralel menggunakan subset data dan subset fitur yang dipilih secara acak. Hasil prediksi akhir diperoleh melalui mekanisme voting mayoritas dari seluruh pohon yang terbentuk. Pendekatan ini efektif dalam mengurangi variansi model dan meningkatkan ketahanan terhadap overfitting.

Model kedua dan ketiga merupakan algoritma berbasis *boosting ensemble*, yaitu *Gradient Boosting* dan *XGBoost*. Kedua algoritma tersebut membangun model secara bertahap (*sequential learning*), di mana setiap model baru dilatih untuk memperbaiki kesalahan prediksi yang dihasilkan oleh model sebelumnya. *Gradient Boosting* melakukan proses optimasi berdasarkan fungsi *loss* menggunakan pendekatan *gradient descent*, sedangkan *XGBoost* merupakan pengembangan dari *Gradient Boosting* yang dilengkapi mekanisme regularisasi, optimasi komputasi, dan penanganan data yang lebih efisien. Kedua algoritma ini dipilih karena dikenal memiliki kemampuan yang baik dalam menangani permasalahan klasifikasi dengan pola data yang kompleks.

Selain *ensemble* homogen berbasis pohon keputusan, penelitian ini juga mengimplementasikan metode *Soft Voting* sebagai bentuk *heterogeneous ensemble*. *Soft Voting* menggabungkan probabilitas prediksi yang dihasilkan oleh beberapa algoritma berbeda, kemudian menentukan kelas akhir berdasarkan rata-rata probabilitas tertinggi. Pada penelitian ini, model penyusun *Soft Voting* terdiri atas *Logistic Regression*, *Decision Tree*, dan *Support Vector Machine* (SVM). Kombinasi tersebut dipilih untuk merepresentasikan tiga pendekatan pembelajaran yang berbeda, yaitu model statistik, model berbasis aturan, dan model berbasis jarak.

Metode *ensemble* terakhir yang digunakan adalah *Stacking*. Berbeda dengan *Soft Voting* yang menggabungkan hasil prediksi secara langsung, *Stacking* menggunakan struktur pembelajaran bertingkat (*multi-level learning*). Pada penelitian ini, level pertama (*base learners*) terdiri atas *Random Forest*, *XGBoost*, dan *Gradient Boosting*. Prediksi yang dihasilkan oleh ketiga model tersebut kemudian digunakan sebagai fitur baru yang menjadi masukan bagi model tingkat kedua (*meta-learner*). Meta-learner yang digunakan adalah *Logistic Regression* dengan parameter maksimum iterasi sebesar 1000. Pendekatan ini memungkinkan model mempelajari cara terbaik untuk mengombinasikan prediksi dari masing-masing *base learner* sehingga dapat menghasilkan performa klasifikasi yang lebih optimal.

Setiap model dilatih menggunakan seluruh variasi teknik resampling yang telah dijelaskan pada tahap sebelumnya, yaitu *Random Oversampling*, *SMOTE*, *ADASYN*, *Random Undersampling*, *Tomek Links*, *ENN*, *SMOTE-Tomek*, dan *SMOTE-ENN*. Selain itu, model juga dilatih menggunakan dataset tanpa melalui resampling sebagai *baseline*. Dengan demikian, diperoleh berbagai kombinasi model dan teknik resampling yang selanjutnya akan dievaluasi untuk menentukan kombinasi dengan performa terbaik dalam mendeteksi transaksi fraud.

Pada tahap ini, seluruh model dilatih menggunakan parameter *default* yang disediakan oleh masing-masing pustaka *machine learning*, kecuali parameter tertentu yang diperlukan agar proses pelatihan dapat berjalan dengan baik, seperti pengaturan *random state* untuk menjaga reproduksibilitas eksperimen dan *max_iter* pada *Logistic Regression* untuk memastikan proses optimasi mencapai konvergensi. Penggunaan parameter default dilakukan secara sengaja karena tujuan utama penelitian ini adalah mengevaluasi pengaruh berbagai teknik *resampling* terhadap performa model *ensemble* dalam menangani ketidakseimbangan kelas, bukan membandingkan hasil optimasi parameter dari masing-masing algoritma. Dengan mempertahankan konfigurasi parameter yang sama pada seluruh eksperimen, perbedaan performa yang dihasilkan dapat lebih merefleksikan dampak dari teknik *resampling* yang diterapkan, sehingga validitas analisis menjadi lebih terjaga. Selain itu, parameter *default* pada algoritma-algoritma yang digunakan telah dirancang berdasarkan praktik umum dan sering dijadikan sebagai titik awal (*baseline configuration*) dalam berbagai penelitian machine learning. Pendekatan ini juga memungkinkan proses evaluasi dilakukan secara lebih adil dan konsisten sebelum tahap optimasi parameter (*hyperparameter tuning*) dilakukan pada model-model dengan performa terbaik. Alur kode untuk *training* setiap kombinasi melalui proses dan cara yang sama sehingga direpresentasikan oleh satu kode semu yang ditunjukkan pada **Kode Semu 3.6**.

Input: Dataset hasil resampling (X_train_resampled, y_train_resampled), Data uji (X_test, y_test)	
1.	BEGIN
2.	INITIALIZE model klasifikasi
3.	LOAD dataset hasil resampling sebagai data training
4.	TRAIN model menggunakan X_train_resampled dan y_train_resampled
5.	STORE model hasil training
6.	APPLY model pada X_test untuk menghasilkan prediksi
7.	END
Output: Model telah melalui proses training	

Kode Semu 3.6 Training Model

3.1.7 Model Testing

Tahap *model testing* dilakukan setelah proses pelatihan model selesai dilaksanakan. Tujuan utama dari tahap ini adalah menguji kemampuan generalisasi model dalam melakukan klasifikasi terhadap data yang belum pernah digunakan selama proses *training*. Kemampuan generalisasi menjadi aspek penting karena model yang memiliki performa tinggi pada data pelatihan belum tentu mampu mempertahankan performanya ketika dihadapkan pada data baru.

Pada penelitian ini, seluruh model *Ensemble Learning* yang telah dilatih menggunakan data *train* hasil resampling diuji menggunakan data uji (*testing set*) yang telah dipisahkan pada tahap *preprocessing*. Data uji tidak mengalami proses resampling maupun modifikasi distribusi kelas sehingga tetap mempertahankan karakteristik asli dataset yang sangat tidak seimbang.

Pendekatan ini dipilih untuk mensimulasikan kondisi operasional yang sesungguhnya, di mana sistem deteksi fraud harus mampu mengidentifikasi sejumlah kecil transaksi fraud yang tersebar di antara transaksi normal dalam jumlah yang jauh lebih besar.

Proses pengujian dilakukan dengan memberikan data fitur pada himpunan data uji sebagai masukan bagi model yang telah dilatih sebelumnya. Selanjutnya, model menghasilkan prediksi kelas untuk setiap transaksi berdasarkan pola yang telah dipelajari selama proses *training*. Hasil prediksi tersebut kemudian disimpan dan dibandingkan dengan label aktual (*ground truth*) yang terdapat pada data uji.

Perbandingan antara hasil prediksi dan label aktual menghasilkan empat kemungkinan klasifikasi, yaitu *True Positive* (TP), *True Negative* (TN), *False Positive* (FP), dan *False Negative* (FN). Informasi tersebut selanjutnya direpresentasikan dalam bentuk *Confusion Matrix* yang menjadi dasar perhitungan berbagai metrik evaluasi pada tahap berikutnya. Dengan menggunakan data uji yang tidak mengalami proses resampling, hasil pengujian yang diperoleh dapat memberikan gambaran yang lebih objektif mengenai kemampuan model dalam mendeteksi transaksi *fraud* pada kondisi nyata.

Tahap ini dilakukan pada seluruh kombinasi model *Ensemble Learning* dan teknik *resampling* yang digunakan dalam penelitian. Hasil prediksi dari masing-masing kombinasi kemudian menjadi masukan pada tahap *Result Evaluation* untuk dilakukan pengukuran dan perbandingan performa menggunakan berbagai metrik klasifikasi. Implementasi *model testing* ditunjukkan pada **Kode Semu 3.7**.

Input: Model Hasil Pelatihan, X_test_scaled, y_test	
1.	BEGIN
2.	LOAD model hasil training
3.	LOAD X_test_scaled sebagai data pengujian
4.	GENERATE hasil prediksi menggunakan model
5.	STORE hasil prediksi ke y_pred
6.	COMPARE y_pred dengan y_test
7.	GENERATE confusion matrix
8.	GENERATE classification report
9.	DISPLAY hasil evaluasi model
10.	END
Output: Hasil prediksi, confusion matrix, classification report	

Kode Semu 3.7 Model testing

3.1.8 Result Evaluation

Tahap *Result Evaluation* dilakukan setelah seluruh kombinasi teknik resampling dan model *ensemble learning* selesai ditrain dan diuji menggunakan data *test*. Pada tahap ini, hasil prediksi dari setiap kombinasi model dan teknik resampling dievaluasi menggunakan metrik *accuracy*, *precision*, *recall*, *F1-score*, *pr-auc*, serta *confusion matrix* yang diperoleh dari *classification report*. Evaluasi dilakukan untuk mengukur kemampuan masing-masing model dalam mendeteksi transaksi *fraud* pada kondisi data yang tetap imbalanced sehingga hasil yang diperoleh dapat merepresentasikan kondisi implementasi di dunia nyata.

Fokus utama penelitian ini adalah meningkatkan sensitivitas atau *recall* terhadap kelas *fraud*. Oleh karena itu, nilai recall digunakan sebagai dasar utama dalam membandingkan performa seluruh kombinasi model dan teknik resampling. Selain melihat performa secara individual, dilakukan juga perbandingan terhadap model *baseline* yang data trainnya tanpa

melalui penerapan teknik *resampling* untuk mengetahui efek yang diberikan oleh setiap teknik *resampling* terhadap nilai *recall* pada masing-masing model.

Berdasarkan hasil evaluasi tersebut, seluruh kombinasi model *Ensemble Learning* dan teknik *resampling* diurutkan berdasarkan nilai *recall*. *Recall* digunakan sebagai dasar pemeringkatan karena penelitian ini berfokus pada kemampuan model dalam mendeteksi transaksi *fraud* sebagai kelas minoritas. Pada kasus deteksi *fraud*, kesalahan *false negative* lebih kritis karena transaksi *fraud* yang tidak terdeteksi dapat menyebabkan kerugian finansial.

Selanjutnya, kombinasi model dengan nilai *recall* tertinggi dipertimbangkan untuk skenario selanjutnya yaitu tahap *hyperparameter tuning*. Namun, untuk memastikan keseimbangan antara kemampuan deteksi dan ketepatan prediksi, dipilih lima kombinasi dengan nilai *F1-score* terbaik. *F1-score* digunakan karena mampu merepresentasikan keseimbangan antara *precision* dan *recall*, sehingga menghindari pemilihan model yang memiliki *recall* tinggi tetapi menghasilkan terlalu banyak *false positive*.

Pemilihan hanya lima kombinasi terbaik juga bertujuan untuk meningkatkan efisiensi proses optimasi, mengingat *hyperparameter tuning* menggunakan *RandomizedSearchCV* membutuhkan waktu dan sumber daya komputasi yang cukup besar. Dengan memfokuskan optimasi pada model yang telah menunjukkan performa paling menjanjikan, proses pencarian parameter optimal dapat dilakukan secara lebih efektif dan efisien.

3.1.9 Hyperparameter Tuning

Tahap *hyperparameter tuning* dilakukan untuk meningkatkan performa model yang telah memperoleh hasil terbaik pada tahap evaluasi sebelumnya. Proses *tuning* hanya diterapkan pada lima kombinasi model dan teknik *resampling* dengan nilai *f1-score* tertinggi pada pengujian sebelumnya yang dilakukan dengan parameter *default* masing-masing model. Metode *hyperparameter tuning* yang digunakan dalam penelitian ini adalah *Randomized Search Cross Validation (RandomizedSearchCV)*. Teknik ini bekerja dengan melakukan pencarian kombinasi *hyperparameter* secara acak dari ruang parameter yang telah ditentukan sebelumnya, kemudian mengevaluasi setiap kombinasi menggunakan proses *cross validation*. Dibandingkan dengan *Grid Search*, metode *Randomized Search* memiliki efisiensi komputasi yang lebih baik karena tidak perlu mengevaluasi seluruh kemungkinan kombinasi parameter.

Pada penelitian ini, proses pencarian parameter dilakukan menggunakan *RandomizedSearchCV* dengan beberapa konfigurasi utama yang ditunjukkan pada **Tabel 3.9**.

Tabel 3.9 Parameter *RandomsearchCV*

Parameter	Nilai	Keterangan
Estimator	Model yang terpilih (top 5 nilai <i>f1-score</i> tertinggi)	Model yang akan dilakukan proses <i>tuning</i> .
Param_distributions	Ruang pencarian parameter	Kumpulan nilai <i>hyperparameter</i> yang akan dipilih secara acak selama proses <i>tuning</i> .

Parameter	Nilai	Keterangan
n_iter	10 (RF, XGB, GB) 5 (Stacking)	Jumlah kombinasi parameter yang diuji secara acak.
scoring	Recall	Metrik evaluasi yang digunakan untuk menentukan kombinasi parameter terbaik.
cv	Stratified K-Fold (3-Fold)	Jumlah fold yang digunakan dalam proses cross validation dengan menjaga proporsi kelas tetap seimbang.
random_state	42	Nilai seed untuk memastikan hasil eksperimen dapat direproduksi.
n_jobs	-1	Menggunakan seluruh core prosesor yang tersedia untuk mempercepat proses tuning.
verbose	1	Menampilkan informasi proses tuning selama eksekusi.

Selain parameter *RandomizedSearchCV*, setiap model memiliki ruang pencarian *hyperparameter* yang berbeda sesuai karakteristik algoritmanya. Ruang pencarian *hyperparameter* dirancang berdasarkan parameter yang paling berpengaruh terhadap performa masing-masing algoritma. Rentang nilai yang digunakan dipilih untuk memberikan keseimbangan antara luasnya eksplorasi parameter dan efisiensi komputasi serta menghasilkan konfigurasi yang lebih baik dibandingkan parameter default. Ruang pencarian parameter tiap model ditunjukkan pada **Tabel 3.10-3.14**.

Tabel 3.10 Ruang pencarian parameter Random Forest

Parameter	Nilai	Keterangan
n_estimators	100-499	Jumlah pohon keputusan yang dibangun dalam Random Forest.
max_depth	None, 10, 20, 30, 50	Kedalaman maksimum setiap pohon keputusan.
min_samples_split	2 – 19	Jumlah minimum sampel yang diperlukan untuk melakukan pemisahan node.
min_samples_leaf	1 – 9	Jumlah minimum sampel yang harus terdapat pada node daun.

max_features	sqrt, log2, None	Jumlah fitur yang dipertimbangkan pada setiap proses pemisahan node.
class_weight	balanced, balanced_subsample, None	Bobot kelas yang digunakan untuk menangani ketidakseimbangan data.

Rentang parameter pada *Random Forest* dipilih untuk memberikan keseimbangan antara kemampuan model dalam mempelajari pola transaksi fraud yang kompleks dan risiko terjadinya *overfitting*. Parameter *n_estimators* dipilih pada rentang 100–499 agar jumlah pohon cukup banyak untuk menghasilkan prediksi yang stabil tanpa meningkatkan waktu komputasi secara berlebihan. Parameter *max_depth* menggunakan nilai *None*, 10, 20, 30, dan 50 untuk mengevaluasi pengaruh kedalaman pohon, mulai dari pohon yang tumbuh bebas hingga pohon dengan kompleksitas yang dibatasi. Nilai *min_samples_split* pada rentang 2–19 dipilih untuk mengontrol jumlah minimum sampel sebelum suatu node dipecah sehingga proses pembentukan pohon tidak terlalu sensitif terhadap variasi data. Selanjutnya, *min_samples_leaf* dengan rentang 1–9 digunakan untuk menghindari terbentuknya leaf yang hanya berisi sedikit sampel sehingga model memiliki kemampuan generalisasi yang lebih baik. Parameter *max_features* dieksplorasi menggunakan *sqrt*, *log2*, dan *None* untuk memperoleh variasi jumlah fitur yang dipertimbangkan pada setiap proses pemisahan node sehingga dapat meningkatkan keberagaman antar pohon. Sementara itu, *class_weight* menggunakan pilihan *balanced*, *balanced_subsample*, dan *None* untuk mengevaluasi pengaruh pemberian bobot yang lebih besar pada kelas fraud mengingat dataset yang digunakan memiliki distribusi kelas yang sangat tidak seimbang. Pemilihan rentang tersebut diharapkan mampu menghasilkan konfigurasi Random Forest yang lebih adaptif terhadap karakteristik data dibandingkan penggunaan parameter *default*.

Tabel 3.11 Ruang pencarian parameter XGBoost

Parameter	Nilai	Keterangan
n_estimators	50-399	Jumlah pohon boosting yang dibangun.
learning_rate	0,01 – 0,30	Laju pembelajaran yang mengontrol kontribusi setiap pohon baru.
max_depth	3 – 9	Kedalaman maksimum setiap pohon keputusan.
min_child_weight	1 – 9	Jumlah minimum bobot sampel yang diperlukan pada node anak.
subsample	0,6 – 1,0	Proporsi data pelatihan yang digunakan pada setiap iterasi boosting.
colsample_bytree	0,5 – 1,0	Proporsi fitur yang digunakan untuk membangun setiap pohon.

gamma	0 – 5	Nilai minimum pengurangan loss yang diperlukan untuk melakukan pemisahan node.
scale_pos_weight	1, 5, 10, 50, 100	Bobot tambahan untuk kelas minoritas guna menangani data tidak seimbang.

Ruang pencarian parameter XGBoost dirancang dengan mempertimbangkan mekanisme *boosting* yang sensitif terhadap konfigurasi *hyperparameter* serta karakteristik dataset yang sangat *imbalanced*. Parameter *n_estimators* dipilih pada rentang 50–399 agar jumlah *boosting iteration* cukup untuk mempelajari pola *fraud* tanpa menyebabkan waktu pelatihan menjadi terlalu tinggi. Nilai *learning_rate* sebesar 0,01–0,30 digunakan untuk mengevaluasi laju pembelajaran dari proses *boosting*, mulai dari pembelajaran yang konservatif hingga lebih agresif. Parameter *max_depth* pada rentang 3–9 dipilih agar kompleksitas pohon tetap terkendali sehingga mampu menangkap pola *fraud* tanpa meningkatkan risiko *overfitting*. Parameter *min_child_weight* dengan rentang 1–9 digunakan untuk mengatur jumlah minimum bobot sampel pada node anak sehingga pembentukan pohon menjadi lebih stabil terhadap *noise*. Parameter *subsample* sebesar 0,6–1,0 dan *colsample_bytree* sebesar 0,5–1,0 dipilih untuk mengurangi korelasi antar pohon melalui pengambilan sebagian data maupun fitur pada setiap iterasi *boosting*. Selain itu, *gamma* pada rentang 0–5 digunakan untuk mengontrol tingkat selektivitas pemisahan node sehingga model tidak mudah membentuk percabangan yang kurang informatif. Terakhir, *scale_pos_weight* menggunakan beberapa nilai diskrit mulai dari 1 hingga 100 sebagai upaya mengevaluasi pengaruh pembobotan kelas minoritas pada proses training, mengingat proporsi transaksi *fraud* dalam dataset jauh lebih kecil dibandingkan transaksi normal. Kombinasi ruang pencarian tersebut diharapkan mampu menghasilkan konfigurasi XGBoost yang optimal dalam meningkatkan sensitivitas deteksi fraud.

Tabel 3.12 Ruang pencarian parameter GradientBoosting

Parameter	Nilai	Keterangan
n_estimators	50-399	Jumlah pohon boosting yang dibangun secara bertahap.
learning_rate	0,01 – 0,31	Laju pembelajaran yang mengontrol kontribusi setiap estimator.
max_depth	2 – 7	Kedalaman maksimum setiap pohon keputusan.
min_samples_split	2 – 19	Jumlah minimum sampel yang diperlukan untuk melakukan pemisahan node.
min_samples_leaf	1 – 19	Jumlah minimum sampel yang harus terdapat pada node daun.
subsample	0,6 – 1,0	Proporsi data pelatihan yang digunakan pada setiap iterasi boosting.

Parameter	Nilai	Keterangan
max_features	sqrt, log2, None	Jumlah fitur yang dipertimbangkan saat membangun pohon.

Ruang pencarian parameter pada *Gradient Boosting* dipilih untuk mengendalikan kompleksitas model yang dibangun secara bertahap pada setiap iterasi *boosting*. Parameter *n_estimators* menggunakan rentang 50–399 untuk mengevaluasi jumlah *estimator* yang cukup dalam memperbaiki kesalahan model sebelumnya tanpa menimbulkan beban komputasi yang terlalu besar. Nilai *learning_rate* sebesar 0,01–0,31 dipilih agar model dapat dievaluasi pada kondisi pembelajaran yang lambat maupun lebih cepat sehingga diperoleh keseimbangan antara kecepatan konvergensi dan kemampuan generalisasi. Parameter *max_depth* pada rentang 2–7 digunakan untuk membatasi kompleksitas setiap pohon keputusan karena *Gradient Boosting* umumnya menggunakan pohon yang relatif dangkal sebagai *weak learner*. Selanjutnya, *min_samples_split* dan *min_samples_leaf* digunakan untuk mengontrol ukuran minimum node sebelum dilakukan pemisahan maupun pembentukan leaf sehingga risiko overfitting akibat terbentuknya cabang yang terlalu spesifik dapat dikurangi. Parameter *subsample* dipilih pada rentang kurang dari atau sama dengan satu untuk mengevaluasi pengaruh *stochastic gradient boosting* dalam meningkatkan kemampuan generalisasi model. Rentang parameter tersebut dipilih agar proses optimasi mampu mengeksplorasi konfigurasi yang cukup beragam namun tetap sesuai dengan karakteristik algoritma *Gradient Boosting* serta efisien secara komputasi.

Tabel 3.13 Ruang pencarian parameter Soft Voting

Base Learner	Parameter	Nilai	Keterangan
Logistic Regression	C	0.01, 0.1, 1, 10, 100	Jumlah pohon boosting yang dibangun secara bertahap.
Logistic Regression	solver	lbfgs, liblinear	Laju pembelajaran yang mengontrol kontribusi setiap estimator.
Decision Tree	max_depth	None, 5, 10, 20	Kedalaman maksimum setiap pohon keputusan.
Decision Tree	min_samples_split	2, 5, 10	Jumlah minimum sampel yang diperlukan untuk melakukan pemisahan node.
Decision Tree	min_samples_leaf	1, 5, 10	Jumlah minimum sampel yang harus terdapat pada node daun.
Decision Tree	class_weight	balanced, None	Bobot kelas untuk menangani ketidakseimbangan data.
SVM	class_weight	balanced	Mengatur bobot kelas minoritas agar lebih diperhatikan selama pelatihan.

Pada model *Soft Voting*, proses *hyperparameter tuning* difokuskan pada parameter yang dimiliki oleh masing-masing *base learner* karena mekanisme *Soft Voting* sendiri hanya menggabungkan probabilitas keluaran dari setiap *classifier*. Oleh karena itu, ruang pencarian parameter mencakup parameter *Logistic Regression*, *Decision Tree*, dan *Support Vector Machine* yang dipilih berdasarkan tingkat pengaruhnya terhadap performa klasifikasi. Pada *Logistic Regression*, parameter regularisasi dieksplorasi untuk memperoleh keseimbangan antara kompleksitas model dan kemampuan generalisasi. Pada *Decision Tree*, parameter seperti kedalaman pohon dan jumlah minimum sampel dipilih untuk mengendalikan kompleksitas struktur pohon sehingga mengurangi risiko overfitting. Sementara itu, pada *Support Vector Machine*, parameter seperti *class_weight* dievaluasi untuk memperoleh bobot kelas yang lebih seimbang untuk kelas minoritas. Pendekatan ini memungkinkan setiap *base learner* memberikan probabilitas prediksi terbaik sebelum digabungkan melalui mekanisme *soft voting* sehingga diharapkan meningkatkan kemampuan model dalam membedakan transaksi *fraud* dan *non-fraud* secara lebih akurat.

Tabel 3.14 Ruang pencarian parameter *Stacking*

Base Learner	Parameter	Ruang Pencarian	Keterangan
Random Forest	Menggunakan ruang pencarian Random Forest	Sesuai Tabel Random Forest (Tabel 3.10)	Optimasi base learner pertama.
XGBoost	Menggunakan ruang pencarian XGBoost	Sesuai Tabel XGBoost (Tabel 3.11)	Optimasi base learner kedua.
Gradient Boosting	Menggunakan ruang pencarian Gradient Boosting	Sesuai Tabel Gradient Boosting (Tabel 3.12)	Optimasi base learner ketiga.
Meta Learner (Logistic Regression)	max_iter	1000	Jumlah maksimum iterasi pelatihan meta learner.

Ruang pencarian parameter pada *Stacking* dirancang dengan melakukan optimasi terhadap seluruh *base learner* yang terdiri atas *Random Forest*, *XGBoost*, dan *Gradient Boosting*, sedangkan *meta learner Logistic Regression* menggunakan konfigurasi yang sederhana. Pemilihan strategi tersebut dilakukan karena performa model *Stacking* sangat dipengaruhi oleh kualitas prediksi yang dihasilkan masing-masing *base learner* sebelum dipelajari kembali oleh *meta learner*. Oleh sebab itu, ruang pencarian parameter untuk *Random Forest*, *XGBoost*, dan *Gradient Boosting* disusun serupa dengan model individual agar setiap algoritma memiliki kesempatan memperoleh konfigurasi terbaik sesuai karakteristiknya. Sementara itu, penggunaan *Logistic Regression* sebagai *meta learner* dipertahankan dengan kompleksitas yang rendah karena perannya bukan membangun pola baru dari data asli, melainkan mempelajari hubungan antar hasil prediksi *base learner*. Pendekatan tersebut diharapkan menghasilkan model *Stacking* yang mampu memanfaatkan keunggulan masing-masing algoritma sekaligus meningkatkan kemampuan generalisasi dalam mendeteksi transaksi *fraud* pada dataset yang sangat tidak seimbang.

Proses evaluasi pada setiap iterasi tuning menggunakan *Stratified K-Fold Cross Validation* untuk menjaga proporsi kelas *fraud* dan *non-fraud* tetap konsisten pada setiap fold.

Metrik yang digunakan sebagai fungsi optimasi adalah *recall* karena penelitian ini berfokus pada kemampuan model dalam mendeteksi transaksi *fraud* dan meminimalkan jumlah *false negative*. Hasil dari tahap tuning berupa kombinasi parameter terbaik dari *range* parameter yang sudah ditentukan untuk masing-masing model yang dipilih berdasarkan nilai *recall* tertinggi selama proses *cross validation*. Model yang telah dioptimasi kemudian diuji kembali menggunakan data test yang sama untuk memperoleh nilai performa akhir yang akan digunakan pada tahap analisis penelitian untuk mengetahui perbandingannya dengan hasil model yang tidak melalui tahap *hyperparameter tuning*. Implementasi *hyperparameter tuning* ditunjukkan pada **Kode Semu 3.8**.

Input: Lima kombinasi model dan teknik resampling terbaik, Dataset hasil resampling dari masing-masing kombinasi, Ruang pencarian hyperparameter (parameter grid), Data testing (X_test_scaled), Label aktual data testing (y_test)	
1.	BEGIN
2.	SELECT lima kombinasi model dan teknik resampling terbaik berdasarkan hasil evaluasi
3.	FOR setiap kombinasi yang terpilih
4.	DEFINE ruang pencarian hyperparameter
5.	INITIALIZE RandomizedSearchCV
6.	SET parameter RandomizedSearchCV
7.	LOAD model yang akan dioptimasi
8.	LOAD dataset hasil resampling
9.	SEARCH kombinasi hyperparameter terbaik
10.	SELECT hyperparameter terbaik berdasarkan metrik evaluasi
11.	BUILD model menggunakan hyperparameter terbaik
12.	TRAIN model menggunakan seluruh data training hasil resampling
13.	APPLY model pada data testing
14.	GENERATE hasil prediksi (y_pred)
15.	COMPARE y_pred dengan y_test
16.	GENERATE confusion matrix
17.	CALCULATE Accuracy, Precision, Recall, dan F1-Score
18.	STORE hyperparameter terbaik dan hasil evaluasi
19.	END FOR
20.	END
Output: Hyperparameter terbaik untuk setiap model, Hasil prediksi pada data testing	

Kode Semu 3.8 Hyperparameter tuning

3.1.10 Analisis and Evaluasi

Tahap akhir dari metodologi yang digunakan dalam penelitian adalah analisis dan evaluasi kinerja model untuk mengukur efektivitas teknik *resampling* dan algoritma *ensemble* yang diusulkan. Analisis juga dilakukan terhadap hasil evaluasi sebelum dan sesudah hyperparameter tuning untuk mengetahui pengaruh optimasi parameter terhadap kemampuan deteksi fraud. Dasar dari seluruh perhitungan evaluasi adalah *Confusion Matrix*, yang memetakan hasil prediksi model ke dalam empat kuadran: *True Positive* (TP), *True Negative* (TN), *False Positive* (FP), dan *False Negative* (FN). Mengingat sifat dataset yang sangat *imbalanced* dengan rasio *fraud* hanya 0.172%, penggunaan metrik Akurasi (*Accuracy*) saja tidak dapat dijadikan tolok ukur utama karena berpotensi memberikan nilai tinggi yang bias terhadap kelas mayoritas. Oleh karena itu, evaluasi difokuskan pada metrik sensitivitas atau *Recall*, yang mengukur kemampuan model dalam mendeteksi seluruh transaksi *fraud* yang

sebenarnya terjadi. Dalam domain deteksi kejahatan finansial, meminimalkan *False Negative* (kegagalan mendeteksi *fraud*) adalah prioritas utama dibandingkan menghindari *False Positive*. Selain itu, metrik *Precision*, *pr-auc*, dan *F1-Score* juga dianalisis untuk melihat keseimbangan antara ketepatan prediksi dan kelengkapan deteksi, serta untuk memastikan bahwa peningkatan sensitivitas tidak menyebabkan lonjakan kesalahan deteksi palsu yang berlebihan. Metrik-metrik yang disebutkan diatas akan didapatkan melalui *classification report* yang menampilkan beberapa metrik performa dalam sebuah tabel.

Hasil analisis kemudian digunakan untuk mengidentifikasi kombinasi teknik *resampling* dan model *ensemble learning* yang memberikan performa paling optimal dalam mendeteksi transaksi *fraud*. Temuan tersebut selanjutnya digunakan untuk menjawab rumusan masalah penelitian serta menyusun rekomendasi mengenai strategi penanganan imbalanced data yang paling efektif pada kasus deteksi penipuan transaksi kartu kredit.

3.1.11 Skenario Pengujian

Pengujian pada penelitian ini dilakukan untuk mengevaluasi pengaruh penerapan teknik *resampling* dan *hyperparameter tuning* terhadap performa model *ensemble learning* dalam mendeteksi transaksi *fraud* pada dataset yang memiliki ketidakseimbangan kelas ekstrem. Skenario pengujian dibagi menjadi tiga tahap utama, yaitu pengujian *baseline*, pengujian dengan *teknik resampling*, dan pengujian dengan *hyperparameter tuning*.

1. Pengujian Baseline (Data tanpa resampling)

Skenario pertama bertujuan untuk memperoleh performa dasar (baseline) dari setiap model *ensemble learning* tanpa menerapkan teknik penyeimbangan kelas. Pada tahap ini, model dilatih menggunakan data hasil *preprocessing* yang masih mempertahankan distribusi kelas asli. Model yang digunakan dalam pengujian baseline meliputi:

- Random Forest
- XGBoost
- Gradient Boosting
- Soft Voting
 - Model Penyusun:
 - Logistic Regression
 - Decision Tree
 - SVM (Support Vector Machine)
- Stacking (Stacked Generalization)
 - Base Learners:
 - Random Forest
 - XGBoost
 - Gradient Boost
 - Meta Learner:
 - Logistic Regression

Hasil pengujian pada skenario ini digunakan sebagai acuan untuk mengevaluasi peningkatan performa yang diperoleh setelah penerapan teknik *resampling*.

2. Pengujian dengan Teknik Resampling

Skenario kedua bertujuan untuk menganalisis pengaruh berbagai teknik resampling terhadap performa model ensemble learning dalam menangani permasalahan ketidakseimbangan kelas. Teknik resampling yang digunakan pada penelitian ini terdiri atas:

- Random Oversampling
- SMOTE (Synthetic Minority Over-sampling Technique)
- ADASYN (Adaptive Synthetic Sampling)
- Random Undersampling
- Tomek-Links
- Edited Nearest Neighbour (ENN)
- SMOTE-Tomek
- SMOTE-ENN

Setiap teknik *resampling* diterapkan pada data *training*, kemudian digunakan untuk melatih seluruh model *ensemble learning* yang telah ditentukan. Selanjutnya model diuji menggunakan data *test* yang sama sehingga hasil yang diperoleh dapat dibandingkan secara objektif. Pengujian ini menghasilkan 45 (40 dengan *resampling*, 5 *baseline*) kombinasi eksperimen yang berasal dari 8 teknik *resampling* dan 5 model *ensemble learning*.

3. Pengujian dengan Hyperparameter Tuning

Skenario ketiga bertujuan untuk mengetahui pengaruh optimasi *hyperparameter* terhadap performa model. Tahap ini dilakukan terhadap lima kombinasi model dan teknik resampling terbaik yang diperoleh pada skenario sebelumnya berdasarkan nilai F1-Score tertinggi.

Proses optimasi dilakukan menggunakan metode *Randomized Search Cross Validation* untuk mencari kombinasi parameter yang paling optimal pada masing-masing model. Parameter yang diuji disesuaikan dengan karakteristik setiap algoritma *ensemble learning*. Setelah proses tuning selesai, model hasil optimasi dievaluasi menggunakan data *test* yang sama dan dibandingkan dengan performa model sebelum tuning. Perbandingan ini dilakukan untuk mengetahui sejauh mana hyperparameter tuning mampu meningkatkan kemampuan model dalam mendeteksi transaksi *fraud*.

3.2 Bahan dan peralatan yang digunakan

Pada penelitian ini, bahan dan peralatan utama yang digunakan adalah perangkat lunak dan perangkat keras. Salah satu faktor kelancaran proses penelitian adalah tingkat kualitas perangkat lunak dan keras yang digunakan, sehingga penting untuk memiliki peralatan yang memadai. **Tabel 3.15** dan **3.16** menunjukkan spesifikasi perangkat lunak dan keras yang digunakan dalam penelitian ini.

Tabel 3.15 Spesifikasi perangkat lunak

Komponen	Spesifikasi
Sistem Operasi	Windows 11 Home Single Language 64-bit (versi 24H2 build 26100.7462)
Integrated Development Environment (IDE)	Visual Studio Code versi 1.107.1, Google Collab, dan Kaggle
Bahasa Pemrograman	Python 3.12.6
Pustaka Pendukung	Pandas, Numpy, Matplotlib, Seaborn, Sklearn

Tabel 3.16 Spesifikasi perangkat keras

Komponen	Spesifikasi
Perangkat	ASUS TUF FX505DT
RAM	16 GB
CPU	AMD Ryzen 5 3550H with Radeon Vega Mobile Gfx (2.10 GHz)
Penyimpanan	1 TB

3.3 Lingkungan Pengembangan Penelitian

Penelitian ini dilaksanakan menggunakan berbagai perangkat lunak dan pustaka (*library*) Python yang mendukung proses pengolahan data, penanganan ketidakseimbangan kelas, pelatihan model klasifikasi, evaluasi kinerja, serta visualisasi hasil. Penggunaan pustaka yang terdokumentasi dengan jelas bertujuan untuk menjamin reproduksibilitas penelitian sehingga proses eksperimen dapat dijalankan kembali pada lingkungan yang serupa. **Tabel 3.17** menyajikan daftar pustaka yang digunakan beserta versi yang digunakan selama penelitian.

Tabel 3.17 Pustaka Pengembangan Penelitian

Pustaka	Fungsi dalam Penelitian	Versi
NumPy	Operasi numerik dan manipulasi array	2.4.6
Pandas	Pengolahan dan analisis data	2.3.3
Scikit-learn	Preprocessing, evaluasi, model machine learning, dan hyperparameter tuning	1.6.1
Imbalanced-learn	Implementasi teknik resampling	0.14.1
XGBoost	Implementasi model XGBoost	3.2.0
Matplotlib	Visualisasi data	3.10.0
Seaborn	Visualisasi statistik	0.13.2
Plotly	Visualisasi interaktif	5.24.1
SciPy	Operasi statistik dan komputasi ilmiah	1.16.3
Warnings	Pengelolaan pesan peringatan	Library bawaan Python

BAB 4 HASIL DAN PEMBAHASAN

4.1 Hasil penelitian

Subbab ini berisikan hasil penelitian yang diperoleh dari serangkaian eksperimen yang telah dilakukan sesuai dengan metodologi penelitian. Hasil yang disajikan meliputi pengujian model *baseline* yang menggunakan data tanpa di *resampling*, penerapan berbagai teknik *resampling*, evaluasi performa model *ensemble learning*, serta hasil optimasi model melalui *hyperparameter tuning*. Penyajian hasil dilakukan untuk memberikan gambaran mengenai performa masing-masing model dalam mendeteksi transaksi *fraud* berdasarkan metrik evaluasi yang telah ditentukan.

4.1.1 Hasil Pengujian Model *Baseline* (Tanpa Penerapan Teknik *Resampling*)

Skenario awal penelitian dilakukan dengan menguji performa seluruh model klasifikasi menggunakan dataset asli tanpa penerapan teknik *resampling*. Pengujian ini bertujuan untuk mengetahui kemampuan dasar masing-masing algoritma dalam mengidentifikasi transaksi *fraud* pada kondisi data yang masih mengalami ketidakseimbangan kelas (*imbalanced dataset*). Berdasarkan hasil analisis distribusi data, jumlah data kelas *non-fraud* mencapai 227.451 transaksi, sedangkan jumlah data *fraud* hanya sebanyak 394 transaksi. Perbedaan jumlah data yang sangat besar tersebut berpotensi menyebabkan model cenderung mempelajari pola dari kelas mayoritas dan mengabaikan karakteristik kelas minoritas. **Tabel 4.1** menunjukkan hasil evaluasi lima model klasifikasi *ensemble learning* yang digunakan pada penelitian ini, yaitu *Random Forest*, *XGBoost*, *Stacking*, *Soft Voting*, dan *Gradient Boosting*. Evaluasi dilakukan menggunakan metrik *Accuracy*, *Precision*, *Recall*, *pr-auc*, dan *F1-Score*.

Tabel 4.1 Hasil pengujian baseline

Model	Recall	Precision	F1-score	PR-AUC	Accuracy
Random Forest	0,8163	0,9412	0,8743	0,8788	0,9996
XGBoost	0,7959	0,8667	0,8298	0,7973	0,9994
Gradient Boosting	0,1837	0,5294	0,2727	0,2123	0,9983
Soft Voting	0,7449	0,9359	0,8295	0,8512	0,9995
Stacking	0,7755	0,9383	0,8492	0,8747	0,9995

Berdasarkan hasil pengujian pada **Tabel 4.1**, seluruh model menghasilkan nilai *accuracy* yang sangat tinggi, yaitu di atas 99%. Sekilas hasil tersebut menunjukkan performa yang sangat baik. Namun pada kasus deteksi *fraud*, *accuracy* bukan merupakan indikator utama karena distribusi kelas yang sangat tidak seimbang dapat menyebabkan nilai *accuracy* menjadi bias. Model dapat memperoleh *accuracy* tinggi hanya dengan memprediksi sebagian besar data sebagai kelas *non-fraud*. Oleh karena itu, penelitian ini lebih memfokuskan analisis pada nilai *recall*. *Recall* menunjukkan kemampuan model dalam mendeteksi transaksi *fraud* yang sebenarnya. Semakin tinggi nilai *recall*, semakin sedikit transaksi *fraud* yang gagal terdeteksi oleh sistem.

Berdasarkan nilai *recall*, model *Random Forest* memperoleh performa terbaik dengan nilai *recall* sebesar 0,8163. Hasil tersebut menunjukkan bahwa *Random Forest* mampu mengidentifikasi sekitar 81,63% transaksi fraud pada data *test*. Selain memiliki *recall* tertinggi, model ini juga menghasilkan *precision* sebesar 0,9412, *pr-auc* sebesar 0,8788, dan *F1-Score* sebesar 0,8743 yang merupakan nilai *F1-Score* tertinggi di antara seluruh model *baseline*. Nilai *precision* yang tinggi juga menunjukkan bahwa sebagian besar transaksi yang diprediksi sebagai fraud memang merupakan transaksi fraud yang sebenarnya.

Model *XGBoost* menempati posisi kedua dengan nilai *recall* sebesar 0,7959 dan *F1-Score* sebesar 0,8298. Meskipun sedikit lebih rendah dibandingkan *Random Forest*, performa *XGBoost* masih baik dalam mendeteksi kelas *fraud*. Hal ini menunjukkan bahwa metode *boosting* masih mampu mempelajari pola fraud meskipun jumlah data minoritas sangat terbatas.

Model *Stacking* memperoleh nilai *recall* sebesar 0,7755 dengan *precision* sebesar 0,9383, *pr-auc* sebesar 0,8747 dan *F1-Score* sebesar 0,8492. Hasil ini menunjukkan bahwa kombinasi beberapa algoritma dasar dalam pendekatan *ensemble* mampu menghasilkan performa yang kompetitif jika dibandingkan dengan model lain. Namun, *recall* yang diperoleh masih berada di bawah *Random Forest* dan *XGBoost* sehingga jumlah transaksi fraud yang belum berhasil diidentifikasi masih lebih banyak dibandingkan dua model tersebut.

Selanjutnya, model *Soft Voting* menghasilkan *recall* sebesar 0,7449 dengan *precision* sebesar 0,9359, *pr-auc* sebesar 0,8512 dan *F1-Score* sebesar 0,8295. Nilai *precision* yang tinggi menunjukkan bahwa prediksi fraud yang dihasilkan relatif akurat, tetapi kemampuan model dalam menemukan seluruh transaksi *fraud* masih lebih rendah jika dibandingkan dengan *Random Forest*, *XGBoost*, dan *Stacking*.

Performa terendah ditunjukkan oleh model *Gradient Boosting* dengan nilai *recall* hanya sebesar 0,1837. Hal tersebut menandakan bahwa model ini hanya mampu mendeteksi sekitar 18,37% transaksi fraud yang terdapat pada data *test*. Nilai *F1-Score* yang diperoleh juga sangat rendah, yaitu sebesar 0,2727. Hasil tersebut mengindikasikan bahwa *Gradient Boosting* mengalami kesulitan dalam mempelajari pola kelas minoritas pada dataset yang sangat tidak seimbang. Model cenderung memprioritaskan prediksi pada kelas mayoritas sehingga sebagian besar transaksi fraud tidak berhasil terdeteksi.

Hasil pengujian model *baseline* menunjukkan bahwa ketidakseimbangan data memiliki pengaruh yang signifikan terhadap kemampuan model dalam mendeteksi transaksi *fraud*. Meskipun beberapa model seperti *Random Forest*, *XGBoost*, dan *Stacking* masih mampu menghasilkan nilai *recall* di atas 0,75, performa yang diperoleh masih berpotensi ditingkatkan melalui penerapan teknik *resampling*. Oleh karena itu, tahap penelitian selanjutnya difokuskan pada penerapan berbagai metode *resampling* untuk meningkatkan representasi kelas *fraud* sehingga model dapat mempelajari pola transaksi fraud secara lebih optimal.

4.1.2 Hasil Penerapan Teknik Resampling terhadap Distribusi Data

Tahap *resampling* dilakukan untuk mengatasi permasalahan ketidakseimbangan kelas (*class imbalance*) yang terdapat pada dataset *Credit Card Fraud Detection*. Berdasarkan hasil analisis distribusi data sebelum *resampling*, jumlah transaksi non-fraud pada data *training* mencapai 227.451 data, sedangkan jumlah transaksi *fraud* hanya sebanyak 394 data. Perbedaan distribusi yang sangat besar tersebut menyebabkan proporsi kelas fraud hanya sekitar 0,17% dari keseluruhan data *training*. Kondisi ini dapat menimbulkan bias pada model klasifikasi

karena algoritma cenderung lebih banyak mempelajari pola dari kelas mayoritas dibandingkan kelas minoritas.

Kondisi tersebut menjadi tantangan dalam kasus deteksi fraud karena model dapat memperoleh nilai akurasi yang sangat tinggi meskipun gagal mendeteksi sebagian transaksi *fraud*. Oleh sebab itu, teknik resampling diterapkan untuk memperbaiki distribusi data sehingga representasi kelas fraud menjadi lebih memadai selama proses pelatihan.

Dalam penelitian ini, seluruh teknik *resampling* menggunakan nilai *sampling_strategy* sebesar 0,3 atau 0,5 (untuk *hybrid sampling*). Nilai tersebut dipilih berdasarkan hasil pengujian yang menunjukkan performa yang lebih baik dibandingkan penggunaan nilai 0,5 pada beberapa teknik, khususnya pada metrik *recall* dan *F1-score*. Selain itu, penggunaan rasio 0,3 dan 0,5 dinilai cukup untuk meningkatkan representasi kelas fraud tanpa mengubah distribusi data secara terlalu ekstrem. **Tabel 4.2** menunjukkan perubahan distribusi data setelah penerapan berbagai teknik resampling yang digunakan dalam penelitian ini.

Tabel 4.2 Distribusi data sebelum dan sesudah resampling

Teknik Resampling	Kelas 0 (Non-Fraud)	Kelas 1 (Fraud)	Total Data
Data Orisinal (Sebelum Resampling)	227.451	394	227.845
Random Oversampling	227.451	68.235	295.686
SMOTE	227.451	68.235	295.686
ADASYN	227.451	68.283	295.734
Random Undersampling	1.313	394	1.707
Tomek-Links	227.434	394	227.828
Edited Nearest Neighbor (ENN)	227.430	394	227.824
SMOTE-ENN	227.451	227.451	454.902
SMOTE-Tomek Links	227.451	227.451	454.902

Berdasarkan **Tabel 4.2**, perubahan distribusi data yang paling terlihat terjadi pada kelompok *oversampling*. *Random Oversampling*, SMOTE, dan ADASYN meningkatkan jumlah data *fraud* dari 394 menjadi lebih dari 68 ribu data, sementara jumlah data *non-fraud* tetap dipertahankan. Dengan bertambahnya jumlah sampel fraud, model memiliki lebih banyak contoh untuk mempelajari karakteristik transaksi yang mencurigakan, namun tidak terlalu banyak sehingga data tidak terlalu bias ke kelas *fraud*.

Pada teknik *Random Undersampling*, jumlah data *non-fraud* berkurang secara signifikan menjadi 1.313 data. Meskipun pendekatan ini mampu mengurangi dominasi kelas mayoritas, pengurangan data yang terlalu besar juga berisiko menghilangkan sebagian informasi penting

yang terdapat pada transaksi normal. Berbeda dengan pendekatan tersebut, *Tomek Links* dan ENN hanya menghasilkan perubahan jumlah data yang relatif kecil. Kedua metode ini lebih berfokus pada pembersihan data yang dianggap ambigu atau berpotensi menjadi noise. Hal ini terlihat dari jumlah data yang dihapus hanya sebagian kecil dari keseluruhan dataset. Sementara itu, SMOTE-ENN dan SMOTE-Tomek menghasilkan distribusi yang seimbang antara kelas *non-fraud* dan *fraud*. Jumlah data pada kedua kelas menjadi sama, yaitu 227.451 data. Kombinasi ini tidak hanya meningkatkan jumlah data *fraud*, tetapi juga melakukan proses pembersihan data sehingga kualitas data latih diharapkan menjadi lebih baik dari sebelumnya.

Setiap teknik menghasilkan karakteristik distribusi yang berbeda. *Oversampling* meningkatkan representasi kelas *fraud*, *undersampling* mengurangi dominasi kelas mayoritas, sedangkan metode *hybrid sampling* menghasilkan distribusi yang lebih seimbang sekaligus melakukan pembersihan data. Perbedaan distribusi tersebut diperkirakan akan memengaruhi kemampuan model dalam mempelajari pola transaksi *fraud*.

4.1.3 Hasil Pengujian Model dengan Data Resampling

4.1.3.1 Hasil Pengujian Kombinasi Model dan Teknik Resampling

Tahap ini bertujuan untuk menampilkan hasil evaluasi berbagai teknik *resampling* terhadap performa model *Ensemble Learning* dalam mendeteksi transaksi *fraud*. Setiap teknik *resampling* diterapkan pada data *training*, kemudian digunakan untuk melatih lima model, yaitu *Random Forest*, *XGBoost*, *Stacking*, *Soft Voting*, dan *Gradient Boosting*. Selanjutnya, model diuji menggunakan data *testing* yang tetap berada pada kondisi asli tanpa *resampling* sehingga hasil evaluasi dapat merepresentasikan kondisi implementasi nyata. **Tabel 4.3** menyajikan rekapitulasi hasil pengujian seluruh kombinasi model dan teknik resampling serta baseline sebagai pembandingan.

Tabel 4.3 Hasil pengujian seluruh kombinasi

Teknik Resampling	Model	Recall	Precision	PR-AUC	F1-score
Baseline	Random Forest	0,8163	0,9412	0,8788	0,8743
	XGBoost	0,7959	0,8667	0,7973	0,8298
	Gradient Boost	0,1837	0,5294	0,2123	0,2727
	Soft Voting	0,7449	0,9359	0,8512	0,8295
	Stacking	0,7755	0,9383	0,8747	0,8492
Random Oversampling	Random Forest	0,8061	0,9518	0,8661	0,8729
	XGBoost	0,8367	0,9011	0,8754	0,8677
	Gradient Boost	0,8878	0,3551	0,8183	0,5073
	Soft Voting	0,8673	0,4857	0,8346	0,6227
	Stacking	0,8265	0,8901	0,8679	0,8571
SMOTE	Random Forest	0,8469	0,8830	0,8732	0,8646
	XGBoost	0,8673	0,7870	0,8768	0,8252
	Gradient Boost	0,8878	0,2852	0,7701	0,4318

Teknik Resampling	Model	Recall	Precision	PR-AUC	F1-score
	Soft Voting	0,8776	0,4574	0,8340	0,6014
	Stacking	0,8571	0,8660	0,8748	0,8615
ADASYN	Random Forest	0,7959	0,8764	0,8680	0,8342
	XGBoost	0,8163	0,7767	0,8527	0,7960
	Gradient Boost	0,8980	0,1064	0,7225	0,1902
	Soft Voting	0,8163	0,5263	0,7673	0,64
	Stacking	0,8980	0,352	0,8681	0,5057
Random Undersampling	Random Forest	0,8878	0,1490	0,7681	0,2551
	XGBoost	0,8979	0,0916	0,7488	0,1664
	Gradient Boost	0,8878	0,0601	0,4887	0,1127
	Soft Voting	0,9184	0,1128	0,7171	0,2009
	Stacking	0,8980	0,0985	0,7615	0,1776
Tomek Links	Random Forest	0,8163	0,9412	0,8723	0,8743
	XGBoost	0,7959	0,8864	0,8044	0,8387
	Gradient Boost	0,1633	0,6154	0,2113	0,2581
	Soft Voting	0,7653	0,9259	0,8548	0,8380
	Stacking	0,8061	0,9186	0,8726	0,8587
ENN	Random Forest	0,8265	0,9101	0,8683	0,8663
	XGBoost	0,8367	0,9111	0,8517	0,8723
	Gradient Boost	0,6224	0,8356	0,5913	0,7135
	Soft Voting	0,7857	0,8953	0,8483	0,8370
	Stacking	0,8061	0,9405	0,8640	0,8681
SMOTE-Tomek Links	Random Forest	0,8163	0,8791	0,8763	0,8466
	XGBoost	0,8776	0,7544	0,8725	0,8113
	Gradient Boost	0,9082	0,1193	0,7455	0,2109
	Soft Voting	0,8776	0,1845	0,7903	0,3050
	Stacking	0,8673	0,85	0,8773	0,8586
SMOTE-ENN	Random Forest	0,8163	0,8989	0,8734	0,8556
	XGBoost	0,8469	0,7411	0,8711	0,7905
	Gradient Boost	0,9082	0,1193	0,7455	0,2109
	Soft Voting	0,8776	0,1845	0,8055	0,3050
	Stacking	0,8469	0,8469	0,8745	0,8469

Berdasarkan **Tabel 4.3**, terlihat bahwa kombinasi model dan teknik *resampling* menghasilkan performa yang sangat beragam. Tidak seluruh teknik *resampling* mampu meningkatkan performa model secara konsisten pada semua algoritma *ensemble learning*. Perbedaan karakteristik masing-masing teknik *resampling* menyebabkan setiap model merespons secara berbeda terhadap perubahan distribusi data yang terjadi selama proses *training*.

Selain itu, kombinasi ENN dengan *XGBoost* juga menunjukkan performa yang baik dengan nilai *F1-Score* sebesar 0,8723. Hasil ini mengindikasikan bahwa proses pembersihan data menggunakan ENN mampu membantu *XGBoost* dalam membangun batas keputusan (*boundary lines*) yang lebih jelas antara kelas *fraud* dan *non-fraud*. ENN bekerja dengan menghapus sampel yang dianggap tidak konsisten dengan mayoritas tetangga terdekatnya sehingga dapat mengurangi *noise* pada data *training*. Hasil lain terlihat pada kombinasi SMOTE dan *Stacking* yang menghasilkan *recall* sebesar 0,8571 dan *F1-Score* sebesar 0,8615. Peningkatan *recall* tersebut menunjukkan bahwa penambahan sampel sintetis melalui SMOTE mampu membantu model mengenali pola transaksi *fraud* dengan lebih baik. Namun, peningkatan *recall* tidak selalu diikuti peningkatan *F1-Score* yang signifikan karena *precision* cenderung mengalami penurunan dibandingkan model *baseline*.

Terdapat beberapa kombinasi yang menghasilkan *recall* yang sangat tinggi tetapi memiliki *precision* yang rendah. Contohnya adalah kombinasi *Random Undersampling* dengan *Soft Voting* yang memperoleh *recall* sebesar 0,9184, namun *precision* hanya sebesar 0,1128. Kondisi tersebut menunjukkan bahwa model berhasil mendeteksi sebagian besar transaksi *fraud*, tetapi menghasilkan jumlah *false positive* yang sangat besar. Akibatnya, banyak transaksi normal yang salah diklasifikasikan sebagai *fraud* sehingga menurunkan kualitas model secara keseluruhan. Fenomena serupa juga terlihat pada beberapa kombinasi yang menggunakan *Random Undersampling* dan *Hybrid Sampling* pada model *Gradient Boosting*. Meskipun *recall* berhasil ditingkatkan hingga di atas angka 0,90, nilai *precision* yang sangat rendah menyebabkan *F1-Score* menjadi rendah. Hasil tersebut menunjukkan bahwa peningkatan *recall* tidak selalu mencerminkan peningkatan performa model secara keseluruhan apabila tidak diimbangi dengan kemampuan mempertahankan *precision*.

Secara umum, hasil pengujian menunjukkan bahwa teknik *resampling* memberikan dampak yang berbeda pada setiap model *ensemble learning*. Beberapa teknik seperti ENN, SMOTE, dan *Random Oversampling* mampu menghasilkan keseimbangan yang baik antara *recall* dan *precision* yang ditunjukkan pada nilai *f1-score* yang cukup tinggi, sedangkan teknik lain seperti *Random Undersampling* cenderung menghasilkan *recall* yang tinggi lebih tinggi dari teknik yang disebutkan sebelumnya, namun mengorbankan *precision* secara signifikan.

4.1.3.2 Hasil Pengujian Berdasarkan Teknik *Resampling*

Untuk memperoleh gambaran yang lebih jelas mengenai pengaruh masing-masing teknik *resampling* terhadap performa model, dilakukan perbandingan berdasarkan rata-rata nilai *precision*, *recall*, *pr-auc* dan *F1-Score* yang dihasilkan oleh seluruh model *ensemble learning* pada setiap teknik *resampling*. Pendekatan ini digunakan untuk mengurangi bias yang mungkin muncul apabila analisis hanya didasarkan pada satu model tertentu, sehingga efektivitas setiap teknik *resampling* dapat diamati secara lebih menyeluruh.

Precision digunakan untuk mengukur ketepatan model dalam mengidentifikasi transaksi *fraud*, *recall* digunakan untuk mengukur kemampuan model mendeteksi seluruh transaksi *fraud*

yang sebenarnya, *pr-auc* mengukur performa keseluruhan model pada berbagai tingkat batas keputusan (*threshold*) yang berbeda sekaligus, sedangkan *F1-Score* digunakan untuk mengevaluasi keseimbangan antara *precision* dan *recall*. Dengan mempertimbangkan keempat metrik tersebut secara bersamaan, karakteristik performa setiap teknik *resampling* dapat diamati dengan lebih komprehensif. **Tabel 4.4** menunjukkan rata-rata performa setiap teknik *resampling* yang diperoleh dari seluruh model yang diuji.

Tabel 4.4 Hasil pengujian per Teknik Resampling

Teknik Resampling	Rata-Rata Recall	Rata-Rata Precision	Rata-Rata PR-AUC	Rata-Rata F1-score
Baseline	0,6633	0,8423	0,7229	0,7311
Random Oversampling	0,8449	0,7168	0,8525	0,7455
SMOTE	0,8673	0,6557	0,8458	0,7169
ADASYN	0,8449	0,5276	0,8157	0,5932
Random Undersampling	0,8980	0,1024	0,6968	0,1825
Tomek Links	0,6694	0,8575	0,7231	0,7336
ENN	0,7755	0,8985	0,8047	0,8314
SMOTE-Tomek	0,8694	0,5575	0,8324	0,6065
SMOTE-ENN	0,8592	0,5581	0,8340	0,5581

Berdasarkan **Tabel 4.4**, terdapat perbedaan karakteristik performa yang cukup jelas antara teknik *oversampling*, *undersampling*, dan *hybrid sampling*. Jika ditinjau dari nilai *recall*, *Random Undersampling* menghasilkan rata-rata *recall* tertinggi sebesar 0,8980. Hasil tersebut menunjukkan bahwa teknik ini mampu meningkatkan sensitivitas model terhadap kelas *fraud* sehingga sebagian besar transaksi *fraud* berhasil terdeteksi. Namun, tingginya *recall* tidak selalu diikuti oleh kualitas klasifikasi yang baik. Hal tersebut terlihat dari nilai rata-rata *F1-Score Random Undersampling* yang hanya mencapai 0,1825 dan menjadi yang terendah di antara seluruh teknik yang diuji. Selain itu, nilai *precision* pada teknik ini juga paling rendah dibandingkan teknik lainnya. Kondisi tersebut menunjukkan bahwa meskipun model mampu mendeteksi lebih banyak transaksi *fraud*, jumlah transaksi normal yang salah diklasifikasikan sebagai *fraud* juga meningkat. Akibatnya, keseimbangan performa model menjadi kurang baik.

Pada teknik *oversampling*, SMOTE menghasilkan rata-rata *recall* tertinggi sebesar 0,8673. Hasil ini menunjukkan bahwa penambahan sampel sintetis menggunakan pendekatan interpolasi mampu meningkatkan representasi kelas *fraud* secara efektif. Walaupun performanya paling baik, fenomena *trade-off* juga tetap terlihat dimana ketika ada kenaikan pada metrik *recall* selalu diikuti dengan penurunan di metrik *precision*.

Jika ditinjau dari keseimbangan performa secara keseluruhan, ENN menunjukkan hasil yang paling baik. Teknik ini menghasilkan rata-rata *F1-Score* tertinggi sebesar 0,8314 dengan rata-rata *recall* sebesar 0,7755. Selain itu, nilai *precision* yang diperoleh merupakan yang paling

tinggi dibandingkan dengan teknik *resampling* lainnya. Hasil tersebut menunjukkan bahwa ENN mampu menjaga keseimbangan antara kemampuan mendeteksi transaksi *fraud* dan ketepatan prediksi *fraud* yang dihasilkan model.

Pada teknik *hybrid sampling*, SMOTE-Tomek menghasilkan rata-rata *recall* sebesar 0,8694 yang sedikit lebih tinggi dibandingkan SMOTE-ENN sebesar 0,8592. Akan tetapi, kedua teknik tersebut menghasilkan rata-rata F1-Score yang relatif lebih rendah dibandingkan teknik *under sampling* ENN maupun *Tomek Links*. Nilai *precision* pada kedua teknik hybrid tersebut juga perlu diperhatikan karena peningkatan *recall* yang cukup tinggi diikuti oleh penurunan yang cukup besar dalam metrik tersebut yang menandakan bahwa hasil prediksi masih banyak yang tidak tepat.

Hasil pengujian menunjukkan bahwa teknik *resampling* yang menghasilkan *recall* tertinggi menghasilkan *precision* dan F1-Score yang lebih rendah. Beberapa teknik seperti *Random Undersampling*, SMOTE, dan SMOTE-Tomek mampu meningkatkan kemampuan deteksi *fraud* secara signifikan, tetapi tidak mampu mempertahankan ketepatan prediksi. Sebaliknya, teknik seperti *Random Oversampling*, ENN, dan *Tomek Links* menunjukkan performa yang lebih seimbang antara *precision*, *recall*, dan F1-Score.

4.1.3.3 Hasil Pengujian Berdasarkan Model Ensemble Learning

Selain menganalisis pengaruh teknik *resampling*, penelitian ini juga melakukan perbandingan performa berdasarkan model *ensemble learning* yang digunakan. Analisis ini bertujuan untuk mengetahui model mana yang secara umum memiliki kemampuan terbaik dalam menangani permasalahan deteksi *fraud* pada berbagai kondisi distribusi data hasil *resampling*. Untuk memperoleh gambaran yang lebih representatif, perbandingan dilakukan menggunakan rata-rata nilai *precision*, *recall*, *pr-auc*, dan *F1-Score* yang diperoleh setiap model dari seluruh kombinasi teknik *resampling* yang diuji. **Tabel 4.5** menunjukkan rata-rata performa masing-masing model *ensemble learning* selama proses eksperimen.

Tabel 4.5 Hasil pengujian per Model

Model	Rata-Rata Recall	Rata-Rata Precision	Rata-Rata PR-AUC	Rata-Rata F1-score
Random Forest	0,8254	0,8256	0,8605	0,7938
XGBoost	0,8413	0,7462	0,8390	0,7553
Gradient Boosting	0,7052	0,3362	0,5895	0,3231
Soft Voting	0,8367	0,5231	0,8115	0,5755
Stacking	0,8424	0,7445	0,8595	0,7426

Berdasarkan **Tabel 4.5**, terlihat bahwa setiap model menunjukkan karakteristik performa yang berbeda. Perbedaan tersebut menunjukkan bahwa respons masing-masing algoritma terhadap perubahan distribusi data akibat *resampling* tidak selalu sama. Beberapa model mampu mempertahankan keseimbangan antara *precision* dan *recall*, sementara model lainnya menunjukkan peningkatan *recall* yang tinggi tetapi disertai penurunan *precision* yang cukup besar.

Jika ditinjau dari nilai *recall*, *Stacking* menghasilkan rata-rata *recall* tertinggi sebesar 0,8424, diikuti oleh XGBoost sebesar 0,8413 dan *Soft Voting* sebesar 0,8367. Hasil tersebut menunjukkan bahwa ketiga model tersebut memiliki kemampuan yang lebih baik relatif dengan model lain dalam penelitian dalam mendeteksi transaksi *fraud* pada berbagai jenis teknik *resampling*. Perbedaan nilai *recall* antar ketiga model tersebut juga relatif kecil, yang mengindikasikan bahwa seluruh model mampu memanfaatkan informasi tambahan yang diberikan oleh teknik *resampling* untuk meningkatkan sensitivitas terhadap kelas *fraud*.

Meskipun demikian, nilai *recall* yang tinggi tidak selalu diikuti oleh kualitas performa prediksi yang baik secara keseluruhan. Hal ini terlihat pada model *Soft Voting* yang menghasilkan rata-rata *recall* sebesar 0,8367, namun hanya memperoleh rata-rata *precision* sebesar 0,5231 dan rata-rata *F1-Score* sebesar 0,5755. Kondisi tersebut menunjukkan bahwa model mampu mendeteksi banyak transaksi *fraud*, tetapi juga menghasilkan jumlah kesalahan prediksi *fraud* yang relatif tinggi. Akibatnya, keseimbangan performa model menjadi kurang optimal dibandingkan model lainnya.

Jika ditinjau dari nilai *F1-Score*, *Random Forest* menghasilkan performa terbaik dengan rata-rata *F1-Score* sebesar 0,7938. Nilai tersebut merupakan yang tertinggi di antara seluruh model yang diuji. Selain itu, *Random Forest* juga menunjukkan keseimbangan yang sangat baik antara *precision* dan *recall*, dengan kedua metrik berada pada kisaran yang hampir sama yaitu di angka 0,82. Hasil ini menunjukkan bahwa performa *Random Forest* cenderung lebih stabil pada berbagai variasi teknik *resampling* dibandingkan model lainnya.

Pola yang cukup menarik juga terlihat pada XGBoost dan *Stacking*. Kedua model menghasilkan rata-rata *recall* yang tinggi serta rata-rata *F1-Score* yang tidak jauh dibawah model dengan *F1-Score* tertinggi yaitu *Random Forest*. Hal ini menunjukkan bahwa performa kedua model cukup konsisten pada berbagai teknik *resampling*. Selain itu, nilai *precision* yang masih berada di atas 0,74 menunjukkan bahwa peningkatan kemampuan deteksi *fraud* tidak diikuti oleh peningkatan kesalahan klasifikasi yang berlebihan. Dengan demikian, XGBoost dan *Stacking* dapat dianggap sebagai model yang mampu mempertahankan keseimbangan performa pada berbagai kondisi distribusi data. Berbeda dengan ketiga model tersebut, *Soft Voting* menunjukkan variasi performa yang lebih besar antar teknik *resampling*. Pada beberapa kombinasi, model ini mampu menghasilkan *recall* yang sangat tinggi, namun peningkatan tersebut sering kali diikuti penurunan *precision* yang signifikan. Pola ini menyebabkan rata-rata *F1-Score* *Soft Voting* menjadi lebih rendah dibandingkan *Random Forest*, XGBoost, maupun *Stacking*.

Model dengan performa terendah pada penelitian ini adalah *Gradient Boosting*. Meskipun beberapa teknik *resampling* mampu meningkatkan *recall* hingga di atas 0,88, rata-rata *precision* yang diperoleh hanya sebesar 0,3362 sehingga menghasilkan rata-rata *F1-Score* sebesar 0,3231. Hasil tersebut menunjukkan bahwa performa *Gradient Boosting* sangat dipengaruhi oleh teknik *resampling* yang digunakan. Pada beberapa kondisi, model menjadi sangat sensitif terhadap kelas *fraud* sehingga mampu meningkatkan *recall* secara signifikan, tetapi peningkatan tersebut tidak diimbangi dengan kemampuan mempertahankan ketepatan prediksi sehingga efeknya terlihat pada nilai *F1-Score* dan *precision* yang sangat rendah.

Secara umum, hasil penelitian menunjukkan bahwa *Random Forest* merupakan model yang paling konsisten dalam mempertahankan keseimbangan antara *precision* dan *recall* pada berbagai teknik *resampling*. Sementara itu, XGBoost dan *Stacking* menunjukkan kemampuan

deteksi *fraud* yang sangat baik dengan nilai recall yang tinggi dan performa yang relatif stabil dibandingkan dengan model lainnya. Di sisi lain, *Soft Voting* dan *Gradient Boosting* cenderung menunjukkan variasi performa yang lebih besar terhadap perubahan distribusi data hasil *resampling*. Temuan ini mengindikasikan bahwa pemilihan model memiliki pengaruh yang sama pentingnya dengan pemilihan teknik *resampling* dalam membangun sistem deteksi *fraud* yang efektif.

4.1.4 Pemeringkatan Kombinasi Model dan Teknik Resampling Berdasarkan Metrik Evaluasi

Setelah dilakukan analisis berdasarkan teknik *resampling* dan model *ensemble learning*, langkah selanjutnya adalah mengidentifikasi kombinasi model dan teknik resampling yang menghasilkan performa terbaik per metriknya. Selain dari alasan tersebut, tahap ini bertujuan untuk menentukan kandidat model yang akan digunakan pada skenario lanjutnya yaitu proses *hyperparameter tuning*. Pada pemilihan lima kombinasi untuk skenario selanjutnya, pemeringkatan utama dilakukan berdasarkan nilai *F1-Score*. Pemilihan *F1-Score* sebagai dasar pemilihan dilakukan karena metrik tersebut mempertimbangkan keseimbangan antara *precision* dan *recall* secara bersamaan. Dalam kasus deteksi *fraud*, model tidak hanya dituntut mampu mendeteksi sebanyak mungkin transaksi *fraud*, tetapi juga harus mampu menjaga ketepatan prediksi agar jumlah transaksi normal yang salah diklasifikasikan sebagai *fraud* tidak terlalu besar. **Tabel 4.6** menunjukkan sepuluh kombinasi model dan teknik resampling dengan nilai *F1-Score* tertinggi yang diperoleh dari seluruh eksperimen.

Tabel 4.6 Pemeringkatan berdasarkan *F1-Score*

No	Teknik Resampling	Model	Recall	Precision	PR-AUC	F1-Score
1	Baseline	Random Forest	0,8163	0,9412	0,8788	0,8743
2	Tomek Links	Random Forest	0,8163	0,9412	0,8723	0,8743
3	Random Oversampling	Random Forest	0,8061	0,9518	0,8661	0,8729
4	ENN	XGBoost	0,8367	0,9111	0,8517	0,8723
5	ENN	Stacking	0,8061	0,9405	0,8640	0,8681
6	Random Oversampling	XGBoost	0,8367	0,9011	0,8754	0,8677
7	ENN	Random Forest	0,8265	0,9101	0,8683	0,8663
8	SMOTE	Random Forest	0,8469	0,8830	0,8732	0,8646
9	SMOTE	Stacking	0,8571	0,8660	0,8748	0,8615
10	Tomek Links	Stacking	0,8061	0,9186	0,8726	0,8587

Berdasarkan **Tabel 4.6**, terlihat bahwa sebagian besar kombinasi dengan *F1-Score* tertinggi didominasi oleh model *Random Forest*, *XGBoost*, dan *Stacking*. Tidak terdapat

kombinasi *Soft Voting* maupun *Gradient Boosting* pada sepuluh peringkat teratas berdasarkan *F1-Score*. Hasil ini menunjukkan bahwa ketiga model tersebut memiliki kemampuan yang lebih konsisten dalam mempertahankan keseimbangan antara *precision* dan *recall* pada berbagai kondisi distribusi data.

Dari sepuluh kombinasi terbaik, empat kombinasi melibatkan model *Random Forest*. Selain memperoleh *F1-Score* tertinggi, model ini juga menunjukkan variasi performa yang relatif kecil pada berbagai teknik *resampling*. Kondisi tersebut mengindikasikan bahwa *Random Forest* memiliki tingkat stabilitas yang baik terhadap perubahan distribusi data yang dihasilkan oleh proses *resampling*. Teknik ENN juga muncul sebanyak tiga kali dalam sepuluh peringkat teratas. Hasil ini sejalan dengan temuan pada subbab sebelumnya yang menunjukkan bahwa ENN menghasilkan rata-rata *F1-Score* tertinggi dibandingkan teknik *resampling* lainnya. Keberadaan ENN pada berbagai model menunjukkan bahwa teknik ini mampu memberikan peningkatan performa secara konsisten tanpa harus mengubah distribusi data secara drastis. Selain itu, teknik *Random Oversampling* dan SMOTE juga beberapa kali muncul pada kelompok peringkat atas. Kedua teknik tersebut menghasilkan *recall* yang relatif tinggi dengan penurunan *precision* yang tidak terlalu drastis. Pola ini menunjukkan bahwa peningkatan jumlah sampel fraud mampu membantu model mempelajari karakteristik kelas minoritas secara lebih baik tanpa menyebabkan penurunan kualitas klasifikasi yang terlalu besar.

Selain melakukan pemeringkatan berdasarkan *F1-Score*, penelitian ini juga mengidentifikasi kombinasi terbaik berdasarkan masing-masing metrik evaluasi. Analisis ini penting karena setiap metrik merepresentasikan tujuan yang berbeda. *precision* menggambarkan ketepatan model ketika memprediksi suatu transaksi sebagai *fraud*, sedangkan *recall* menggambarkan kemampuan model dalam menemukan seluruh transaksi *fraud* yang sebenarnya terjadi. Selanjutnya akan membahas mengenai kombinasi terbaik berdasarkan *recall*. **Tabel 4.7** menunjukkan sepuluh kombinasi dengan nilai *recall* tertinggi dari seluruh eksperimen yang dilakukan.

Tabel 4.7 Pemeringkatan berdasarkan Recall

No	Teknik Resampling	Model	Recall	Precision	PR-AUC	F1-Score
1	Random Undersampling	Soft Voting	0,9184	0,1128	0.7171	0,2009
2	SMOTE-ENN	Gradient Boosting	0,9082	0,1193	0.7455	0,2109
3	SMOTE-Tomek	Gradient Boosting	0,9082	0,1193	0,7455	0,2109
4	ADASYN	Stacking	0,8980	0,3520	0,8681	0,5057
5	ADASYN	Gradient Boosting	0,8980	0,1064	0,7225	0,1903
6	Random Undersampling	Stacking	0,8980	0,0985	0,7615	0,1776
7	Random Undersampling	XGBoost	0,8980	0,0917	0,7488	0,1664

No	Teknik Resampling	Model	Recall	Precision	PR-AUC	F1-Score
8	Random Oversampling	Gradient Boosting	0,8878	0,3551	0,8183	0,5073
9	SMOTE	Gradient Boosting	0,8878	0,2852	0,7701	0,4318
10	Random Undersampling	Gradient Boosting	0,8878	0,0602	0,7681	0,1127

Berdasarkan Tabel 4.7, terlihat bahwa kombinasi dengan *recall* tertinggi didominasi oleh teknik resampling yang secara agresif meningkatkan representasi kelas minoritas, seperti *Random Undersampling*, SMOTE-ENN, dan SMOTE-Tomek. Selain itu, model *Soft Voting* dan *Gradient Boosting* muncul lebih sering pada kelompok *recall* tertinggi dibandingkan pada kelompok *F1-Score* tertinggi.

Kombinasi *Random Undersampling* dengan *Soft Voting* menghasilkan *recall* tertinggi sebesar 0,9184. Artinya, lebih dari 91% transaksi *fraud* berhasil dideteksi oleh model. Hasil ini menunjukkan kemampuan yang sangat baik dalam meminimalkan *false negative*, yaitu transaksi *fraud* yang gagal terdeteksi oleh sistem. Teknik *Random Undersampling* tidak muncul pada kelompok kombinasi terbaik berdasarkan *F1-Score* meskipun menghasilkan *recall* yang sangat tinggi pada beberapa model. Hal tersebut terjadi karena nilai *precision* yang rendah menyebabkan *F1-Score* menurun secara signifikan. Temuan ini menunjukkan bahwa peningkatan kemampuan deteksi *fraud* saja belum cukup untuk menghasilkan performa model yang baik apabila tidak disertai kemampuan mempertahankan ketepatan prediksi.

Pola yang muncul pada pemeringkatan *recall* berbeda dengan pemeringkatan *F1-Score*. Kombinasi-kombinasi yang mendominasi *recall* tertinggi tidak muncul pada kelompok peringkat atas *F1-Score*. Sebagai contoh, kombinasi *Random Undersampling* dengan *Soft Voting* menghasilkan *recall* tertinggi sebesar 0,9184, namun hanya memperoleh *precision* sebesar 0,1128 dan *F1-Score* sebesar 0,2009. Pola serupa juga terlihat pada kombinasi *Random Undersampling* dengan XGBoost maupun beberapa kombinasi yang melibatkan *Gradient Boosting*. Hal ini menunjukkan bahwa peningkatan kemampuan mendeteksi *fraud* sering kali diikuti oleh penurunan *precision*, yang menandakan bahwa model menjadi lebih sensitif terhadap *fraud*, tetapi juga lebih sering mengklasifikasikan transaksi normal sebagai *fraud*. Temuan tersebut menunjukkan adanya *trade-off* antara *precision* dan *recall*. Kombinasi dengan *precision* sangat tinggi cenderung lebih konservatif dalam memberikan label *fraud* sehingga menghasilkan lebih sedikit *false positive*, sedangkan kombinasi dengan *recall* sangat tinggi cenderung lebih agresif dalam mendeteksi *fraud* sehingga mampu menemukan lebih banyak kasus *fraud* tetapi berpotensi meningkatkan *false positive*. Oleh karena itu, penggunaan *F1-Score* sebagai dasar utama pemeringkatan menjadi relevan karena metrik ini mempertimbangkan kedua aspek tersebut secara bersamaan.

Selanjutnya adalah pemeringkatan berdasarkan metrik *precision*, yang menggambarkan ketepatan model ketika memprediksi suatu transaksi sebagai *fraud*. Semakin rendah *precision*, maka semakin buruk performa model dalam hal ketepatan prediksi (banyak *false positive*). Tabel 4.8 menunjukkan sepuluh kombinasi dengan nilai *precision* tertinggi dari seluruh eksperimen yang dilakukan.

Tabel 4.8 Pemeringkatan berdasarkan Precision

No	Teknik Resampling	Model	Recall	Precision	PR-AUC	F1-Score
1	Random Oversampling	Random Forest	0,8061	0,9518	0,8661	0,8729
2	Baseline	Random Forest	0,8163	0,9412	0,8788	0,8743
3	Tomek Links	Random Forest	0,8163	0,9412	0,8723	0,8743
4	ENN	Stacking	0,8061	0,9405	0,8640	0,8681
5	Baseline	Stacking	0,7755	0,9383	0,8747	0,8492
6	Baseline	Soft Voting	0,7449	0,9359	0,8512	0,8295
7	Tomek Links	Soft Voting	0,7653	0,9259	0,8548	0,8380
8	Tomek Links	Stacking	0,8061	0,9186	0,8726	0,8587
9	ENN	XGBoost	0,8367	0,9111	0,8517	0,8723
10	ENN	Random Forest	0,8265	0,9101	0,8683	0,8663

Berdasarkan **Tabel 4.8**, terlihat bahwa model *Random Forest* mendominasi peringkat atas *precision*. Tiga posisi teratas seluruhnya ditempati oleh *Random Forest* dengan teknik *Baseline*, *Tomek Links*, dan *Random Oversampling*. Hal ini menunjukkan bahwa *Random Forest* memiliki kemampuan yang sangat baik dalam mengurangi jumlah *false positive* sehingga prediksi *fraud* yang dihasilkan cenderung lebih akurat. Kombinasi *Random Oversampling* dengan *Random Forest* menghasilkan *precision* tertinggi sebesar 0,9518. Nilai tersebut menunjukkan bahwa lebih dari 95% transaksi yang diprediksi sebagai *fraud* benar-benar merupakan transaksi *fraud*. Hasil ini mengindikasikan bahwa kombinasi tersebut sangat efektif dalam menjaga ketepatan prediksi sekaligus tetap mempertahankan *recall* pada tingkat yang cukup baik secara keseluruhan jika dibandingkan dengan kombinasi lain.

Secara keseluruhan, hasil penelitian menunjukkan bahwa kombinasi terbaik berdasarkan *precision*, *recall*, dan *F1-Score* tidak selalu berasal dari kombinasi model dan teknik *resampling* yang sama. Kombinasi *Random Oversampling* dengan *Random Forest* unggul dalam *precision*, *Random Undersampling* dengan *Soft Voting* unggul dalam *recall*, sedangkan *Baseline Random Forest* dan *Tomek Links Random Forest* menghasilkan *F1-Score* tertinggi. Pola ini mengindikasikan bahwa pemilihan model terbaik perlu disesuaikan dengan tujuan sistem yang akan dibangun. Namun, karena penelitian ini membutuhkan model yang mampu mendeteksi *fraud* secara efektif sekaligus menjaga ketepatan prediksi, maka *F1-Score* digunakan sebagai dasar utama dalam menentukan kandidat model untuk skenario *hyperparameter tuning*.

Berdasarkan hasil evaluasi keseluruhan, lima kombinasi dengan nilai *F1-Score* tertinggi dipilih untuk mengikuti skenario *hyperparameter tuning*, yaitu *Baseline-Random Forest*, *Tomek Links-Random Forest*, *Random Oversampling-Random Forest*, *ENN-XGBoost*, dan *ENN-Stacking*. Pemilihan dilakukan berdasarkan *F1-Score* dan bukan *recall* karena kombinasi yang ingin dipilih adalah yang terbaik performanya sehingga tidak hanya berfokus pada kemampuan

mendeteksi *fraud*, tetapi juga mempertimbangkan ketepatan prediksi yang dihasilkan model. Hasil eksperimen menunjukkan bahwa beberapa kombinasi dengan *recall* sangat tinggi justru memiliki *precision* yang rendah sehingga menghasilkan *F1-Score* yang buruk. Oleh karena itu, penggunaan *F1-Score* sebagai dasar pemilihan dianggap lebih representatif dalam mengidentifikasi model yang memiliki performa seimbang dan layak untuk dioptimasi lebih lanjut melalui proses *hyperparameter tuning*.

4.1.5 Hasil Pengujian Model dengan Hyperparameter Tuning

Setelah diperoleh lima kombinasi model dan teknik *resampling* terbaik berdasarkan nilai *F1-Score*, tahap selanjutnya adalah menguji skenario berikutnya yaitu melakukan optimasi model melalui proses *hyperparameter tuning*. Tahap ini bertujuan untuk mencari konfigurasi parameter yang mampu meningkatkan performa model khususnya pada metrik *recall* dibandingkan konfigurasi *default* yang digunakan pada tahap eksperimen sebelumnya.

Proses tuning dilakukan menggunakan metode *Randomized Search Cross Validation* dengan ruang pencarian parameter yang telah ditentukan pada masing-masing model. Evaluasi dilakukan menggunakan data *testing* yang sama dengan tahap sebelumnya sehingga hasil yang diperoleh dapat dibandingkan secara langsung dengan performa model sebelum tuning. Dengan pendekatan ini, pengaruh *hyperparameter tuning* terhadap kemampuan deteksi *fraud* dapat diamati secara lebih objektif.

Berbeda dengan tahap seleksi kandidat model yang menggunakan *F1-Score* sebagai dasar pemilihan karena mempertimbangkan keseimbangan antara *precision* dan *recall*, pada tahap *hyperparameter tuning* metrik yang digunakan sebagai parameter scoring adalah *recall*. Pemilihan *recall* sebagai fokus optimasi dilakukan karena lima kombinasi yang masuk ke tahap tuning telah menunjukkan performa yang baik dan relatif seimbang untuk metrik lainnya pada tahap sebelumnya. Oleh karena itu, proses tuning diarahkan untuk meningkatkan kemampuan model dalam mendeteksi transaksi *fraud* sebanyak mungkin tanpa harus melakukan pencarian ulang terhadap model yang sudah terbukti memiliki keseimbangan performa yang baik. Dalam kasus deteksi *fraud*, peningkatan *recall* menjadi penting karena kesalahan dalam mengidentifikasi transaksi *fraud* (*false negative*) dapat menimbulkan risiko yang lebih besar dibandingkan kesalahan mendeteksi transaksi normal sebagai *fraud* (*false positive*). **Tabel 4.9** menunjukkan perbandingan performa sebelum dan sesudah dilakukan *hyperparameter tuning* pada lima kombinasi terbaik yang diperoleh pada tahap evaluasi sebelumnya.

Tabel 4.9 Hasil Skenario Hyperparameter Tuning

Teknik Resampling	Model	Skenario	Recall	Precision	PR-AUC	F1-Score
Baseline	Random Forest	Default	0,8163	0,9412	0,8788	0,8743
		Hyperparameter tuning	0,8265	0,6750	0,8101	0,7431
Tomek Links	Random Forest	Default	0,8163	0,9412	0,8723	0,8743

Teknik Resampling	Model	Skenario	Recall	Precision	PR-AUC	F1-Score
		Hyperparameter Tuning	0,8265	0,6983	0,8177	0,7570
Random Oversampling	Random Forest	Default	0,8061	0,9518	0,8661	0,8729
		Hyperparameter Tuning	0,8469	0,8557	0,8689	0,8513
ENN	XGBoost	Default	0,8367	0,9111	0,8517	0,8723
		Hyperparameter Tuning	0,8469	0,8218	0,8707	0,8342
ENN	Stacking	Default	0,8061	0,9405	0,8640	0,8681
		Hyperparameter Tuning	0,7959	0,8966	0,8664	0,8432

Berdasarkan **Tabel 4.9**, terlihat bahwa *hyperparameter tuning* memberikan pengaruh yang berbeda pada setiap kombinasi model dan teknik *resampling*. Perubahan performa tidak selalu mengarah pada peningkatan seluruh metrik evaluasi. Pada beberapa kombinasi, tuning berhasil meningkatkan *recall*, namun diikuti penurunan *precision* yang cukup besar sehingga menyebabkan *F1-Score* menurun.

Pola pertama yang terlihat adalah adanya peningkatan *recall* pada hampir seluruh model setelah dilakukan tuning. Kombinasi *Baseline-Random Forest* dan *Tomek Links-Random Forest* mengalami peningkatan *recall* dari 0,8163 menjadi 0,8265. Peningkatan yang lebih besar juga terjadi pada *Random Oversampling-Random Forest* yang meningkat dari 0,8061 menjadi 0,8469, serta ENN-XGBoost yang meningkat dari 0,8367 menjadi 0,8469. Hasil ini menunjukkan bahwa penggunaan *recall* sebagai parameter scoring pada proses tuning berhasil mengarahkan model untuk menjadi lebih sensitif dalam mendeteksi transaksi fraud.

Meskipun demikian, peningkatan *recall* tersebut tidak selalu menghasilkan peningkatan performa secara keseluruhan. Pada kombinasi *Baseline-Random Forest* dan *Tomek Links-Random Forest*, peningkatan *recall* diikuti oleh penurunan *precision* yang cukup signifikan, masing-masing dari 0,9412 menjadi 0,6750 dan dari 0,9412 menjadi 0,6983. Akibatnya, nilai *F1-Score* mengalami penurunan yang cukup besar menjadi 0,7431 dan 0,7570. Pola ini menunjukkan bahwa model menjadi lebih agresif dalam memberikan prediksi *fraud* sehingga jumlah transaksi normal yang salah diklasifikasikan sebagai *fraud* meningkat.

Pola serupa juga terlihat pada kombinasi ENN-XGBoost. Meskipun *recall* meningkat dari 0,8367 menjadi 0,8469, *precision* menurun dari 0,9111 menjadi 0,8218 sehingga *F1-Score*

turun dari 0,8723 menjadi 0,8342. Kondisi ini menunjukkan bahwa peningkatan kemampuan mendeteksi *fraud* diperoleh dengan mengorbankan sebagian ketepatan prediksi.

Berbeda dengan kombinasi lainnya, *Random Oversampling-Random Forest* menunjukkan perubahan performa yang relatif lebih seimbang. *Recall* meningkat sebesar 0,0408 poin, sementara *precision* hanya mengalami penurunan sebesar 0,0961 poin. Akibatnya, nilai *F1-Score* hanya mengalami penurunan kecil dari 0,8729 menjadi 0,8513. Dibandingkan kombinasi tuning lainnya, hasil ini menunjukkan bahwa *Random Oversampling-Random Forest* mampu mempertahankan keseimbangan performa yang lebih baik setelah proses optimasi dilakukan.

Sementara itu, kombinasi ENN-*Stacking* menunjukkan pola yang berbeda. Setelah tuning dilakukan, nilai *recall* sedikit menurun dari 0,8061 menjadi 0,7959 dan *precision* menurun dari 0,9405 menjadi 0,8966. Penurunan pada kedua metrik tersebut menyebabkan *F1-Score* turun menjadi 0,8432. Hasil ini menunjukkan bahwa konfigurasi parameter hasil tuning tidak selalu lebih baik dibandingkan konfigurasi default yang digunakan sebelumnya, meskipun proses optimasi telah difokuskan pada peningkatan *recall*.

Jika ditinjau berdasarkan nilai *recall* setelah tuning, *Random Oversampling-Random Forest* dan ENN-XGBoost menghasilkan *recall* tertinggi sebesar 0,8469. Akan tetapi, jika mempertimbangkan keseimbangan performa melalui *F1-Score*, kombinasi *Random Oversampling-Random Forest* tetap menjadi yang terbaik dengan *F1-Score* sebesar 0,8513, diikuti oleh ENN-*Stacking* sebesar 0,8432 dan ENN-XGBoost sebesar 0,8342. Hasil tersebut menunjukkan bahwa kombinasi *Random Oversampling* dengan *Random Forest* merupakan kandidat terbaik setelah proses optimasi dilakukan karena mampu meningkatkan kemampuan deteksi *fraud* sekaligus mempertahankan keseimbangan performa yang lebih baik dibandingkan kombinasi lainnya.

Secara umum, hasil penelitian menunjukkan bahwa *hyperparameter tuning* berhasil meningkatkan kemampuan deteksi *fraud* yang direpresentasikan oleh *recall* pada sebagian besar kombinasi model. Temuan ini sejalan dengan penggunaan *recall* sebagai parameter scoring selama proses optimasi. Namun, peningkatan tersebut sering kali diikuti oleh penurunan *precision* sehingga menyebabkan *F1-Score* menurun. Temuan ini menunjukkan bahwa proses optimasi parameter cenderung menggeser karakteristik model menjadi lebih sensitif terhadap kelas *fraud*, tetapi tidak selalu mampu mempertahankan keseimbangan performa antara *recall* dan *precision*.

4.1.5.1 Konfigurasi Parameter Optimal

Hasil proses *hyperparameter tuning* selain menghasilkan perubahan performa model juga menghasilkan konfigurasi parameter optimal untuk masing-masing kombinasi model dan teknik *resampling* sesuai dengan ruang pencarian parameter yang ditentukan. Konfigurasi tersebut diperoleh melalui proses *RandomizedSearchCV* dengan fungsi evaluasi *recall* dan digunakan sebagai parameter akhir pada model yang dievaluasi pada tahap pengujian. **Tabel 4.10** menunjukkan parameter Terbaik *Random Forest-Baseline*.

Tabel 4.10 Parameter terbaik Baseline-Random Forest

Hyperparameter	Nilai Terbaik
class_weight	balanced

Hyperparameter	Nilai Terbaik
max_depth	10
max_features	None
min_samples_leaf	7
min_samples_split	13
n_estimators	363

Konfigurasi terbaik pada *Random Forest-Baseline* menunjukkan bahwa proses tuning memilih kedalaman pohon yang relatif terbatas, yaitu 10 tingkat, serta menggunakan mekanisme pembobotan kelas (`class_weight = balanced`). Penggunaan pembobotan kelas menunjukkan bahwa model perlu memberikan perhatian lebih besar terhadap kelas *fraud* yang jumlahnya jauh lebih sedikit dibandingkan kelas *non-fraud*. Selain itu, nilai `min_samples_leaf` dan `min_samples_split` yang relatif besar menunjukkan bahwa model cenderung membentuk pohon yang lebih sederhana untuk mengurangi risiko overfitting pada data yang tidak mengalami proses resampling. Selanjutnya, **Tabel 4.11** menunjukkan parameter terbaik *Random Forest-Tomek Links*.

Tabel 4.11 Parameter Terbaik Tomek Links-Random Forest

Hyperparameter	Nilai Terbaik
class_weight	balanced
max_depth	10
max_features	None
min_samples_leaf	7
min_samples_split	13
n_estimators	363

Kombinasi *Tomek Links* menghasilkan konfigurasi terbaik yang identik dengan *Baseline Random Forest*. Hasil ini menunjukkan bahwa proses pembersihan data menggunakan *Tomek Links* tidak mengubah karakteristik distribusi data secara signifikan. Oleh karena itu, model tetap memerlukan konfigurasi parameter yang sama untuk mencapai performa *recall* terbaik. Selanjutnya, **Tabel 4.12** menunjukkan parameter terbaik *Random Forest-Random Oversampling*.

Tabel 4.12 Parameter Terbaik Random Oversampling-Random Forest

Hyperparameter	Nilai Terbaik
class_weight	None
max_depth	30
max_features	sqrt

Hyperparameter	Nilai Terbaik
min_samples_leaf	8
min_samples_split	8
n_estimators	221

Berbeda dengan dua kombinasi sebelumnya, model hasil *Random Oversampling* tidak lagi menggunakan pembobotan kelas (`class_weight = None`). Hal ini menunjukkan bahwa setelah distribusi kelas diperbaiki melalui proses *oversampling*, model tidak lagi memerlukan mekanisme penyeimbangan kelas tambahan. Selain itu, nilai `max_depth` yang meningkat menjadi 30 menunjukkan bahwa model membangun pohon yang lebih dalam untuk menangkap pola yang lebih kompleks pada data hasil *oversampling*. Penggunaan `max_features = sqrt` mengindikasikan bahwa pada setiap proses pemisahan node, model hanya mempertimbangkan sebagian fitur yang dipilih secara acak, sehingga dapat meningkatkan keragaman antar pohon dan mengurangi korelasi dalam *ensemble*. Sementara itu, nilai `min_samples_leaf` dan `min_samples_split` yang lebih besar membantu menjaga agar pohon tidak terlalu spesifik terhadap data *training*. Dengan jumlah pohon sebanyak 221 *estimator*, konfigurasi ini mampu menghasilkan performa yang lebih seimbang setelah proses tuning dibandingkan kombinasi lainnya. Selanjutnya, **Tabel 4.13** menunjukkan *parameter* terbaik *XGBoost-ENN*.

Tabel 4.13 Parameter Terbaik ENN-XGBoost

Hyperparameter	Nilai Terbaik
colsample_bytree	0,6294
gamma	3,3126
learning_rate	0,1035
max_depth	8
min_child_weight	6
n_estimators	155
scale_pos_weight	50
subsample	0,6739

Pada kombinasi ENN dan XGBoost, proses tuning menghasilkan nilai `scale_pos_weight` sebesar 50. Parameter ini menunjukkan bahwa model memberikan bobot yang jauh lebih besar pada kelas *fraud* selama proses *training*. Selain itu, penggunaan nilai `gamma` yang relatif tinggi menyebabkan proses pemisahan node menjadi lebih selektif sehingga membantu mengurangi kompleksitas model. Kombinasi parameter tersebut berkontribusi terhadap peningkatan Recall dari 0,8367 menjadi 0,8469 setelah tuning dilakukan. Selanjutnya, **Tabel 4.14 - 4.16** menunjukkan parameter terbaik *Stacking* dengan *base learner* *Random Forest*, *XGBoost*, dan *Gradient Boosting* yang dikombinasikan dengan ENN.

Tabel 4.14 Parameter Terbaik Base Learner ENN-Stacking Random Forest

Hyperparameter	Nilai Terbaik
class_weight	None
max_depth	20
max_features	None
min_samples_leaf	8
min_samples_split	5
n_estimators	459

Tabel 4.15 Base Learner ENN-Stacking XGBoost

Hyperparameter	Nilai Terbaik
colsample_bytree	0,9693
gamma	0,0039
learning_rate	0,3077
max_depth	3
min_child_weight	6
n_estimators	302
scale_pos_weight	50
subsample	0,6092

Tabel 4.16 Base Learner ENN-Stacking Gradient Boosting

Hyperparameter	Nilai Terbaik
learning_rate	0,0240
max_depth	5
max_features	None
min_samples_leaf	15
min_samples_split	13
n_estimators	104
subsample	0,9933

Pada model *Stacking*, proses tuning menghasilkan konfigurasi yang berbeda pada masing-masing *base learner*. *Random Forest* menggunakan kedalaman pohon sebesar 20 dengan jumlah estimator yang relatif besar, yaitu 459 pohon, untuk meningkatkan kemampuan model dalam menangkap pola pada data hasil *resampling* ENN. Berbeda dengan konfigurasi sebelumnya, model ini tidak menggunakan pembobotan kelas maupun pembatasan fitur melalui

`max_features`, yang menunjukkan bahwa distribusi data hasil ENN sudah cukup mendukung proses pembelajaran tanpa penyesuaian tambahan.

Pada *base learner* XGBoost, nilai `scale_pos_weight` sebesar 50 tetap menunjukkan fokus model terhadap kelas *fraud*. Namun, kedalaman pohon yang lebih dangkal (`max_depth` = 3) dikombinasikan dengan `learning_rate` yang relatif tinggi memungkinkan model melakukan pembelajaran secara lebih efisien tanpa membentuk struktur yang terlalu kompleks. Nilai `colsample_bytree` yang mendekati 1 menunjukkan bahwa hampir seluruh fitur digunakan pada setiap pohon yang dibangun.

Sementara itu, *base learner Gradient Boosting* menggunakan `learning_rate` yang rendah dan jumlah estimator yang cukup banyak untuk menghasilkan proses pembelajaran yang lebih bertahap dan stabil. Kombinasi parameter tersebut membantu mengurangi risiko *overfitting* sekaligus mempertahankan kemampuan generalisasi model.

Ketiga *base learner* tersebut kemudian digabungkan melalui *meta-learner Logistic Regression* untuk menghasilkan prediksi akhir. Meskipun konfigurasi hasil tuning menghasilkan struktur *ensemble* yang lebih kompleks dan meningkatkan sensitivitas terhadap kelas *fraud*, peningkatan performa secara keseluruhan tidak melampaui konfigurasi *default*. Hal ini terlihat dari penurunan *F1-Score* dari 0,8681 menjadi 0,8432 setelah proses tuning dilakukan.

4.2 Pembahasan

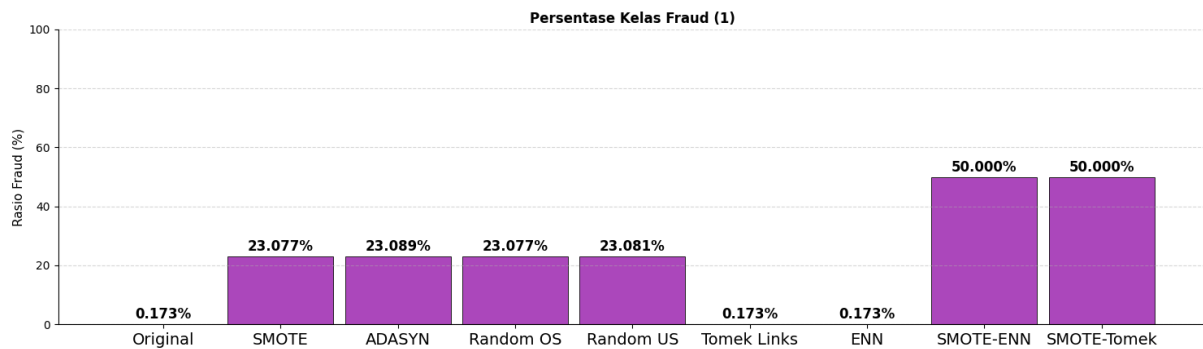
Subbab ini berisikan analisis dan interpretasi terhadap hasil penelitian yang telah diperoleh. Pembahasan dilakukan untuk menjelaskan pengaruh teknik *resampling* terhadap distribusi data dan performa model, membandingkan kinerja model *ensemble learning* yang digunakan, serta mengevaluasi dampak hyperparameter tuning terhadap kemampuan deteksi *fraud*. Melalui pembahasan ini, diharapkan dapat diperoleh pemahaman yang lebih mendalam mengenai faktor-faktor yang memengaruhi performa model serta alasan yang mendasari hasil yang diperoleh pada penelitian ini.

4.2.1 Analisis Pengaruh Teknik Resampling terhadap Distribusi Data

Salah satu tantangan utama pada dataset *Credit Card Fraud Detection* yang digunakan dalam penelitian ini adalah terjadinya ketidakseimbangan kelas (*class imbalance*) yang sangat ekstrem. Sebelum proses *resampling* dilakukan, data *training* terdiri atas 227.451 transaksi *non-fraud* dan hanya 394 transaksi *fraud*. Dengan demikian, proporsi kelas *fraud* hanya sekitar 0,17% dari keseluruhan data *training* dan perbandingan rasio antara data *fraud* dengan *non-fraud* adalah 1 banding 577. Hal itu menandakan bahwa pada setiap 1 data *fraud* terdapat 577 data *non-fraud*. Kondisi tersebut menyebabkan model klasifikasi berpotensi mengalami bias terhadap kelas mayoritas karena sebagian besar pola yang dipelajari berasal dari transaksi mayoritas tersebut yaitu *non-fraud*. Akibatnya, model dapat menghasilkan akurasi yang tinggi namun gagal mendeteksi sebagian besar transaksi *fraud* yang fatal jika terjadi di situasi dunia nyata sehingga metrik *recall* menjadi fokus utama penelitian ini.

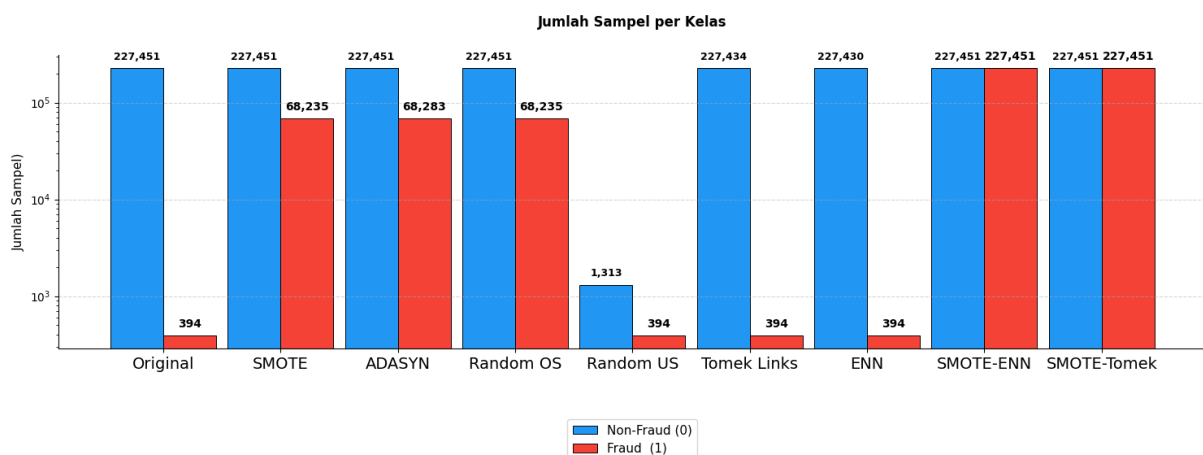
Untuk mengatasi permasalahan tersebut, penelitian ini menerapkan delapan teknik *resampling* yang terdiri atas kelompok *oversampling*, *undersampling*, dan *hybrid sampling*. Seluruh teknik *oversampling* menggunakan nilai *sampling strategy* sebesar 0,3 dan 0,5 (untuk *hybrid sampling*) yang dipilih berdasarkan hasil pengujian awal karena mampu memberikan

keseimbangan yang lebih baik antara peningkatan representasi kelas *fraud* dan kestabilan distribusi data.



Gambar 4.1 Perbandingan Rasio Fraud

Terlihat pada **Gambar 4.1** bahwa kelompok *oversampling* dan *Random Undersampling* menghasilkan rasio kelas minoritas sekitar 23%, sedangkan metode *hybrid sampling* menghasilkan distribusi yang sepenuhnya seimbang dengan rasio 50:50. Sebaliknya, *Tomek Links* dan ENN hanya mengubah distribusi data dalam jumlah yang sangat kecil sehingga proporsi kelas fraud hampir tidak berubah.

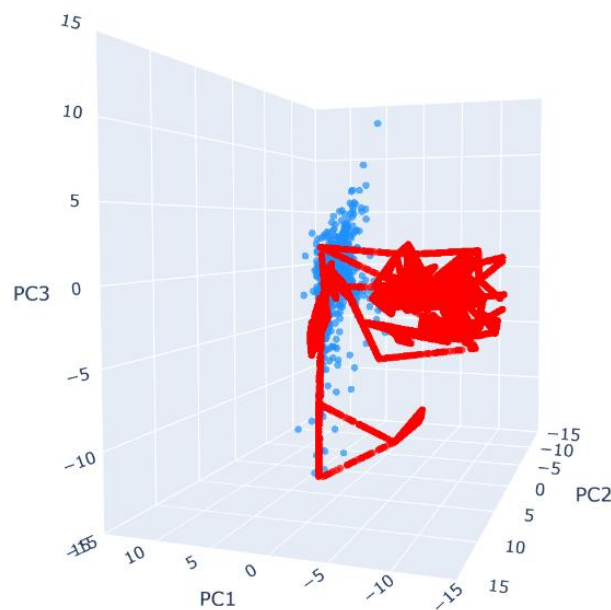


Gambar 4.2 Perbandingan Distribusi Data

Gambar 4.2 menunjukkan kelompok *oversampling* yang terdiri atas *Random Oversampling*, SMOTE, dan ADASYN menunjukkan perubahan distribusi yang paling signifikan pada kelas *fraud*. Jumlah transaksi fraud meningkat dari 394 data menjadi lebih dari 68 ribu data, sementara jumlah transaksi *non-fraud* tetap dipertahankan pada 227.451 data. Kondisi ini menyebabkan representasi kelas *fraud* meningkat hampir 173 kali lipat dibandingkan distribusi awal.

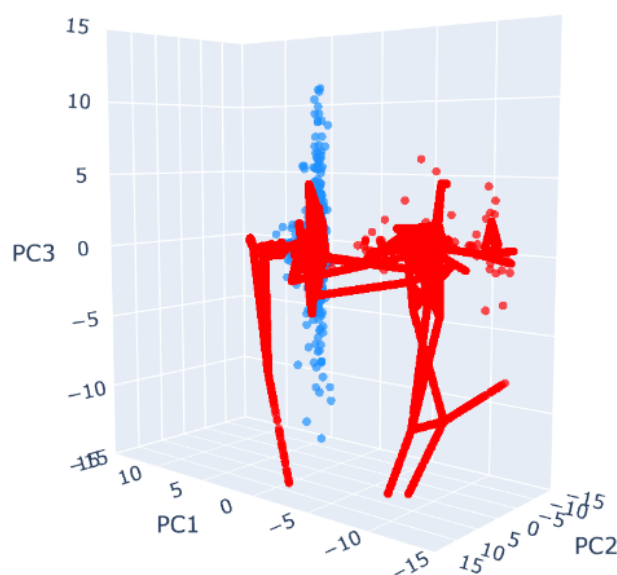
Peningkatan jumlah sampel minoritas memberikan lebih banyak contoh bagi model untuk mempelajari pola transaksi *fraud*. Pada dataset yang sangat tidak seimbang, algoritma klasifikasi sering kali mengalami kesulitan membentuk batas keputusan atau *decision boundary* yang mampu memisahkan kedua kelas secara optimal karena jumlah contoh *fraud* yang sangat terbatas. Dengan bertambahnya jumlah sampel *fraud*, area ruang fitur yang sebelumnya jarang terwakili menjadi lebih padat sehingga model memperoleh informasi yang lebih kaya mengenai karakteristik transaksi *fraud*.

Meskipun menghasilkan distribusi akhir yang hampir sama, ketiga teknik *oversampling* memiliki mekanisme pembentukan data yang berbeda. *Random Oversampling* hanya menduplikasi sampel *fraud* yang sudah ada sehingga tidak menambahkan informasi baru ke dalam dataset. Sebaliknya, SMOTE membuat sampel sintetis melalui interpolasi antar tetangga terdekat pada kelas minoritas sehingga distribusi *fraud* menjadi lebih bervariasi. ADASYN mengembangkan konsep tersebut dengan menghasilkan lebih banyak sampel sintetis pada area yang dianggap sulit dipelajari oleh model. Oleh karena itu, ADASYN menghasilkan jumlah data *fraud* sedikit lebih banyak dibandingkan SMOTE, yaitu 68.283 data dibandingkan 68.235 data. Perbedaan tersebut menunjukkan bahwa ADASYN melakukan penyesuaian distribusi secara adaptif berdasarkan tingkat kesulitan klasifikasi masing-masing wilayah data.



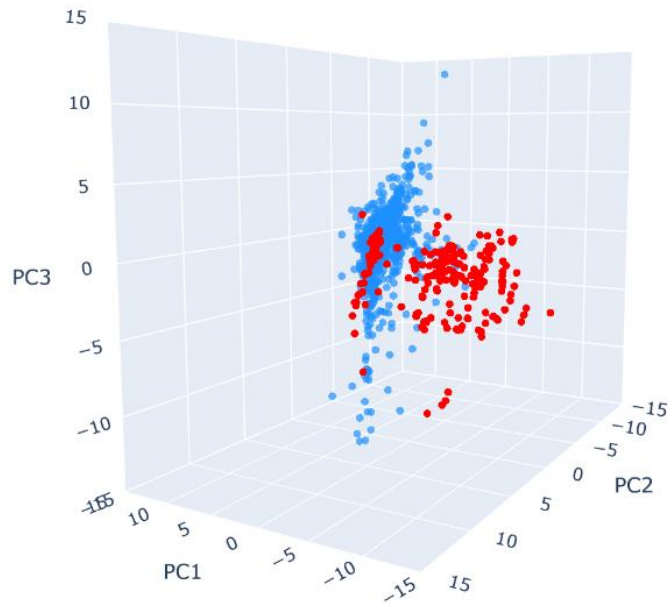
Gambar 4.3 Perbandingan persebaran Data setelah SMOTE

Biru = non-fraud, Merah = fraud



Gambar 4.4 Perbandingan persebaran Data setelah ADASYN

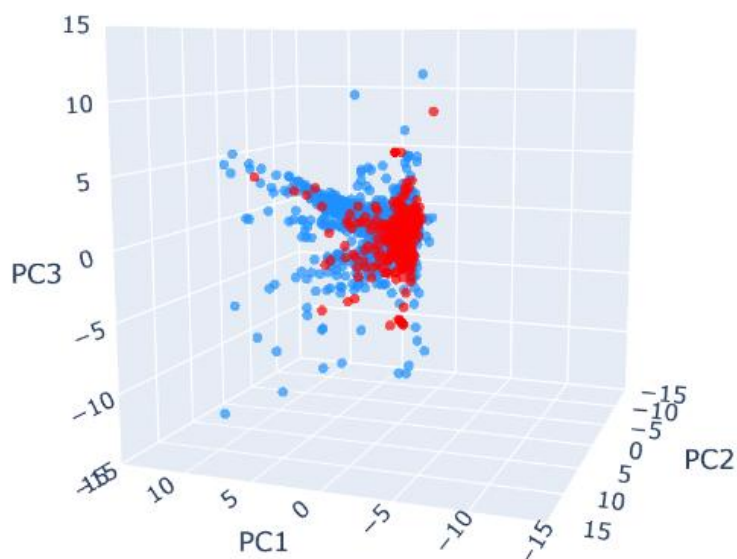
Biru = non-fraud, Merah = fraud



Gambar 4.5 Perbandingan persebaran Data setelah Random Oversampling

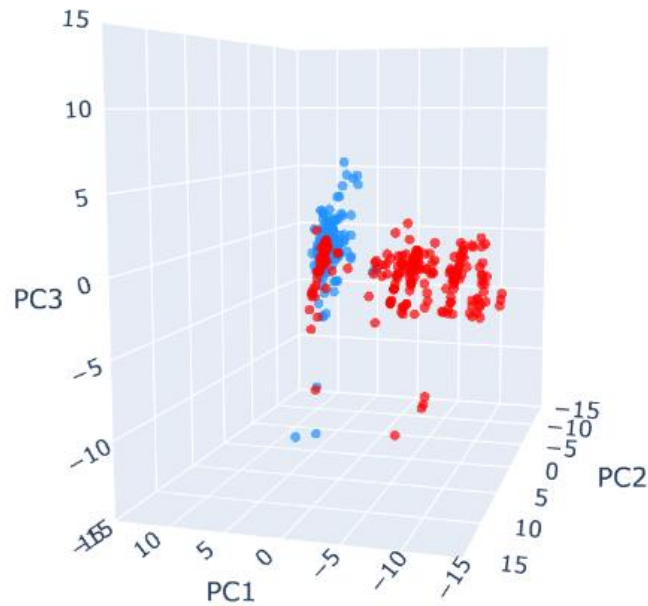
Biru = non-fraud, Merah = fraud

Berbeda dengan *oversampling*, *Random Undersampling* mengurangi jumlah data non-fraud secara drastis dari 227.451 menjadi hanya 1.313 data. Jumlah data *fraud* tetap dipertahankan sebanyak 394 data sehingga rasio kedua kelas menjadi jauh lebih seimbang. Namun, pendekatan ini menghilangkan lebih dari 99% sampel kelas mayoritas. pengurangan data dalam jumlah sangat besar berpotensi menyebabkan hilangnya informasi penting yang diperlukan untuk merepresentasikan variasi transaksi normal. Akibatnya, batas keputusan yang dipelajari model menjadi kurang stabil dan lebih rentan menghasilkan prediksi *fraud* pada transaksi yang sebenarnya normal. Fenomena ini menjelaskan mengapa teknik *Random Undersampling* pada hasil eksperimen menghasilkan recall yang sangat tinggi namun sering diikuti penurunan *precision* yang signifikan pada berbagai model.



Gambar 4.6 Persebaran Data Sebelum Sampling

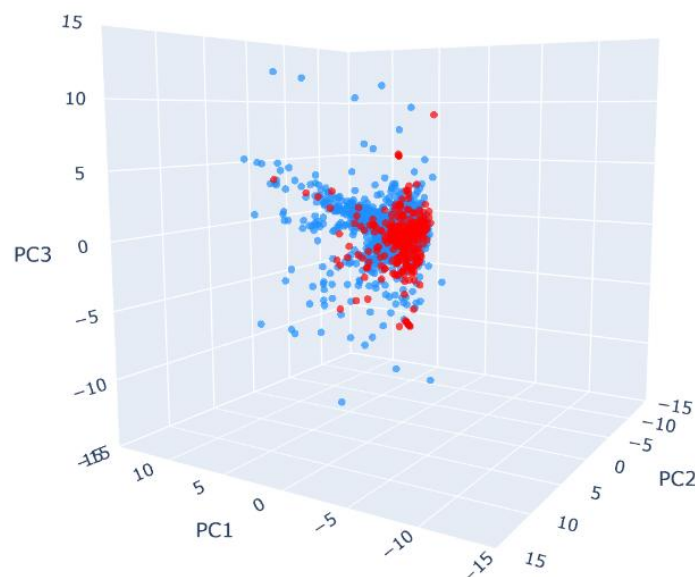
Biru = non-fraud, Merah = fraud



Gambar 4.7 Persebaran Data setelah Random Undersampling

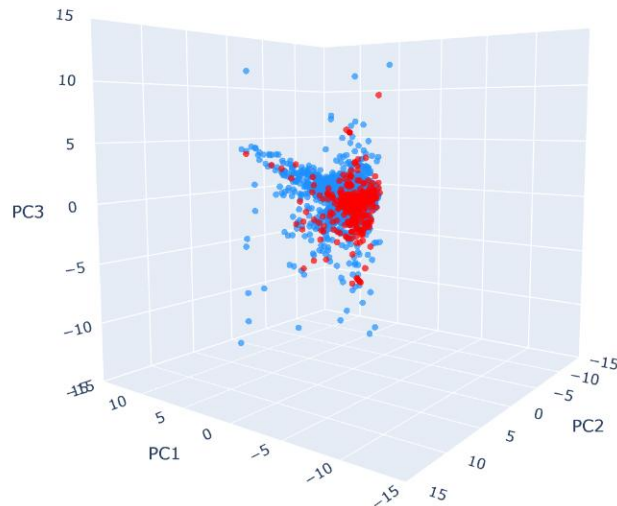
Biru = non-fraud, Merah = fraud

Untuk *Tomek Links* dan ENN, kedua teknik tersebut menghasilkan perubahan distribusi yang sangat kecil. *Tomek Links* hanya menghapus 17 data *non-fraud*, sedangkan ENN menghapus 21 data *non-fraud* dari total lebih dari 227 ribu data. Hal itu menandakan bahwa lebih dari 99,99% data tetap dipertahankan. Hasil tersebut menunjukkan bahwa sebagian besar data pada dataset telah memiliki pemisahan kelas yang relatif baik. Fungsi utama kedua metode tersebut bukan untuk menyeimbangkan distribusi kelas, melainkan melakukan pembersihan data atau *data cleaning* dengan menghilangkan sampel yang berada pada wilayah ambigu atau dianggap sebagai *noise*. Oleh karena itu, perubahan distribusi yang dihasilkan relatif kecil meskipun dapat memberikan dampak terhadap kualitas batas keputusan yang dibentuk model.



Gambar 4.8 Persebaran Data setelah Tomek Links

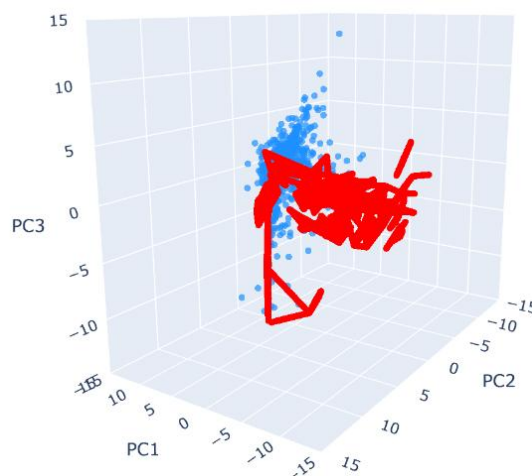
Biru = non-fraud, Merah = fraud



Gambar 4.9 Persebaran Data setelah ENN

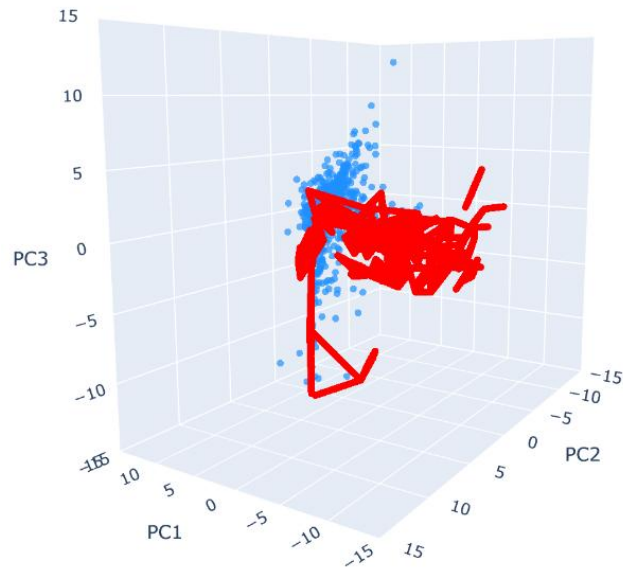
Biru = non-fraud, Merah = fraud

Pada kelompok *hybrid sampling*, SMOTE-ENN dan SMOTE-Tomek menghasilkan perubahan distribusi yang paling agresif dibandingkan teknik lainnya. Kedua metode menghasilkan jumlah data yang sama pada kelas *fraud* dan *non-fraud*, yaitu masing-masing 227.451 data. Dengan demikian, distribusi data menjadi sepenuhnya seimbang dengan rasio 50:50. Distribusi yang seimbang tersebut diperoleh melalui kombinasi dua tahapan. Tahap pertama adalah penggunaan SMOTE untuk meningkatkan jumlah sampel *fraud* secara sintesis. Lalu pada tahap kedua dilakukan proses pembersihan data menggunakan ENN atau Tomek Links untuk menghapus sampel yang berada pada area tumpang tindih antar kelas. Pendekatan ini tidak hanya meningkatkan representasi kelas minoritas, tetapi juga berupaya memperjelas batas keputusan dengan mengurangi keberadaan sampel yang ambigu. Kombinasi *oversampling* dan pembersihan data mampu menghasilkan distribusi yang lebih representatif dibandingkan penggunaan salah satu pendekatan secara terpisah. Namun, distribusi yang terlalu seimbang juga dapat menyebabkan model menjadi lebih sensitif terhadap kelas *fraud* sehingga berpotensi meningkatkan jumlah *false positive* ketika dievaluasi pada data *testing* yang tetap memiliki distribusi kelas yang tidak seimbang.



Gambar 4.10 Persebaran Data setelah SMOTE-ENN

Biru = non-fraud, Merah = fraud



Gambar 4.11 Persebaran Data setelah SMOTE-Tomek

Biru = non-fraud, Merah = fraud

Berdasarkan hasil distribusi yang diperoleh, dapat disimpulkan bahwa setiap kelompok teknik *resampling* menghasilkan karakteristik perubahan data yang berbeda. *Oversampling* berfokus pada peningkatan representasi kelas *fraud* tanpa mengurangi data mayoritas. *Undersampling* mengurangi dominasi kelas mayoritas dengan risiko kehilangan informasi penting. Sementara itu, *hybrid sampling* mengombinasikan peningkatan jumlah sampel *fraud* dengan proses pembersihan data sehingga menghasilkan distribusi yang paling seimbang.

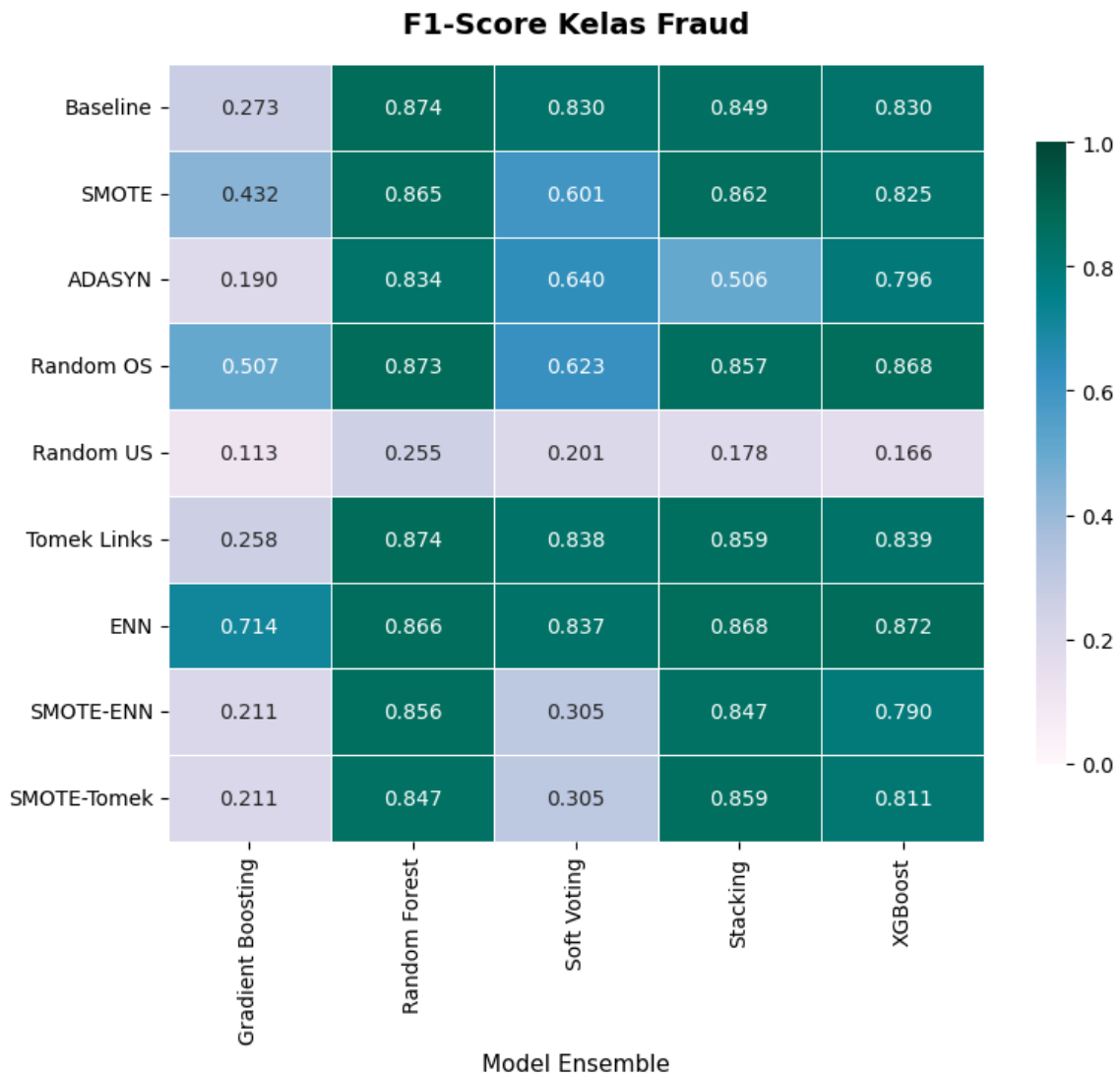
Perbedaan distribusi tersebut menjadi faktor utama yang memengaruhi kemampuan model dalam mempelajari pola transaksi *fraud* pada tahap pengujian berikutnya. Semakin besar representasi kelas *fraud* yang tersedia selama proses pelatihan, semakin besar pula peluang model untuk mengenali pola transaksi *fraud*. Namun, peningkatan representasi yang terlalu agresif juga berpotensi menurunkan kemampuan model dalam membedakan transaksi *fraud* dan *non-fraud* secara akurat ketika dihadapkan pada distribusi data dunia nyata yang tetap tidak seimbang.

4.2.2 Pengaruh Teknik Resampling terhadap Kinerja Model Ensemble Learning

Tahap berikutnya adalah menganalisis bagaimana perubahan distribusi data memengaruhi kinerja model *ensemble learning* dalam mendeteksi transaksi fraud. Evaluasi dilakukan menggunakan metrik *recall*, *precision*, dan *F1-Score* karena ketiga metrik tersebut lebih representatif dibandingkan *accuracy* pada kasus klasifikasi tidak seimbang atau *imbalanced classification*.

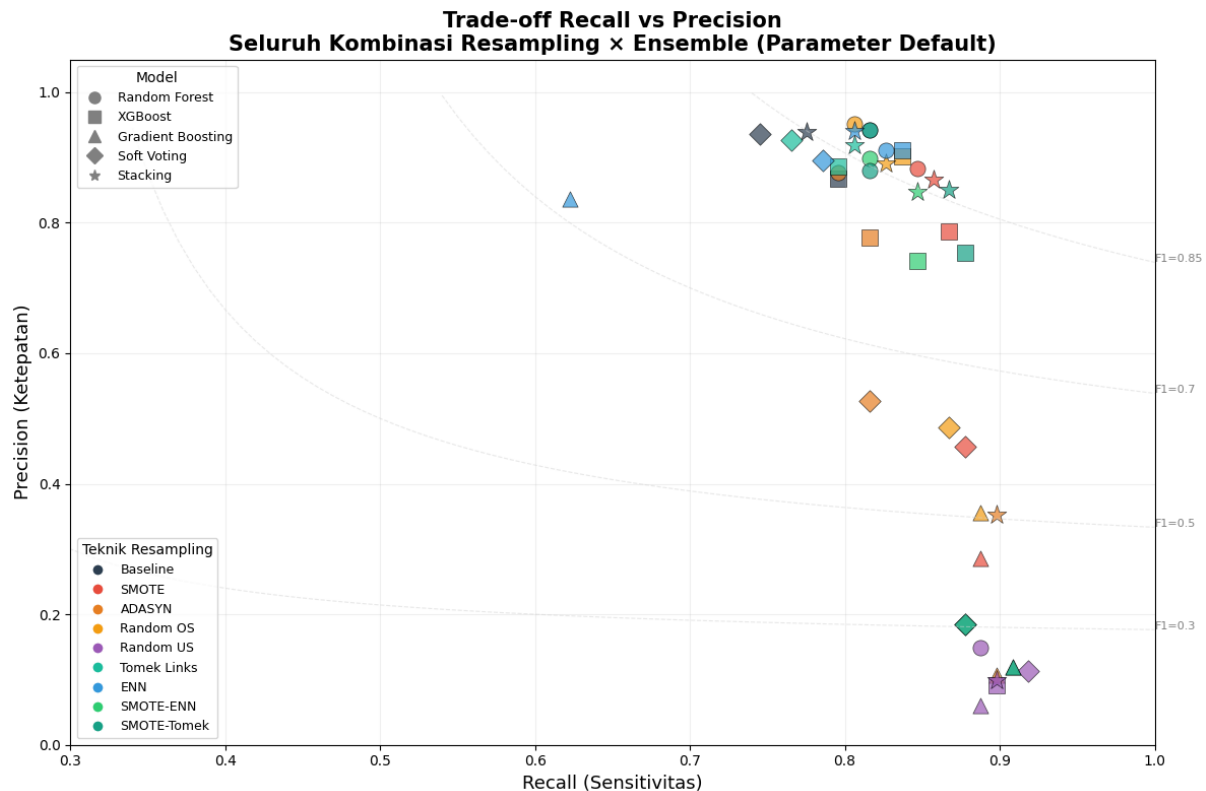
Pada dataset *fraud detection*, *accuracy* cenderung memberikan gambaran yang menyesatkan karena jumlah transaksi *non-fraud* mendominasi lebih dari 99% keseluruhan data. Kondisi ini menyebabkan model dapat memperoleh nilai *accuracy* yang sangat tinggi meskipun gagal mendeteksi sebagian besar transaksi fraud. Oleh karena itu, pembahasan difokuskan pada *recall* yang menunjukkan kemampuan model mendeteksi *fraud*, *precision* yang menggambarkan tingkat ketepatan prediksi *fraud*, serta *F1-Score* yang merepresentasikan keseimbangan antara keduanya.

Hasil eksperimen menunjukkan bahwa pengaruh teknik *resampling* terhadap performa model tidak selalu bersifat positif. Beberapa teknik mampu meningkatkan *recall* secara signifikan namun mengorbankan *precision*, sedangkan teknik lainnya menghasilkan keseimbangan yang lebih baik sehingga meningkatkan *F1-Score*. Berdasarkan rata-rata nilai metrik berdasarkan teknik *resampling*, teknik ENN menghasilkan *F1-Score* tertinggi sebesar 0,8314, diikuti *Random Oversampling* sebesar 0,7455 dan *Tomek Links* sebesar 0,7336. Sebaliknya, *Random Undersampling* menghasilkan *F1-Score* rata-rata terendah yaitu hanya 0,1825 meskipun memiliki *Recall* rata-rata tertinggi sebesar 0,8980. **Gambar 4.12** menunjukkan *heatmap* hasil kombinasi berdasarkan *F1-Score*.



Gambar 4.12 Heatmap berdasarkan F1-Score

Temuan bahwa ketika nilai *recall* naik selalu diikuti dengan penurunan *precision* menunjukkan bahwa peningkatan *recall* yang tinggi tidak selalu menghasilkan model yang lebih baik dan terdapat *trade-off* pada nilai kedua metrik tersebut. Pada sistem deteksi *fraud*, model yang terlalu agresif mengklasifikasikan transaksi sebagai *fraud* memang mampu menangkap lebih banyak kasus *fraud* yang ditandai dengan nilai *recall* yang tinggi, namun juga diikuti dengan meningkatnya jumlah *false positive* secara signifikan sehingga *precision* menurun drastis. **Gambar 4.13** menunjukkan pola *trade-off* antara *precision* dan *recall*.



Gambar 4.13 Scatter Plot Trade-off Precision dan Recall

Kelompok *oversampling*, yang terdiri atas *Random Oversampling*, SMOTE, dan ADASYN berhasil meningkatkan *recall* dibandingkan model *baseline*. Sebagai contoh pada model *Random Forest*, *recall* meningkat dari 0,8163 menjadi 0,8469 setelah menggunakan SMOTE. Pada *Stacking*, *recall* meningkat dari 0,7755 menjadi 0,8571. Pada XGBoost, *Recall* meningkat dari 0,7959 menjadi 0,8673. Peningkatan tersebut terjadi karena hasil *oversampling* menambah representasi kelas *fraud* selama proses *training*. Dengan jumlah sampel *fraud* yang lebih banyak, model memperoleh informasi yang lebih banyak mengenai pola transaksi *fraud* sehingga batas keputusan yang dibentuk menjadi lebih sensitif terhadap kelas minoritas. Namun, peningkatan *recall* tersebut diikuti penurunan *precision* pada sebagian besar model. Sebagai contoh, *precision* XGBoost turun dari 0,8667 pada *baseline* menjadi 0,7870 setelah menggunakan SMOTE. Penurunan yang lebih drastis terjadi pada Soft Voting, yaitu dari 0,9359 menjadi 0,4574. Fenomena tersebut menunjukkan bahwa *oversampling* menyebabkan model menjadi lebih sensitif dalam memberikan label *fraud*. Akibatnya, jumlah *fraud* yang berhasil ditemukan meningkat, tetapi jumlah transaksi normal yang salah diklasifikasikan sebagai *fraud* juga meningkat.

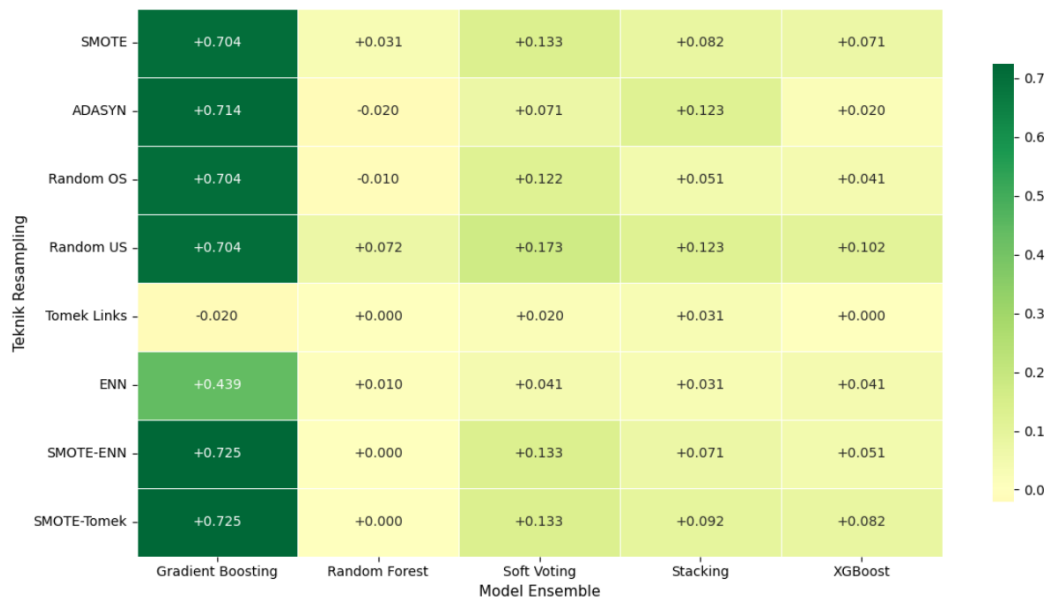
Di antara seluruh teknik *oversampling*, *Random Oversampling* menunjukkan performa paling stabil dalam hal keseimbangan antar memprediksi kelas *fraud* dan ketepatan dalam memprediksi. Teknik ini menghasilkan rata-rata *F1-Score* sebesar 0,7455 dan menempati beberapa posisi teratas pada kombinasi terbaik, termasuk *Random Oversampling–Random Forest* dengan *F1-Score* 0,8729 serta *Random Oversampling–XGBoost* dengan *F1-Score* 0,8677. Untuk ADASYN, teknik tersebut menunjukkan kecenderungan menghasilkan *precision* yang lebih rendah dibandingkan SMOTE maupun *Random Oversampling*. Contoh

pada kombinasi ADASYN–*Stacking*, *recall* mencapai 0,8980 tetapi *precision* turun menjadi 0,3520 sehingga *F1-Score* hanya 0,5057. Hal ini mengindikasikan bahwa sampel sintetis yang dibangkitkan secara adaptif pada wilayah sulit dapat memperluas area kelas *fraud* secara berlebihan dan meningkatkan kesalahan klasifikasi.

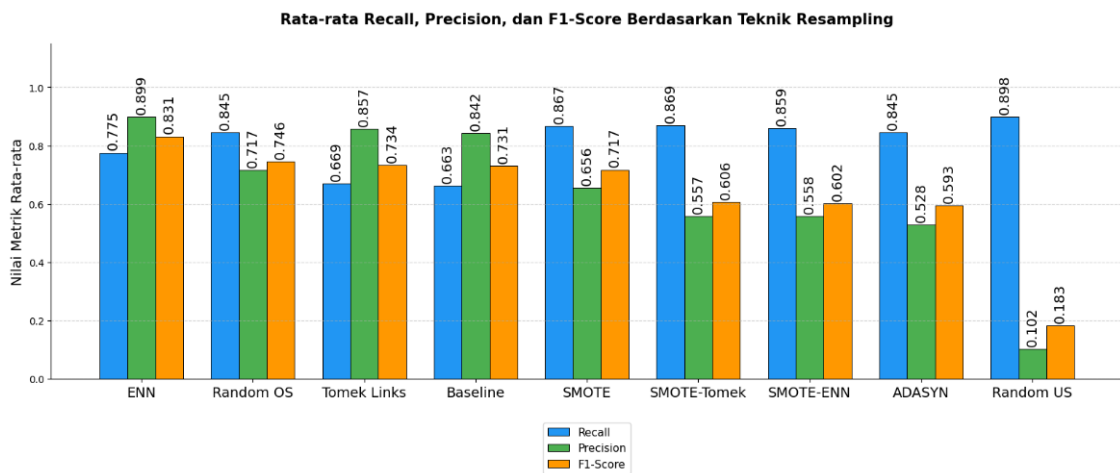
Untuk Kelompok *undersampling*, teknik-teknik tersebut menunjukkan karakteristik yang sangat berbeda dibandingkan *oversampling*. *Random Undersampling* menghasilkan *recall* tertinggi hampir pada seluruh model kecuali dengan *Gradient Boosting*. *Soft Voting* mencapai *recall* 0,9184, sedangkan XGBoost dan *Stacking* masing-masing mencapai *Recall* 0,8980. Meskipun demikian, *precision* mengalami penurunan yang sangat drastis. Pada kombinasi dengan XGBoost, *precision* hanya mencapai 0,0917. Pada *Stacking* juga turun menjadi 0,0985. Akibatnya, seluruh kombinasi *Random Undersampling* menghasilkan *F1-Score* yang sangat rendah. Fenomena ini dapat dijelaskan melalui distribusi data hasil *undersampling* yang hanya menyisakan 1.707 sampel dari lebih dari 227 ribu data *training* orisinal. Sebagian besar informasi mengenai transaksi normal hilang selama proses *training* sehingga model tidak lagi mampu membentuk representasi yang baik terhadap kelas mayoritas. Akibatnya, model cenderung menganggap banyak transaksi normal sebagai *fraud*.

Berbeda dengan *Random Undersampling*, *Tomek Links* dan ENN menghasilkan performa yang jauh lebih baik. Kedua teknik tersebut tidak mengurangi data mayoritas secara agresif, melainkan hanya menghapus data yang berada pada wilayah perbatasan kelas. Hasil eksperimen menunjukkan bahwa ENN menghasilkan rata-rata *F1-Score* tertinggi dibandingkan seluruh teknik *resampling* yaitu sebesar 0,8314. Kombinasi ENN–XGBoost menghasilkan *F1-Score* 0,8723, sedangkan ENN–*Stacking* menghasilkan *F1-Score* 0,8681. Peningkatan tersebut menunjukkan bahwa pembersihan data atau *noise cleaning* lebih efektif dibandingkan penyeimbangan distribusi secara ekstrem. Dengan menghapus sampel ambigu di sekitar batas kelas, model memperoleh batas keputusan yang lebih jelas sehingga *precision* dan *recall* dapat meningkat secara bersamaan.

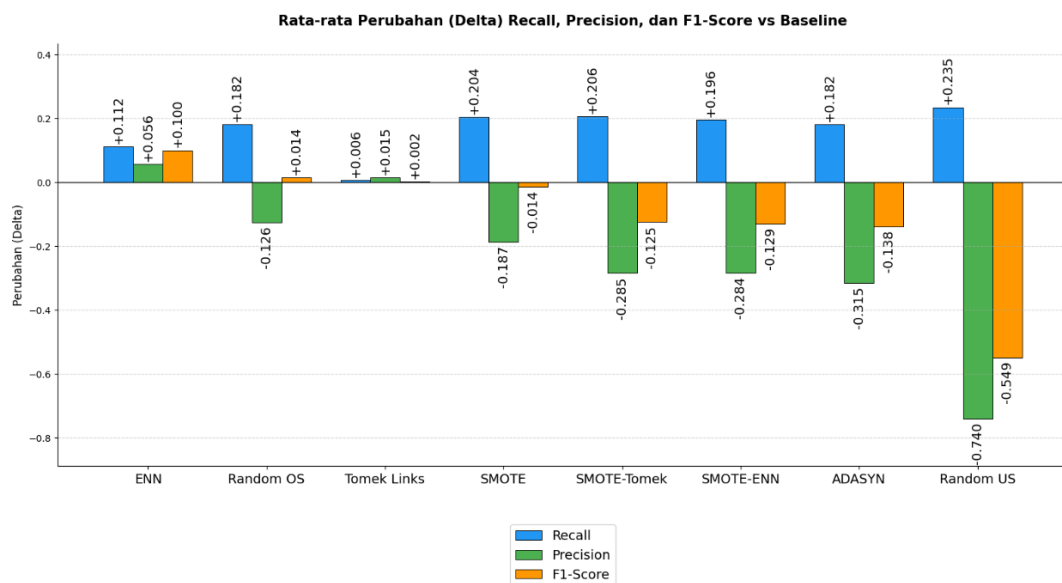
Pada kelompok *hybrid* sampling, SMOTE-ENN dan SMOTE-Tomek menghasilkan distribusi data yang sepenuhnya seimbang dengan rasio perbandingan 50:50. Kondisi tersebut memberikan peluang terbaik bagi model untuk mempelajari kedua kelas secara setara. Namun hasil eksperimen menunjukkan bahwa distribusi yang sangat seimbang tidak selalu menghasilkan performa terbaik. Pada XGBoost, *recall* meningkat menjadi 0,8776 setelah menggunakan SMOTE-Tomek, tetapi *precision* turun menjadi 0,7544. Pada *Soft Voting*, *recall* mencapai 0,8776 tetapi *precision* hanya 0,1845 sehingga *F1-Score* turun drastis menjadi 0,3050. Hasil ini mengindikasikan bahwa proses *oversampling* yang terlalu agresif dapat mengubah distribusi asli data secara signifikan. Ketika model dilatih menggunakan data yang sepenuhnya seimbang tetapi diuji pada data yang tetap tidak seimbang, model cenderung menghasilkan lebih banyak prediksi *fraud* dibandingkan kondisi sebenarnya. Meskipun demikian, kombinasi SMOTE-Tomek–*Stacking* tetap menunjukkan performa kompetitif dengan *F1-Score* sebesar 0,8586. Temuan ini menunjukkan bahwa model yang lebih kompleks seperti *Stacking* mampu memanfaatkan informasi tambahan dari data sintetis dengan lebih baik dibandingkan model *ensemble* lainnya.



Gambar 4.14 Heatmap Perubahan Recall terhadap Baseline



Gambar 4.15 Rata-rata Metrik Berdasarkan Teknik Resampling



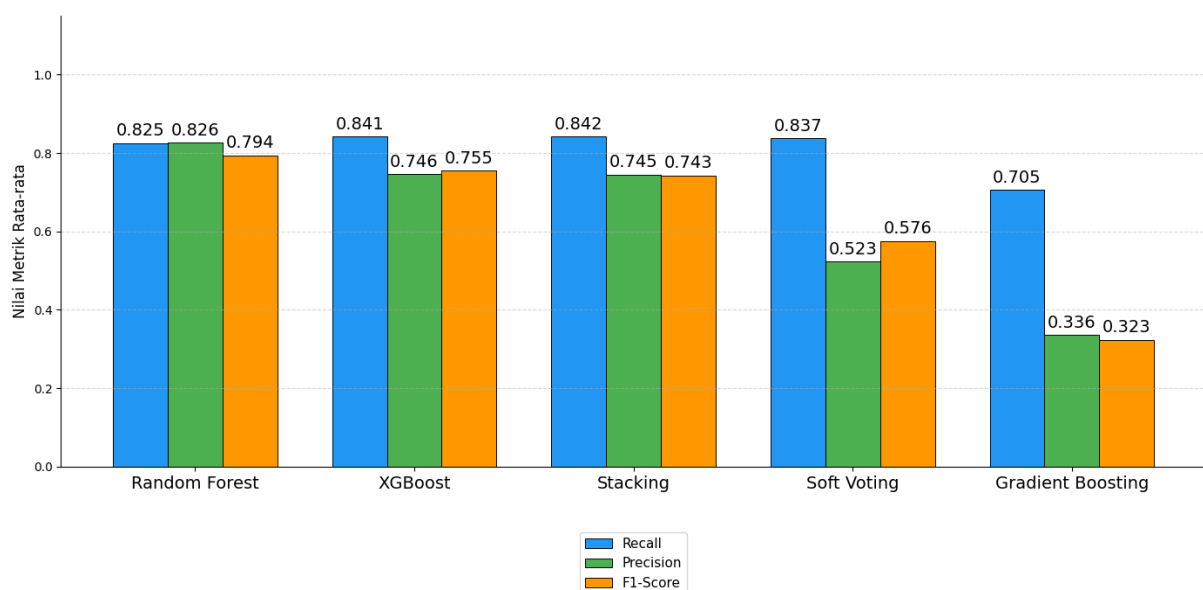
Gambar 4.16 Rata-rata perubahan Metrik terhadap Baseline

Hasil penelitian menunjukkan bahwa pengaruh teknik *resampling* terhadap kinerja model *ensemble learning* sangat bergantung pada karakteristik teknik *resampling* dan mekanisme pembelajaran model yang digunakan. Teknik yang menghasilkan distribusi paling seimbang tidak selalu memberikan performa terbaik. ENN terbukti menjadi teknik yang paling konsisten dalam meningkatkan F1-Score karena mampu membersihkan data tanpa mengubah distribusi secara ekstrem. Sebaliknya, *Random Undersampling* menghasilkan *recall* tertinggi tetapi *precision* terendah akibat hilangnya sebagian besar informasi kelas mayoritas. Secara keseluruhan, hasil penelitian menunjukkan bahwa peningkatan kualitas representasi data melalui pembersihan noise lebih efektif dibandingkan sekadar menyeimbangkan jumlah kelas.

4.2.3 Analisis Performa Model Ensemble Learning

Tujuan utama penelitian ini salah satunya adalah mengevaluasi kemampuan berbagai metode *ensemble learning* dalam mendeteksi transaksi *fraud* pada dataset *Credit Card Fraud Detection* yang memiliki tingkat ketidakseimbangan kelas yang sangat tinggi. Untuk mencapai tujuan tersebut, lima model *ensemble learning* diuji pada berbagai kondisi distribusi data hasil penerapan teknik *resampling*, yaitu *Random Forest* sebagai representasi metode *bagging*, *Gradient Boosting* dan *XGBoost* sebagai representasi *boosting*, *Stacking* sebagai *heterogeneous ensemble*, serta *Soft Voting* sebagai kombinasi beberapa classifier.

Hasil eksperimen menunjukkan bahwa setiap model memiliki karakteristik yang berbeda dalam menghadapi permasalahan ketidakseimbangan kelas. Perbedaan tersebut terlihat dari variasi nilai *recall*, *precision*, dan *F1-Score* yang dihasilkan pada berbagai teknik *resampling*. Secara umum, *Random Forest* menunjukkan performa paling stabil dengan *F1-Score* tinggi pada hampir seluruh skenario pengujian. Sementara itu, *XGBoost* dan *Stacking* menunjukkan kemampuan adaptasi yang baik terhadap perubahan distribusi data, khususnya ketika dikombinasikan dengan teknik pembersihan data seperti ENN. Sebaliknya, *Gradient Boosting* dan *Soft Voting* menunjukkan sensitivitas yang lebih tinggi terhadap perubahan distribusi data sehingga performanya lebih bervariasi. Untuk memberikan gambaran umum performa masing-masing model, rata-rata metrik seluruh berdasarkan model divisualisasikan pada **Gambar 4.17**.



Gambar 4.17 Rata-rata Nilai Metrik Berdasarkan Model

Gambar 4.17 menunjukkan bahwa *Random Forest* secara konsisten menghasilkan performa tertinggi, sedangkan *Soft Voting* menghasilkan performa terendah berdasarkan *F1-Score*. Perbedaan ini menunjukkan bahwa mekanisme pembelajaran yang digunakan masing-masing model memiliki pengaruh yang signifikan terhadap kemampuan mendeteksi *fraud*.

Random Forest merupakan metode *ensemble* berbasis *bagging* yang membangun sejumlah *decision tree* secara independen menggunakan subset data dan subset fitur yang berbeda. Pendekatan ini menghasilkan model yang lebih *robust* terhadap *overfitting* dibandingkan *single decision tree*. Berdasarkan hasil pengujian, *Random Forest* mempunyai nilai rata-rata *F1-Score* dan *precision* terbaik yang artinya model ini menghasilkan performa paling stabil serta tingkat ketepatan dalam memprediksi terbaik di antara seluruh model yang diuji. Pada kondisi *baseline* tanpa resampling, model ini memperoleh *recall* sebesar 0,8163 dan *F1-Score* sebesar 0,8743 yang merupakan nilai tertinggi pada penelitian ini. Bahkan setelah diterapkan berbagai teknik *resampling*, performa *Random Forest* tetap berada pada rentang yang relatif sama. Stabilitas tersebut menunjukkan bahwa *Random Forest* telah mampu mempelajari pola *fraud* dengan baik meskipun distribusi data masih sangat tidak seimbang. Karakteristik ini sesuai dengan teori *bagging* yang menyatakan bahwa proses *bootstrap sampling* menghasilkan variasi data *training* pada setiap pohon sehingga model lebih tahan terhadap dominasi kelas mayoritas. Selain itu, mekanisme voting antar pohon membantu mengurangi pengaruh *noise* maupun *outlier* yang sering muncul pada data transaksi keuangan. Oleh karena itu, perubahan distribusi akibat *resampling* tidak memberikan dampak yang terlalu besar terhadap performa model. Hal ini menunjukkan bahwa *Random Forest* memiliki kemampuan generalisasi yang sangat baik pada kasus *fraud* detection.

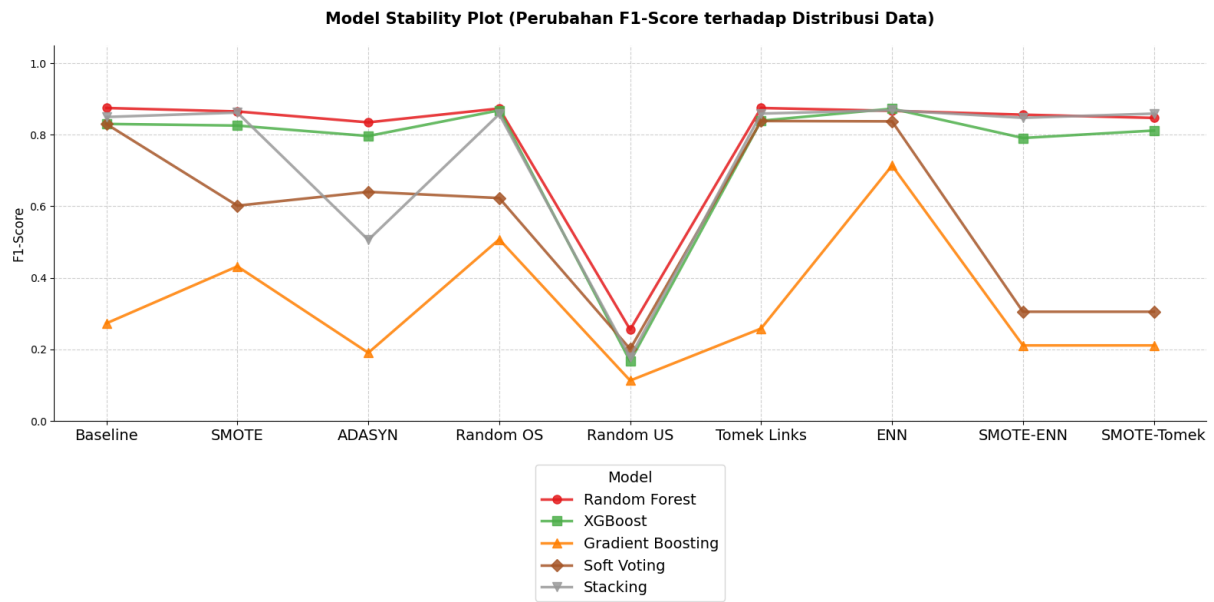
Berbeda dengan *Random Forest*, *Gradient Boosting* menunjukkan performa awal yang relatif rendah pada data tidak seimbang. Pada kondisi *baseline*, model hanya menghasilkan *recall* sebesar 0.5294 dan *F1-Score* sekitar 0,27 yang jauh di bawah model lainnya. Nilai tersebut menunjukkan bahwa model mengalami kesulitan dalam mempelajari pola *fraud* ketika distribusi data sangat didominasi oleh transaksi *non-fraud*. Fenomena tersebut dapat dijelaskan oleh mekanisme *boosting* yang membangun pohon secara berurutan dengan fokus memperbaiki kesalahan model sebelumnya. Pada dataset yang sangat tidak seimbang, sebagian besar kesalahan berasal dari kelas minoritas sehingga model cenderung bias terhadap kelas mayoritas. Namun, performa *Gradient Boosting* meningkat secara signifikan setelah diterapkan beberapa teknik *resampling*, khususnya ENN. Pada kombinasi tersebut *F1-Score* meningkat menjadi 0.7135 dan *recall* menjadi 0.6224. Walaupun hasil tersebut belum optimal, peningkatan tersebut menunjukkan bahwa kualitas data *training* memiliki pengaruh yang sangat besar terhadap performa *Gradient Boosting*. Ketika *noise* dan data ambigu dihapus melalui ENN, model mampu membentuk batas keputusan yang lebih jelas sehingga kemampuan deteksi *fraud* meningkat secara signifikan.

Untuk XGBoost yang merupakan pengembangan dari *Gradient Boosting* dengan mengintegrasikan regularisasi, optimisasi berbasis gradient, serta mekanisme penanganan *missing value* secara otomatis. Hasil eksperimen menunjukkan bahwa XGBoost memiliki performa yang sangat kompetitif pada seluruh teknik *resampling*. Model ini secara konsisten menghasilkan *recall* tinggi tanpa mengalami penurunan *precision* yang terlalu besar. XGBoost memperoleh peningkatan performa yang cukup signifikan ketika dikombinasikan dengan teknik pembersihan data seperti ENN dan *Tomek Links*. Kombinasi terbaik diperoleh pada penggunaan ENN dengan *F1-Score* sebesar 0,8723 dan *recall* sebesar 0.8367. Nilai tersebut

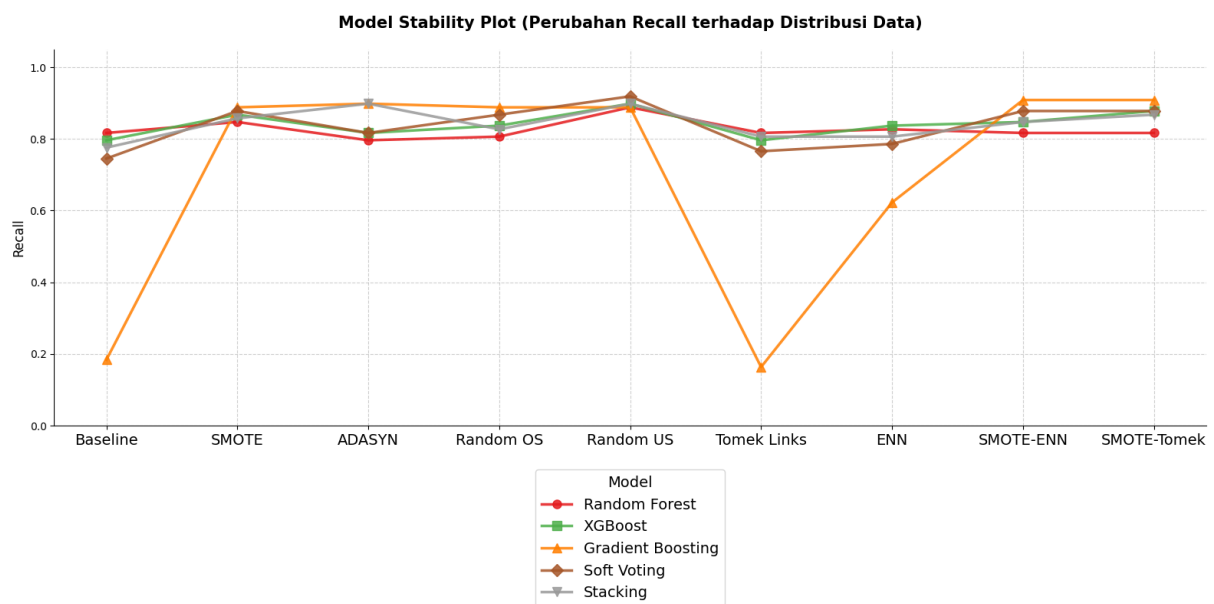
sangat dekat dengan performa terbaik yaitu Random Forest dan menunjukkan bahwa XGBoost lebih diuntungkan oleh data yang bersih dibandingkan data yang hanya seimbang secara kuantitas. Keunggulan XGBoost berasal dari kemampuannya memfokuskan pembelajaran pada sampel yang sulit diklasifikasikan melalui optimisasi fungsi *loss*. Pada kasus *fraud detection*, mekanisme tersebut memungkinkan model memberikan perhatian lebih besar terhadap transaksi *fraud* yang jumlahnya sangat sedikit. Selain itu, regularisasi yang diterapkan dalam XGBoost membantu mengurangi risiko *overfitting* akibat penambahan data sintetis pada teknik *oversampling* seperti SMOTE dan ADASYN. Penelitian ini menunjukkan bahwa XGBoost merupakan model *boosting* yang paling efektif untuk mendeteksi *fraud* pada dataset yang tidak seimbang.

Model selanjutnya adalah *Stacking* yang merupakan metode *ensemble* heterogen yang menggabungkan prediksi beberapa model dasar melalui *meta-learner*. *Stacking* memperoleh nilai rata-rata recall tertinggi yang artinya dalam mendeteksi data *fraud*, model ini mempunyai performa terbaik. Pada penelitian ini, *Stacking* mengombinasikan *Random Forest*, XGBoost, dan *Gradient Boosting* sebagai *base-learner* dan *Logistic Regression* sebagai *meta-learner*. Keunggulan utama *Stacking* terletak pada kemampuannya memanfaatkan kekuatan berbagai algoritma sekaligus dan mengurangi kelemahan individual yang dimiliki setiap algoritma. Apabila satu model gagal mengenali pola tertentu, model lain dapat memberikan informasi tambahan yang digunakan oleh *meta-learner* untuk menghasilkan keputusan akhir. Hasil pengujian menunjukkan bahwa performa *Stacking* meningkat secara signifikan setelah diterapkan teknik *resampling*, terutama SMOTE dan ENN. Pada kombinasi ENN, model menghasilkan *recall* sebesar 0.8061 dan *F1-Score* sebesar 0,8681 yang termasuk dalam lima besar kombinasi terbaik dalam penelitian ini berdasarkan *F1-Score*. Pada kombinasi dengan SMOTE, nilai *recall* menunjukkan kenaikan dari 0,7755 pada *baseline* menjadi 0.8571. Performa tersebut menunjukkan bahwa *keberagaman* antar *base learner* berhasil meningkatkan kemampuan deteksi *fraud*. *Meta-learner* mampu memanfaatkan informasi yang diperoleh dari berbagai model sehingga menghasilkan keputusan yang lebih akurat dibandingkan sebagian model individual. Temuan ini mendukung teori *ensemble learning* yang menyatakan bahwa kombinasi beberapa model yang berbeda dapat meningkatkan kemampuan generalisasi sistem klasifikasi.

Terakhir, ada model *Soft Voting* yang bekerja dengan menggabungkan probabilitas prediksi dari beberapa *classifier* kemudian memilih kelas dengan probabilitas rata-rata tertinggi. Secara teoritis, pendekatan ini mampu meningkatkan stabilitas prediksi. Namun hasil eksperimen menunjukkan bahwa *Soft Voting* menghasilkan performa yang kedua paling rendah dibandingkan model lainnya, hanya mengungguli *Gradient Boosting*. Kemungkinan penyebab hal tersebut adalah sensitivitas terhadap distribusi probabilitas yang dihasilkan masing-masing *classifier*. Ketika salah satu model menghasilkan probabilitas yang bias akibat perubahan distribusi data hasil *resampling*, maka probabilitas rata-rata yang digunakan *Soft Voting* juga ikut terpengaruh. Akibatnya, *recall* memang meningkat pada beberapa teknik *resampling*, namun *precision* mengalami penurunan yang sangat besar sehingga *F1-Score* tetap rendah. Hal tersebut menunjukkan bahwa agregasi probabilitas sederhana belum mampu memanfaatkan informasi antar model secara optimal pada kasus *fraud detection* yang datanya sangat tidak seimbang serta menunjukkan *Soft Voting* kurang *robust* dibandingkan *Random Forest*, XGBoost, maupun *Stacking* dalam menghadapi perubahan distribusi kelas. Namun, *Soft Voting* masih mendapat performa yang cukup baik jika dipasangkan dengan *Tomek Links* dan ENN dengan nilai *F1-Score* sebesar 0.8380 dan 0.8370.



Gambar 4.18 Stability Plot Model Berdasarkan F1-Score



Gambar 4.19 Stability Plot Model Berdasarkan Recall

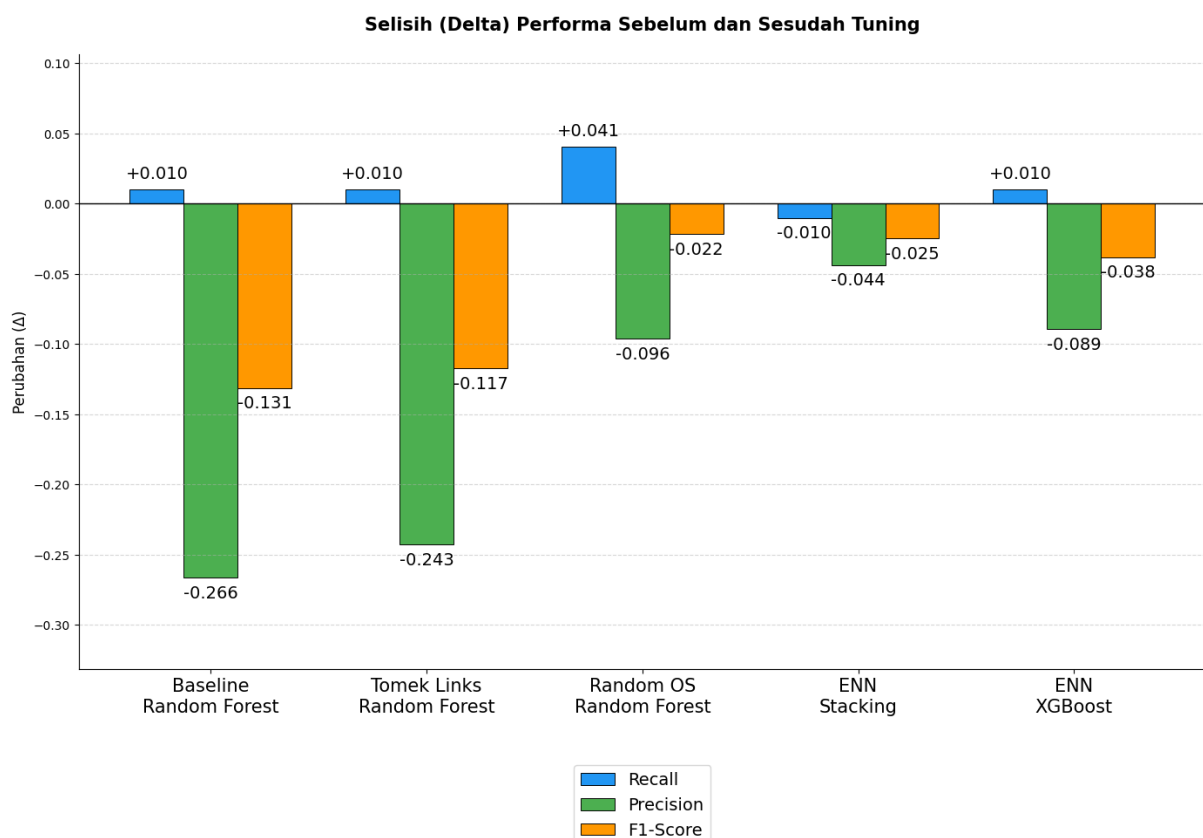
Berdasarkan **Gambar 4.18** dan **4.19**, Random Forest menunjukkan fluktuasi performa yang paling kecil. Hal ini menunjukkan bahwa model tersebut memiliki kemampuan generalisasi yang baik dan tidak terlalu bergantung pada teknik penyeimbangan data tertentu. Sebaliknya, *Gradient Boosting* menunjukkan fluktuasi terbesar yang mengindikasikan sensitivitas tinggi terhadap perubahan distribusi data. Untuk implementasi pada kondisi nyata, model yang stabil lebih diinginkan karena memiliki risiko penurunan performa yang lebih kecil ketika dihadapkan pada perubahan karakteristik data di masa mendatang.

Hasil penelitian menunjukkan bahwa tidak terdapat satu model yang unggul mutlak pada seluruh kondisi. Namun berdasarkan konsistensi performa, keseimbangan *recall* dan *precision*,

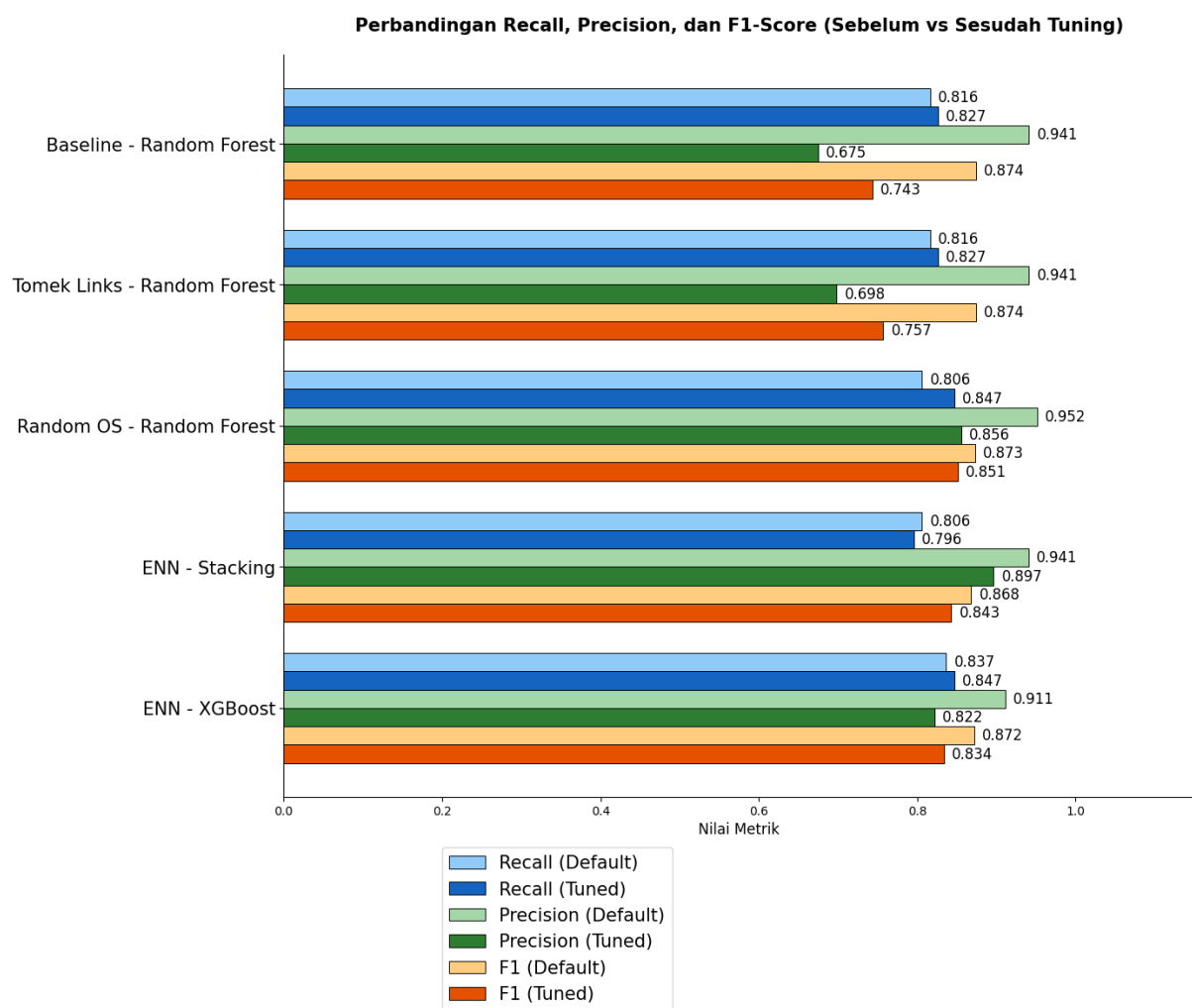
serta kestabilan terhadap berbagai teknik *resampling*, Random Forest dan XGBoost merupakan model yang paling menjanjikan. Random Forest unggul dari sisi ketahanan dan kestabilan performa, sedangkan XGBoost menunjukkan kemampuan yang sangat baik dalam memanfaatkan data hasil pembersihan seperti ENN. Stacking juga menunjukkan performa kompetitif dengan kemampuan mendeteksi data *fraud* terbaik namun peforma ketepatan dalam memprediksi masih lebih buruk dibanding Random Forest dan XGBoost sehingga nilai *F1-Score* sedikit lebih buruk. Namun, *Stacking* masih memiliki potensi peningkatan lebih lanjut.

4.2.4 Pengaruh Hyperparameter Tuning terhadap Kinerja Model Ensemble Learning

Setelah diperoleh lima kombinasi model dan teknik *resampling* terbaik berdasarkan nilai *F1-Score* pada tahap sebelumnya, dilakukan proses *hyperparameter* tuning menggunakan *Randomized Search Cross Validation*. Tujuan dari tahap ini adalah mengetahui pengaruh *hyperparameter* tuning terhadap performa model dan mencari konfigurasi parameter yang mampu menghasilkan performa klasifikasi yang lebih optimal dibandingkan konfigurasi *default*. *Hyperparameter* tuning berfungsi untuk menyesuaikan kompleksitas model dengan karakteristik data sehingga model dapat menghasilkan kemampuan generalisasi yang lebih baik. Pada penelitian ini, proses tuning dilakukan dengan mempertimbangkan kebutuhan utama sistem *fraud detection*, yaitu memaksimalkan kemampuan model dalam mendeteksi transaksi *fraud*. Oleh karena itu, metrik *recall* menjadi fokus utama dalam evaluasi karena kesalahan berupa transaksi *fraud* yang tidak terdeteksi atau *false negative* memiliki risiko yang lebih besar dibandingkan transaksi normal yang salah ditandai sebagai *fraud* atau *false positive*.



Gambar 4.20 Selisih Nilai Sebelum dan Sesudah Tuning



Gambar 4.21 Perbandingan Sebelum dan Sesudah Tuning

Berdasarkan gambar tersebut, tidak ditemukan kombinasi yang mengalami peningkatan nilai *F1-Score* setelah proses tuning. Penurunan terbesar terjadi pada kombinasi *Random Forest* tanpa *resampling* dengan selisih sebesar 0,1312. Sebaliknya, penurunan terkecil terjadi pada kombinasi *Random Forest* dengan *Random Oversampling* sebesar 0,0216. Namun, dari perspektif deteksi *fraud*, hasil ini tidak menunjukkan bahwa tuning gagal meningkatkan kemampuan model. Pada beberapa kombinasi, *recall* mengalami peningkatan setelah tuning. Hal ini menunjukkan bahwa model menjadi lebih mampu mengenali transaksi *fraud*, meskipun peningkatan tersebut diikuti *trade-off* bertambahnya jumlah *false positive* yang menyebabkan *precision* dan *F1-Score* menurun. Temuan ini menunjukkan bahwa konfigurasi *default* yang digunakan pada beberapa model sebenarnya telah berada pada kondisi yang mendekati optimum untuk karakteristik dataset *fraud detection* yang digunakan. Ruang perbaikan yang dapat dicapai melalui tuning relatif terbatas sehingga perubahan parameter justru menyebabkan penurunan keseimbangan antara *recall* dan *precision*.

Pada kombinasi *Random Forest-baseline*, *recall* meningkat dari 0,8163 menjadi 0,8265 setelah dilakukan tuning. Akan tetapi, peningkatan tersebut diikuti penurunan *precision* yang sangat signifikan dari 0,9412 menjadi 0,6750. Akibatnya, nilai *F1-Score* turun dari 0,8743 menjadi 0,7431. Fenomena yang sama juga terjadi pada *Random Forest* dengan *Tomek Links*.

Recall meningkat dari 0,8163 menjadi 0,8265, namun *precision* turun dari 0,9412 menjadi 0,6983 sehingga menghasilkan penurunan *F1-Score* sebesar 0,1173 poin. Hasil tersebut menunjukkan bahwa model menjadi lebih agresif dalam mengklasifikasikan transaksi sebagai *fraud*. Dari sudut pandang *fraud detection*, kondisi ini memiliki keuntungan karena lebih banyak transaksi *fraud* berhasil ditemukan. Namun, konsekuensinya adalah meningkatnya jumlah transaksi normal yang salah diklasifikasikan sebagai *fraud* sehingga nilai *precision* menurun.

Random Forest merupakan model dengan performa terbaik pada tahap pengujian awal. Nilai *F1-Score* sebesar 0,8743 pada model *baseline* dan kombinasi *Tomek Links* merupakan nilai tertinggi yang diperoleh selama eksperimen. Kinerja yang tinggi sebelum tuning menunjukkan bahwa parameter *default Random Forest* sudah mampu menghasilkan keseimbangan yang baik antara bias dan varians. Ketika proses tuning dilakukan, model cenderung menghasilkan konfigurasi yang lebih sensitif terhadap kelas minoritas sehingga *recall* meningkat, tetapi kemampuan mempertahankan *precision* menurun. Hal ini sejalan dengan teori ensemble bagging yang menyatakan bahwa *Random Forest* secara alami memiliki kemampuan generalisasi yang tinggi melalui mekanisme *bootstrap aggregation* sehingga sering kali tidak memerlukan optimasi parameter yang terlalu agresif untuk menghasilkan performa yang baik.

Pada kombinasi XGBoost dengan ENN, *recall* meningkat dari 0,8367 menjadi 0,8469. Akan tetapi polanya tetap sama dimana *precision* mengalami penurunan dari 0,9111 menjadi 0,8218 sehingga *F1-Score* turun dari 0,8723 menjadi 0,8342. Peningkatan *recall* tersebut menunjukkan bahwa tuning berhasil meningkatkan sensitivitas model terhadap pola transaksi *fraud*. Hal tersebut menandakan lebih banyak transaksi *fraud* yang berhasil dikenali oleh model setelah dilakukan optimasi parameter. Penurunan *precision* menunjukkan bahwa konfigurasi hasil tuning menghasilkan *decision boundary* yang lebih longgar. Model menjadi lebih mudah mengklasifikasikan suatu transaksi sebagai *fraud* sehingga jumlah *fraud* yang terdeteksi meningkat, namun pada saat yang sama jumlah *false positive* juga bertambah. Meskipun terjadi penurunan *F1-Score*, nilai *recall* hasil tuning merupakan salah satu *recall* tertinggi di antara kombinasi model terbaik yang diuji. Karena fokus utama penelitian adalah meminimalkan jumlah *fraud* yang tidak terdeteksi, hasil ini menunjukkan bahwa tuning pada XGBoost memberikan manfaat dalam meningkatkan kemampuan deteksi *fraud* meskipun keseimbangan antara *recall* dan *precision* menjadi sedikit berkurang.

Pada kombinasi *Stacking* dengan ENN, nilai *recall* menurun dari 0,8061 menjadi 0,7959 dan *precision* menurun dari 0,9405 menjadi 0,8966. Penurunan kedua metrik tersebut menyebabkan *F1-Score* turun dari 0,8681 menjadi 0,8432. Berbeda dengan *Random Forest* dan XGBoost yang masih menunjukkan peningkatan *recall*, tuning pada model *Stacking* justru tidak mampu meningkatkan kemampuan deteksi *fraud* maupun ketepatan klasifikasi *fraud*. Proses tuning pada kombinasi ini tidak memberikan keuntungan baik dari sisi *recall* maupun *F1-Score*. Hal ini dapat terjadi karena *Stacking* merupakan model *ensemble* bertingkat yang telah mengombinasikan informasi dari beberapa *base learner*. Struktur tersebut pada dasarnya sudah menghasilkan representasi data yang kompleks sehingga perubahan parameter pada salah satu komponen dapat mengganggu keseimbangan yang telah terbentuk sebelumnya. Performa *Stacking* lebih dipengaruhi oleh kualitas dan keberagaman model penyusun dibandingkan perubahan *hyperparameter* individual.

Dari hasil-hasil tersebut, dapat diketahui bahwa *hyperparameter tuning* tidak selalu menjamin peningkatan performa model. Terdapat beberapa faktor yang dapat menjelaskan hal tersebut. Yang pertama adalah dataset yang digunakan telah melalui proses *resampling* dan *preprocessing* sehingga kualitas data yang diberikan kepada model sudah cukup baik. Dalam kondisi tersebut, parameter *default* dari algoritma *ensemble* modern sering kali sudah mampu menghasilkan performa yang mendekati optimum. Kemudian ruang pencarian parameter pada *Randomized Search* bersifat acak sehingga kombinasi parameter terbaik belum tentu ditemukan. Parameter yang dipilih merupakan kombinasi terbaik dari sampel parameter yang diuji, bukan dari seluruh kemungkinan kombinasi yang tersedia. Lalu karakteristik data *fraud detection* yang sangat tidak seimbang menyebabkan model cenderung sensitif terhadap perubahan parameter. Sedikit perubahan pada kedalaman pohon, jumlah estimator, atau *learning rate* dapat menghilangkan keseimbangan antara *recall* dan *precision* secara signifikan. Selanjutnya, proses tuning yang berorientasi pada peningkatan kemampuan deteksi *fraud* dapat menghasilkan model yang lebih sensitif terhadap kelas minoritas. Akibatnya, *recall* meningkat tetapi *precision* menurun karena jumlah *false positive* bertambah. Kondisi ini terlihat pada beberapa kombinasi model yang berhasil meningkatkan *recall* namun mengalami penurunan *F1-Score*. Terakhir, proses tuning dilakukan dengan menggunakan fungsi objektif yang berfokus pada performa *cross validation*. Kondisi terbaik pada data *testing* belum tentu identik dengan kondisi terbaik pada data *training* sehingga dapat terjadi penurunan performa ketika model diterapkan pada data yang belum pernah dilihat sebelumnya.

Dalam konteks industri perbankan, peningkatan *false positive* memang dapat menimbulkan biaya operasional tambahan berupa investigasi transaksi yang sebenarnya tidak bermasalah. Akan tetapi, dibandingkan risiko kerugian akibat transaksi *fraud* yang lolos dari deteksi, peningkatan *recall* sering kali dianggap lebih penting. Maka dari itu, hasil tuning yang menghasilkan *recall* lebih tinggi tetap memiliki nilai praktis meskipun *F1-Score* mengalami penurunan. Oleh karena itu, pemilihan model terbaik perlu mempertimbangkan kebutuhan implementasi. Jika prioritas utama adalah menjaga keseimbangan antara *recall* dan *precision*, maka konfigurasi *default* lebih sesuai. Namun, jika prioritas utama adalah meminimalkan jumlah *fraud* yang lolos dari deteksi, maka konfigurasi hasil tuning yang menghasilkan *recall* lebih tinggi dapat menjadi alternatif yang dapat dipertimbangkan.

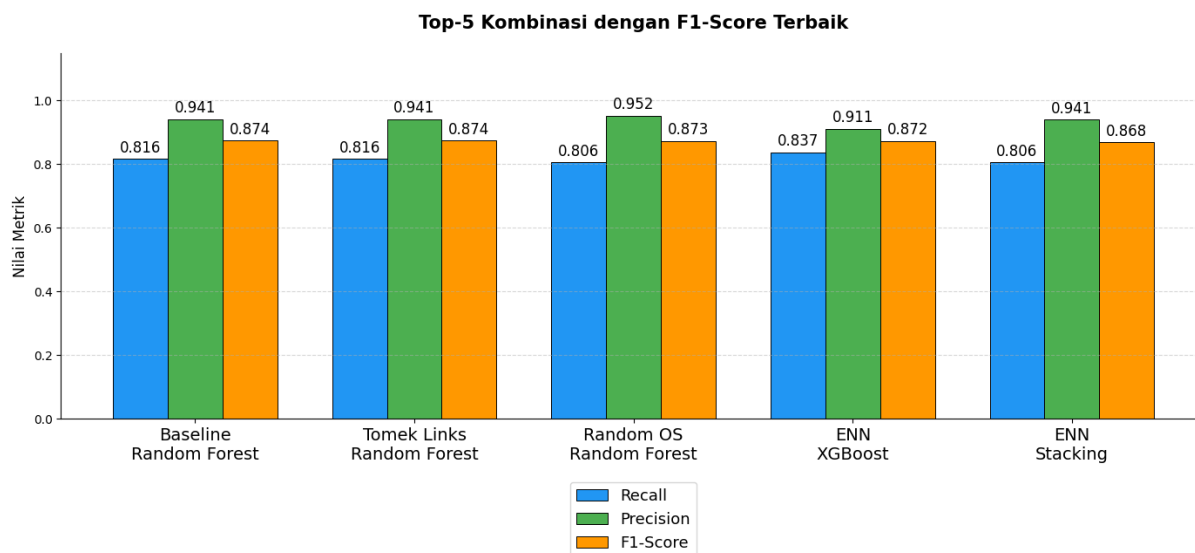
Secara keseluruhan, hasil penelitian mengindikasikan bahwa pemilihan teknik *resampling* yang tepat memiliki pengaruh yang lebih besar terhadap peningkatan performa dibandingkan proses tuning yang dilakukan setelahnya. Keberhasilan sistem deteksi *fraud* pada penelitian ini lebih banyak ditentukan oleh kemampuan model dalam menangani ketidakseimbangan kelas dibandingkan optimasi parameter lanjutan.

4.2.5 Analisis Kombinasi Model dan Teknik Resampling Terbaik

Setelah dilakukan serangkaian pengujian terhadap berbagai kombinasi model *ensemble learning* dan teknik *resampling*, langkah selanjutnya adalah mengidentifikasi kombinasi yang menghasilkan performa terbaik dalam mendeteksi transaksi *fraud*. Analisis ini bertujuan untuk menentukan konfigurasi yang paling efektif berdasarkan hasil evaluasi yang telah dilakukan serta memahami faktor-faktor yang menyebabkan kombinasi tersebut mampu menghasilkan performa yang lebih unggul dibandingkan kombinasi lainnya.

Pada penelitian *fraud detection*, kemampuan model dalam mendeteksi transaksi *fraud* merupakan aspek yang paling penting karena kegagalan mendeteksi transaksi *fraud* atau *false*

negative dapat menimbulkan kerugian yang lebih besar dibandingkan kesalahan mengklasifikasikan transaksi normal sebagai *fraud* atau false positive. Oleh karena itu, metrik *recall* menjadi fokus utama dalam penelitian ini. Namun, *recall* yang tinggi juga harus diimbangi dengan *precision* yang baik agar model tidak menghasilkan terlalu banyak *false positive*. Kombinasi terbaik adalah kombinasi yang mampu mencapai *recall* setinggi mungkin tanpa mengorbankan *precision* secara signifikan. *F1-Score* tetap digunakan sebagai metrik pendukung karena mampu menggambarkan keseimbangan antara *recall* dan *precision*. Berdasarkan hasil pengujian seluruh kombinasi model dan teknik resampling, diperoleh lima kombinasi dengan nilai *F1-Score* tertinggi sebagaimana ditunjukkan pada **Gambar 4.22**.



Gambar 4.22 Kombinasi dengan F1-Score Terbaik

Jika hanya dilihat berdasarkan *F1-Score*, *Random Forest-Baseline* dan *Random Forest-Tomek Links* merupakan kombinasi terbaik. Namun, karena fokus utama penelitian ini adalah memaksimalkan *sensitivitas* model atau kemampuan deteksi *fraud*, nilai *recall* perlu menjadi pertimbangan utama dalam menentukan kombinasi yang paling efektif. Dari lima kombinasi terbaik tersebut, XGBoost-ENN menghasilkan *recall* tertinggi yaitu sebesar 0,8367. Nilai ini lebih tinggi dibandingkan *recall* yang diperoleh *Random Forest-Baseline* maupun *Random Forest-Tomek Links* yang masing-masing sebesar 0,8163. Meskipun *precision* XGBoost-ENN sedikit lebih rendah yaitu 0,9111, nilainya masih tergolong sangat baik dan tidak menunjukkan penurunan yang drastis. Hal ini menunjukkan bahwa apabila prioritas utama adalah mendeteksi sebanyak mungkin transaksi *fraud* tanpa mengorbankan *precision* secara berlebihan, maka kombinasi XGBoost-ENN dapat dianggap sebagai kombinasi yang paling efektif. Sementara itu, *Random Forest-Baseline* dan *Random Forest-Tomek Links* tetap menjadi kombinasi dengan keseimbangan *recall* dan *precision* terbaik berdasarkan *F1-Score*. Selain itu, hasil tersebut menunjukkan bahwa performa terbaik tidak selalu diperoleh dari teknik *resampling* yang menghasilkan distribusi data paling seimbang. Sebaliknya, kombinasi terbaik justru diperoleh dari teknik yang mampu mempertahankan karakteristik asli data sambil mengurangi *noise* dan sampel ambigu yang berpotensi mengganggu proses pembelajaran model.

Random Forest merupakan model yang paling konsisten menghasilkan performa tinggi pada hampir seluruh teknik *resampling* yang diuji. Bahkan tanpa penerapan teknik *resampling*, model ini mampu memperoleh *F1-Score* sebesar 0,8743 yang menjadi salah satu nilai tertinggi

pada penelitian ini. Keunggulan Random Forest dapat dijelaskan melalui mekanisme *bagging* yang digunakan. Random Forest membangun banyak pohon keputusan menggunakan sampel *bootstrap* yang berbeda-beda kemudian menggabungkan hasil prediksi seluruh pohon melalui mekanisme voting. Pendekatan ini menghasilkan model yang memiliki varians rendah dan kemampuan generalisasi yang baik. Pada dataset *Credit Card Fraud Detection*, sebagian besar fitur telah mengalami transformasi PCA sehingga pola antar kelas relatif terpisah dengan baik pada ruang fitur. Kondisi tersebut memungkinkan *Random Forest* membangun batas keputusan yang efektif tanpa memerlukan penyesuaian distribusi data yang ekstrem. Hal ini terlihat dari performa *Random Forest* pada kondisi *baseline* mampu mengungguli sebagian besar kombinasi yang menggunakan teknik *resampling*. Temuan tersebut menunjukkan bahwa pada dataset ini, kemampuan model dalam memanfaatkan struktur data yang tersedia lebih berpengaruh dibandingkan proses penyeimbangan distribusi kelas secara agresif. Meskipun, jika fokus utama diarahkan pada Recall, Random Forest bukanlah kombinasi dengan kemampuan deteksi fraud tertinggi.

Kombinasi *Random Forest* dengan *Tomek Links* menghasilkan nilai *F1-Score* yang identik dengan model *baseline*, yaitu sebesar 0,8743. *Tomek Links* hanya menghapus sebagian kecil sampel mayoritas yang berada pada area tumpang tindih antar kelas. Berdasarkan hasil distribusi data, jumlah data *non-fraud* hanya berkurang dari 227.451 menjadi 227.434 sampel. Hasil tersebut menunjukkan bahwa keberadaan *noise* pada dataset relatif rendah. Distribusi data asli sudah cukup representatif sehingga penghapusan sejumlah kecil sampel ambigu tidak memberikan perubahan performa yang signifikan. Namun, keberhasilan *Tomek Links* mempertahankan performa yang sama dengan *baseline* menunjukkan bahwa teknik ini mampu melakukan pembersihan data tanpa merusak distribusi asli dataset.

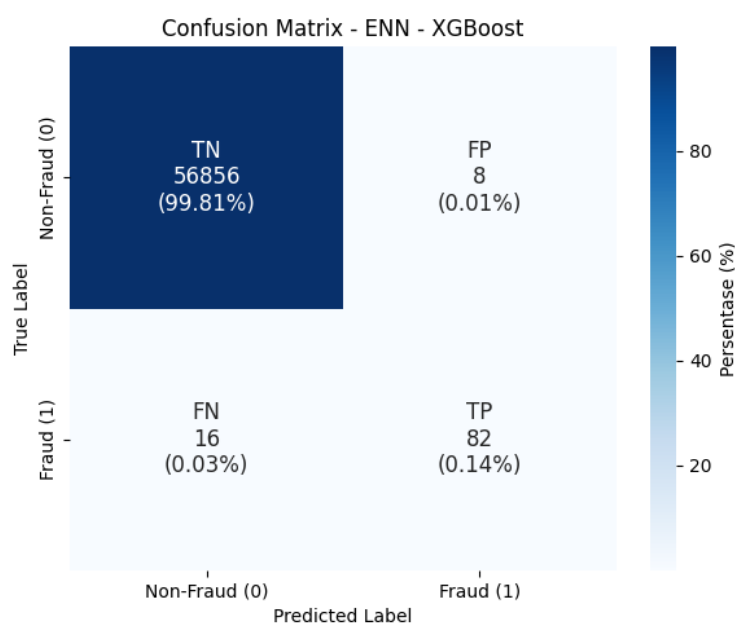
Kombinasi *Random Forest* dengan *Random Oversampling* memperoleh *F1-Score* sebesar 0,8729 dengan *precision* tertinggi sebesar 0,9518. Nilai *Precision* tersebut merupakan yang tertinggi di antara seluruh kombinasi yang diuji. Hal ini menunjukkan bahwa *Random Oversampling* berhasil meningkatkan representasi kelas *fraud* tanpa menghasilkan peningkatan *false positive* yang berlebihan. Berbeda dengan SMOTE dan ADASYN yang menghasilkan data sintesis, *Random Oversampling* hanya melakukan duplikasi terhadap sampel *fraud* yang telah ada sehingga risiko pembentukan pola sintesis yang tidak representatif tidak ada pada penelitian ini. Namun, Recall yang diperoleh hanya sebesar 0,8061, lebih rendah dibandingkan beberapa kombinasi lainnya. Oleh karena itu, meskipun kombinasi ini sangat baik dalam menjaga *precision*, kemampuannya dalam mendeteksi fraud tidak menjadi yang terbaik.

Dua kombinasi terbaik berikutnya diperoleh oleh XGBoost-ENN dan *Stacking*-ENN dengan nilai *F1-Score* masing-masing sebesar 0,8723 dan 0,8681. Berbeda dengan *Tomek Links* yang hanya menghapus pasangan sampel ambigu, ENN melakukan proses pembersihan yang lebih agresif dengan menghapus sampel yang tidak sesuai dengan mayoritas tetangga terdekatnya. Hasil penelitian menunjukkan bahwa pendekatan tersebut sangat efektif ketika dikombinasikan dengan model berbasis *boosting* dan *stacking*. Pada XGBoost, pembersihan data oleh ENN membantu proses *boosting* untuk lebih fokus mempelajari pola *fraud* yang benar-benar representatif. Hal ini tercermin dari nilai recall sebesar 0,8367 yang merupakan nilai tertinggi di antara lima kombinasi terbaik, dengan *precision* yang tetap tinggi yaitu sebesar 0,9111. Untuk konteks *fraud detection*, hasil ini sangat penting karena menunjukkan bahwa model mampu mendeteksi lebih banyak transaksi *fraud* tanpa mengalami penurunan ketepatan prediksi yang terlalu besar. Sementara itu, pada *Stacking*, data yang lebih bersih memungkinkan

base learner menghasilkan prediksi yang lebih akurat sehingga *meta learner* dapat membangun keputusan akhir yang lebih efektif. Temuan ini menunjukkan bahwa teknik pembersihan data lebih bermanfaat dibandingkan teknik penyeimbangan distribusi pada model yang memiliki sensitivitas tinggi terhadap *noise*.

Jika dianalisis secara keseluruhan, terdapat beberapa karakteristik yang dimiliki oleh kombinasi terbaik pada penelitian ini. Pertama, seluruh kombinasi terbaik mampu mempertahankan *precision* di atas 0,90. Hal ini menunjukkan bahwa model-model tersebut tetap mampu menjaga tingkat *false positive* pada level yang rendah. Kedua, seluruh kombinasi terbaik memiliki *recall* pada rentang 0,80–0,84. Nilai tersebut menunjukkan kemampuan deteksi *fraud* yang cukup tinggi sekaligus tetap mempertahankan *precision* yang baik. Ketiga, kombinasi yang paling menarik dari perspektif *fraud detection* adalah kombinasi yang mampu meningkatkan *recall* tanpa menyebabkan penurunan *precision* yang drastis. Dalam hal ini, XGBoost-ENN menunjukkan karakteristik yang paling sesuai karena menghasilkan *recall* tertinggi sebesar 0,8367 dengan *Precision* yang masih mencapai 0,9111.

Berdasarkan seluruh hasil eksperimen, terdapat dua sudut pandang dalam menentukan kombinasi terbaik. Jika penilaian dilakukan berdasarkan keseimbangan *recall* dan *precision* yang direpresentasikan oleh *F1-Score*, maka *Random Forest* tanpa *resampling* dan *Random Forest* dengan *Tomek Links* merupakan kombinasi terbaik dengan nilai *F1-Score* sebesar 0,8743. Jika prioritasnya adalah memaksimalkan kemampuan deteksi *fraud* tanpa memikirkan ketepatan prediksi, maka kombinasi *Random Undersampling-Soft Voting* merupakan pilihannya dengan nilai *recall* sebesar 0.9184. Namun, jika prioritas utama adalah memaksimalkan kemampuan deteksi *fraud* dengan tetap mempertahankan *precision* pada tingkat yang baik, maka kombinasi XGBoost-ENN merupakan pilihan yang lebih unggul. Kombinasi ini menghasilkan *recall* tertinggi sebesar 0,8367 dengan *Precision* sebesar 0,9111, sehingga mampu mendeteksi banyak transaksi *fraud* tanpa menghasilkan peningkatan *false positive* yang berlebihan. Oleh karena itu, dapat disimpulkan bahwa kombinasi model dan teknik *resampling* yang paling efektif untuk tujuan *fraud detection* pada penelitian ini adalah XGBoost-ENN karena menghasilkan *recall* tertinggi dengan *precision* yang tetap tinggi.



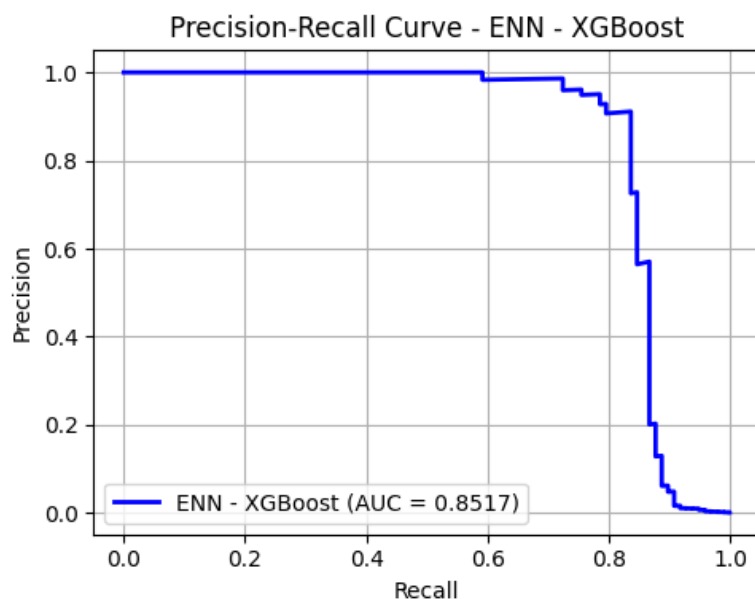
Gambar 4.23 Confusion Matrix Kombinasi Terbaik

Berdasarkan confusion matrix yang ditunjukkan pada **Gambar 4.23**, kombinasi XGBoost-ENN berhasil mengklasifikasikan 56.856 transaksi *non-fraud* sebagai *non-fraud* (*True Negative*) dan 82 transaksi *fraud* sebagai *fraud* (*True Positive*). Sementara itu, terdapat 8 transaksi *non-fraud* yang salah diklasifikasikan sebagai *fraud* (*False Positive*) dan 16 transaksi *fraud* yang salah diklasifikasikan sebagai *non-fraud* (*False Negative*).

Jumlah *false positive* yang sangat rendah menunjukkan bahwa model memiliki kemampuan yang baik dalam membedakan transaksi *non-fraud* dan transaksi *fraud*. *False positive* umumnya terjadi ketika karakteristik suatu transaksi *non-fraud* memiliki pola yang menyerupai transaksi *fraud* sehingga model memberikan prediksi positif pada transaksi yang sebenarnya *non-fraud*. Kondisi ini dapat disebabkan oleh adanya transaksi *non-fraud* yang memiliki nilai atribut yang berada di sekitar batas keputusan (*decision boundary*) antara kedua kelas. Meskipun demikian, jumlah *false positive* yang hanya mencapai 8 transaksi dari lebih dari 56 ribu transaksi *non-fraud* menunjukkan bahwa proses pembersihan data menggunakan ENN berhasil mengurangi keberadaan sampel ambigu dan *noise* pada data *training*. ENN bekerja dengan menghapus sampel yang tidak sesuai dengan mayoritas tetangga terdekatnya sehingga batas antar kelas menjadi lebih jelas. Akibatnya, XGBoost dapat membangun struktur pohon keputusan yang lebih representatif dan tidak mudah menganggap transaksi normal sebagai *fraud*. Nilai *precision* sebesar 0,9111 memperkuat temuan tersebut karena lebih dari 91% transaksi yang diprediksi *fraud* memang benar merupakan *fraud*.

Di sisi lain, masih terdapat 16 transaksi *fraud* yang diklasifikasikan sebagai *non-fraud* atau *false negative*. *False negative* umumnya terjadi akibat adanya karakteristik transaksi *fraud* yang sangat mirip dengan transaksi *non-fraud* sehingga model kesulitan menemukan pola pembeda yang cukup kuat. Meskipun ENN mampu mengurangi *noise* pada data, teknik tersebut tidak menambah jumlah sampel *fraud* seperti metode *oversampling*. Akibatnya, variasi pola *fraud* yang sangat beragam belum tentu seluruhnya terwakili dalam data *training*. Selain itu, sifat *fraud* yang terus berkembang (*evolving fraud pattern*) menyebabkan sebagian transaksi *fraud* memiliki karakteristik yang sengaja dirancang menyerupai transaksi *non-fraud* untuk menghindari sistem deteksi. Pada kondisi tersebut, model cenderung memberikan prediksi *non-fraud* karena probabilitas yang dihasilkan belum cukup tinggi untuk melewati ambang keputusan kelas *fraud*.

Meskipun demikian, model tetap mampu mendeteksi 82 dari 98 transaksi *fraud*, yang menghasilkan *recall* sebesar 83,67%. Hasil ini menunjukkan bahwa sebagian besar transaksi *fraud* berhasil dikenali dengan tetap mempertahankan *precision* yang tinggi. Kinerja tersebut menunjukkan adanya kerja sama antara ENN yang meningkatkan kualitas data melalui penghapusan *noise* dan XGBoost yang mampu mempelajari pola kompleks pada data. Oleh karena itu, distribusi kesalahan pada *confusion matrix* memperlihatkan bahwa kombinasi XGBoost dan ENN mampu memberikan keseimbangan yang baik antara meminimalkan *fraud* yang tidak terdeteksi dan menghindari *false positive* yang berlebihan, sehingga sesuai untuk diterapkan pada sistem deteksi fraud yang memprioritaskan sensitivitas terhadap kelas *fraud*.



Gambar 4.24 Grafik PR-AUC Kombinasi Terbaik

Untuk melengkapi evaluasi performa model, penelitian ini juga menggunakan kurva *Precision-Recall* dan nilai *Precision-Recall Area Under Curve* (PR-AUC). Berdasarkan **Gambar 4.24**, kombinasi ENN-XGBoost menghasilkan nilai PR-AUC sebesar 0,8517, yang menunjukkan kemampuan model dalam mempertahankan keseimbangan antara *precision* dan *recall* pada berbagai nilai *threshold*. Kurva memperlihatkan bahwa nilai *precision* tetap berada pada kisaran yang sangat tinggi ketika *recall* meningkat hingga sekitar 0,75, yang mengindikasikan bahwa sebagian besar transaksi yang diprediksi sebagai *fraud* benar-benar merupakan transaksi *fraud* meskipun model berhasil mendeteksi sebagian besar kasus positif. Setelah melewati titik tersebut, nilai *precision* mengalami penurunan yang cukup tajam seiring meningkatnya *recall*. Kondisi ini menunjukkan adanya *trade-off*, yaitu untuk mendeteksi lebih banyak transaksi *fraud*, model mulai menghasilkan lebih banyak *false positive*. Meskipun demikian, luas area di bawah kurva yang relatif tinggi menunjukkan bahwa kombinasi ENN-XGBoost mampu mempertahankan performa klasifikasi yang baik pada berbagai nilai *threshold*. Hasil evaluasi menggunakan PR-AUC juga memperkuat analisis pada *confusion matrix* sebelumnya. Nilai PR-AUC sebesar 0,8517 menunjukkan bahwa model tidak hanya memiliki kemampuan yang baik dalam mengidentifikasi transaksi *fraud* berdasarkan *threshold* tertentu, tetapi juga tetap mampu menjaga keseimbangan antara *precision* dan *recall* pada berbagai *threshold* klasifikasi. Hal ini mengindikasikan bahwa kombinasi ENN-XGBoost memiliki kemampuan generalisasi yang baik terhadap kelas *fraud* pada dataset yang sangat tidak seimbang, sehingga semakin memperkuat alasan pemilihannya sebagai kombinasi model terbaik dalam penelitian ini.

Halaman ini sengaja dikosongkan

BAB 5 KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil penelitian mengenai analisis efektivitas teknik *resampling* dalam menangani *imbalanced* data untuk meningkatkan *sensitivitas* model ensemble learning pada dataset *Credit Card Fraud Detection*, diperoleh beberapa kesimpulan sebagai berikut.

1. Penerapan teknik *resampling* secara umum mampu meningkatkan kemampuan model dalam mendeteksi transaksi *fraud* dibandingkan kondisi *baseline* pada beberapa model *ensemble learning* yang ditunjukkan oleh peningkatan nilai *recall*. Hasil pengujian menunjukkan bahwa teknik *resampling* dapat meningkatkan representasi kelas minoritas sehingga model menjadi lebih sensitif terhadap pola transaksi *fraud*. Namun, peningkatan *recall* tidak selalu diikuti oleh peningkatan *precision* dan *F1-Score*. Beberapa teknik *resampling* menghasilkan *recall* yang tinggi tetapi menyebabkan peningkatan *false positive* sehingga *precision* menurun. Oleh karena itu, pengaruh *resampling* terhadap performa model sangat bergantung pada karakteristik teknik *resampling* dan model *ensemble learning* yang digunakan. Dengan demikian, teknik *resampling* berpengaruh terhadap performa model dalam menangani *imbalanced data*, terutama dalam meningkatkan sensitivitas terhadap kelas *fraud*.
2. Efektivitas teknik *resampling* dalam menangani *imbalanced* data memiliki hasil yang berbeda untuk setiap model. Berdasarkan rata-rata performa seluruh model, teknik *Edited Nearest Neighbor* (ENN) menghasilkan *F1-Score* tertinggi sebesar 0,8314 dengan *precision* sebesar 0,8985, menunjukkan bahwa teknik pembersihan data lebih efektif dibandingkan teknik yang berfokus pada penyeimbangan distribusi kelas secara agresif. Sementara itu, *Random Undersampling* menghasilkan *recall* tertinggi sebesar 0,8980, namun *precision* yang sangat rendah menyebabkan kualitas prediksi secara keseluruhan menurun. Hasil penelitian juga menunjukkan bahwa teknik *hybrid sampling* seperti SMOTE-ENN dan SMOTE-Tomek tidak selalu menghasilkan performa terbaik meskipun mampu meningkatkan *recall*. Temuan ini mengindikasikan bahwa peningkatan kualitas data melalui pengurangan *noise* lebih berpengaruh dibandingkan hanya menyeimbangkan jumlah sampel antar kelas.
3. Berdasarkan nilai *F1-Score*, kombinasi terbaik diperoleh oleh *Random Forest* tanpa *resampling* dan *Random Forest* dengan *Tomek Links* yang sama-sama menghasilkan *F1-Score* sebesar 0,8743 dengan *recall* sebesar 0,8163, dan *precision* sebesar 0,9412. Namun, karena pada penelitian ini memprioritaskan kemampuan deteksi *fraud*, kombinasi *XGBoost* dengan ENN dapat dianggap sebagai kombinasi yang paling efektif. Kombinasi tersebut menghasilkan *recall* tertinggi di antara lima kombinasi terbaik berdasarkan *F1-Score*, yaitu sebesar 0,8367, dengan *precision* yang tetap tinggi sebesar 0,9111, *pr-auc* sebesar 0,8517, dan *F1-Score* sebesar 0,8723. Hasil tersebut menunjukkan bahwa kombinasi *XGBoost* dan ENN mampu mendeteksi lebih banyak transaksi *fraud* tanpa mengorbankan ketepatan prediksi secara signifikan sehingga paling sesuai dengan tujuan utama sistem *fraud detection*.
4. Penerapan *hyperparameter tuning* menggunakan *RandomizedSearchCV* tidak menghasilkan peningkatan nilai *F1-Score* pada seluruh lima kombinasi terbaik yang

diuji. Seluruh kombinasi mengalami penurunan *F1-Score* setelah tuning, yang menunjukkan bahwa konfigurasi parameter *default* pada model-model tersebut telah mampu menghasilkan performa yang mendekati optimal untuk karakteristik dataset yang digunakan. Meskipun demikian, tuning berhasil meningkatkan *recall* pada beberapa kombinasi, khususnya *Random Forest* dan *XGBoost*. *Recall* tertinggi setelah tuning diperoleh oleh *Random Forest* dengan *Random Oversampling* dan *XGBoost* dengan ENN yang sama-sama mencapai nilai 0,8469. Hasil ini menunjukkan bahwa *hyperparameter tuning* dapat meningkatkan sensitivitas model terhadap transaksi *fraud*, tetapi peningkatan tersebut umumnya diperoleh dengan konsekuensi menurunnya ketepatan prediksi sehingga keseimbangan performa keseluruhan menjadi berkurang.

5.2 Saran

Berdasarkan hasil penelitian yang telah dilakukan, terdapat beberapa saran yang dapat dipertimbangkan untuk pengembangan penelitian selanjutnya.

1. Penelitian selanjutnya dapat menguji teknik penanganan imbalanced data lain, seperti *Borderline-SMOTE*, *KMeans-SMOTE*, *SVMSMOTE*, *NearMiss*, atau kombinasi *hybrid sampling* lainnya untuk mengetahui pengaruhnya terhadap kemampuan deteksi *fraud*.
2. Penelitian berikutnya dapat menambahkan model *ensemble learning* maupun algoritma *machine learning* lain yang belum diuji pada penelitian ini, seperti *CatBoost*, *LightGBM*, *Balanced Random Forest*, *Easy Ensemble*, atau metode berbasis *deep learning* sehingga diperoleh perbandingan yang lebih komprehensif.
3. Proses *hyperparameter tuning* dapat menggunakan metode optimasi yang lebih eksploratif seperti *Optuna*, *Bayesian Optimization*, *Genetic Algorithm*, atau *Particle Swarm Optimization* untuk memperoleh kombinasi parameter yang lebih optimal dibandingkan *RandomizedSearchCV*.
4. Penelitian selanjutnya dapat mengevaluasi model menggunakan metrik lain, seperti *Area Under Precision-Recall Curve* (PR-AUC), *Matthews Correlation Coefficient* (MCC), *Balanced Accuracy*, maupun analisis *cost-sensitive learning* yang mempertimbangkan biaya kesalahan klasifikasi antara *false positive* dan *false negative*.
5. Untuk implementasi pada lingkungan nyata, penelitian selanjutnya dapat mengembangkan sistem *fraud detection* berbasis *real-time streaming data* serta mengintegrasikan mekanisme *threshold optimization* sehingga model dapat disesuaikan dengan kebutuhan operasional institusi keuangan yang berbeda-beda.

DAFTAR PUSTAKA

- Abi Nader, E., & Soufi, A. (2026). Modeling occupant air-conditioning behavior in mediterranean offices with manual HVAC control: A case study conducted in montpellier during summer 2025. *Energy and Buildings*, 365, 117666. <https://doi.org/10.1016/J.ENBUILD.2026.117666>
- Ahmed, K. H., Axelsson, S., Li, Y., & Sagheer, A. M. (2025). A credit card fraud detection approach based on ensemble machine learning classifier with hybrid data sampling. *Machine Learning with Applications*, 20, 100675. <https://doi.org/10.1016/j.mlwa.2025.100675>
- Alqahtani, A. M., Elbisy, M. S., & Osra, F. A. (2023). *INDIAN JOURNAL OF SCIENCE AND TECHNOLOGY Comparing Between Support Vector Machine and Gradient Boosted Trees Models for Prediction of Wave Overtopping at Coastal Structures with Composite Slope*. <https://doi.org/10.17485/IJST/v16i33.919>
- Alsorory, H. S., & Alshraideh, M. (2024). Boosting Software Fault Prediction: Addressing Class Imbalance With Enhanced Ensemble Learning. *Applied Computational Intelligence and Soft Computing*, 2024(1). <https://doi.org/10.1155/2024/2959582>
- Arnold, C., Biedebach, L., K pfer, A., & Neunhoeffler, M. (2024). The role of hyperparameters in machine learning models and how to tune them. *Political Science Research and Methods*, 12(4), 841–848. <https://doi.org/10.1017/psrm.2023.61>
- Arshad, I., Umair, M., Jan, F., Iftikhar, H., Canas Rodrigues, P., Ivan Gonzales Medina, R., & Linkolk Lopez-Gonzales, J. (2025). Performance of Classification Algorithms Under Class Imbalance: Simulation and Real-World Evidence. *IEEE Access*, 13, 179672–179685. <https://doi.org/10.1109/ACCESS.2025.3620264>
- Azim Mim, M., Majadi, N., & Mazumder, P. (2024). A soft voting ensemble learning approach for credit card fraud detection. *Heliyon*, 10(3). <https://doi.org/10.1016/j.heliyon.2024.e25466>
- Bagga, S., Goyal, A., Gupta, N., & Goyal, A. (2020). Credit Card Fraud Detection using Pipeling and Ensemble Learning. *Procedia Computer Science*, 173, 104–112. <https://doi.org/10.1016/j.procs.2020.06.014>
- Bodker, A., Connolly, P., Sing, O., Hutchins, B., Townsley, M., & Drew, J. (2022). Card-not-present fraud: using crime scripts to inform crime prevention initiatives. *Security Journal*, 36(4), 693–711. <https://doi.org/10.1057/s41284-022-00359-w>
- Bonamutial, M., & Prasetyo, S. Y. (2023). Exploring the Impact of Feature Data Normalization and Standardization on Regression Models for Smartphone Price Prediction. *Proceedings of 2023 International Conference on Information Management and Technology, ICIMTech 2023*, 294–298. <https://doi.org/10.1109/ICIMTech59029.2023.10277860>
- Bonde, L., & Bichanga, A. K. (2025). Improving Credit Card Fraud Detection with Ensemble Deep Learning-Based Models: A Hybrid Approach Using SMOTE-ENN. *Journal of Computing Theories and Applications*, 2(3), 383–394. <https://doi.org/10.62411/jcta.12021>

- Btoush, E. A. L. M., Zhou, X., Gururajan, R., Chan, K. C., Genrich, R., & Sankaran, P. (2023). A systematic review of literature on credit card cyber fraud detection using machine and deep learning. *PeerJ Computer Science*, 9. <https://doi.org/10.7717/PEERJ-CS.1278>
- Cervantes, J., Garcia-Lamont, F., Rodríguez-Mazahua, L., & Lopez, A. (2020). A comprehensive survey on support vector machine classification: Applications, challenges and trends. *Neurocomputing*, 408, 189–215. <https://doi.org/10.1016/j.neucom.2019.10.118>
- Chauhan, S., & Lee, S. (2022). Machine Learning-Based Anomaly Detection for Multivariate Time Series with Correlation Dependency. *IEEE Access*, 10, 132062–132070. <https://doi.org/10.1109/ACCESS.2022.3230352>
- Chen, B., Hu, Y., Zhang, G., Cheng, D., & Zhang, S. (2025). Certainty-aware support vector machines for fair classification. *Neurocomputing*, 655. <https://doi.org/10.1016/j.neucom.2025.131330>
- Chhabra, R., Goswami, S., & Ranjan, R. K. (2024). A voting ensemble machine learning based credit card fraud detection using highly imbalance data. *Multimedia Tools and Applications*, 83(18), 54729–54753. <https://doi.org/10.1007/s11042-023-17766-9>
- Chung, J., & Lee, K. (2023). Credit Card Fraud Detection: An Improved Strategy for High Recall Using KNN, LDA, and Linear Regression. *Sensors*, 23(18). <https://doi.org/10.3390/s23187788>
- Danquah, R. A. (2020). *Handling Imbalanced Data: A Case Study for Binary Class Problems*. <http://arxiv.org/abs/2010.04326>
- Dorabiala, O., Aravkin, A. Y., & Kutz, J. N. (2024). Ensemble Principal Component Analysis. *IEEE Access*, 12, 6663–6671. <https://doi.org/10.1109/ACCESS.2024.3350984>
- Feudjio, C., Noyum, V. D., Mofendjou, Y. P., Rockefeller, & Fokoué, E. (2021). *A Novel Use of Discrete Wavelet Transform Features in the Prediction of Epileptic Seizures from EEG Data*. <http://arxiv.org/abs/2102.01647>
- Franceschi, L., Donini, M., Perrone, V., Klein, A., Archambeau, C., Seeger, M., Pontil, M., & Frasconi, P. (2025). *Hyperparameter Optimization in Machine Learning*. <https://doi.org/10.1561/22000000088>
- Gyu Kim, H., & Park, Y. (2025). Calibrating F1 Scores for Fair Performance Comparison of Binary Classification Models With Application to Student Dropout Prediction. *IEEE Access*, 13, 136554–136567. <https://doi.org/10.1109/ACCESS.2025.3594735>
- Ileberi, E., & Sun, Y. (2024). Advancing Model Performance with ADASYN and Recurrent Feature Elimination and Cross-Validation in Machine Learning-Assisted Credit Card Fraud Detection: A Comparative Analysis. *IEEE Access*, 12, 133315–133327. <https://doi.org/10.1109/ACCESS.2024.3457922>
- Jiang, J., Zhang, C., Ke, L., Hayes, N., Zhu, Y., Qiu, H., Zhang, B., Zhou, T., & Wei, G. W. (2025). A review of machine learning methods for imbalanced data challenges in chemistry. In *Chemical Science* (Vol. 16, Number 18, pp. 7637–7658). Royal Society of Chemistry. <https://doi.org/10.1039/d5sc00270b>

- Kette, E. K. D. (2025). HYPERPARAMETER OPTIMIZATION IN MACHINE LEARNING MODELS ON SKY SURVEY DATA CLASSIFICATION. *Jurnal Komputer Dan Informatika*, 13(2), 77–84. <https://doi.org/10.35508/jicon.v13i2.18493>
- Khalid, A. R., Owoh, N., Uthmani, O., Ashawa, M., Osamor, J., & Adejoh, J. (2024). Enhancing Credit Card Fraud Detection: An Ensemble Machine Learning Approach. *Big Data and Cognitive Computing*, 8(1). <https://doi.org/10.3390/bdcc8010006>
- Khan, A. A., Chaudhari, O., & Chandra, R. (2024). A review of ensemble learning and data augmentation models for class imbalanced problems: Combination, implementation and evaluation. In *Expert Systems with Applications* (Vol. 244). Elsevier Ltd. <https://doi.org/10.1016/j.eswa.2023.122778>
- Khan, S. S., Sudan, J. S., Pathak, A., Pandit, R., Rane, P., & Kumawat, A. K. (2024). A Review of EEG Artifact Removal Methods for Brain-Computer Interface Applications. In *Ingenierie des Systemes d'Information* (Vol. 29, Number 1, pp. 247–252). International Information and Engineering Technology Association. <https://doi.org/10.18280/isi.290124>
- Khan, T. A., Tulsi, J., Alam, M., Kadir, K., Ali, K. M., & Mazliham, M. S. (2025). Analysis and visualization of fraud detection patterns through data mining and classification using MLP and hybrid deep learning model. *Egyptian Informatics Journal*, 32. <https://doi.org/10.1016/j.eij.2025.100829>
- Kumari, U., Pramod, P. V., & Nair, P. (n.d.). *AMRITA SCHOOL OF PHYSICAL SCIENCES AMRITA VISHWA VIDYAPEETHAM BONAFIDE CERTIFICATE AMRITA SCHOOL OF PHYSICAL SCIENCES AMRITA VISHWA VIDYAPEETHAM*.
- Lavazza, L., & Morasca, S. (2023). Common Problems With the Usage of F-Measure and Accuracy Metrics in Medical Research. In *IEEE Access* (Vol. 11, pp. 51515–51526). Institute of Electrical and Electronics Engineers Inc. <https://doi.org/10.1109/ACCESS.2023.3278996>
- Liang, J., Pan, L., Shao, Y., Xu, S., Yang, Z., & Nai, W. (2022). Logistic Regression Based on Halton Sequence Initialized Lion Swarm Optimization Algorithm. *2022 7th International Conference on Big Data Analytics, ICBDA 2022*, 154–159. <https://doi.org/10.1109/ICBDA55095.2022.9760337>
- Lin, G. M., & Zeng, H. C. (2021). Electrocardiographic Machine Learning to Predict Mitral Valve Prolapse in Young Adults. *IEEE Access*, 9, 103132–103140. <https://doi.org/10.1109/ACCESS.2021.3098039>
- Liu, H., Tian, Y., & Yu, D. (2024). Prediction of cardiovascular and cerebrovascular diseases based on machine learning models. *Applied and Computational Engineering*, 46(1), 35–44. <https://doi.org/10.54254/2755-2721/46/20241068>
- Liu, W., Fan, H., Xia, M., & Pang, C. (2022). Predicting and interpreting financial distress using a weighted boosted tree-based tree. *Engineering Applications of Artificial Intelligence*, 116. <https://doi.org/10.1016/j.engappai.2022.105466>

- Lokanan, M. E., & Sharma, K. (2022). Fraud prediction using machine learning: The case of investment advisors in Canada. *Machine Learning with Applications*, 8, 100269. <https://doi.org/10.1016/j.mlwa.2022.100269>
- Maharana, K., Mondal, S., & Nemade, B. (2022). A review: Data pre-processing and data augmentation techniques. *Global Transitions Proceedings*, 3(1), 91–99. <https://doi.org/10.1016/j.gltp.2022.04.020>
- Mienye, I. D., & Jere, N. (2024). A Survey of Decision Trees: Concepts, Algorithms, and Applications. *IEEE Access*, 12, 86716–86727. <https://doi.org/10.1109/ACCESS.2024.3416838>
- Mienye, I. D., & Sun, Y. (2022). A Survey of Ensemble Learning: Concepts, Algorithms, Applications, and Prospects. In *IEEE Access* (Vol. 10, pp. 99129–99149). Institute of Electrical and Electronics Engineers Inc. <https://doi.org/10.1109/ACCESS.2022.3207287>
- Mohawesh, R., Maqsood, S., Salameh, H. B., & Elrefae, G. (2026). Leveraging GraphSAGE and large language models with cross-attention for transaction fraud detection: Evidence from credit card and PaySim datasets. *Information Processing and Management*, 63(7). <https://doi.org/10.1016/j.ipm.2026.104860>
- Mullick, S. S., Datta, S., Dhekane, S. G., & Das, S. (2020). Appropriateness of performance indices for imbalanced data classification: An analysis. *Pattern Recognition*, 102, 107197. <https://doi.org/10.1016/J.PATCOG.2020.107197>
- Nouri, A., Merzah, M., Mosayyebpour, S., Mousa, R., & Hesarak, S. (n.d.). *Evaluation Metrics in Learning Systems: A survey*.
- Odeyale, K. M., Moruff, O. A., Taofeekat, S. I. T., & Kayode, S. M. (2024). A support vector machine credit card fraud detection model based on high imbalance dataset. *Journal of Computers for Society*, 5(2), 85–94. <https://doi.org/10.17509/jcs.v5i2.70802>
- Onwudebelu, U., Fasola, O. O., Ugonna, E. C., & Woods, N. C. (2026). A behavior-aware intelligent decision framework for real-time credit card fraud detection using deep learning and adaptive user profiling. *Engineering Applications of Artificial Intelligence*, 181. <https://doi.org/10.1016/j.engappai.2026.115641>
- Paldino, G. M., Lebichot, B., Le Borgne, Y. A., Siblini, W., Oblé, F., Boracchi, G., & Bontempi, G. (2022). The role of diversity and ensemble learning in credit card fraud detection. *Advances in Data Analysis and Classification*, 18(1), 193–217. <https://doi.org/10.1007/s11634-022-00515-5>
- Prasad, P. Y., Chowdarv, A. S., Bavitha, C., Mounisha, E., & Reethika, C. (2023). A Comparison Study of Fraud Detection in Usage of Credit Cards using Machine Learning. *7th International Conference on Trends in Electronics and Informatics, ICOEI 2023 - Proceedings*, 1204–1209. <https://doi.org/10.1109/ICOEI56765.2023.10125838>
- Qaddoura, R., & Biltawi, M. M. (2022). Improving Fraud Detection in An Imbalanced Class Distribution Using Different Oversampling Techniques. *1st International Engineering Conference on Electrical, Energy, and Artificial Intelligence, EICEEAI 2022*. <https://doi.org/10.1109/EICEEAI56378.2022.10050500>

- Quan, M. (2022). An Advanced Hybrid Logistic Regression Model for Static and Dynamic Mixed Data Classification. *IEEE Access*, 10, 73623–73634. <https://doi.org/10.1109/ACCESS.2022.3187767>
- Richardson, E., Trevizani, R., Greenbaum, J. A., Carter, H., Nielsen, M., & Peters, B. (2024). The receiver operating characteristic curve accurately assesses imbalanced datasets. *Patterns*, 5(6), 100994. <https://doi.org/10.1016/J.PATTER.2024.100994>
- Rizka, A. S., & Sari, V. (2025). Penerapan Teknik Soft Voting Ensemble pada Klasifikasi Rating Film. *Indonesian Journal of Applied Statistics*, 8(1), 24. <https://doi.org/10.13057/ijas.v8i1.100904>
- Rizky, M. H., Faisal, M. R., Budiman, I., Kartini, D., & Abadi, F. (2024). Effect of Hyperparameter Tuning Using Random Search on Tree-Based Classification Algorithm for Software Defect Prediction. *IJCCS (Indonesian Journal of Computing and Cybernetics Systems)*, 18(1), 95. <https://doi.org/10.22146/ijccs.90437>
- Shah, V., Passi, K. (2021). Data Balancing for Credit Card Fraud Detection Using Complementary Neural Networks and SMOTE Algorithm. In: Chaubey, N., Parikh, S., Amin, K. (eds) Computing Science, Communication and Security. COMS2 2021. Communications in Computer and Information Science, vol 1416. Springer, Cham. https://doi.org/10.1007/978-3-030-76776-1_1
- Siam AM, Bhowmik P, Uddin MP (2025) Hybrid feature selection framework for enhanced credit card fraud detection using machine learning models. *PLOS ONE* 20(7): e0326975. <https://doi.org/10.1371/journal.pone.0326975>
- Souza, V. F. C., Cicalese, F., Laber, E. S., & Molinaro, M. (2025). Decision trees with short explainable rules. *Theoretical Computer Science*, 1047. <https://doi.org/10.1016/j.tcs.2025.115344>
- Subaşı, N. (2024). Comprehensive Analysis of Grid and Randomized Search on Dataset Performance. *European Journal of Engineering and Applied Sciences*, 7(2), 77–83. <https://doi.org/10.55581/ejeas.1581494>
- Sun, Z., Wang, G., Li, P., Wang, H., Zhang, M., & Liang, X. (2024). An improved random forest based on the classification accuracy and correlation measurement of decision trees. *Expert Systems with Applications*, 237. <https://doi.org/10.1016/j.eswa.2023.121549>
- Swana, E. F., Doorsamy, W., & Bokoro, P. (2022). Tomek Link and SMOTE Approaches for Machine Fault Classification with an Imbalanced Dataset. *Sensors*, 22(9). <https://doi.org/10.3390/s22093246>
- Tufail, S., Iqbal, H., Tariq, M., & Sarwat, A. I. (2025). A Hybrid Machine Learning-Based Framework for Data Injection Attack Detection in Smart Grids Using PCA and Stacked Autoencoders. *IEEE Access*, 13, 33783–33798. <https://doi.org/10.1109/ACCESS.2025.3543751>
- Ünalın, S., Günay, O., Akkurt, I., Gunoglu, K., & Tekin, H. O. (2024). A comparative study on breast cancer classification with stratified shuffle split and K-fold cross validation via ensembled machine learning. *Journal of Radiation Research and Applied Sciences*, 17(4), 101080. <https://doi.org/10.1016/j.jrras.2024.101080>

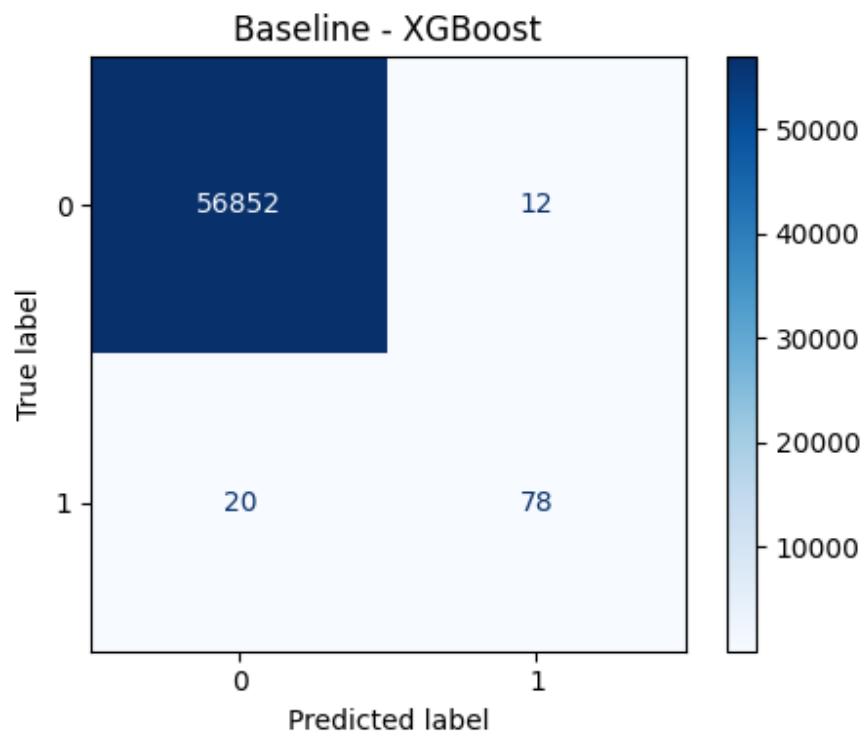
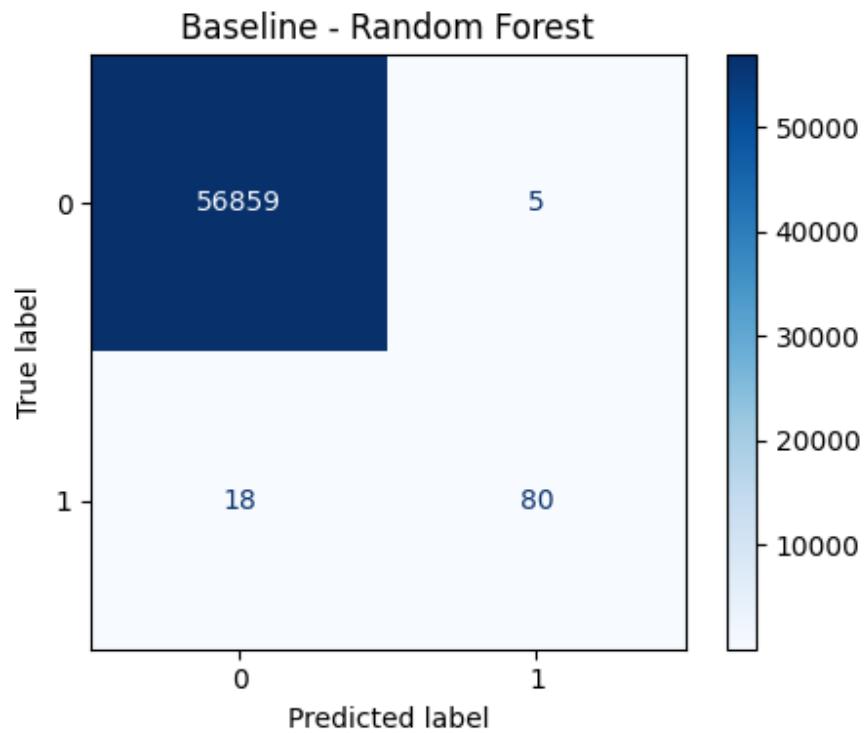
- Xu, B., Wang, Y., Liao, X., & Wang, K. (2023). Efficient fraud detection using deep boosting decision trees. *Decision Support Systems*, 175. <https://doi.org/10.1016/j.dss.2023.114037>
- Zavvar, M., Jafari, M., Pour, N. M., Kiaei, A. A., Zavvar, M. H., Heidari, A., & Navimipour, N. J. (2026). A hybrid deep learning framework using synthetic oversampling, autoencoder, convolutional neural networks, and an attention mechanism for credit card fraud detection. *Journal of Big Data*, 13(1). <https://doi.org/10.1186/s40537-025-01331-2>
- Zhang, J., Fang, Z., Tang, Y., & Ni, L. (2026). TS-GNN: Modeling temporal-semantic discrepancy for graph-based credit card fraud detection. *Expert Systems with Applications*, 331. <https://doi.org/10.1016/j.eswa.2026.133233>
- Zhang, Y., Deng, L., & Wei, B. (2024). Imbalanced Data Classification Based on Improved Random-SMOTE and Feature Standard Deviation. *Mathematics*, 12(11). <https://doi.org/10.3390/math12111709>
- Zian, S., Kareem, S. A., & Varathan, K. D. (2021). An Empirical Evaluation of Stacked Ensembles with Different Meta-Learners in Imbalanced Classification. *IEEE Access*, 9, 87434–87452. <https://doi.org/10.1109/ACCESS.2021.3088414>

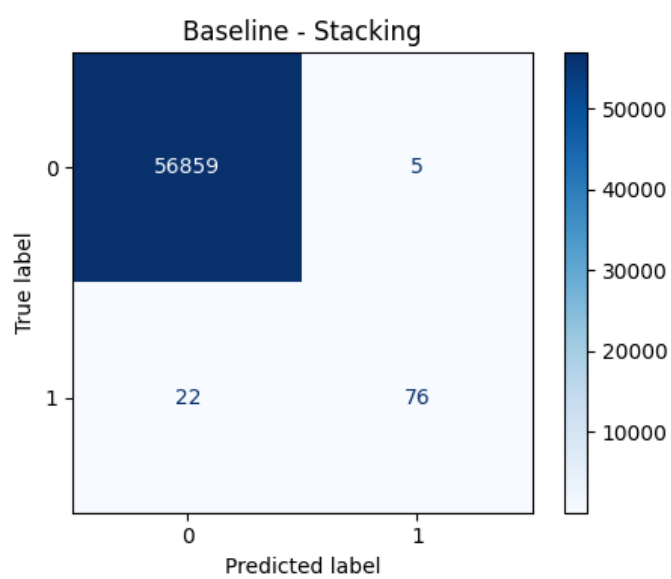
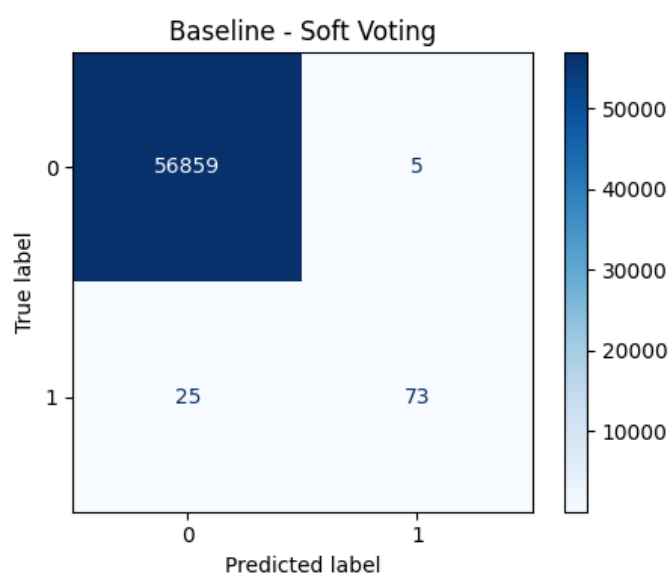
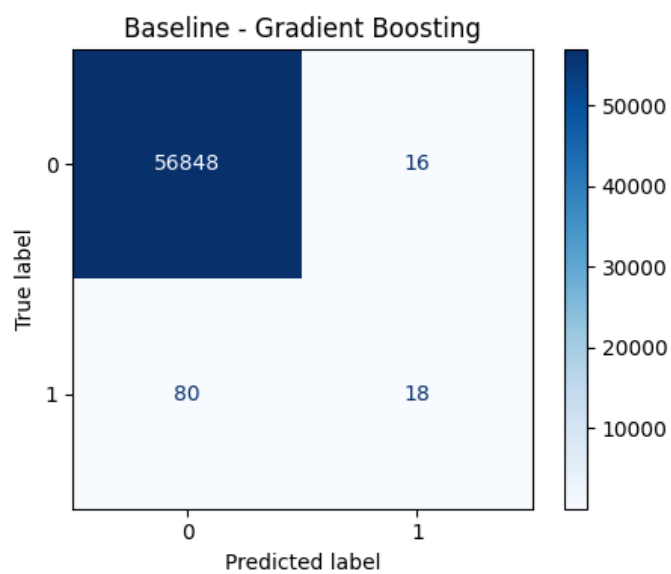
LAMPIRAN

Link Repository Github : <https://github.com/davinfrp/Tugas-Akhir-S1>

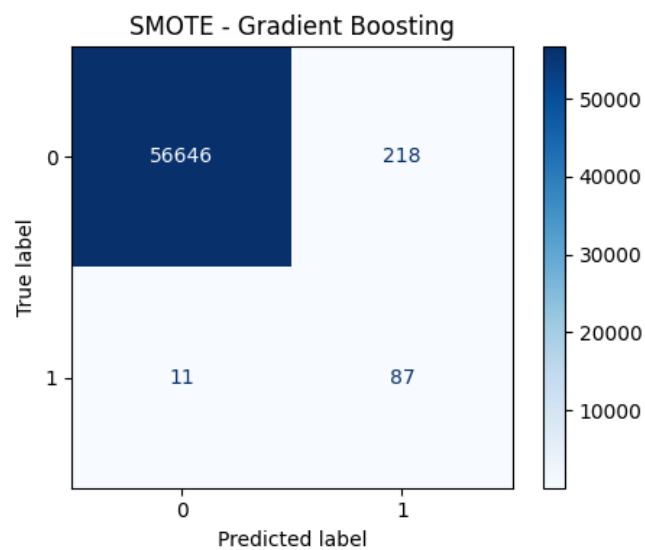
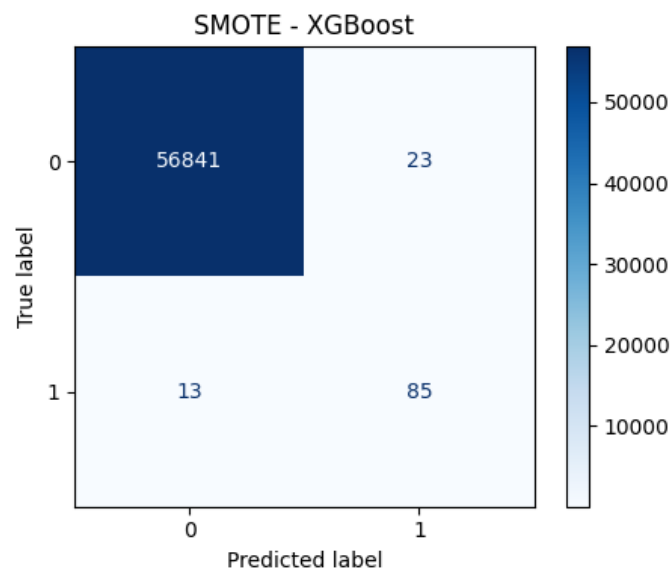
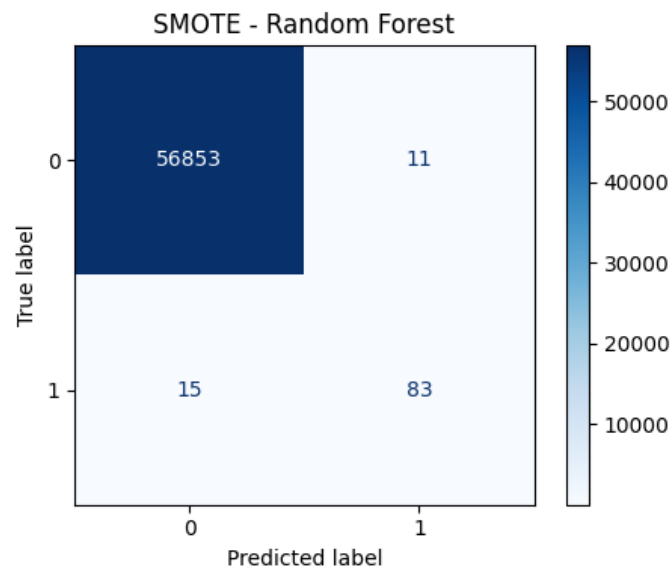
1. Confusion Matrix Setiap Pengujian

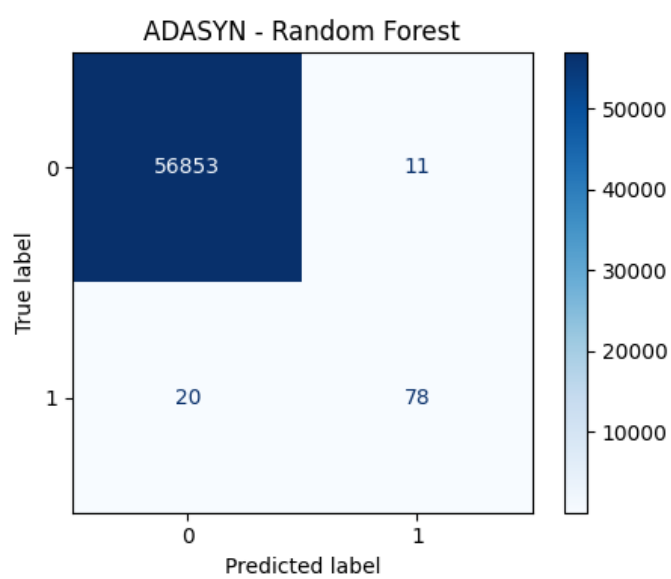
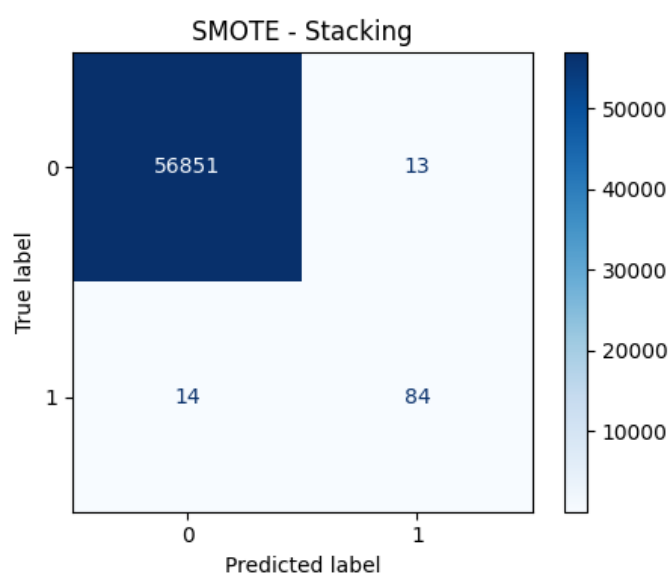
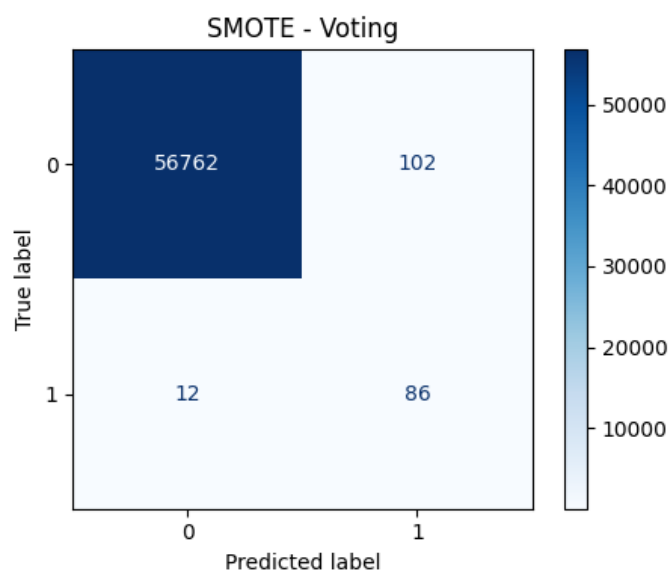
A. Pengujian Skenario Baseline

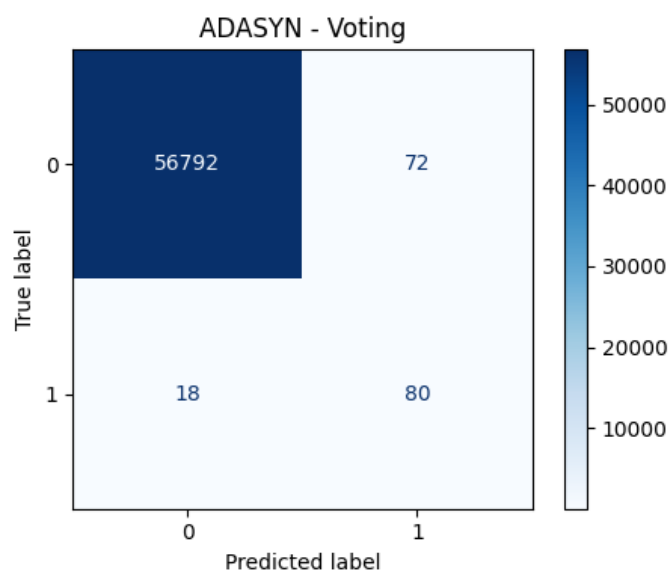
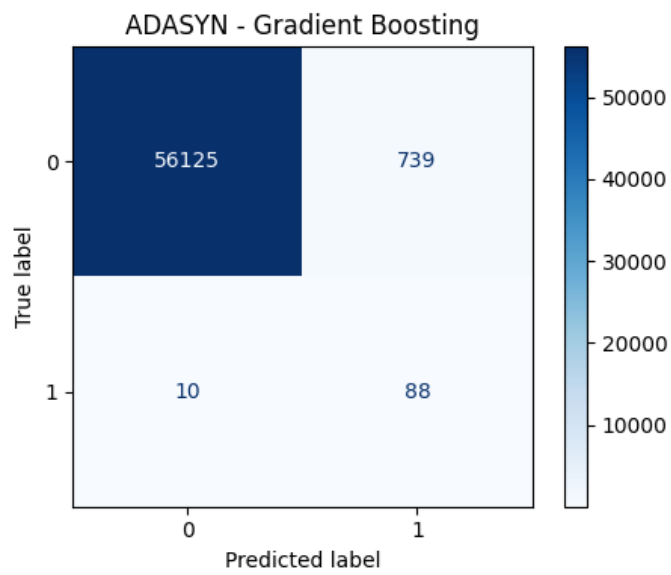
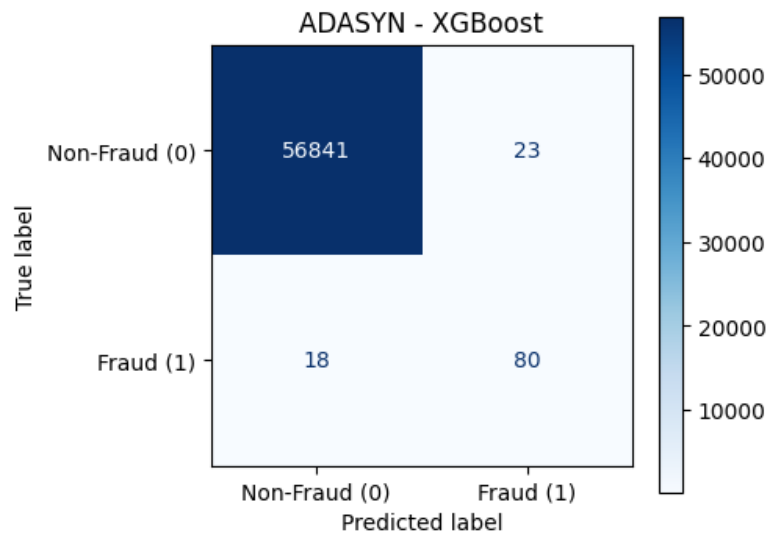


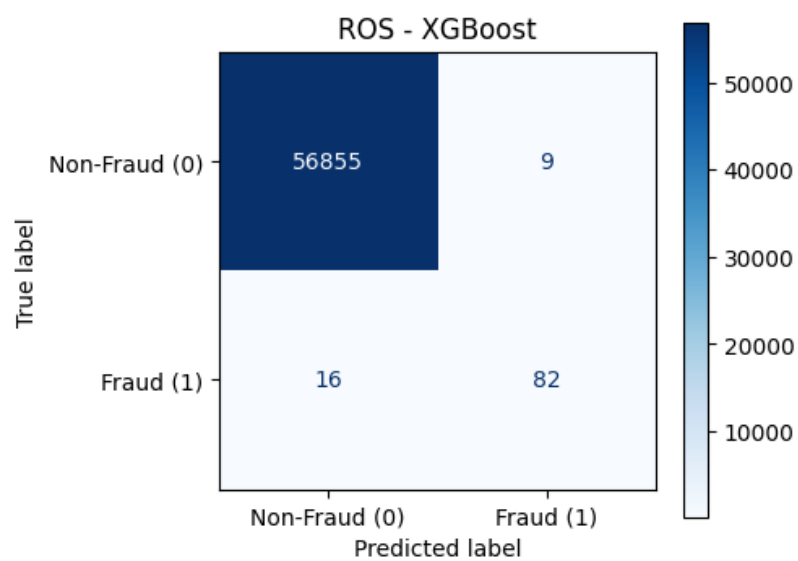
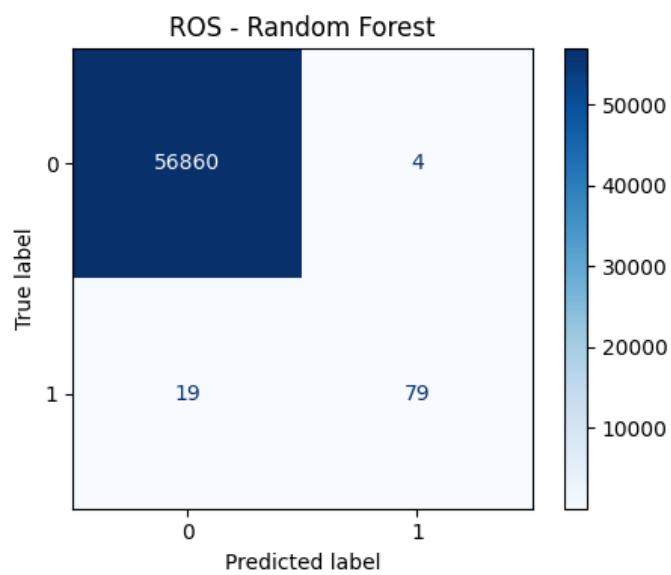
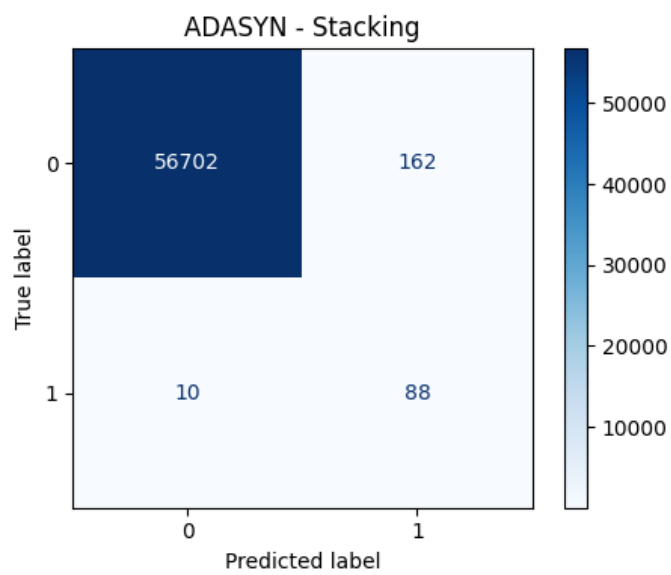


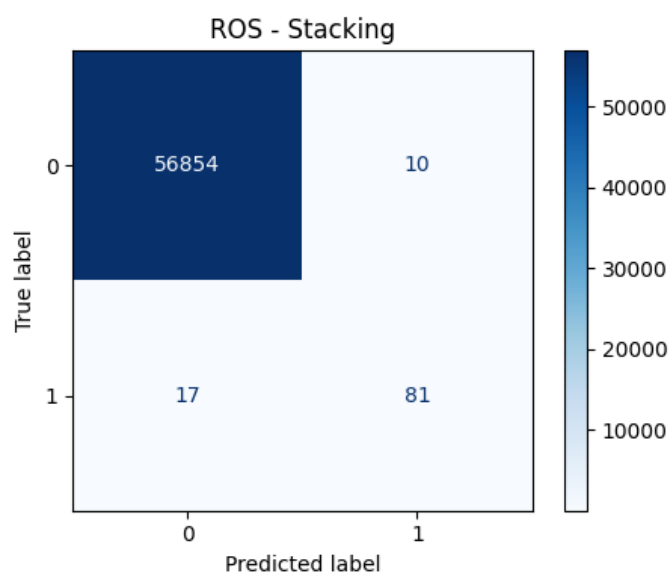
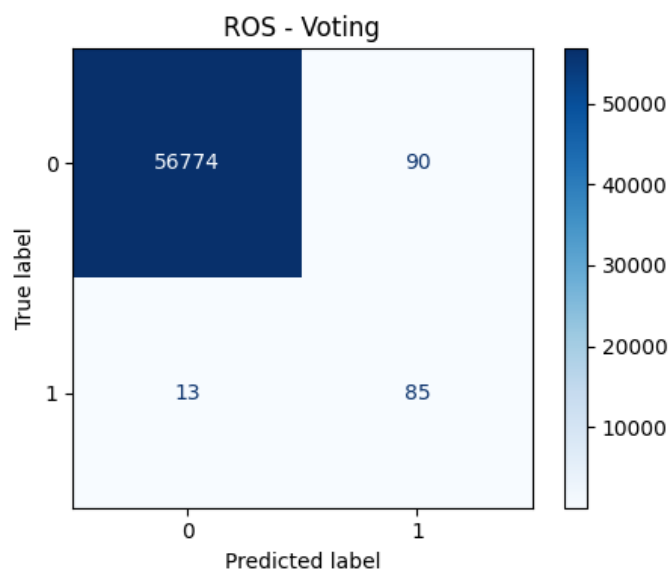
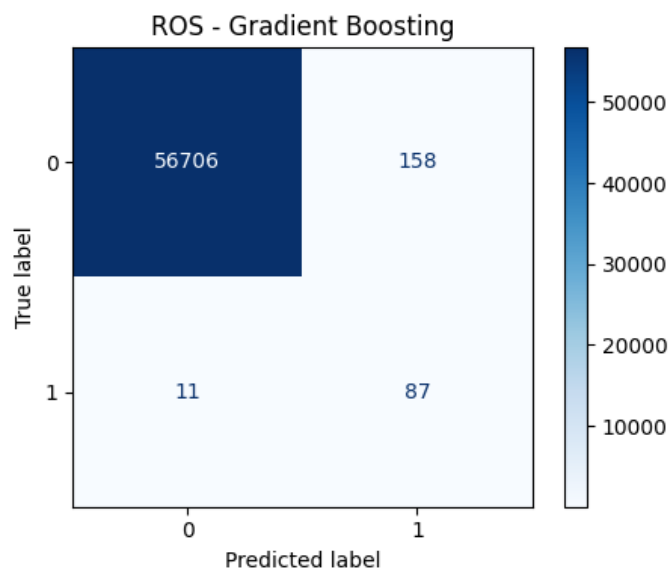
B. Pengujian Skenario Teknik Resampling

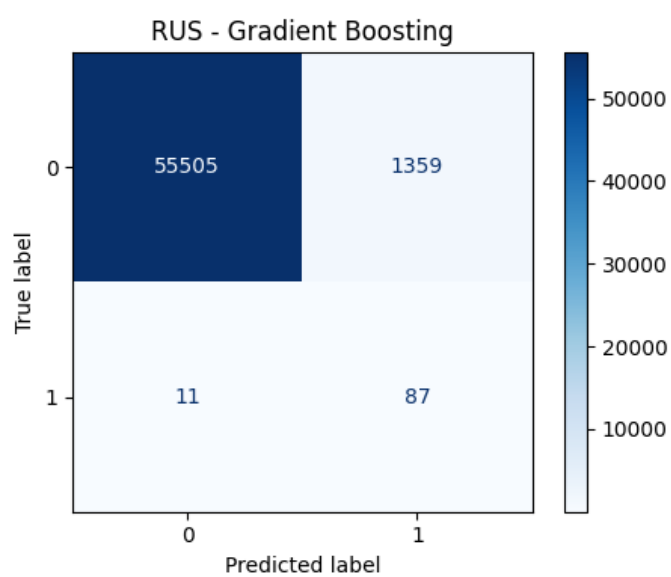
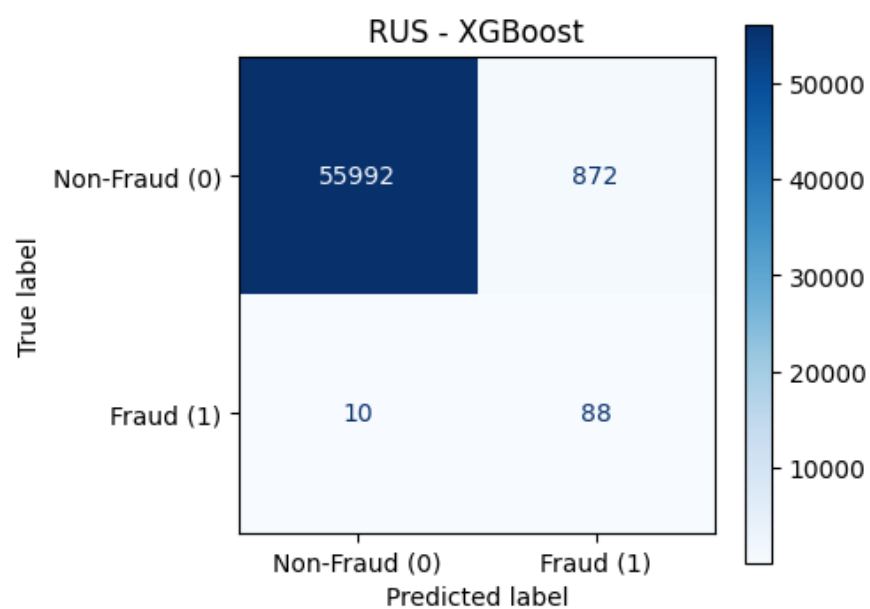
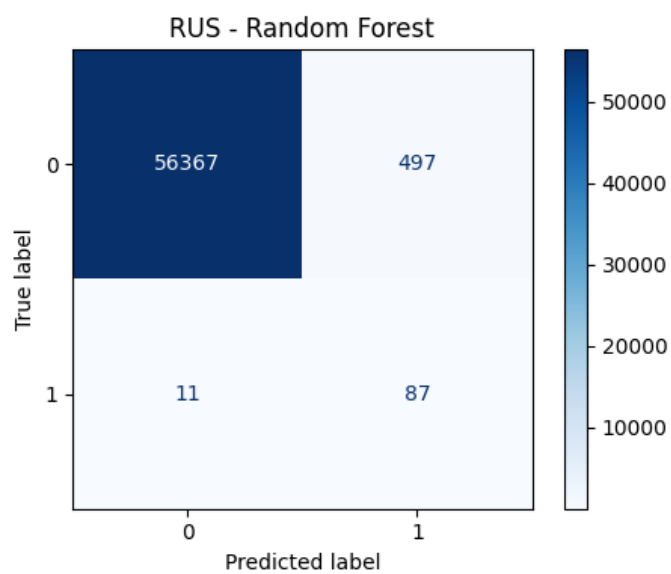


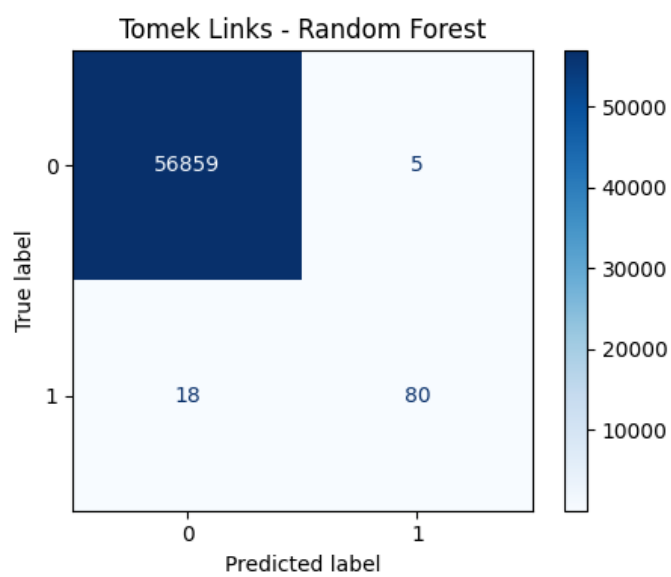
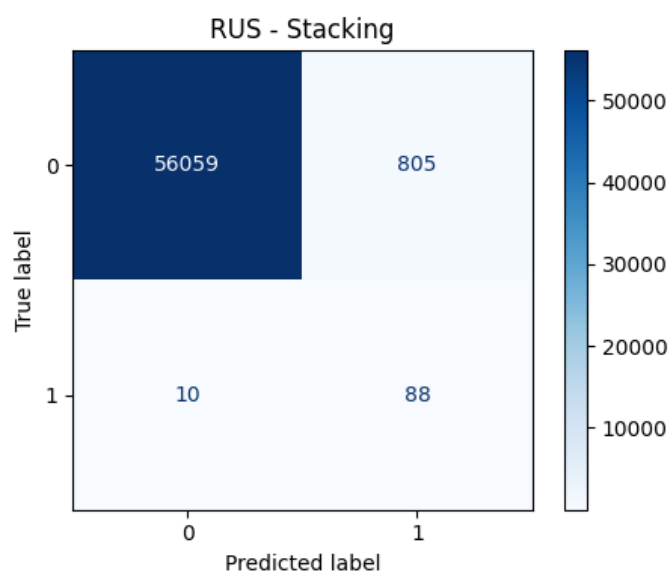
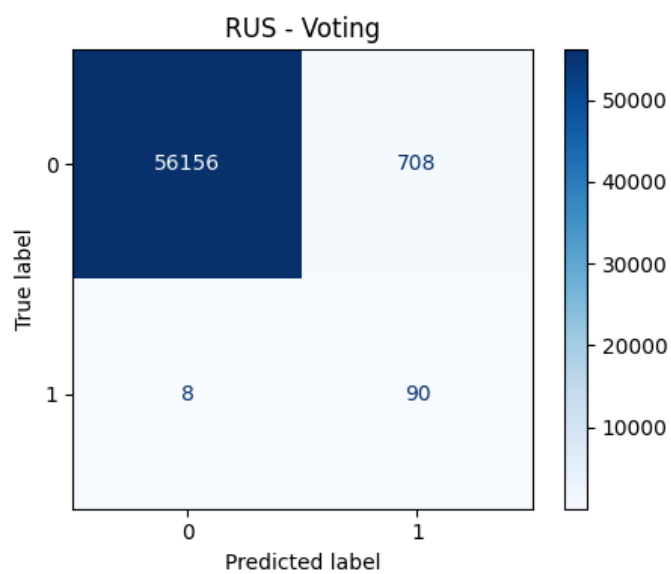


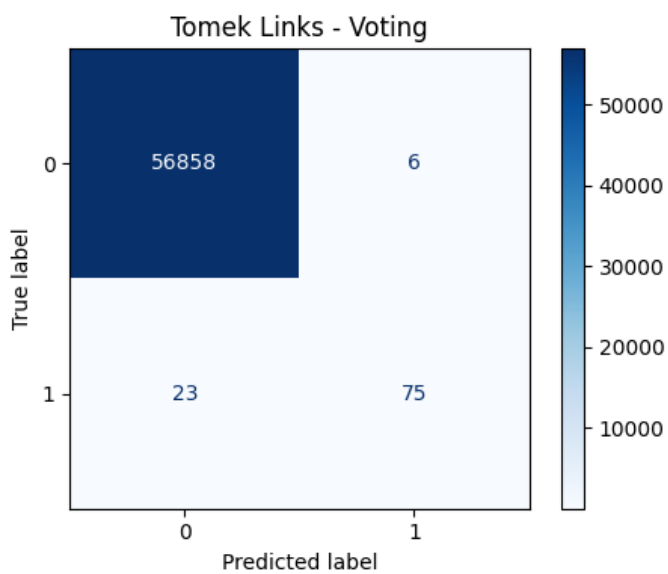
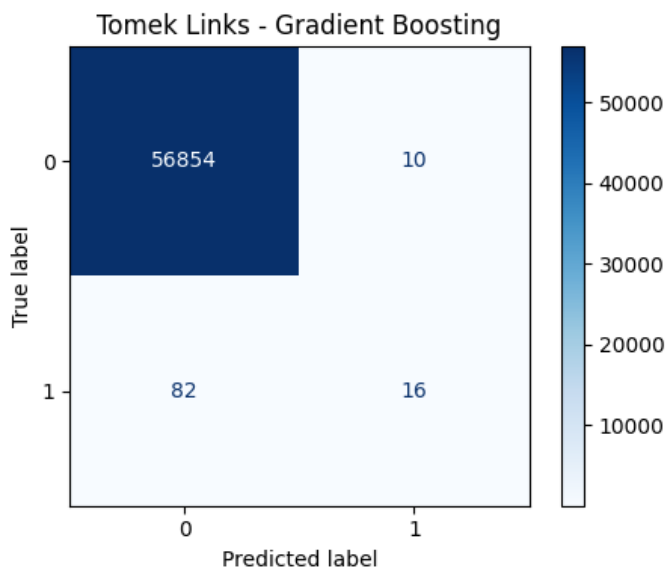
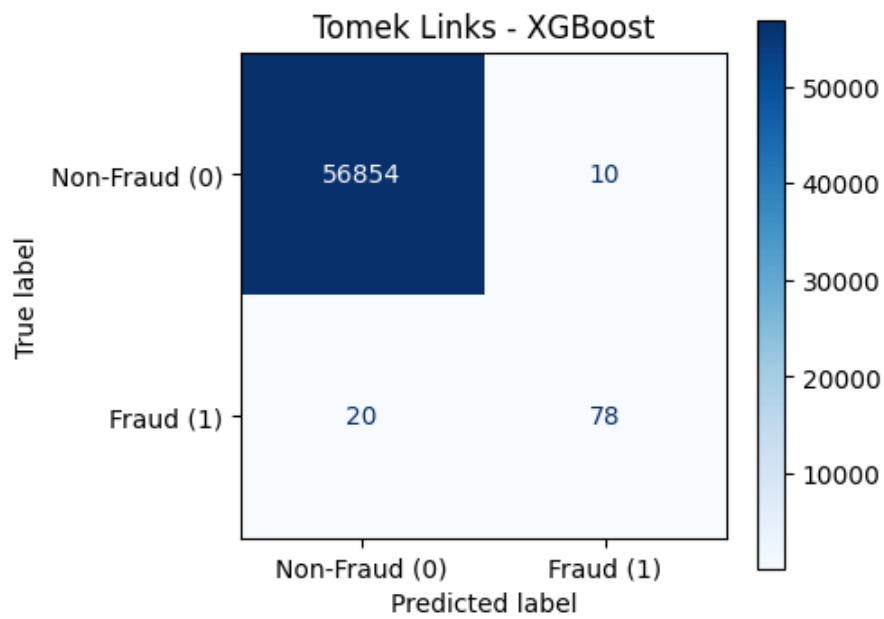


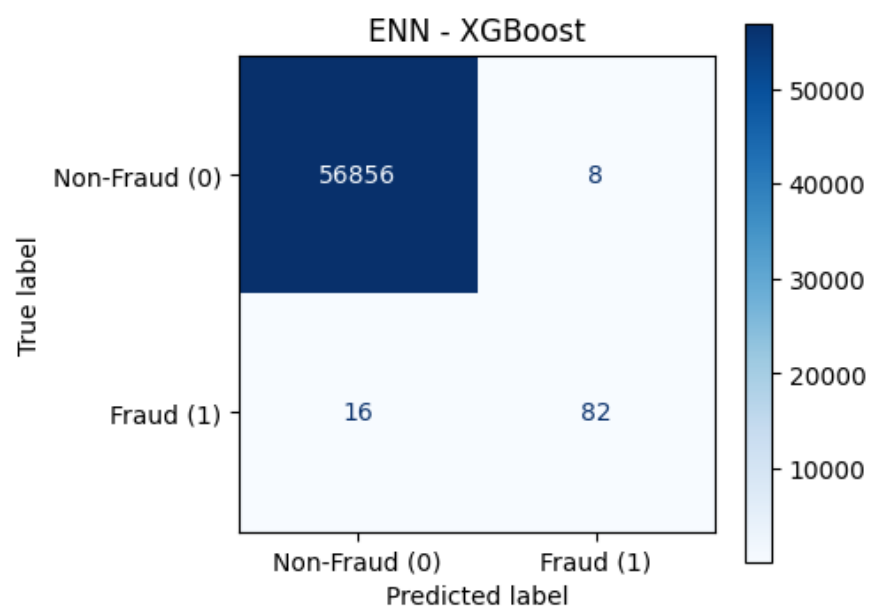
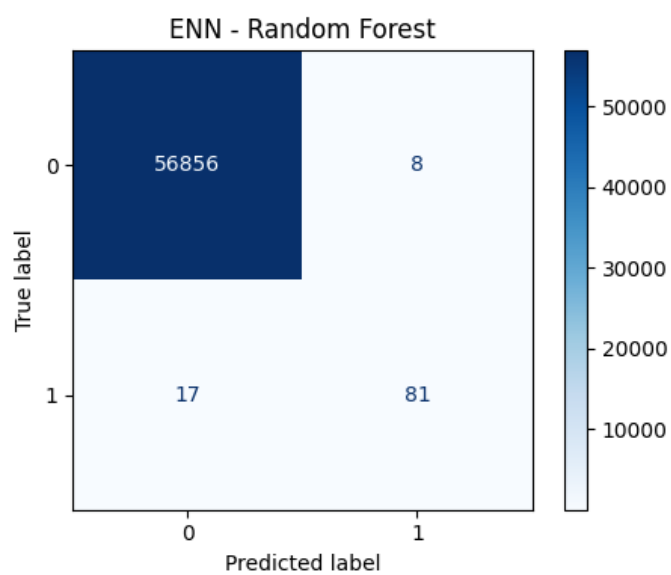
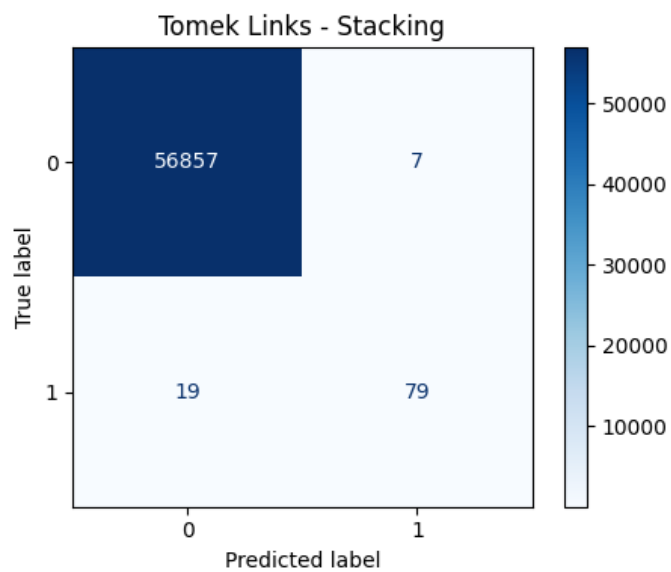


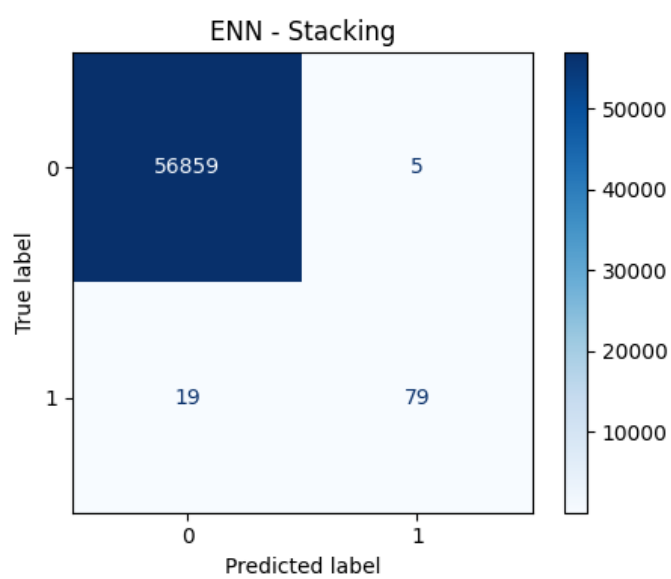
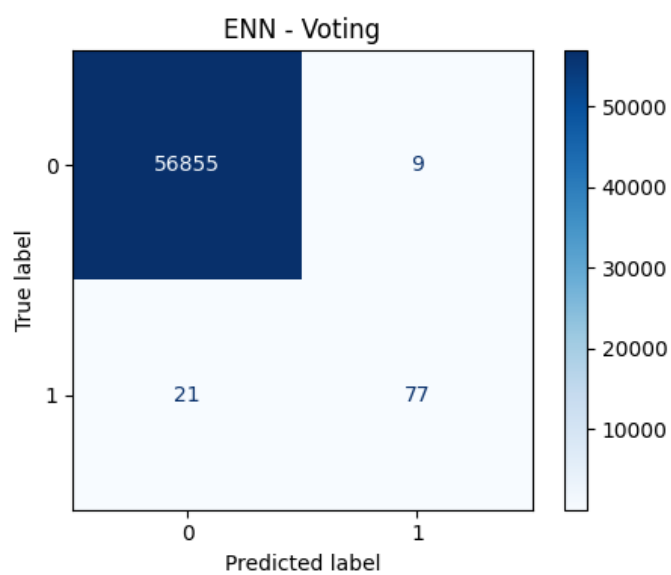
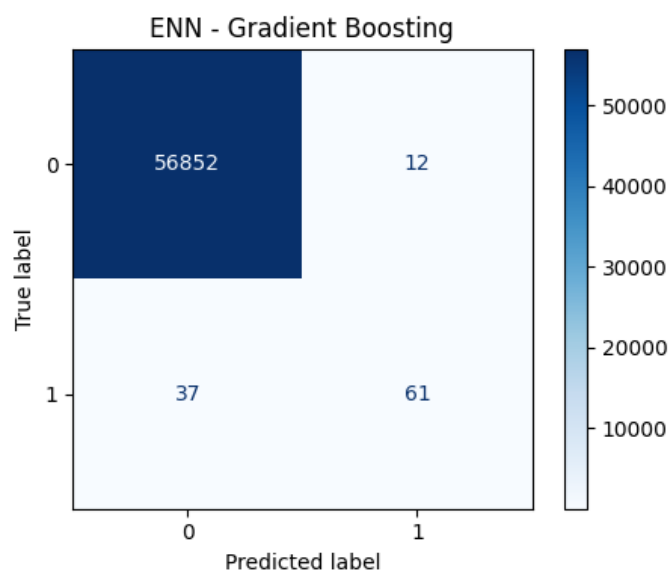


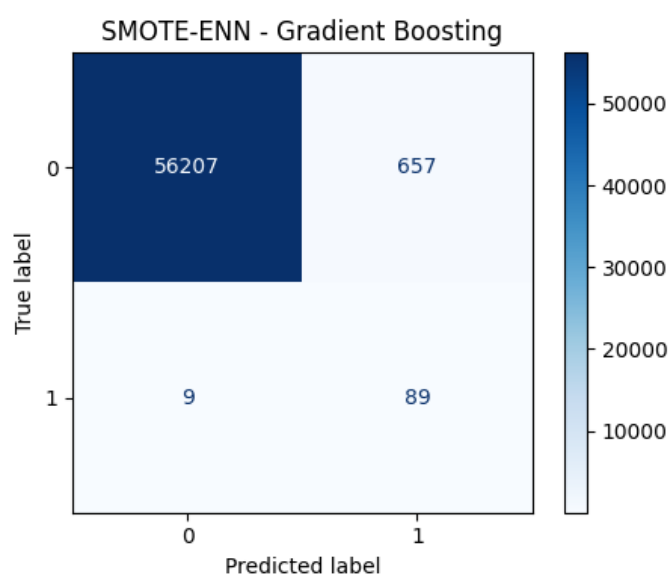
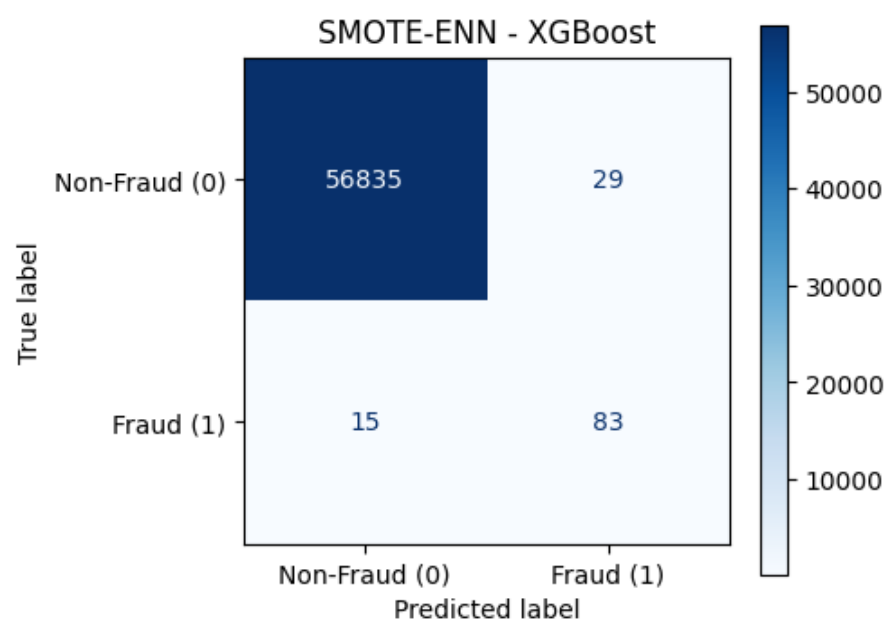
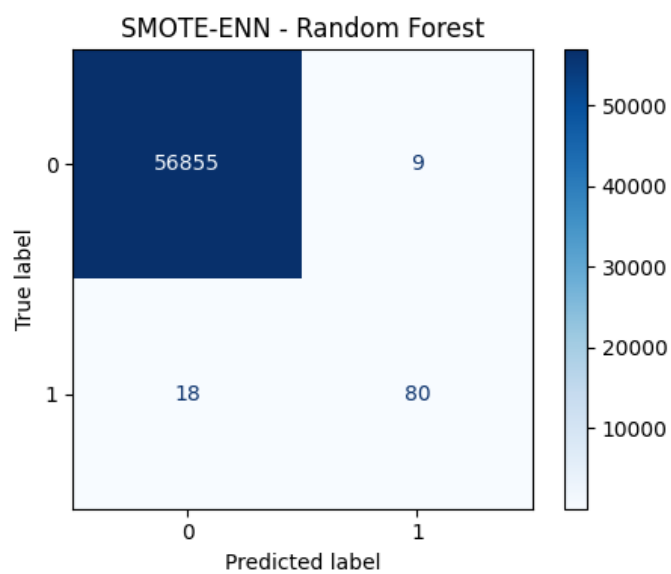


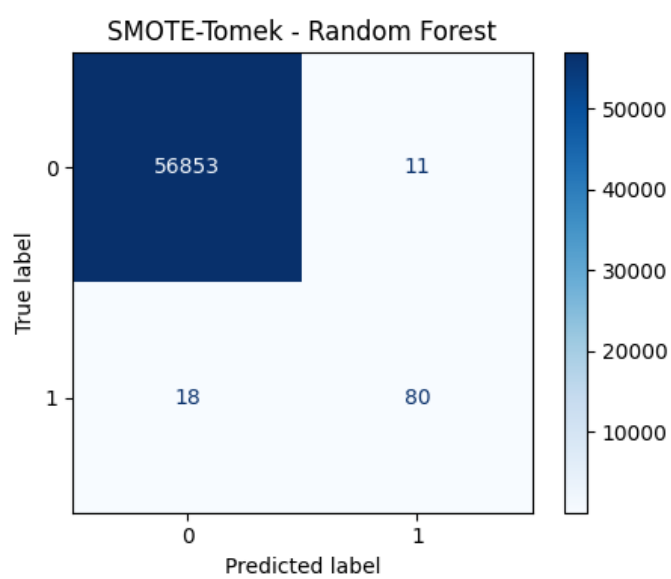
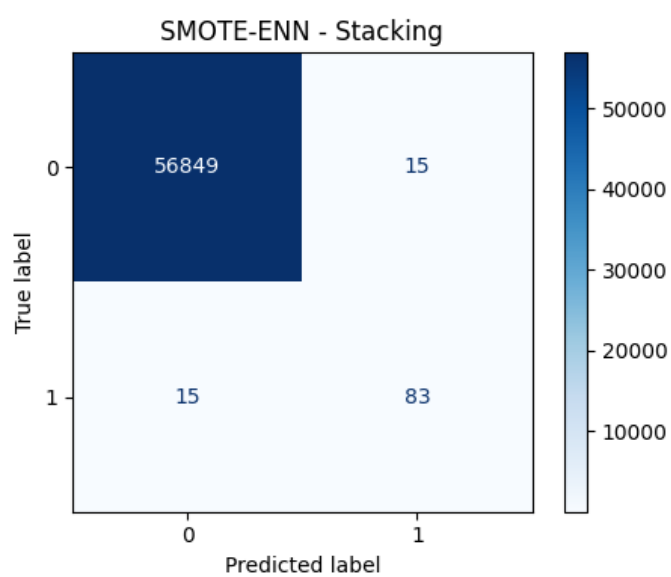
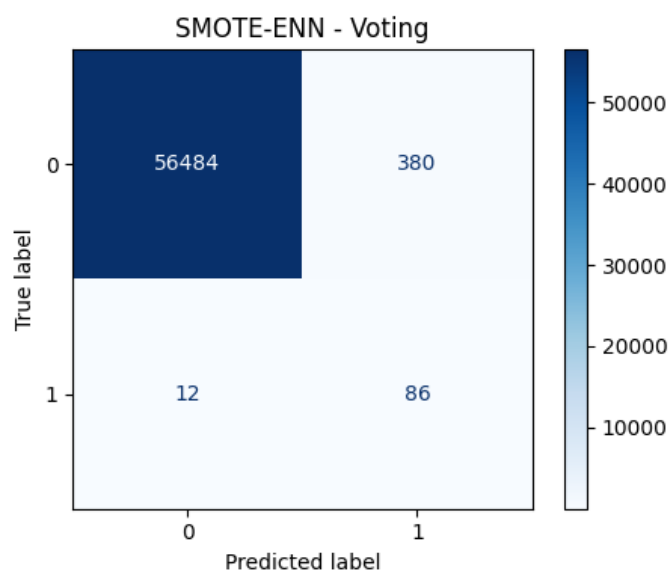


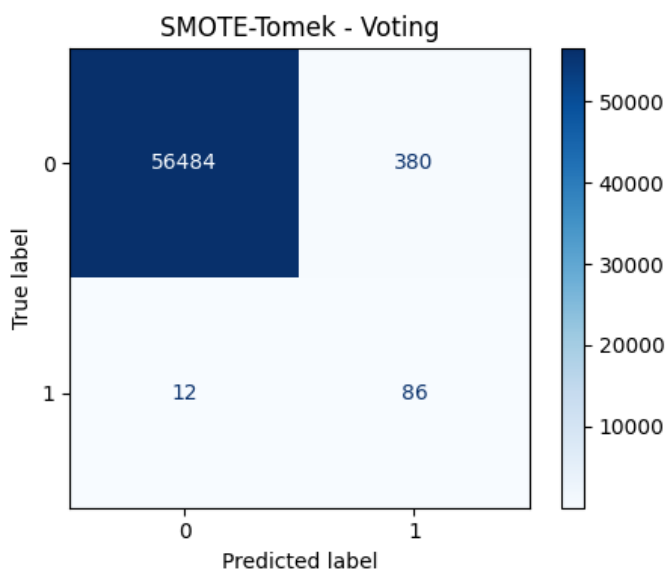
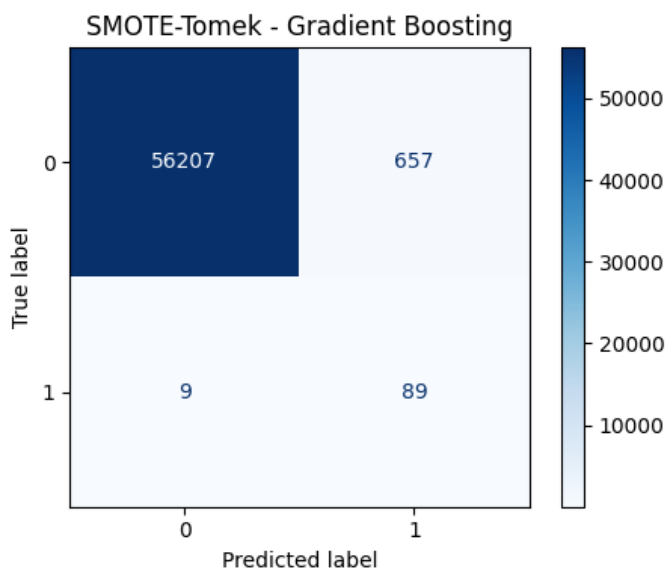
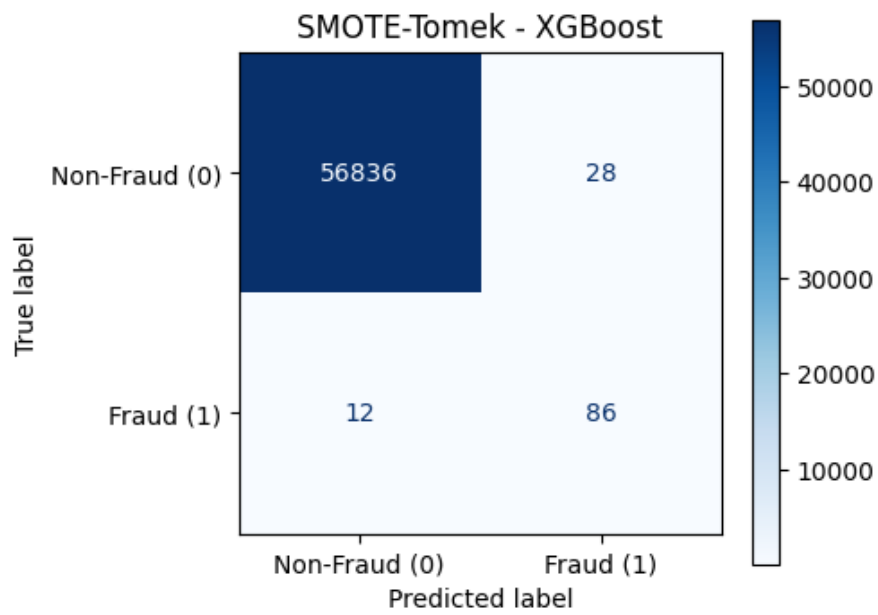


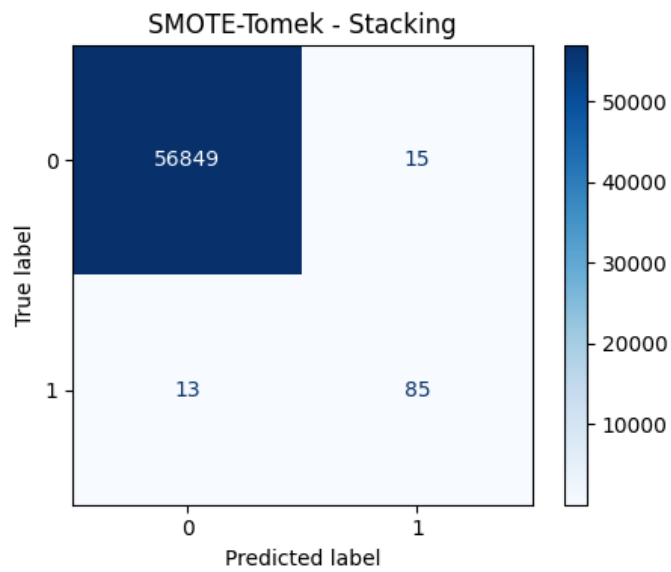




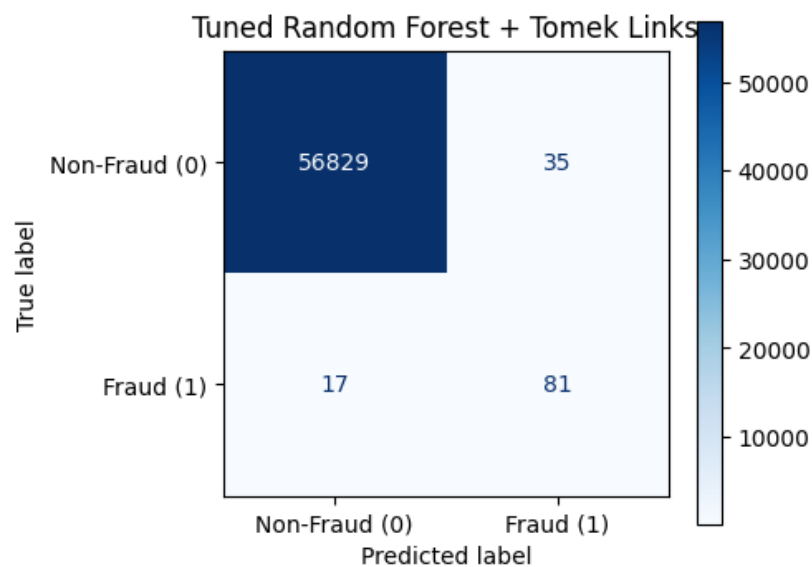
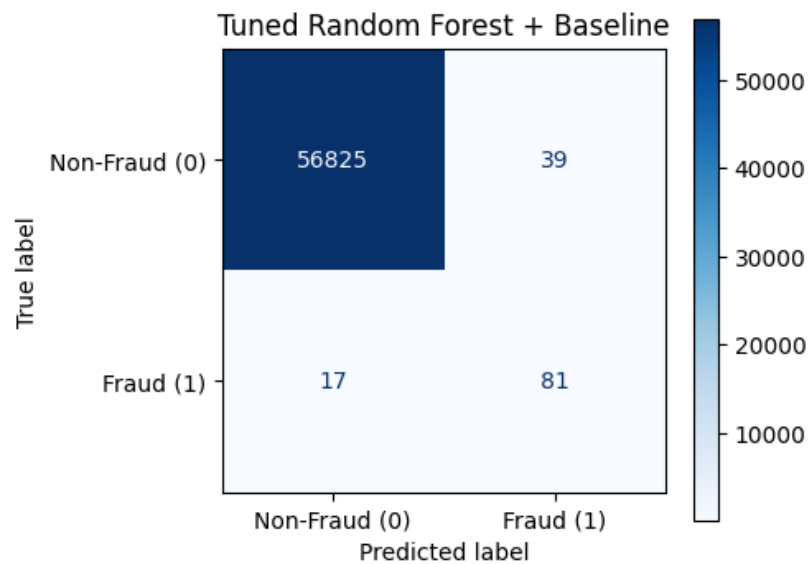


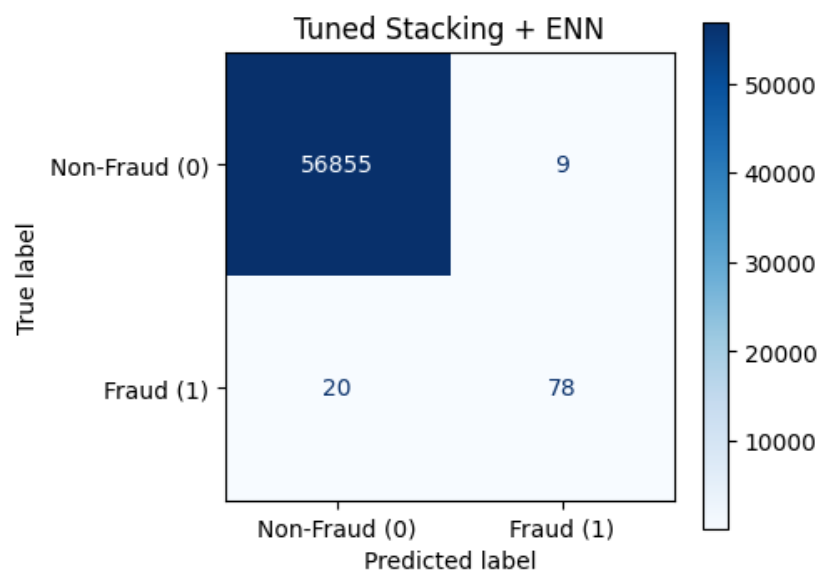
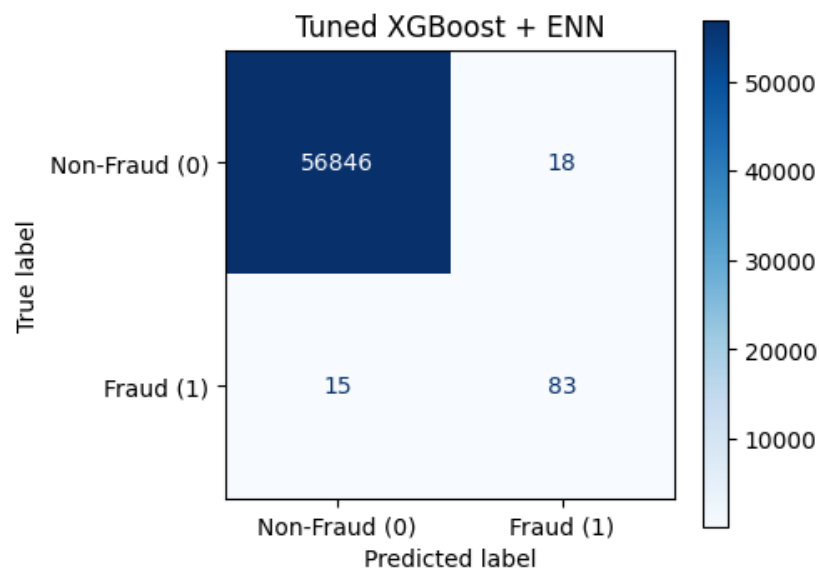
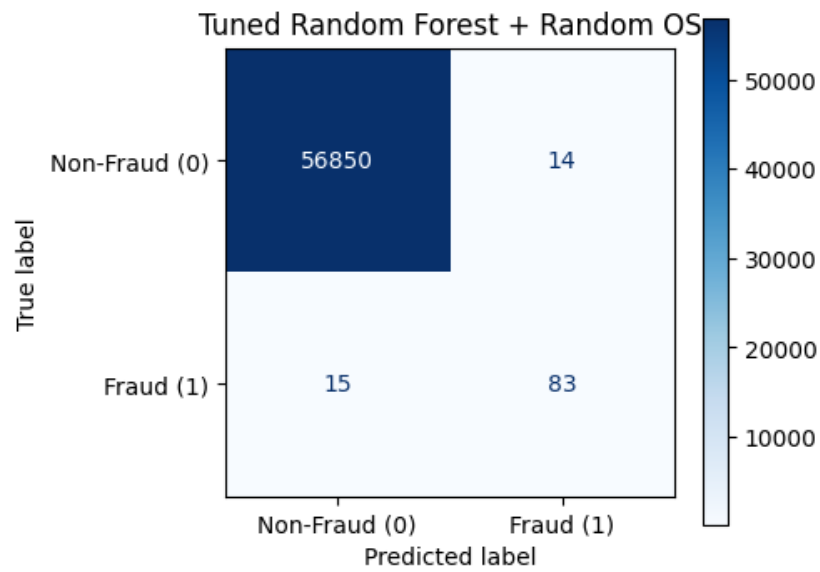






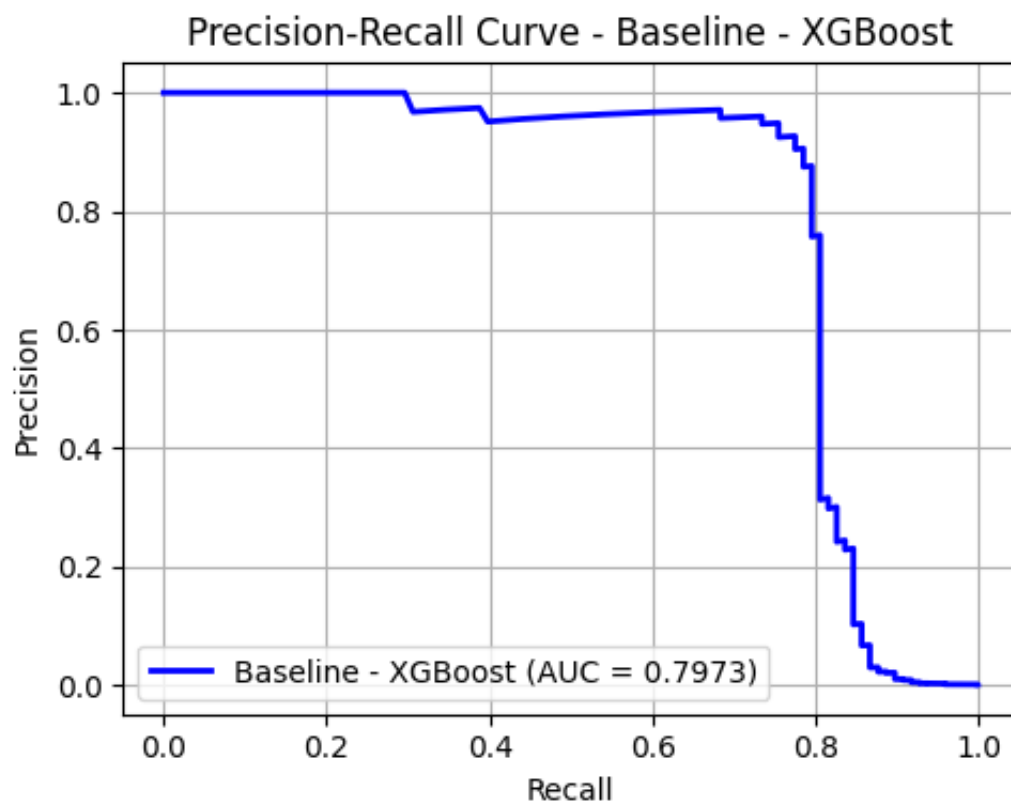
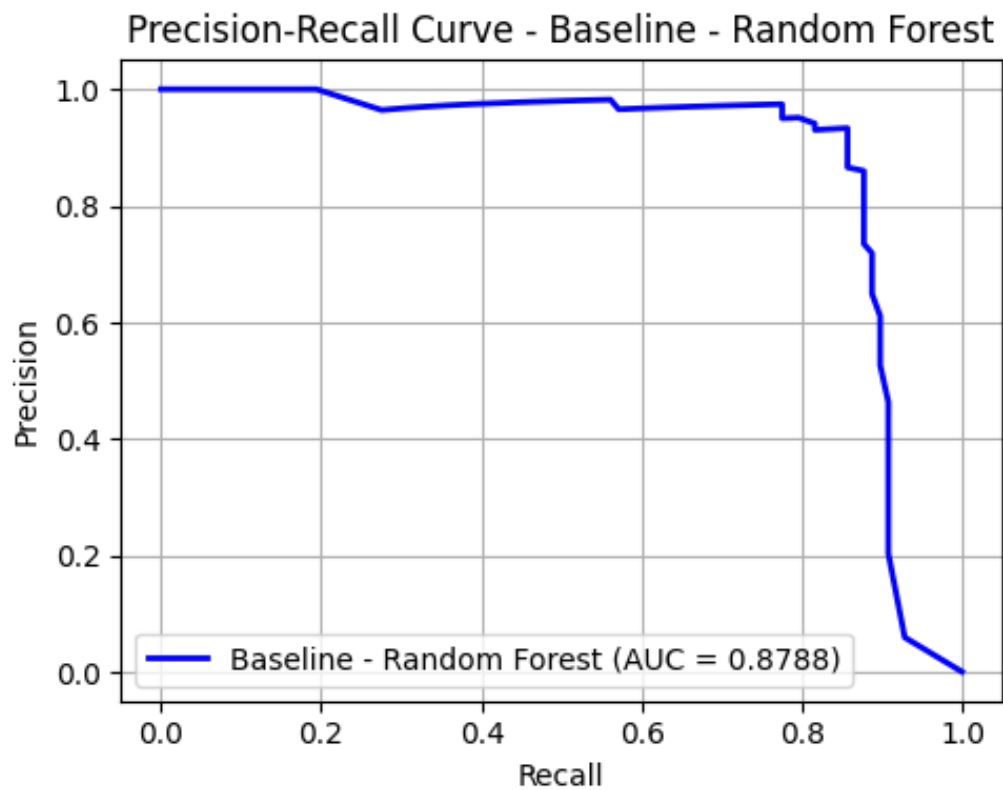
C. Pengujian Skenario Hyperparameter Tuning

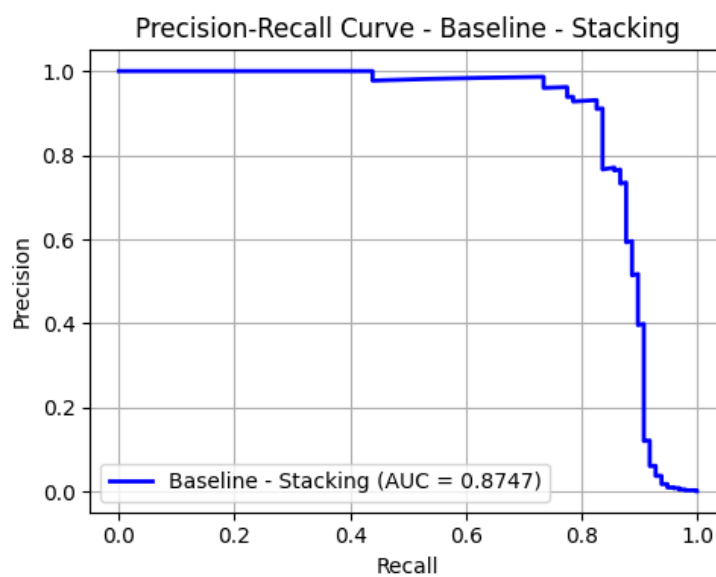
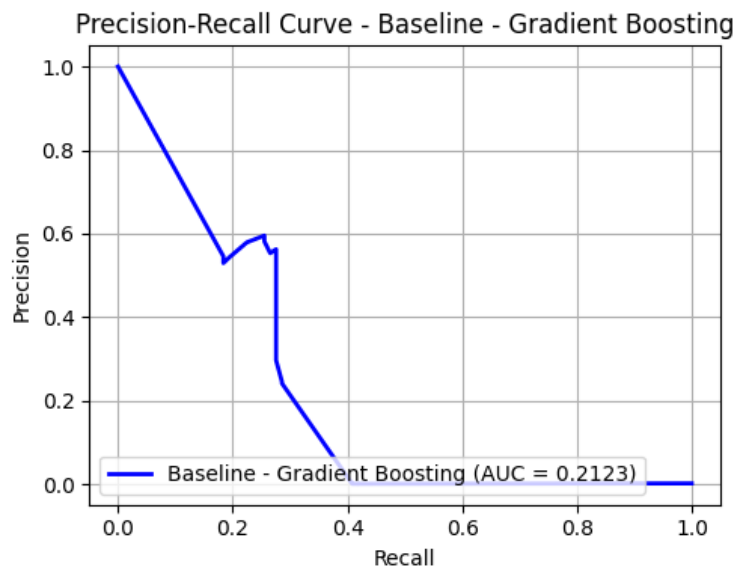
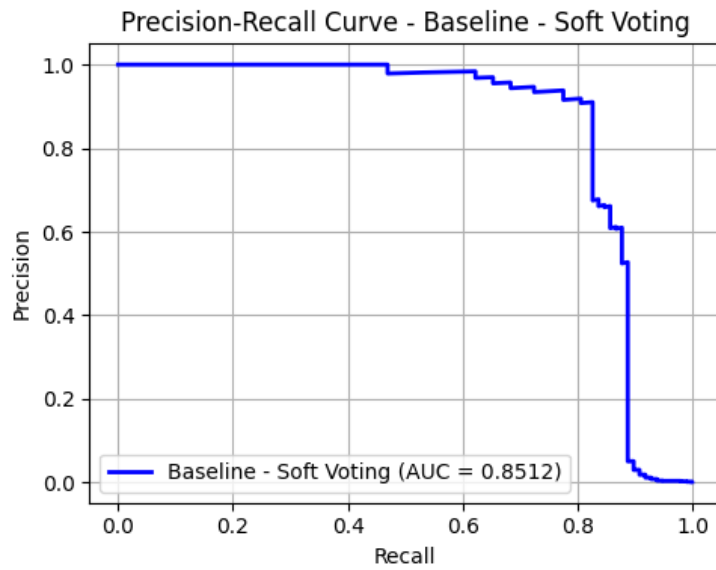




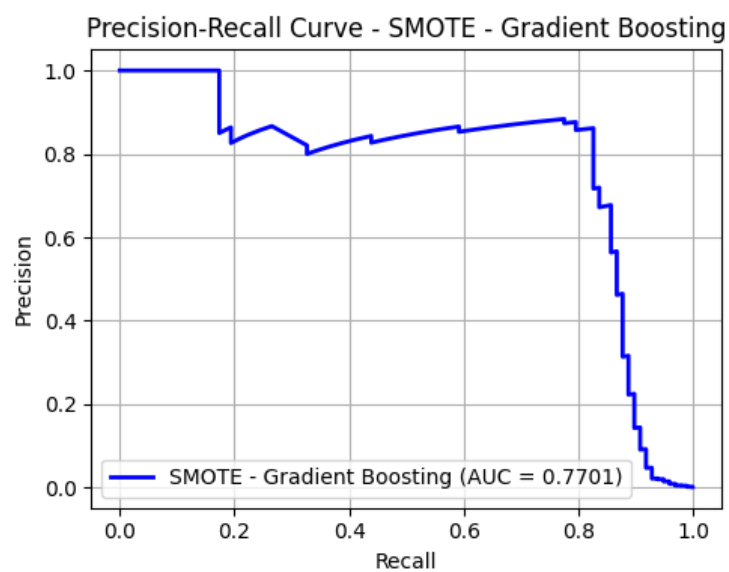
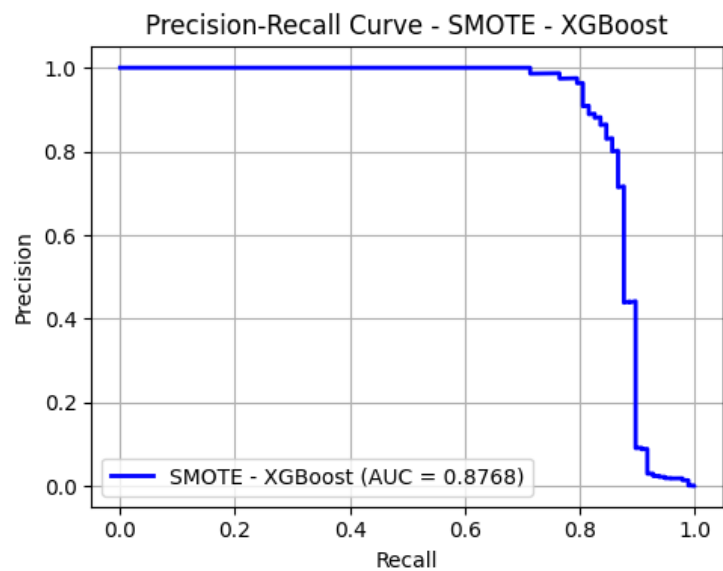
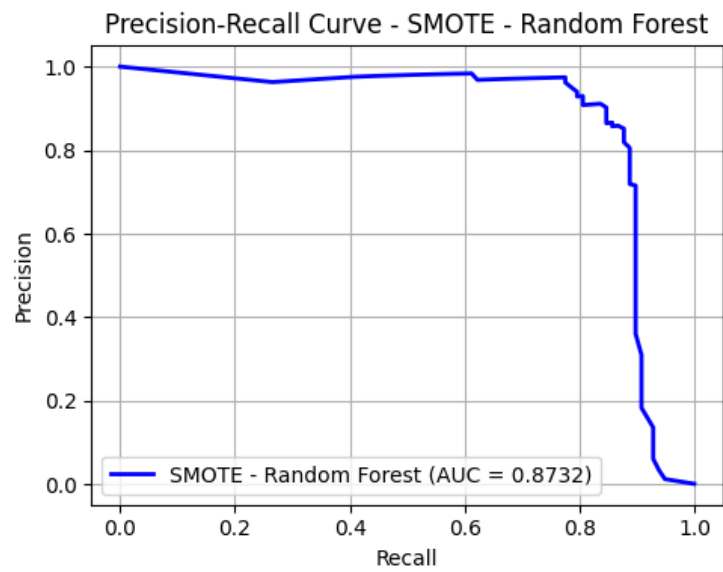
2. Grafik PR-AUC Setiap Kombinasi Pengujian

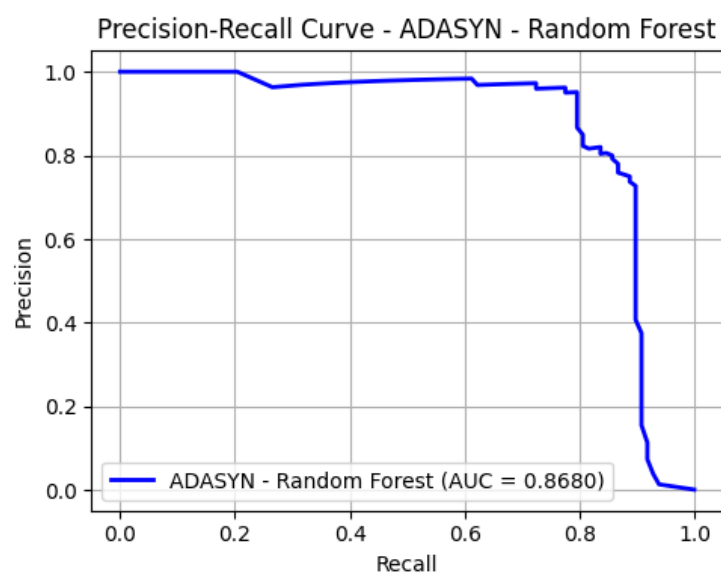
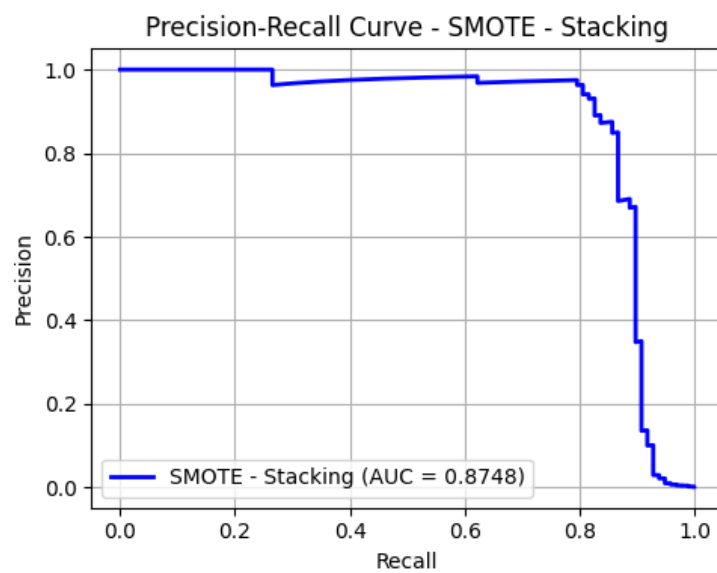
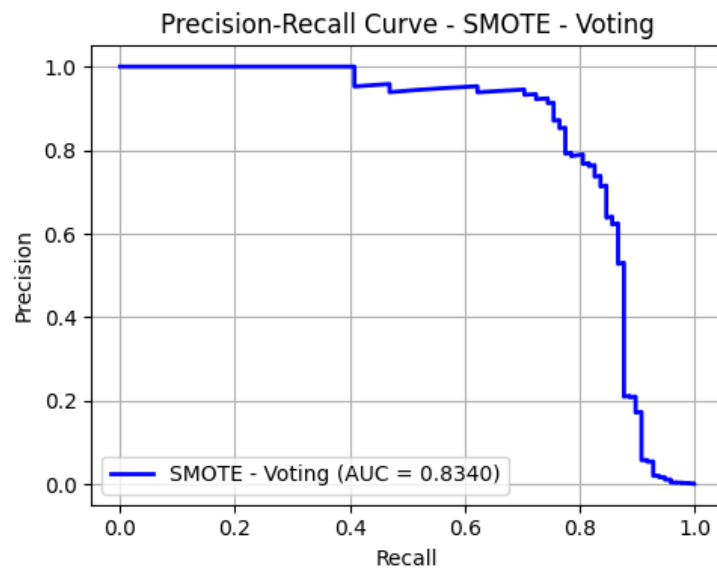
A. Skenario Pengujian Baseline

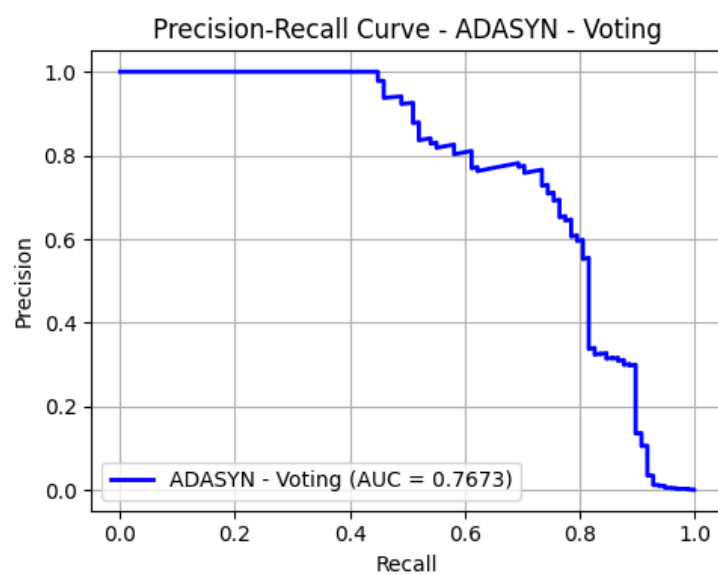
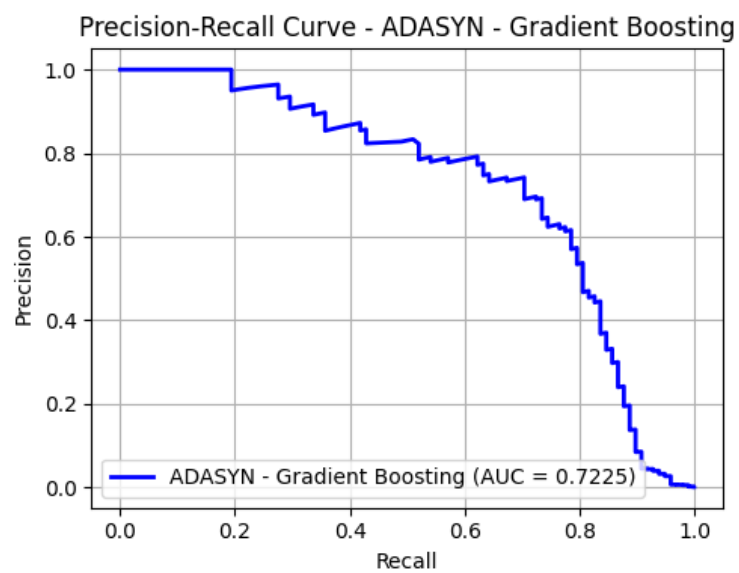
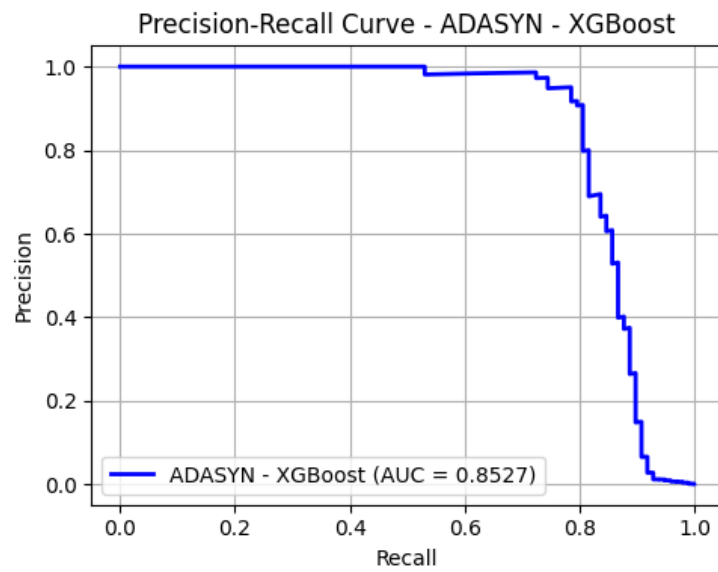


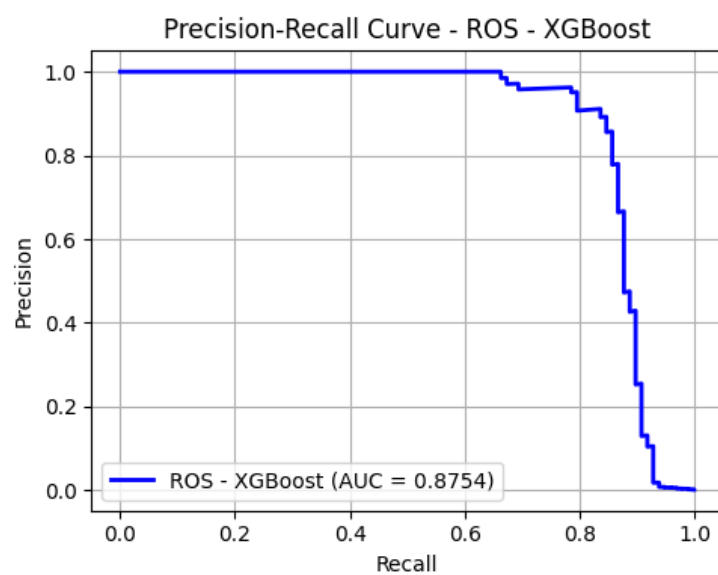
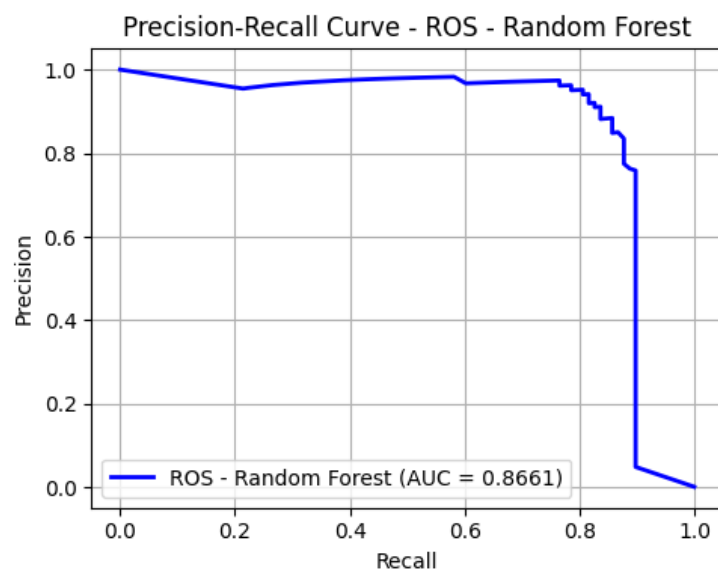
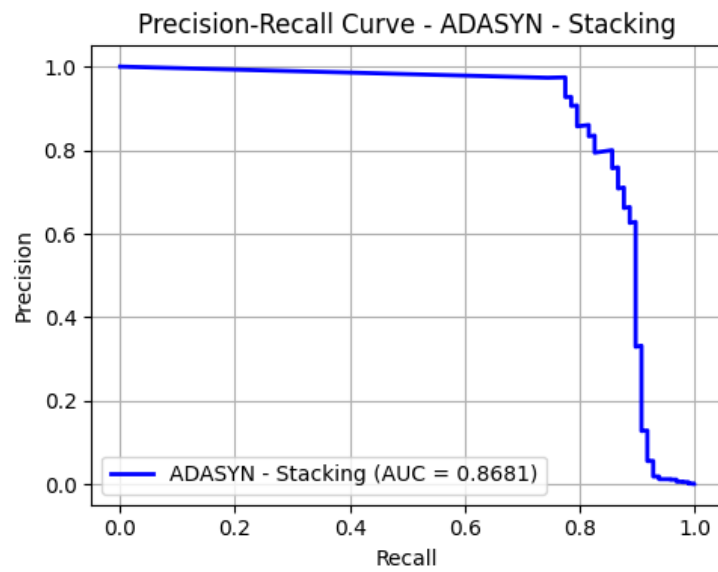


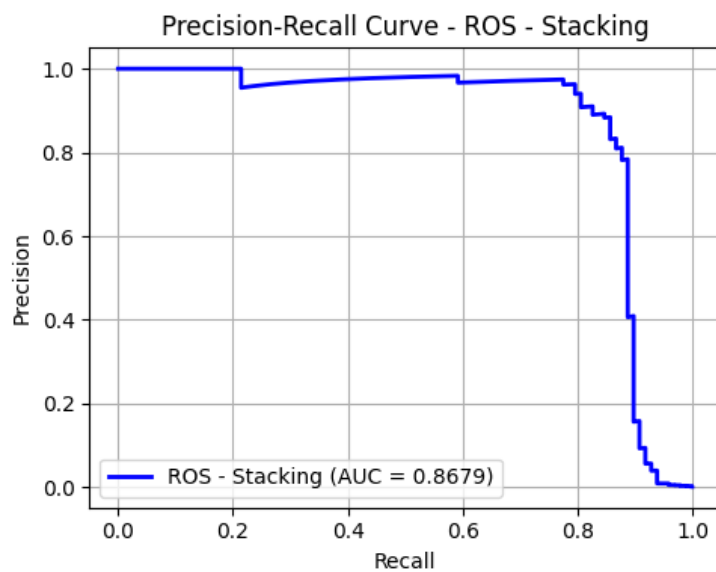
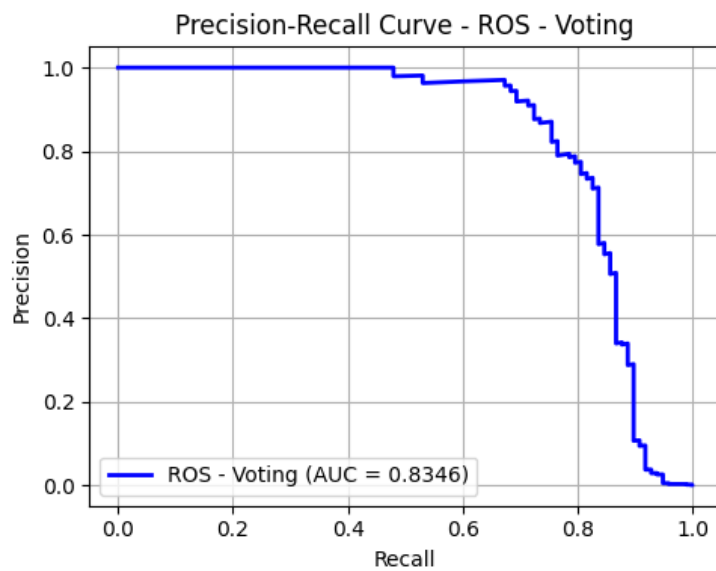
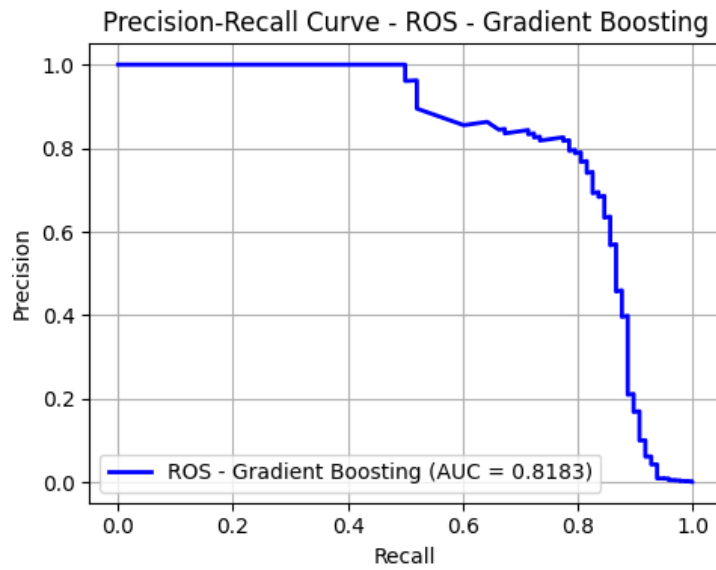
B. Skenario Pengujian Teknik Resampling

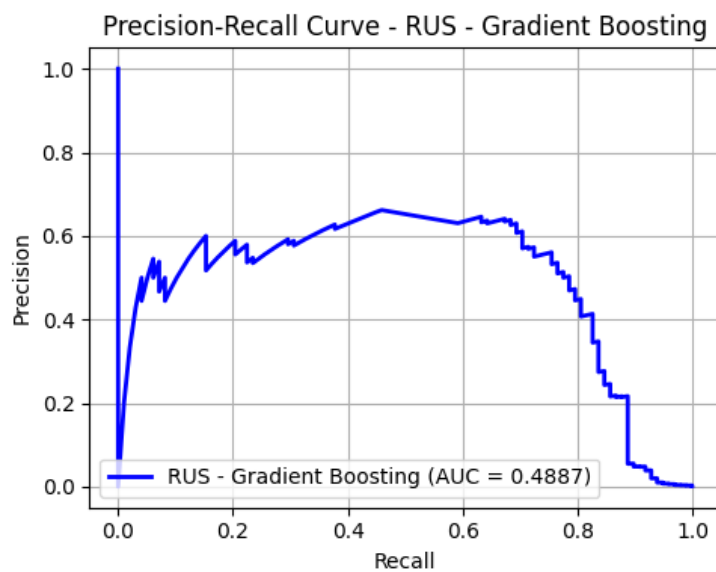
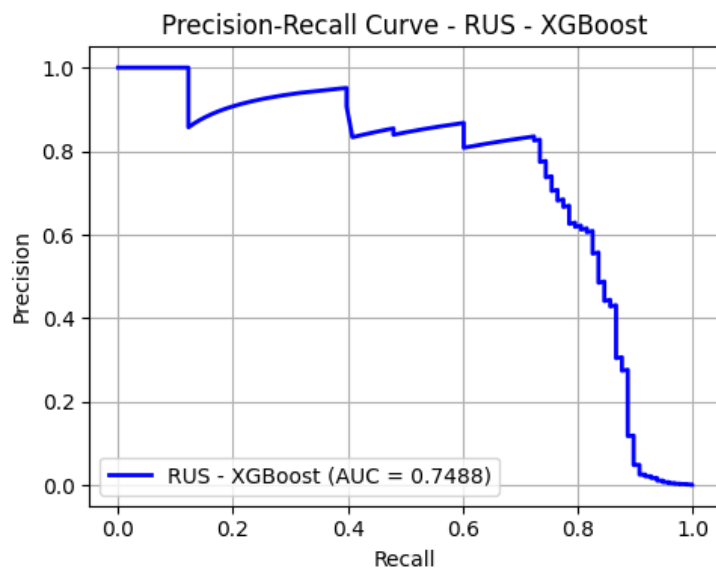
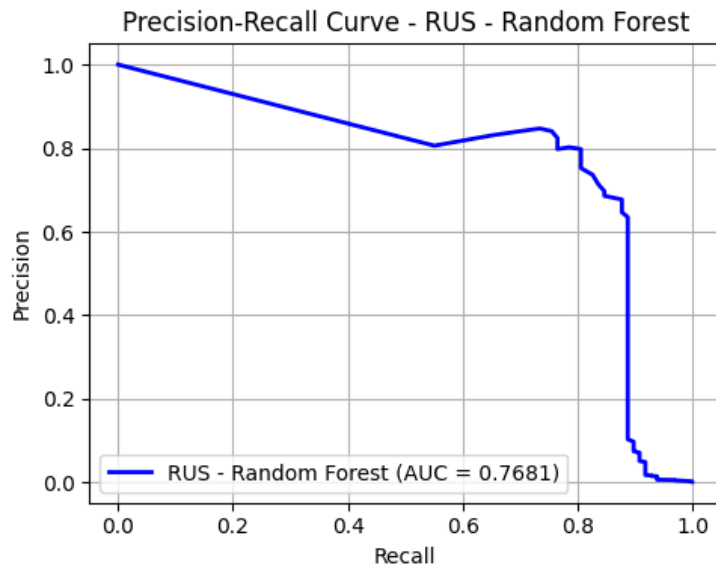


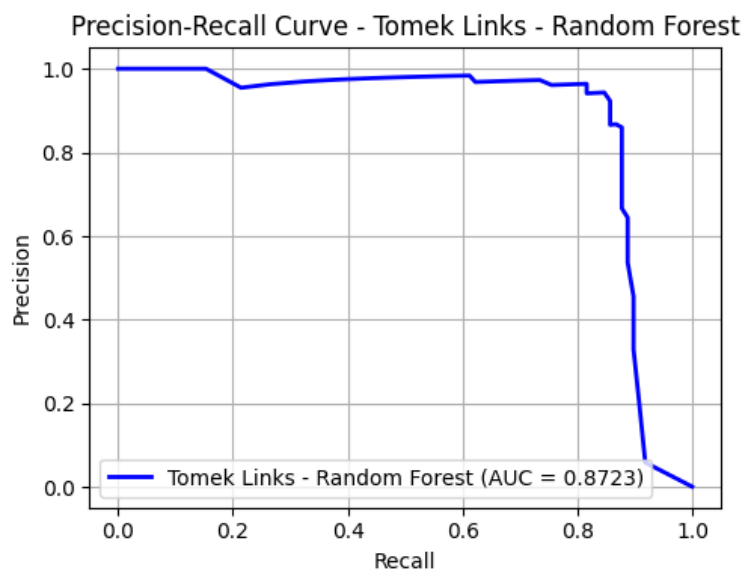
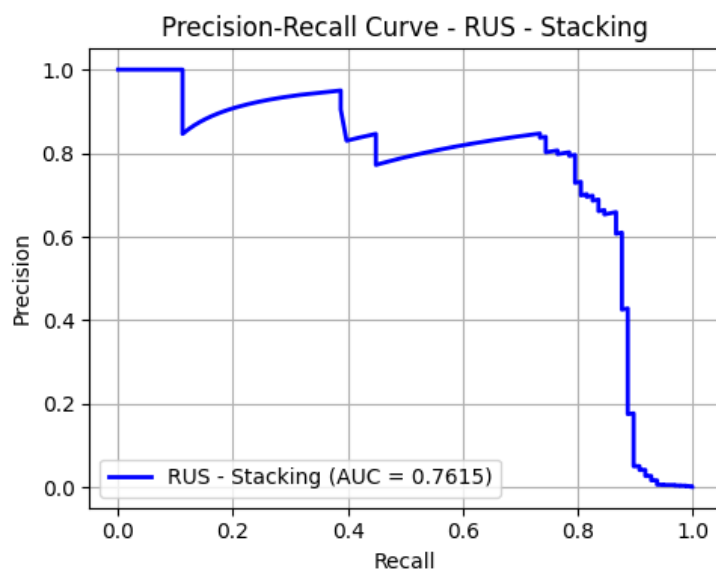
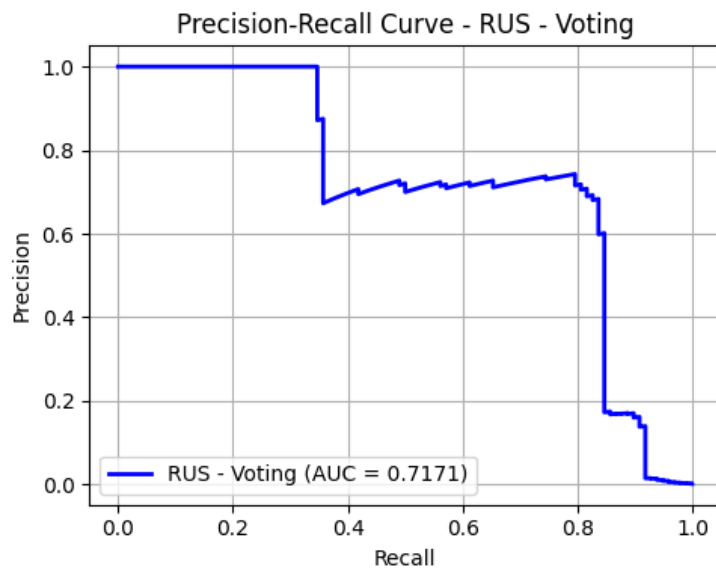


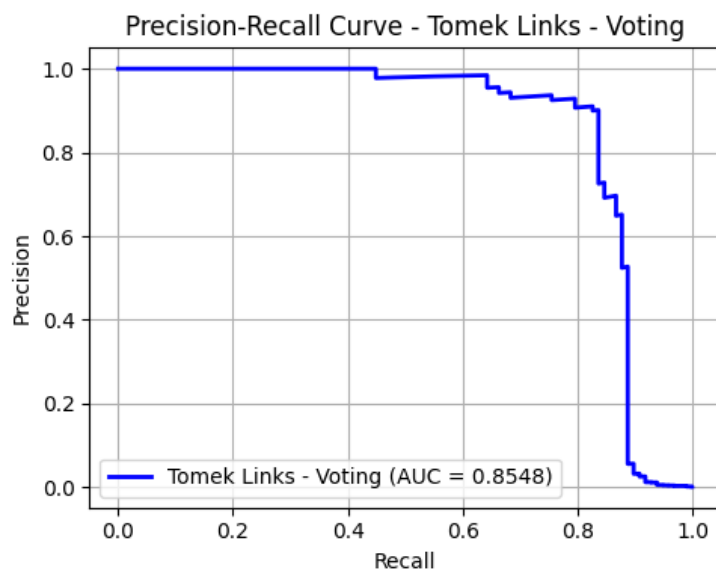
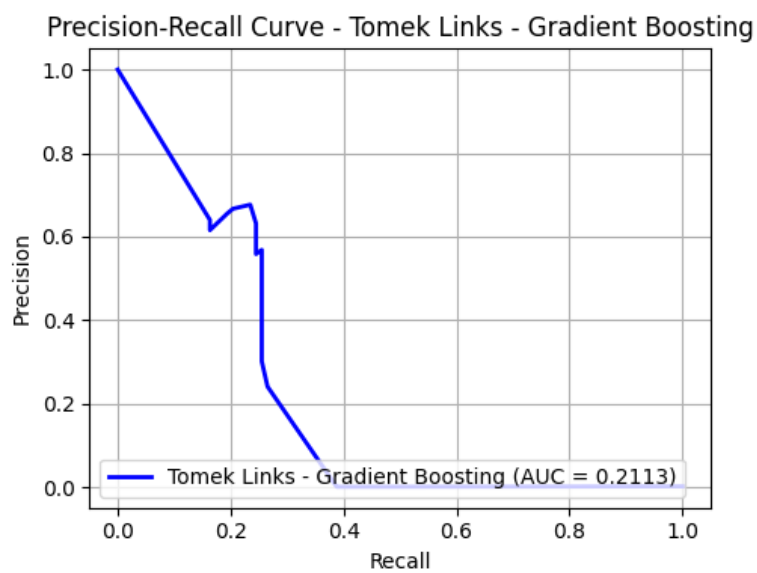
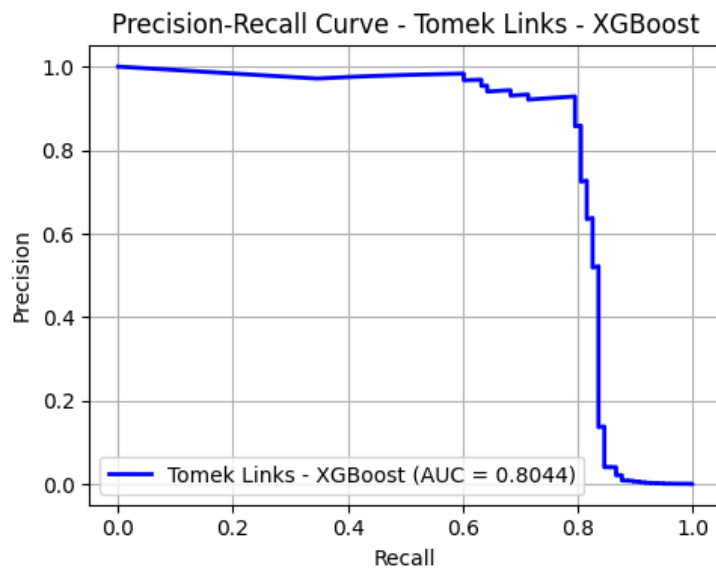


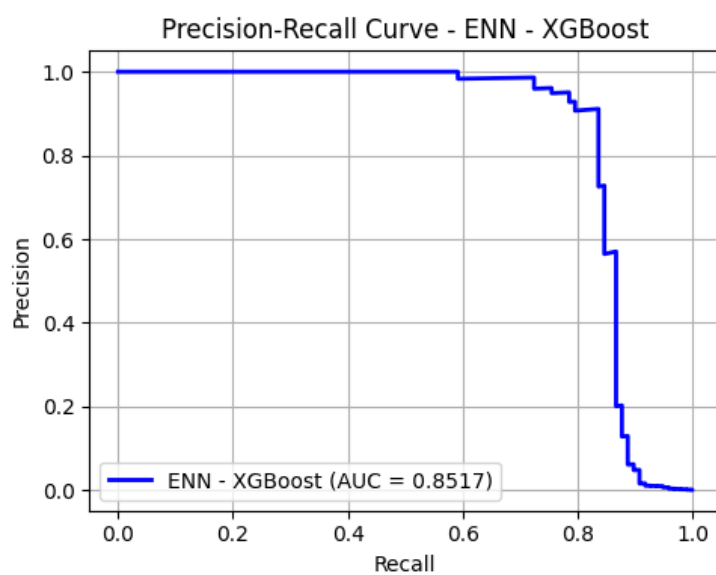
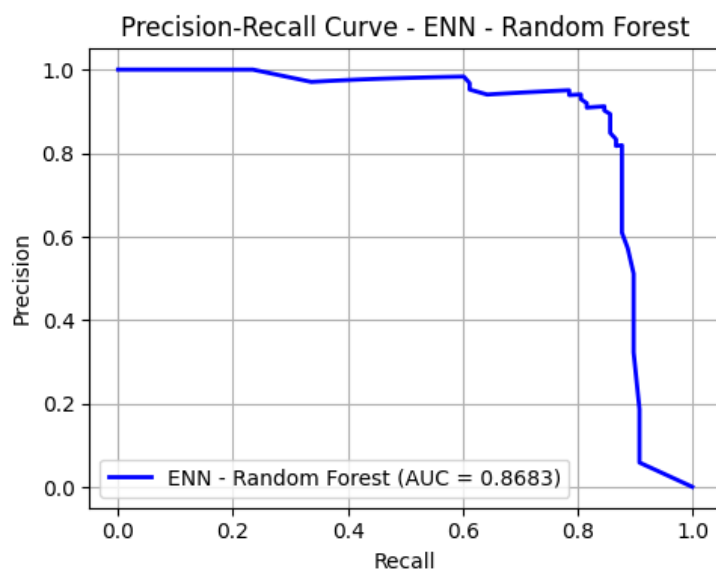
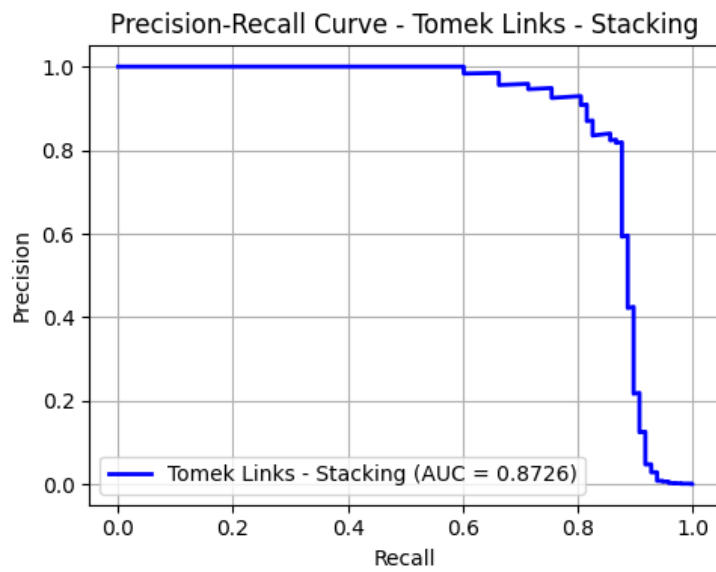


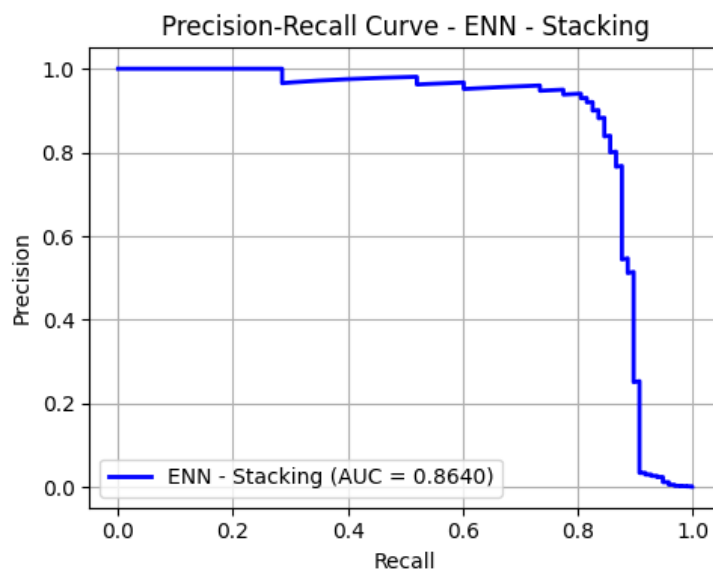
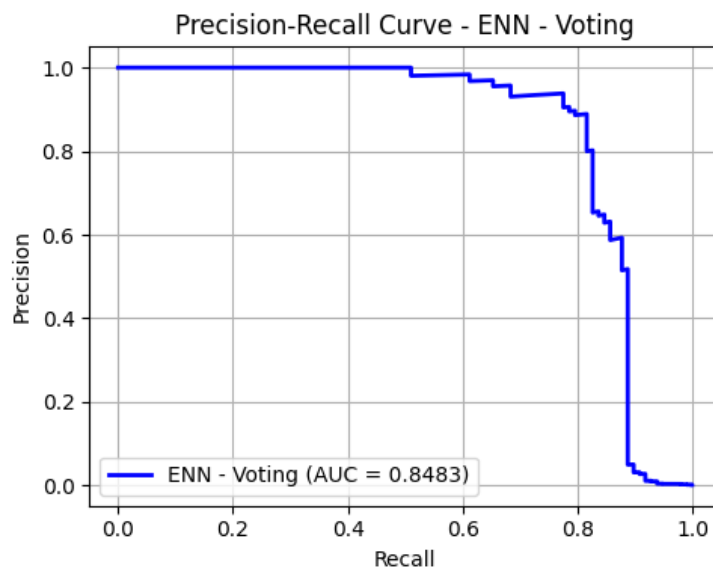
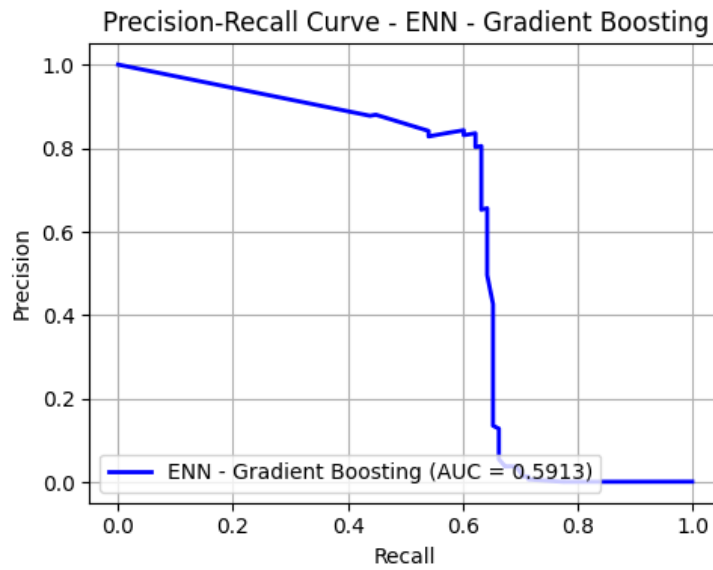


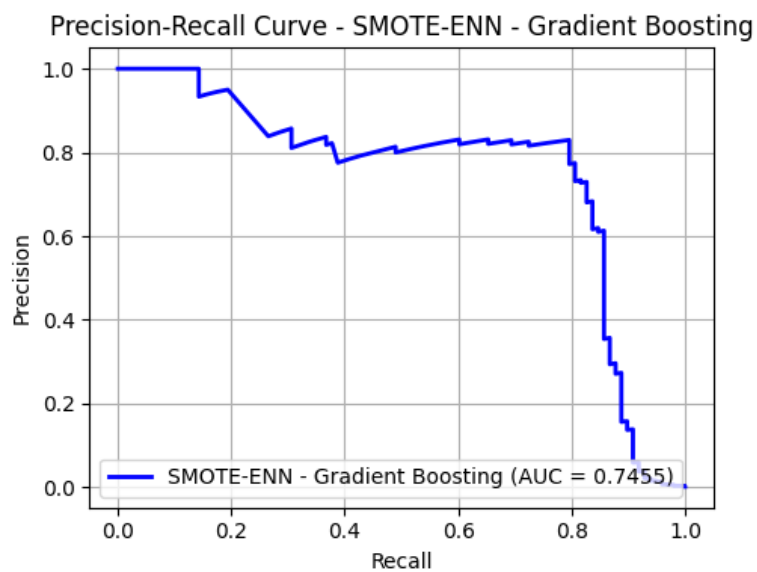
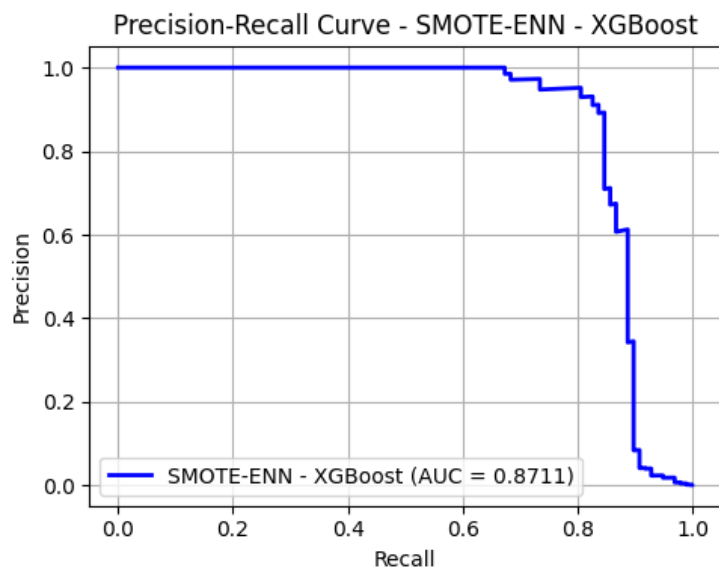
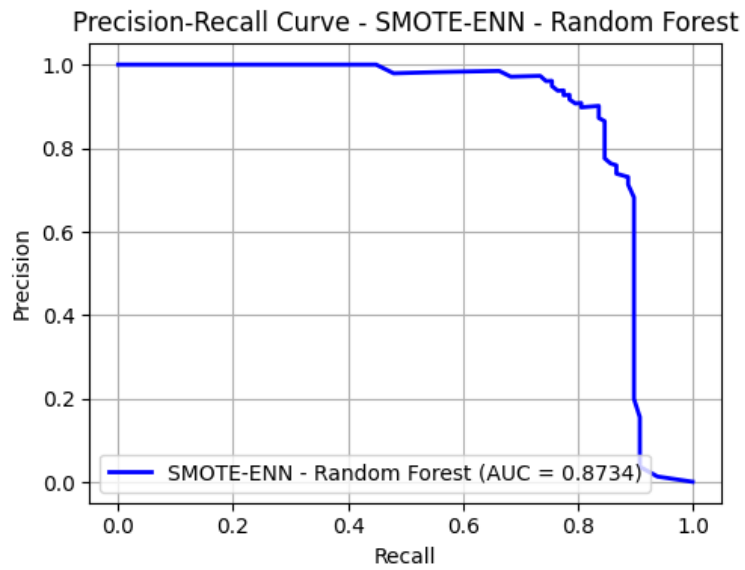


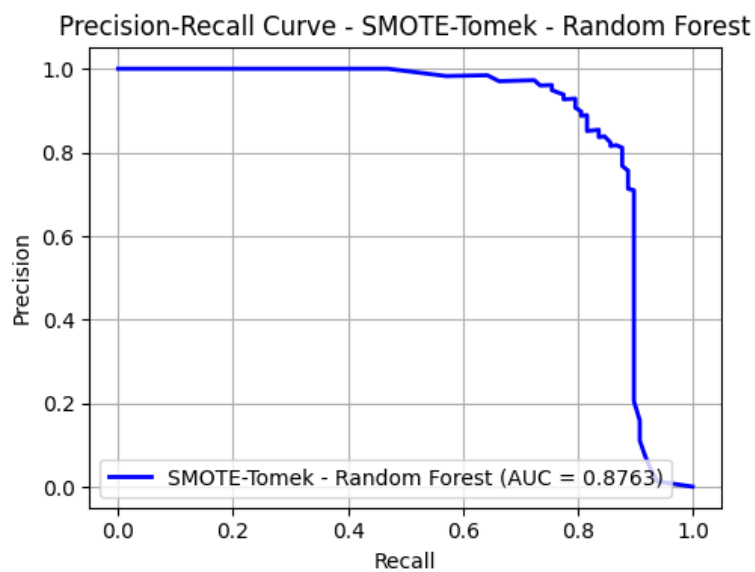
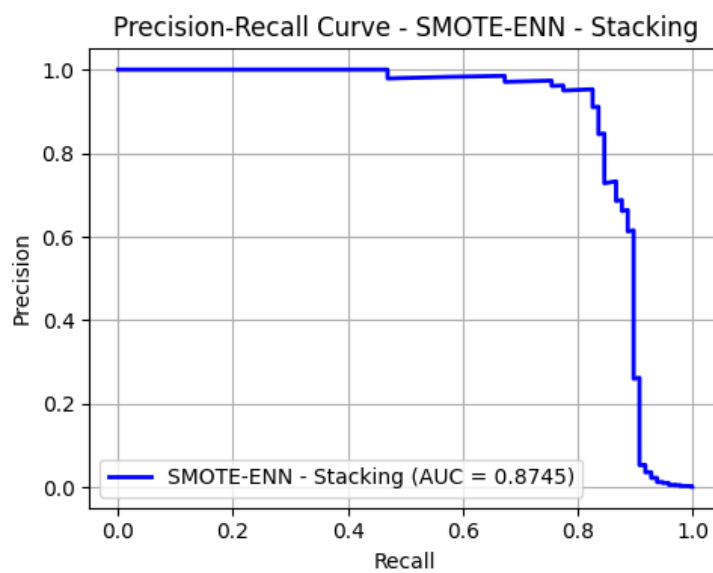
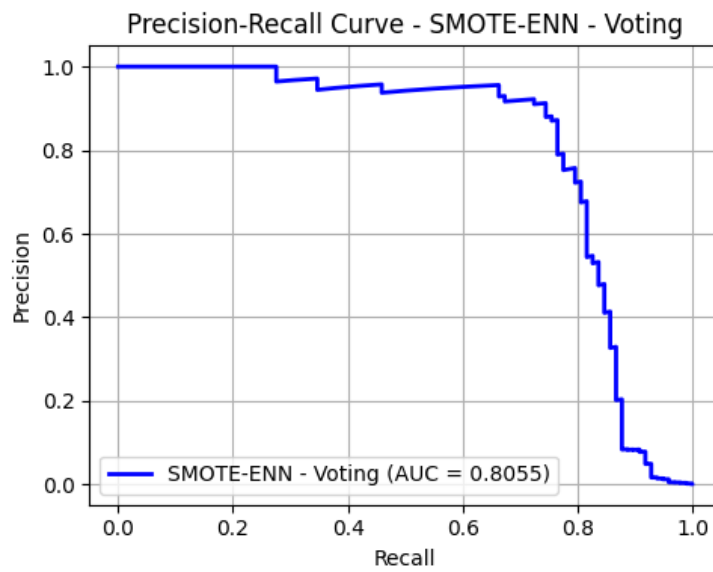


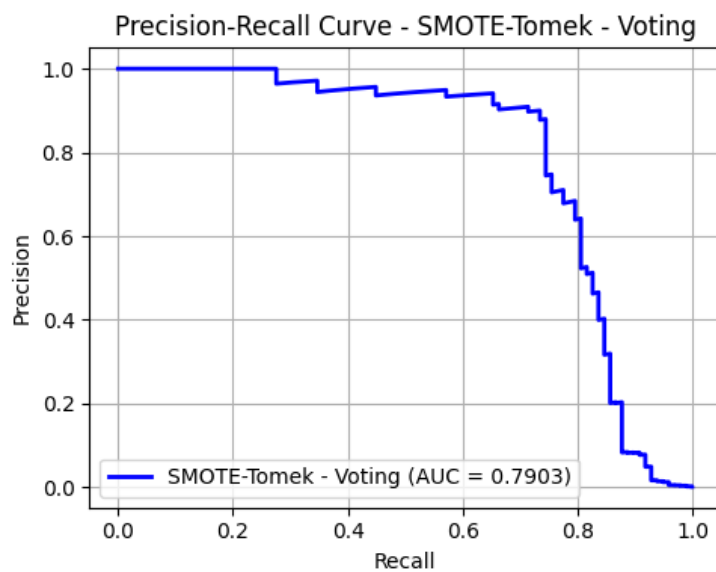
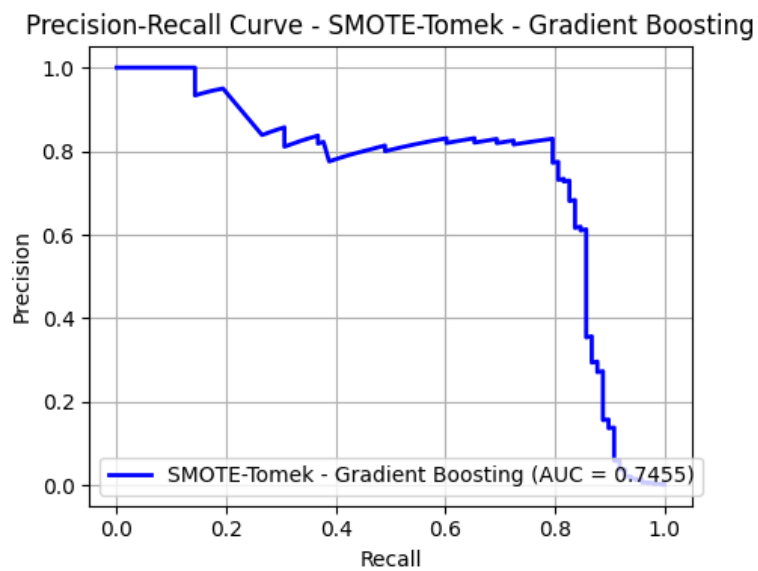
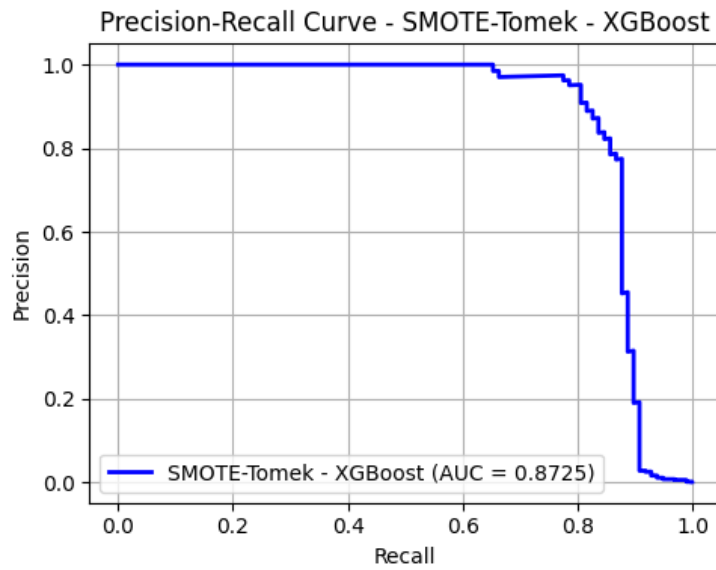


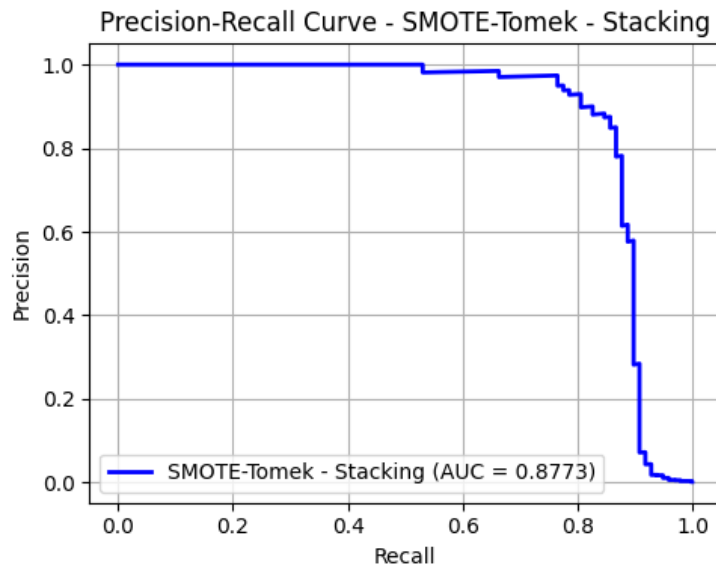






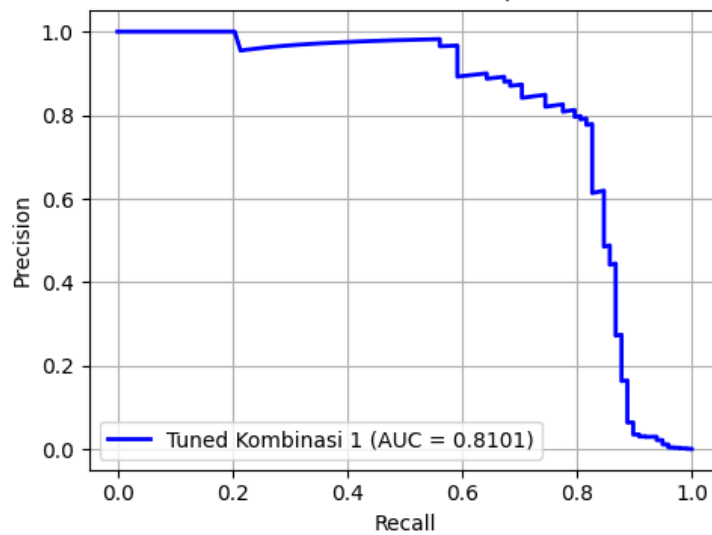




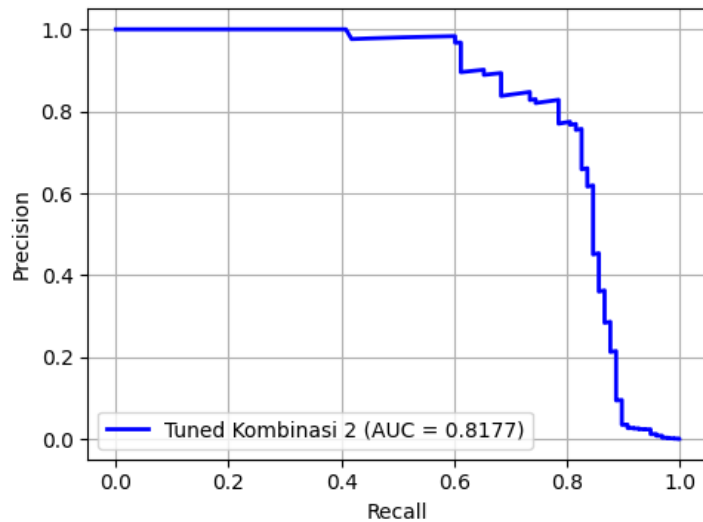


C. Pengujian Skenario Hyperparameter Tuning

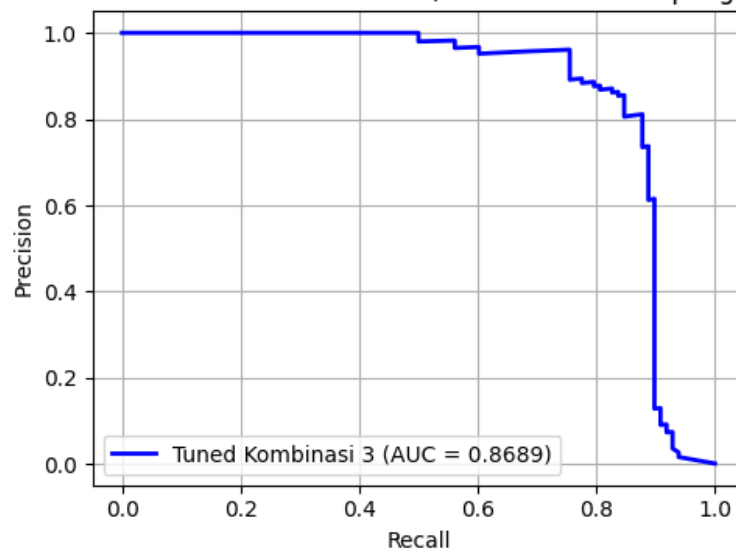
Precision-Recall Curve - Tuned Kombinasi 1 (Baseline x Random Forest)



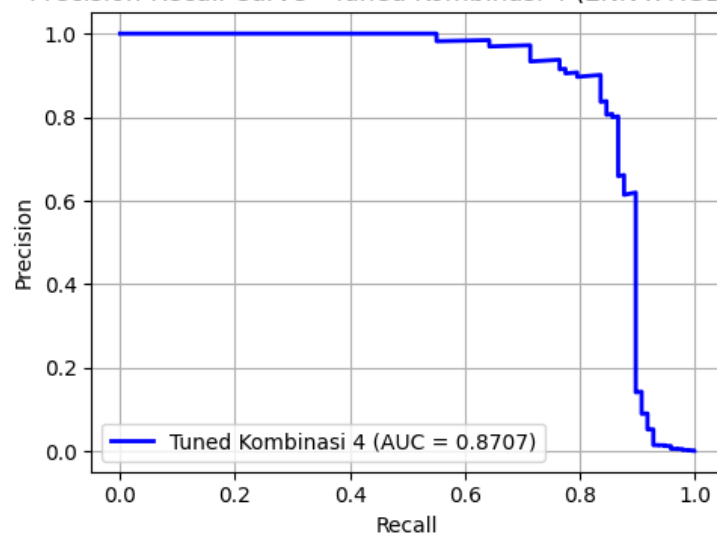
Precision-Recall Curve - Tuned Kombinasi 2 (ENN x XGBoost)



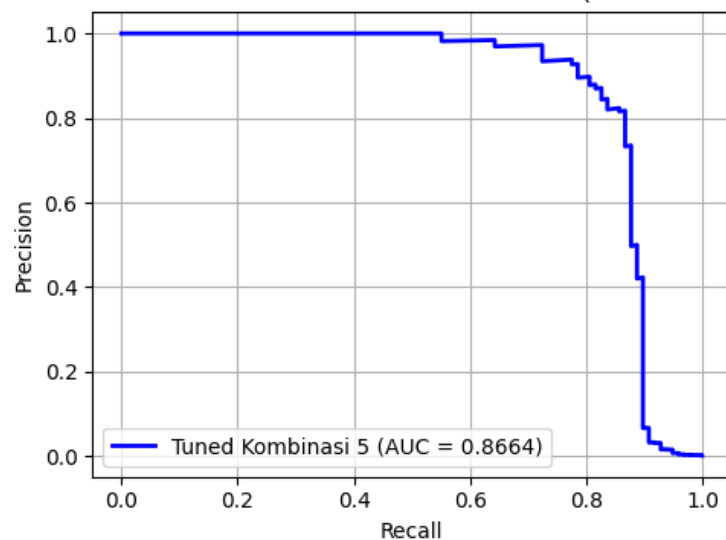
Precision-Recall Curve - Tuned Kombinasi 3 (Random Oversampling x Random Forest)



Precision-Recall Curve - Tuned Kombinasi 4 (ENN x XGBoost)



Precision-Recall Curve - Tuned Kombinasi 5 (ENN x Stacking)



BIODATA PENULIS



Davin Fisabilillah Reynard Putra atau biasa dipanggil Davin dilahirkan di Bekasi, 8 Juli 2004. Penulis merupakan anak pertama dari 3 bersaudara. Penulis telah menempuh pendidikan formal yaitu di TK ABC Kids, SD Labs School Kaizen, SMP Islam Al-Azhar 19 dan SMAN 1 Cileungsi. Setelah lulus dari SMAN tahun 2022, Penulis melanjutkan studi pendidikan tinggi setelah diterima di Departemen Teknik Informatika, Fakultas Teknologi Elektro dan Informatika Cerdas, Institut Teknologi Sepuluh Nopember (ITS), Surabaya.

Selama masa perkuliahan di Departemen Teknik Informatika, penulis aktif di beberapa organisasi mahasiswa serta mengikuti kegiatan pengembangan diri. Penulis menjadi panitia Schematics selama 2 tahun dimana penulis memegang jabatan sebagai Staff dan Kepala Sub Divisi di divisi operasional. Penulis bertanggung jawab dalam persiapan dan jalannya kegiatan Schematics Reeva dari sisi operasional event serta mengkoordinasikan staff dan pengunjung saat event sedang berlangsung. Penulis juga bergabung ke Himpunan Mahasiswa Teknik Computer-Informatika (HMTIC) sebagai staff divisi kompetisi Departemen Student Talent & Interests (STI). Selama menjadi bagian dari HMTIC, penulis mengorganisasikan kompetisi intra departemen seperti TC Grandcup dan Quadrathlon yang dimana penulis dipercaya sebagai Project Officer. Selain itu, penulis juga terpilih sebagai *Organizing Komitee* (OC) pada kegiatan Nawasena dan OKKBK yang merupakan kegiatan pengembangan sumber daya manusia di Departemen Teknik Informatika ITS.